

MEASURING AND MITIGATING SOLUTION MODE COLLAPSE IN RLVR

Anonymous authors

Paper under double-blind review

ABSTRACT

A language model (LM) can usually answer the same question in more than one way, but reinforcement learning with verifiable rewards (RLVR) is indifferent to which correct answer a model produces. A solution will earn the same reward whether it is the thousandth copy of a familiar answer or one the model has never produced before. Yet, there is potential value in having the model retain multiple correct solutions as it is trained. For instance, multiple modes may give users a choice and provide problem-solving strategies that improve overall model performance. Here, we measure solution diversity through RLVR training using MODEBENCH, a set of reasoning tasks, where the verifier returns both correctness and mode discovered. We find that RLVR post-training concentrates probability onto fewer correct modes even as accuracy holds or improves, and moreover, that frontier models are already highly concentrated. We then introduce Re:Max, which uses the verifier to index a memory containing one verified example per discovered mode and rehearses those modes uniformly. A solution found once is, therefore, practiced as often as one found repeatedly. Across three model scales, two RL objectives, and harder task constructions, rehearsing verified successes raises how often a policy succeeds, and rehearsing them uniformly is what preserves how many ways it can succeed.

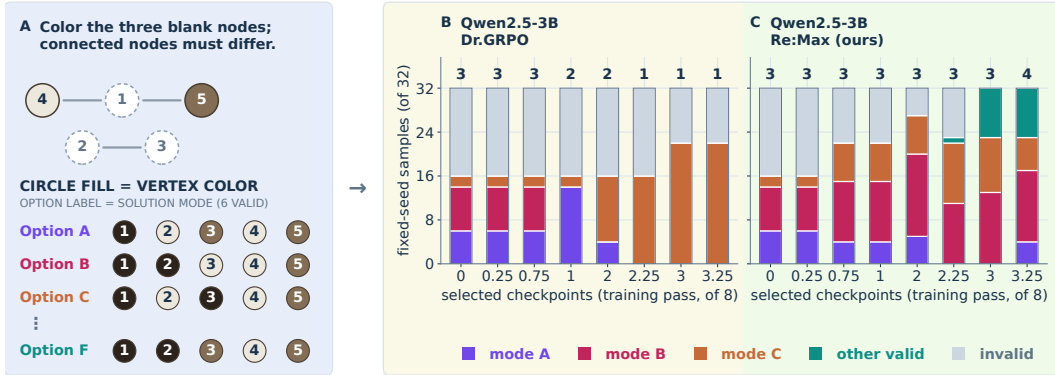


Figure 1: **Motivation.** Consider the Graph coloring problem shown in A (left), with six valid solutions. We evaluate Qwen2.5-3B across training, tracking the number and diversity of correct solutions. At each training pass, we show 32 sampled solutions, pooling eight samples from four matched runs. Training with Dr. GRPO (B) improves accuracy, but the model progressively collapses onto a single solution. In contrast, Re:Max (C) improves accuracy and expands the diversity of correct solutions.

1 INTRODUCTION

We usually evaluate reinforcement learning with verifiable rewards (RLVR) post-training by how often a model succeeds. But test-time search and reranking depend on having alternatives to choose from (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Wang



ModeBench dataset and models



Code repository

Links anonymized for review.

et al., 2023). This raises a question: as a model gets better at finding a correct answer during RLVR, *what happens to other correct alternatives?*

RLVR trains against an executable check (Guo et al., 2025), but its binary reward does not distinguish a solution the model has produced a thousand times from one it has never produced before. Nothing in the objective asks it to maintain diversity. Fig. 1 shows what this can look like on a graph-coloring problem with six valid answers: under Dr.GRPO (Liu et al., 2025), the policy becomes more accurate while its correct outputs concentrate onto a single mode.

Mode collapse, or narrowing of the output space, has been observed in alignment (Kirk et al., 2024; Cui et al., 2025), creative generation (Doshi & Hauser, 2024), brainstorming and ideation (Anderson et al., 2024), public argument (Kim et al., 2026), and LM-assisted writing (Abdulhai et al., 2026). Quantifying this narrowing requires deciding when two outputs are actually the same answer. Token-level diversity (Li et al., 2016; Zhu et al., 2018) counts two spellings of one solution as variety, while a judge that groups answers by meaning (Farquhar et al., 2024; Zheng et al., 2023) puts a second model between the data and the claim.

Here, we investigate the prevalence and impact of mode collapse in problem-solving tasks under RLVR post-training. Our contributions are threefold:

1. **MODEBENCH**: We contribute a benchmark for empirically studying mode collapse in reasoning tasks. **MODEBENCH** provides both output correctness and maintains a unique identity for each available modal solution to a given problem. The benchmark spans five executable domains, each with five difficulty levels (Sec. 2).
2. We conduct comprehensive experiments on **MODEBENCH** across 4 open-weight model families and 7 frontier models. Across open-weight models and at 3 model scales, the number of solution modes collapsed in every domain under both Dr.GRPO and GRPO. Further, in inference, all frontier models we measured reached fewer than 1.5 effective modes (Sections 5.1 and 2.3).
3. We propose *mode replay*: store one verified exemplar per mode and rehearse modes equally. Adding replay to MaxRL (Tajwar et al., 2026) gives Re:Max, which improves accuracy and correct-mode diversity across scales and task levels. A matched weighting ablation isolates the contribution of equal rehearsal to additional verified modes (Sections 3 and 5.2–5.3).

Overall, our findings shed light on solution mode collapse and suggest that incorporating mode replay as part of RLVR post-training can lead to both increased diversity of solutions and improved model performance.

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

Producing a benchmark to study solution mode collapse is challenging for two reasons. First, because we must be able to control that a specific question has several valid discrete solutions. Second, because we must be able to identify, from reasoning traces, which solution was reached. For example, in Fig. 2D, we show algebra problems in our benchmark. Note that we must both generate questions for which multiple solutions exist and, given a correct answer, be able to determine which solution mode is present.

Ensuring these two properties hold is domain-specific. Fig. 2 illustrates a subset of solution modes across the considered domains; e.g., in **Countdown** and **MathIR**, modes mean the model reached

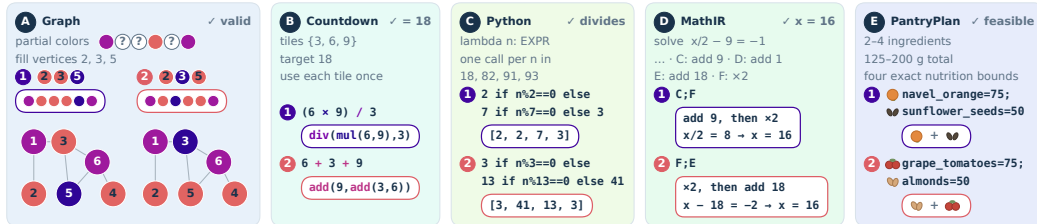


Figure 2: **MODEBENCH** comprises questions with many acceptable solutions across domains. (1) and (2) identify different solution modes across the five domains, which include Graph coloring, a normalized computation tree, executable Python code, algebraic problems and constraint optimization.

the same answer by a different route, whereas in Graph and PantryPlan, modes mean a different valid answer altogether. App. B and App. C give the per-domain rules and the prompts.

For each domain, we provide a verifier that evaluates whether a solution is correct and which solution mode a solution belongs to. This was built by creating in advance a comprehensive set of potential solutions to the problem, which at training time allows us to quickly sort the reasoning traces associated with a solution into a pre-computed solution mode. Precisely, for a prompt x with specification s_x , the validator maps an output y either to failure $\perp$ or to a key $V(y, s_x) \in \mathcal{C}_x$. Two accepted outputs are the same mode if and only if their keys agree. The full set of valid keys $\mathcal{C}_x$ is never revealed, neither its members nor its size. A policy, and the memory that stores its successes, only ever sees keys for answers the policy actually produced.

2.1 SUCCESS AND VERIFIED-MODE DIVERSITY

To measure performance on MODEBENCH, we must consider two distinct properties: how often the output is accurate, and, given that it is right, which of the available right answers has been given. A benchmark reporting only accuracy cannot capture the failure we study.

That obvious way to measure modal diversity is to count distinct correct answers in K samples. Let P be the chance a sample is correct and q_c be how that correct probability is divided among the valid answers. Then

$$\underbrace{\Pr[D_K > 0]}_{\text{pass@K}} = 1 - (1 - P)^K, \quad \underbrace{\mathbb{E}[D_K]}_{\text{distinct@K}} = \sum_c [1 - (1 - Pq_c)^K], \quad (1)$$

where D_K counts the distinct correct answers. However, both metrics increase with P , and thus a policy that is right more often may have higher pass@K and distinct@K while producing less diverse solutions.

We therefore propose an alternative metric, pairwise correct-mode diversity:

$$\text{PCMD} = 1 - \sum_c q_c^2 = \Pr[Z_1 \neq Z_2 \mid \text{both responses verified}], \quad (2)$$

measuring the joint probability of different solution modes given two correct responses. A PCMD of zero thus implies every correct answer belongs to the same solution mode, and an even spread over m yields a PCMD of $1 - 1/m$, e.g., if answers spread evenly across two possible solution modes, then the PCMD is $1/2$. We note that when infinite solution modes are possible, the maximum PCMD converges to 1, and that when the model outputs no correct solutions, we cannot calculate the metric. We only calculate PCMD when models yield at least 30 correct solutions for a specific problem, to avoid excessively noisy measurements.

Fig. 3 shows how PCMD captures diversity of responses independently of accuracy. GPT-5.6 Sol has near-perfect pass@8 for all five tasks considered, yet much weaker models in the

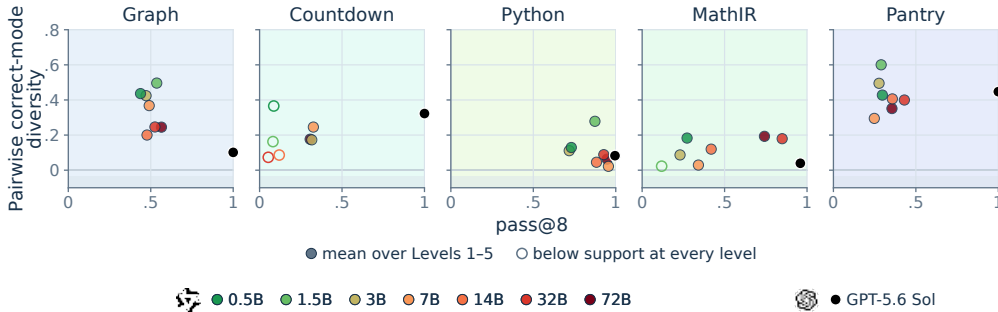


Figure 3: **Higher performance is not associated with increased solution mode diversity.** We show the relationship between a performance-related metric (pass@8; x -axis) and our proposed solution mode diversity metric (PCMD; y -axis) for GPT-5.6 Sol (in black), and Qwen2.5-Instruct models of various sizes (where parameter count is indicated by color). Note that accuracy and model size are not correlated with solution mode diversity.

Qwen2.5-Instruct family have higher PCMD. For the context of this work, we define a *solution mode collapse* as a decline in PCMD during RLVR.

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

We construct MODEBENCH so that tasks can be made more or less challenging, which helps prevent saturation. For each domain, we include 5 independent difficulty levels, e.g. for our Graph domain we allow more hidden vertices, and for Countdown we give the model more operands. Each difficulty level holds constant (1) our definition of mode, (2) our approximate distribution of modes, (3) our verifier constant, and (4) the sample size of 384 train and 128 test problems. To ensure (2) holds, and moreover, that the problems sample are in fact more challenging, we fit each level with consideration to $\text{pass}@1$ and $\text{pass}@8$ of a reference model, e.g. Level 4 was tuned such that a Qwen2.5-Instruct-7B model has the same $\text{pass}@1$ and $\text{pass}@8$ scores as a Qwen2.5-Instruct-0.5B on Level 1.

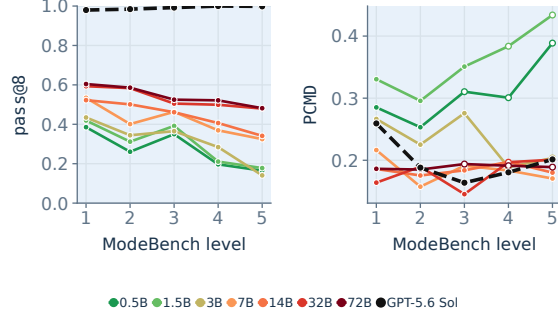


Figure 4: **Harder problems of the same domain exhibit fewer solution modes.** When we ask the Qwen2.5-Instruct family and GPT-5.6 Sol increasingly difficult questions from harder levels of MODEBENCH, $\text{pass}@8$ declined and so too does PCMD. For our benchmark, as problems get harder, solution mode collapse worsens.

LMs with a higher parameter count generally have higher $\text{pass}@8$ and can solve more problems. Yet, in Fig. 4, for our harder problems, we find that solution mode diversity declines, even as our underlying mode count per level remains stable. This is most noticeable when frontier models are evaluated. Even though correctness is saturated, we are able to use PCMD to isolate solution mode collapse (App. N), and this phenomena happen across MODEBENCH. Frontier models answer .991 of questions from level 1-3 correctly, but their mean PCMD is .232, which is below the .570 of a 1.5B model that answers .273 of its own prompts.

2.3 CONCENTRATION IN FRONTIER MODELS

Repeated sampling can increase solution coverage (Brown et al., 2024; Yue et al., 2025), but our discovery curves flatten far below the available modes (Fig. 5). On one Level-3 Graph problem, GPT-5.6 Sol answers correctly in all 1,024 draws yet produces only one of six valid colorings. For Python, 512 draws recover one to four return vectors from hundreds available:

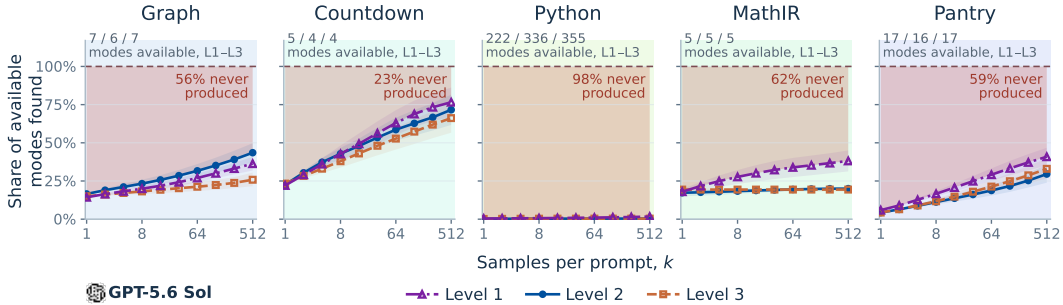


Figure 5: **Asking the model more times may not recover all available modes.** We ask each frontier model the same question eight times in our early experiments. However, a potential critique could be that we simply need to ask the model more times. So, we ask GPT-5.6 Sol, with medium reasoning on; 32 problems per domain and level, 512 responses each. Curves are distinct verified modes in random k -subsets as a share of the modes those problems admit, and the shaded band is what no level ever produced. Shading is the pointwise 95% problem bootstrap. Printed counts are the enumerated support those problems admit, in Level 1/2/3 order, and every panel shares one scale (App. K.3).

under 2% of the support, and under 1% at Levels 2 and 3. All seven hosted deployments narrow from Level 1 to Level 3 despite near-perfect correctness (Fig. 16).

Larger sampling budgets also leave trained local policies concentrated: quadrupling to $K = 32$ changes diversity by thousandths, and MathIR at 64 draws remains as concentrated as at eight (Apps. J and K). Disabling reasoning reduces both correctness and diversity (Table 56, App. O.6). Mixing independent models recovers more modes than either alone (App. L.2); raising temperature also adds modes, at an accuracy cost (Fig. 38).

3 RE:MAX: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

3.1 THEORETICAL INTUITION FOR MODE COLLAPSE

Binary rewards treat all correct modes alike, while fresh samples follow the policy’s current preferences. A mode with an early probability advantage is therefore sampled and reinforced more often (Fig. 6). Under independent logit updates, this feedback can produce winner-take-all concentration. Changing the binary-reward estimator alone does not change the trajectory of this concentration, as the following result makes precise.

For one isolated prompt, let a finite categorical policy have $m \geq 2$ correct modes and $n \geq 1$ incorrect categories, with $p = \text{softmax}(z)$, correctness $P = \sum_c p_c$, and conditional mode probabilities $q_c = p_c/P$. Each category has an independent logit. Draw responses $Z_1, \dots, Z_G$ independently with $G \geq 2$, with binary rewards R_i and $R = \sum_i R_i$. Assume finite initial logits, common response-length normalization, and on-policy Euclidean expected-gradient flow without KL, entropy, momentum, or preconditioning (Assumption P.1).

Theorem 3.1 (Conditional inertness of outcome-binary objectives). *Under Assumption P.1, let $a(\cdot, \cdot)$ be any detached advantage measurable with respect to the group’s binary reward vector, applied as $\hat{g}_a = G^{-1} \sum_i a(R_i, R) \nabla_z \log p_{Z_i}$ at the on-policy point. Then*

$$\mathbb{E}[\hat{g}_a] = c_a(P) \nabla_z P, \quad c_a(P) = \frac{1}{G} \left(\frac{\mathbb{E}[R a(1, R)]}{P} - \frac{\mathbb{E}[(G - R) a(0, R)]}{1 - P} \right). \quad (3)$$

If $c_a > 0$ on $(0, 1)$, then under the time change $d\tau = c_a(P)P(1 - P) dt$ the conditional distribution q obeys Equation (18) exactly, for every such a . Dr.GRPO, reward-standardized GRPO, a leave-one-out baseline, the centered binary MaxRL estimator, and any reweighting shaped by a pass@ k target are points in this class. The choice among them rescales optimization time along the correctness axis and leaves both the trajectory in q and the identity of the surviving mode exactly where they were.

The conditional dynamics are $dq_c/d\tau = q_c(q_c - \sum_d q_d^2)$: above-average modes grow at the expense of others. With a unique initial maximum and unbounded τ , one mode eventually dominates (Theorem P.4). The proof of Theorem 3.1 is in App. P.3. These are statements about the specified logit geometry, not guarantees for stochastic neural training. Replay changes the available targets by retaining successes that fresh sampling may stop producing.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

Replay revisits a discovered success even when fresh sampling no longer produces it, drawing on established uses of memory in RL (Lin, 1992; Mnih et al., 2015; Rolnick et al., 2019). Under a fixed-bank analysis, a constant replay dose keeps every retained mode above a positive probability floor throughout training (Theorem P.20).

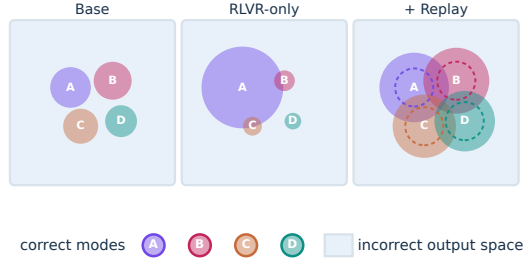


Figure 6: **RLVR early "winner" (A) becomes the predominant way to solve.** Under the base model, we see that before RLVR, many modes exist. However, RLVR-only training concentrates it on the mode found first. Our uniform replay approach keeps every banked mode and leaves room to find more.

For each training prompt x , we maintain an initially empty, bounded bank $\mathcal{B}_x$. The first time a verified mode appears, we store one exemplar e_c that realizes it (subsequent rediscoveries do not modify the bank). A deterministic schedule revisits nonempty banks throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (4)$$

Uniform weighting breaks the link between discovery frequency and rehearsal frequency. Once a mode enters the bank, it is rehearsed as often as every other retained mode, so frequently sampled modes receive no additional replay.

Adding this loss to MaxRL (Tajwar et al., 2026) gives Re:Max; adding it to Dr.GRPO (Liu et al., 2025) gives Re:Dr. Banked responses contribute only through the separate replay loss, leaving the on-policy reward group unchanged. Banks contain only training-prompt responses, so held-out evaluation measures the learned policy (App. D). Fig. 7 illustrates the two learning paths. Unlike methods that modify fresh rewards or advantages to favor rare outcomes (Song et al., 2025; Li et al., 2026a), mode identity determines which successes memory retains and rehearses.

4 EXPERIMENTAL DESIGN

We compare Dr.GRPO (Liu et al., 2025) and MaxRL (Tajwar et al., 2026), each with and without replay, on all five MODEBENCH domains. Level 1 uses Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B with five seeds, eight training epochs, and $G = 16$ fresh rollouts per prompt. Replay contributes a likelihood gradient on at most $M = 16$ stored exemplars within the same optimizer step (Table 6). Optimizer updates and fresh-rollout counts are matched; total FLOPs and wall-clock time are not. The zero-derivative control retains replay computation but does not redirect that work to extra task-gradient updates, fresh rollouts, or longer training. These comparisons test the replay mechanism, not superiority at equal total compute (App. Q.1). We repeat the comparison at Levels 2 and 3 with Qwen2.5-0.5B. At Level 1 and 0.5B, we also test frequency-weighted replay, UCPO (Lochab et al., 2026), RLEP (Zhang et al., 2025), GAPO (Anschel et al., 2025), SetPO (Li et al., 2026a), and fixed Semantic-MaxEnt.

Our separate inference-only studies evaluate GPT-5.6 Sol, GPT-5.4, Grok 4.3, Kimi K3, and Claude Opus 4.8 on Levels 1–3, with eight responses per prompt and a fixed formatting normalizer (App. O.6). Open-weight evaluations span the available scales of SmoLLM2, Qwen2.5, Falcon3, and OLMo-2 across Levels 1–5 (App. B.5). Deployment identifiers and protocols are recorded with these results; no training or replay is applied in these studies.

5 RESULTS

We test three claims: RLVR concentrates correct responses onto fewer modes; verified replay improves accuracy and diversity; and balancing replay across modes contributes additional diversity. We then examine harder task levels and compare replay with anchoring to the initial policy.

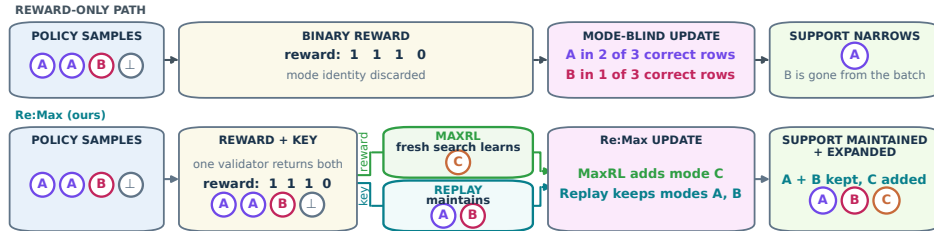


Figure 7: **Replay preserves successes that on-policy learning may stop seeing.** MaxRL learns from the policy’s current samples, while replay stores one verified exemplar per discovered mode and revisits those exemplars with equal weight. Together, these two learning paths allow Re:Max to preserve previously discovered solution modes while continuing to explore for new ones.

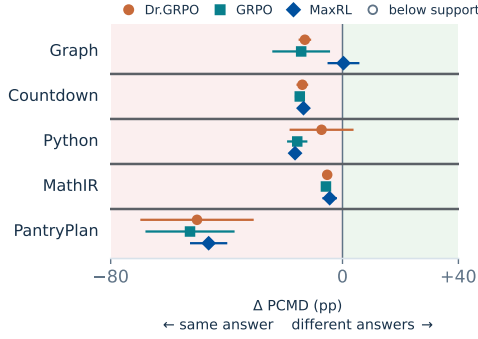


Figure 8: **Training reduces solution mode diversity.** Change in PCMD at Qwen2.5-3B, Level 1, across three objectives and five domains. Marks are five-seed means with nominal 95% intervals. Diversity falls in 14 of 15 comparisons. Other scales are in Fig. 26.

5.1 RLVR CONCENTRATES SOLUTION MODES

RLVR replaces diverse correct attempts with the same repeated answers. We compare each policy’s solution-mode distribution before and after training using PCMD. We evaluate whether training under Dr.GRPO, GRPO, and MaxRL results in fewer modes across five domains and three model scales. Fig. 8 shows Qwen2.5-3B; Fig. 26 gives the other scales. The starkest case is `Python`, where Qwen2.5-3B goes from solving 73 of 128 held-out prompts in multiple ways to as few as 22 after Dr.GRPO post-training. At Qwen2.5-3B, PCMD falls in 14 of the 15 objective–domain comparisons.

As outputs concentrate, rollouts under RLVR lose training signal. A group containing only successes or only failures gives zero fresh task advantage under Dr.GRPO and MaxRL. Across all 3,072 Qwen2.5-0.5B `Python` updates, the five-seed mean mixed-group fraction is 1.5% under Dr.GRPO versus 18.5% under Re:Dr. Saved control completions also become byte-identical within each prompt by steps 288–480 and remain so through training. Replay supplies a separate likelihood gradient even when the fresh task gradient vanishes; the complete audit is in App. H.3.

5.2 REUSE RAISES ACCURACY; EQUAL WEIGHTING PRESERVES DIVERSITY

The combined method improves accuracy and reasoning diversity across objectives and scales. We add the same verified-mode replay mechanism to Dr.GRPO and MaxRL and compare each against its matched no-replay control across five MODEBENCH domains and three model scales. Replay improves the averages on both axes at each scale (Fig. 9). At Qwen2.5-0.5B, Re:Dr improves `pass@8` by .337 and PCMD by .294 over Dr.GRPO; Re:Max improves them by .358 and .320 over the same control (Table 1).

Balancing retained modes contributes additional diversity. We change only the replay target weights: uniform over stored modes versus proportional to their verified fresh-rollout counts. Bank construction, scheduling, and update code stay fixed, although the resulting policies can discover different bank contents. At Qwen2.5-0.5B, Level 1, we compare five matched seeds Table 1: **Compared objectives under Qwen2.5-0.5B, MODEBENCH Level 1.** Differences from terminal Dr.GRPO; Avg weights five domains equally within seed (five seeds). The separate untrained-checkpoint row is the initial checkpoint minus terminal Dr.GRPO. Bold marks the best trained method; dashes indicate insufficient correct-response pairs. Colors scale within each column.

	Δ pass@8						Δ PCMD					
	Count			Pantry			Count			Pantry		
	Graph	down	Python	MathIR	Plan	Avg	Graph	down	Python	MathIR	Plan	Avg
Re:Dr (<i>ours</i>)	+ .646	+ .191	+ .356	+ .285	+ .207	+ .337	+ .559	+ .480	+ .082	+ .017	+ .334	+ .294
Re:Max (<i>ours</i>)	+ .621	+ .185	+ .509	+ .277	+ .199	+ .358	+ .525	+ .504	+ .249	+ .005	+ .317	+ .320
GAPO	+ .175	+ .011	+ .009	− .054	+ .279	+ .084	+ .131	+ .340	.000	.000	+ .653	+ .225
SetPO	+ .064	− .050	+ .497	+ .051	+ .154	+ .143	+ .039	+ .007	+ .013	+ .003	+ .266	+ .066
MaxRL (no replay)	+ .213	+ .101	.000	− .066	+ .009	+ .051	+ .086	+ .015	.000	.000	.000	+ .020
RLEP-Dr	+ .098	+ .103	.000	+ .126	.000	+ .066	+ .035	+ .012	.000	− .001	.000	+ .009
UCPO	− .003	+ .133	.000	+ .031	+ .053	+ .043	− .003 ²	+ .019	.000 ²	− .001	.000 ²	+ .009
Semantic-MaxEnt	+ .002	+ .141	.000	− .121	+ .201	+ .045	+ .004	+ .020	.000	+ .001	—	+ .006
GRPO	+ .037	+ .159	.000	− .052	+ .023	+ .034	+ .008	+ .001	.000	+ .001	.000	+ .002
Untrained checkpoint	+ .132	− .383	− .172	− .292	+ .380	− .067	+ .108	—	—	+ .027	+ .591	+ .242

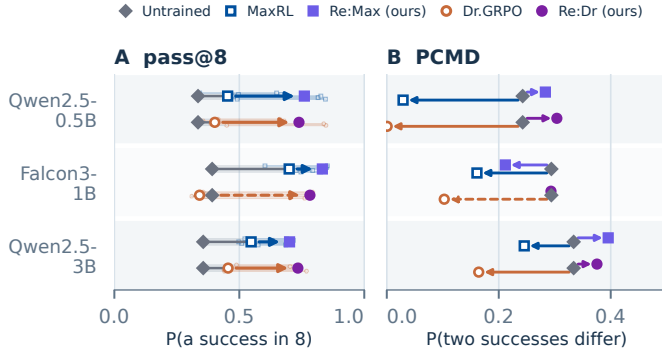


Figure 9: **Replay improves accuracy and diversity.** Diamonds mark initial models; open/filled marks are without/with replay. (A) averages five domains within seed; (B) uses the domains where all arms clear the support bar. Faint paths show seeds. Solid arrows use five seeds; thinner pairs print their counts.

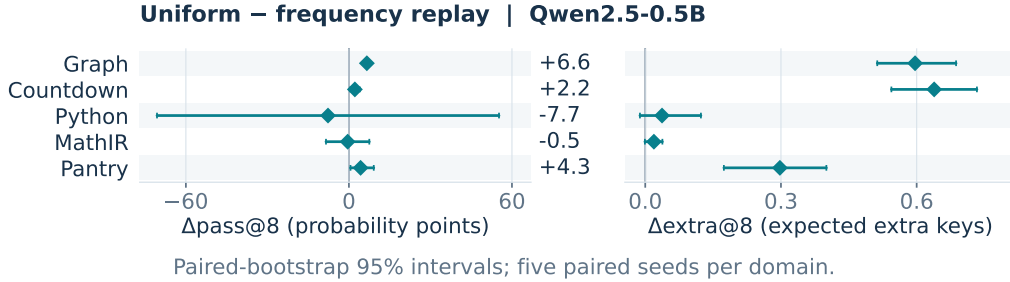


Figure 10: **Uniform weighting preserves more verified alternatives.** Uniform minus frequency-weighted replay at Qwen2.5-0.5B, Level 1, with five matched seeds. Extra modes are $B_8 = \text{distinct}@8 - \text{pass}@8$. Bars show nominal 95% paired-seed bootstrap intervals. The mean gain is .318 extra modes, with an inconclusive correctness effect. Extra modes remain coupled to correctness; they do not measure success-conditional PCMD.

across all five domains. Because these runs lack resampled PCMD estimates, we measure extra verified modes beyond the first, $B_8 = \text{distinct}@8 - \text{pass}@8$.

Uniform weighting adds .318 extra modes per prompt, with a nominal 95% paired-seed bootstrap interval [.272, .358], averaging domains equally within seed (Fig. 10). Graph, Countdown, and PantryPlan drive the gain; Python and MathIR are inconclusive. The $\text{pass}@8$ difference is .010 with interval [-.119, .139], establishing neither a correctness gain nor equivalence. The ablation supports balancing as a source of additional verified diversity, while its correctness effect remains unresolved. Reusing verified successes connects to rejection-sampling fine-tuning (Dong et al., 2023); the full ablation is in App. F.6.

Other diversity mechanisms give smaller aggregate gains. Table 1 compares UCPO, RLEP, GAPO, SetPO, and fixed Semantic-MaxEnt. Re:Max has the largest mean gain in both $\text{pass}@8$ (.358) and PCMD (.320) over Dr.GRPO. GAPO is the strongest alternative on mean PCMD (.225) and leads on PantryPlan (.653), showing that the aggregate advantage does not imply dominance in every domain.

5.3 REPLAY REMAINS EFFECTIVE BEYOND THE BASE TRAINING SETTING

Replay’s gains persist on harder task constructions. We repeat the matched comparisons within each of the first three MODEBENCH levels at Qwen2.5-0.5B. Replay improves average $\text{pass}@8$ and PCMD under both objectives at every level (Fig. 11).

Reference KL and replay preserve different sources of diversity. A KL penalty anchors the policy to its initial distribution; replay rehearses modes discovered during training. Beyond $\beta = .01$ in our sweep, increasing the KL coefficient preserves more diversity at lower correctness (Fig. 12). On the four-domain aggregate, matching replay’s diversity with KL costs correctness. The analysis in App. P.12 distinguishes the reference’s initial diversity from the modes the bank discovers.

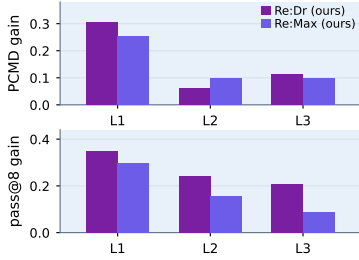


Figure 11: **Replay across task levels.** Re:Dr and Re:Max gains over their matched fresh objectives at Qwen2.5-0.5B, Levels 1–3. App. F.2 gives domain-level results.

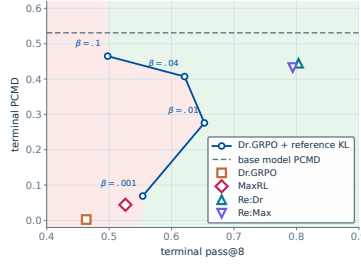


Figure 12: **Reference KL trades correctness for diversity.** Four-domain means (excluding Python) at Qwen2.5-0.5B across KL coefficients β . Green marks replay’s correctness advantage at matched diversity (App. P.12).

6 RELATED WORK

Alignment narrows what a model produces. RLHF and reasoning RL reduce output diversity, entropy and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). The literature establishes the impact of solution mode collapse. Our work tries to quantify the impact and establish both a benchmark for evaluation and a potential path forward to resolve solution mode collapse.

What is a different answer? Diversity can only be measured once sameness is defined. Semantic clustering infers it from language (Farquhar et al., 2024), which is general but depends on a judge. MODEBENCH takes identity from the same executable check that assigns reward, so two outputs differ exactly when they execute differently; narrower, cheaper, and shared by training and evaluation. Alternatives matter because sampling-based selection consumes proposals (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024).

Expanding diversity. Outcome bonuses, set prediction, distribution matching and uniform-correct objectives all reshape the fresh update (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026), and MaxRL combines objectives across sampling budgets (Tajwar et al., 2026); all act on what the policy samples now, the pressure our analysis identifies. Memory sidesteps it, but its weighting decides what survives: replay mitigates forgetting (Lin, 1992; Rolnick et al., 2019) and RLEP reuses successful reasoning trajectories (Zhang et al., 2025), yet both revisit successes in proportion to how they were collected. Balancing modes independently of discovery frequency separates our memory from those and connects it to behavioral archives in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION

MODEBENCH separates reliable answering from verified-mode diversity. Re:Max retains discovered successes after fresh sampling stops producing them, improving accuracy and diversity across our controlled comparisons. The weighting ablation supports equal rehearsal as a source of additional verified modes; its correctness effect remains unresolved.

Limits. Replay retains only modes that sampling discovers. Our theory assumes independent categorical logits, and our benchmark uses executable mode identities. Distinct modes need not have equal practical value, so broader applications require both a defensible notion of equivalence and evaluation of the alternatives’ usefulness. Future work can test whether replay can maintain a chosen distribution over solution modes in open-ended tasks.

AI USE STATEMENT

Generative AI assisted manuscript preparation, mathematical analysis, and supporting code. The authors reviewed and verified the resulting content and take responsibility for the final manuscript.

ETHICS STATEMENT

The benchmark and experiment-specific data are synthetic; this study collects no human-subject or personal data. More verified modes are not inherently better: a distinct execution may be risky or costly even when it passes the validator, and PCMD does not measure downstream value. Systems that evaluate or optimize diversity should report correctness, invalidity, and application-specific costs alongside mode counts.

REPRODUCIBILITY STATEMENT

Every reported number is computed from a versioned result file. App. S records protocols, hashes, validations, and regeneration commands, and links the public dataset and model archive; App. B records split identities. The AI Use Statement above discloses generative-AI assistance.

REFERENCES

- Marwa Abdulhai, Isadora White, Yanming Wan, Ibrahim Qureshi, Joel Leibo, Max Kleiman-Weiner, and Natasha Jaques. How LLMs distort our written language, 2026. URL <https://arxiv.org/abs/2603.18161>.
- Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98):1–76, 2021. URL <https://jmlr.org/papers/v22/19-736.html>.
- Barrett R. Anderson, Jash Hemant Shah, and Max Kreminski. Homogenization effects of large language models on human creative ideation. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 413–425. Association for Computing Machinery, 2024. doi: 10.1145/3635636.3656204.
- Oron Anschel, Alon Shoshan, Adam Botach, Shunit Haviv Hakimi, Asaf Gendler, Emanuel Ben Baruch, Nadav Bhonker, Igor Kviatkovsky, Manoj Aggarwal, and Gerard Medioni. Group-aware reinforcement learning for output diversity in large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 32394–32415, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.1649.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models, 2025. URL <https://arxiv.org/abs/2508.10751>.
- C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404. URL <https://academic.oup.com/biomet/article-abstract/26/4/404/291538>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025. URL <https://arxiv.org/abs/2505.22617>.
- Xingyu Dang, Christina Baek, J. Zico Kolter, and Aditi Raghunathan. Assessing diversity collapse in reasoning. In *ICLR Workshop on Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL <https://openreview.net/forum?id=AMiKsHLjQh>.
- Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward rAnked FineTuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023. URL <https://arxiv.org/abs/2304.06767>.
- Anil R. Doshi and Oliver P. Hauser. Generative ai enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28):eadn5290, 2024. doi: 10.1126/sciadv.adn5290.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580–586, 2021. doi: 10.1038/s41586-020-03157-9.
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024. doi: 10.1038/s41586-024-07421-0.

- Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized Markov decision processes. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2160–2169. PMLR, 2019. URL <https://proceedings.mlr.press/v97/geist19a.html>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- Josef Hofbauer and Karl Sigmund. *Evolutionary Games and Population Dynamics*. Cambridge University Press, Cambridge, United Kingdom, 1998.
- Yekyung Kim, Yapei Chang, Chau Minh Pham, and Mohit Iyyer. Argument collapse: LLMs flatten long-form public debate, 2026. URL <https://arxiv.org/abs/2606.01736>.
- Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=PXD3FAVHJT>.
- Chenyi Li, Yuan Zhang, Bo Wang, Guoqing Ma, Wei Tang, Haoyang Huang, and Nan Duan. SetPO: Set-level policy optimization for diversity-preserving LLM reasoning, 2026a. URL <https://arxiv.org/abs/2602.01062>.
- Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 110–119, 2016. doi: 10.18653/v1/N16-1014. URL <https://aclanthology.org/N16-1014/>.
- Long Li, Zhijian Zhou, Jiaran Hao, Jason Klein Liu, Yanting Miao, Wei Pang, Xiaoyu Tan, Wei Chu, Zhe Wang, Shirui Pan, Chao Qu, and Yuan Qi. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward, 2025. URL <https://arxiv.org/abs/2509.07430>.
- Xiaozhe Li, Yang Li, Xinyu Fang, Shengyuan Ding, Peiji Li, Yongkang Chen, Yichuan Ma, Tianyi Lyu, Linyang Li, Dahua Lin, Qipeng Guo, Qingwen Liu, and Kai Chen. Beyond mode collapse: Distribution matching for diverse reasoning, 2026b. URL <https://arxiv.org/abs/2605.19461>.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992. doi: 10.1007/BF00992699.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2503.20783>.
- Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking RLVR’s indifference to diversity, 2026. URL <https://arxiv.org/abs/2605.00365>.
- Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 6820–6829. PMLR, 2020. URL <https://proceedings.mlr.press/v119/mei20b.html>.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. doi: 10.1038/nature14236.

- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL <https://arxiv.org/abs/1504.04909>.
- Gabriel Pereyra, George Tucker, Jan Chorowski, Łukasz Kaiser, and Geoffrey Hinton. Regularizing neural networks by penalizing confident output distributions. *arXiv preprint arXiv:1701.06548*, 2017. URL <https://arxiv.org/abs/1701.06548>.
- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1811.11682>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Abhijeet Sinha, Sundari Elango, and Dianbo Liu. Expected return causes outcome-level mode collapse in reinforcement learning and how to fix it with inverse probability scaling, 2026. URL <https://arxiv.org/abs/2601.21669>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Yuda Song, Julia Kempe, and Remi Munos. Outcome-based exploration for LLM reasoning, 2025. URL <https://arxiv.org/abs/2509.06941>.
- Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.02710v3>. Version 3.
- Nino Vieillard, Tadashi Kozuno, Bruno Scherrer, Olivier Pietquin, Rémi Munos, and Matthieu Geist. Leverage the average: An analysis of KL regularization in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/8e2c381d4dd04f1c55093f22c59c3a08-Abstract.html>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.
- Hongzhi Zhang, Jia Fu, Jingyuan Zhang, Kai Fu, Qi Wang, Fuzheng Zhang, and Guorui Zhou. RLEP: Reinforcement learning with experience replay for LLM reasoning, 2025. URL <https://arxiv.org/abs/2507.07451>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 36, 2023. URL <https://arxiv.org/abs/2306.05685>.
- Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. Texygen: A benchmarking platform for text generation models. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pp. 1097–1100, 2018. doi: 10.1145/3209978.3210080. URL <https://arxiv.org/abs/1802.01886>.

APPENDIX ORGANIZATION

The supplement is organized in eight parts, mapped here to their pages.

Part I. Scope and measurement	
A Metric and Objective Details	17
A.1 Measuring success and diversity	17
A.2 Endpoints and the reported diversity axis	18
A.3 Why balance within the replay bank	19
Part II. Benchmark and method	
B Benchmark and Evaluation Protocol	21
B.1 Split identities and executable validation	21
B.2 Python execution contract	22
B.3 Matched level constructions	22
B.4 Frozen base models across all five levels	23
B.5 Other model families at matched scale	24
B.6 Training and evaluation settings	24
C Exact Domain Prompts	27
C.1 Graph coloring	28
C.2 Countdown	28
C.3 Python factors	28
C.4 MathIR action menu	28
C.5 PantryPlan	29
D Replay Algorithm	29
D.1 Fresh task advantages	29
D.2 State and optimizer update	30
D.3 Admission, capacity, and deterministic scheduling	30
D.4 Matched controls and recoverable state	30
D.5 What the bank needs from a verifier	31
D.6 What the bank does not rank	32
E Fixed Semantic MaxEnt Comparator	32
Part III. Training results	
F Empirical Results and Comparisons	34
F.1 Terminal comparisons across scales	34
F.2 Terminal comparison on the harder level	37
F.3 Accuracy and verified modes throughout training	39
F.4 Supporting comparators	39
F.5 Diversity-preserving comparators	42
F.6 Uniform versus fresh-frequency replay	43
G Fixed Semantic-Entropy Factorials	44
H Collapse and Replay Mechanism Checks	44
H.1 Conditional concentration from saved training outputs	45
H.2 Diversity before and after RLVR-only training	47
H.3 Fresh-gradient degeneracy and observed output collapse	50
H.4 Whether the controls trained, and what a learning rate can reach	50
H.5 Fixed-bank exemplar score trajectories	53
Part IV. What diversity represents	
I Matched Prompt-Hint Ablation	55
I.1 Removing strategy hints from unchanged problems	55
J The Decoding Grid Behind the Objection	58
K Sampling-Budget Ablation	59
K.1 Protocol and estimands	59
K.2 Hosted discovery: guidance, formatting, and late modes	65
K.3 All five domains and all three levels at 512 draws	65
K.4 Observed sampling-budget effects	66
L Inference Follow-ups: Adaptation, Model Mixtures, and Coarser Keys	68
L.1 Pantry portfolios under changed requirements	68
L.2 Complementarity across hosted models	70
L.3 Sensitivity to task-relevant equivalences	70
M Portfolio Survival under Withdrawn Options	70
M.1 Two withdrawals at once	73
M.2 Decoding settings as a substitute	74
M.3 Recovery after the change	74
Part V. Fixed deployments	
N Inference-Only Concentration in a Hosted Model	77
N.1 Hosted concentration on the success-conditional axis	79
O Comparison across Hosted Deployments	80
O.1 A separate Python prompt sensitivity test	81
O.2 A matched requested-temperature sensitivity test	83
O.3 GPT-5.6 Sol: temperature, pass@8 and verified modes	83
O.4 Native provider outcomes and undefined concentration	86
O.5 The revised Python wording on a second deployment	91
O.6 Matched reasoning-control comparison	92
Part VI. Idealized analysis	
P Mode Collapse and Verified Replay	95
P.1 Categorical abstraction and mean updates	95
P.2 Winner-take-all collapse in the categorical mean flow	97
P.3 Conditional trajectories of outcome-binary mean flows	98
P.4 Neural score identities and the limits of conditional inertness	99
P.5 Diversity at finite steps and sampling budgets	101
P.6 A replay signal when fresh groups supply none	103

756	P.7	Fixed-bank replay and qualitative retention	104
757	P.8	When replay reverses concentration, and how fast	106
758	P.9	Complete responses and finite evaluation visibility	108
759	P.10	When a replay gradient improves a neural exemplar	109
760	P.11	Reference KL retains the reference, not the discovery	110
761	P.12	Two ceilings, and which one a domain sits under	113
762	P.13	Why a memory rather than an anchor, once the reference is collapsed	115
763	P.14	Scope relative to entropy, KL, admission, and neural optimization	117
764	Q	What Verified Replay Costs	118
765	Q.1	What the control matches, and what it does not	118
766	Q.2	Token-based cost accounting and its limits	118
767	Q.3	The missing comparison at equal total compute	119
768	Part VII. Extended comparison		
769	R	Diversity-Preserving RLVR: Extended Comparison	121
770	Part VIII. Records		
771	S	Source Integrity and Reproducibility	123
772	S.1	Admissible sources and missing checkpoints	123
773	S.2	Conflicted initial checkpoints, admitted by provenance	123
774	S.3	Conflicted registered checkpoints, admitted by provenance	124
775	S.4	Data, models, and scientific records	124
776	S.5	Exact continuation and validation	124
777			
778			
779			
780			
781			
782			
783			
784			
785			
786			
787			
788			
789			
790			
791			
792			
793			
794			
795			
796			
797			
798			
799			
800			
801			
802			
803			
804			
805			
806			
807			
808			
809			

NOTATION

Symbols are used consistently across the benchmark, the algorithm and the idealized analysis. Several letters are conventionally overloaded — B , M , m and τ each carry more than one meaning in this area — so this table is the paper’s usage, and nothing below departs from it.

Table 2: **Notation.** Symbols reused across the benchmark, the algorithm, and the idealized analysis.

Symbol	Meaning
<i>Benchmark and metrics</i>	
$V(y, s_x)$	Validator: a canonical execution key, or $\perp$ for failure
$\mathcal{C}_x$	Set of valid keys for prompt x (never revealed to the learner)
m	Number of correct modes a prompt admits, $m = \mathcal{C}_x $
P	Probability that a sampled response is verified correct
q_c	Conditional probability of key c given correctness
Z_i	Canonical key of the i th draw, or $\perp$
$C(q)$	Conditional collision $\sum_c q_c^2$; the concentration measure
PCMD	Success-conditional diversity $1 - C(q)$; the reported diversity axis
$N_{\text{eff}}^{(2)}$	Effective number of verified modes, $1/(1 - \text{PCMD})$
K	Draws per evaluation group ($K = 8$ for every headline number)
n_c, n_+	Draws verified on key c , and verified in all, $n_+ = \sum_c n_c$
D_K, F_K	Distinct verified keys, and correct fraction, in K draws
$\bar{D}_K$	Expected distinct verified keys, $\mathbb{E}[D_K] = \text{distinct}@K$
B	Extra verified modes, $\text{distinct}@K - \text{pass}@K$
<i>Algorithm</i>	
$\mathcal{B}_x$	Replay bank for prompt x ; $M = 16$ its capacity
k	Number of keys a bank holds, $k = \mathcal{B}_x \leq M$; separately, the budget index in $\text{pass}@k$
$P_{\mathcal{B}}$	Total probability mass of banked keys
e_c	Stored exemplar realizing key c ; $ e_c $ its token length
ℓ_c	Mean response-token log probability of the exemplar e_c
G	Fresh rollout group size ($G = 16$ in training)
R_i, R	Binary reward of response i , and group success count $\sum_i R_i$
A_i	Fresh task advantage of response i
σ	Serialized round-robin replay cursor
η, κ	Scale and clip constants of the Semantic-MaxEnt comparator
<i>Idealized analysis</i>	
$\mathcal{A}$	Category set, correct modes $\mathcal{C}$ and incorrect $\mathcal{N}$
$c_G(P), \Psi$	Mean-flow correctness coefficient and its potential
ρ	Effective replay dose in the mean flow; λ_{replay} its implemented weight
τ	Effective optimization time in the mean flow
C_T	Energy constant in the retention bound
μ, β	Reference policy and its KL coefficient in (A4')
D	Reverse divergence $\text{KL}(p \parallel \mu)$ to that reference

Part I. Scope and measurement

A METRIC AND OBJECTIVE DETAILS

Correctness and verified-mode diversity are distinct properties of a policy. For a prompt x with reference specification s_x , the validator maps a response to either failure or a canonical key:

$$V(\cdot, s_x) : \mathcal{Y} \rightarrow \mathcal{C}_x \cup \{\perp\}.$$

Here $\mathcal{C}_x$ is the set of valid solution keys and $\perp$ denotes failure. The learner observes a key only for a response it generates; neither the full set $\mathcal{C}_x$ nor its size is revealed.

For $Y \sim \pi_\theta(\cdot | x)$, the policy induces both a total probability of verified success and a distribution over how that probability is allocated:

$$P_\theta^+(x) = \Pr[V(Y, s_x) \neq \perp], \quad p_\theta^+(c | x) = \Pr[V(Y, s_x) = c | V(Y, s_x) \neq \perp].$$

The main text writes these two quantities as P and q_c , dropping the policy and the prompt where they are clear. The conditional distribution is defined only when $P_\theta^+(x) > 0$; when $P_\theta^+(x) = 0$, all three sampling metrics below are zero, while PCMD is undefined rather than zero. The first quantity measures correctness. The second describes the distribution of verified behaviors conditional on being correct. We call concentration of p_θ^+ onto fewer keys *mode collapse*, particularly when P_θ^+ remains stable or increases. That undefinedness sets a per-prompt threshold, and the noise in an average taken over few such prompts sets two more, which App. A.2 states: a prompt defines PCMD only from two verified responses, a reported value rests on at least 30 such prompts, and a paired difference, whose two sides are read on the same prompts, on at least 20. Moving correct probability onto keys that already hold more of it can only lower $\text{PCMD} = 1 - \sum_c q_c^2$ (Lemma A.1), so the concentration defined here always registers as the fall in PCMD (2) that the main text reports. The converse does not follow: PCMD summarizes an allocation in one number, so a fall in it need not arise from any such ordered movement of mass. Neither statement claims that a key reached probability zero, which finite samples cannot establish.

A.1 MEASURING SUCCESS AND DIVERSITY

Write n_c for the number of a prompt’s draws verified on key c , and $n_+ = \sum_c n_c$ for its verified draws in all. Where $n_+ \geq 2$, the estimator of (2) is the fraction of verified pairs that differ:

$$\widehat{\text{PCMD}} = 1 - \frac{\sum_c \binom{n_c}{2}}{\binom{n_+}{2}} = 1 - \frac{\sum_c n_c(n_c - 1)}{n_+(n_+ - 1)}, \quad (5)$$

which is unbiased for (2) at every $n_+ \geq 2$: conditioning on the number of verified draws leaves that many independent labels drawn from q , so every ordered pair agrees with probability $\sum_c q_c^2$. What the estimator runs on is the verified count n_+ , not the draw budget: under the pooling rule below a local prompt’s four $K = 8$ groups contribute 32 draws, a hosted prompt contributes its eight, and n_+ counts the verified ones. The identity is promptwise, and does not survive joint selection on other conditions or dependent prompts. Two equivalent readings are sometimes useful: *rarefied-distinct@2* = $1 + \text{PCMD}$ is the expected number of distinct modes in exactly two verified draws, which fixes sampling depth rather than success count; and $N_{\text{eff}}^{(2)} = 1/(1 - \text{PCMD})$ is an effective number of verified modes. Both are monotone transforms of PCMD and order policies identically.

Every printed PCMD is an average of that estimator. Where a protocol draws more than one group per prompt, those draws are pooled before scoring — q is a property of the policy and prompt rather than of the draw budget, so pooling estimates the same quantity from more pairs and defines it far more often. The local evaluations draw four groups of eight and pool to 32; the hosted deployments answer each prompt eight times, with nothing to pool. Either way a reported value is the unweighted mean over the prompts where PCMD is defined, carried with its across-prompt standard error. Prompts are weighted equally on purpose. Weighting a prompt by how many verified draws it contributed — or, equivalently, pooling verified pairs across prompts before taking the ratio, as a collision counter naturally does — lets the prompts a policy solves most often dominate the estimate, which reintroduces the accuracy dependence PCMD exists to remove.

For K independent samples $Y_{1:K}$, let

$$D_K = |\{V(Y_i, s_x) : i \leq K\} \setminus \{\perp\}|, \quad F_K = K^{-1} \sum_i \mathbf{1}[V(Y_i, s_x) \neq \perp].$$

MODEBENCH reports three sampling averages with different units. $\text{distinct}@K = \mathbb{E}[D_K]$ is an expected count in $[0, K]$ of unique verified keys; $\text{pass}@K = \Pr[D_K > 0]$ and $\text{mean}@K = \mathbb{E}[F_K]$ are probabilities in $[0, 1]$.

Lemma A.1 (Success, sampled diversity, and PCMD). *Fix a prompt, a finite or countable valid-key set, and $K \geq 1$ independent draws from the same fixed sampling law. Let $\mu_c = \Pr[V(Y, s_x) = c]$ and $P = \sum_c \mu_c$. Then*

$$\text{pass}@K = 1 - (1 - P)^K, \quad \bar{D}_K := \text{distinct}@K = \sum_c [1 - (1 - \mu_c)^K]. \quad (6)$$

For a finite valid-key set of size $m \geq 1$ and $P > 0$, write $q_c = \mu_c/P$. At fixed P , concentration lowers $\bar{D}_K$: if q majorizes q' (its decreasingly sorted partial sums are at least those of q'), then $\bar{D}_K(P, q) \leq \bar{D}_K(P, q')$. In particular,

$$1 - (1 - P)^K \leq \bar{D}_K(P, q) \leq m[1 - (1 - P/m)^K]. \quad (7)$$

For $K \geq 2$, the lower bound is attained only by concentration on one key, and the upper bound only by uniform q . For $K = 1$, $\bar{D}_1 = P$ for every q . At fixed q , increasing P cannot decrease $\bar{D}_K$. By contrast $\text{PCMD}(q) = 1 - \sum_c q_c^2$ does not involve P : it is unchanged when correctness rises or falls at a fixed allocation. Concentration lowers it: if q majorizes q' then $\text{PCMD}(q) \leq \text{PCMD}(q')$, strictly unless q' is a permutation of q . It runs from 0 at a point mass to $1 - 1/m$ at the uniform allocation, so its ceiling is set by the number of valid keys rather than by correctness.

Proof. All K draws fail with probability $(1 - P)^K$. For each key c , its appearance indicator has expectation $1 - (1 - \mu_c)^K$; summing indicators gives the second identity, also for countably many keys by nonnegativity. For fixed $P > 0$ and $K \geq 2$, the function $f(v) = 1 - (1 - Pv)^K$ is strictly concave on $[0, 1]$. Transferring mass from a larger coordinate to a smaller one without reversing their order increases $\sum_c f(q_c)$; successive such transfers give the majorization assertion. The point mass and uniform vector are its extreme cases, giving the bounds and equality conditions. Finally, $\partial \bar{D}_K / \partial P = K \sum_c q_c (1 - Pq_c)^{K-1} \geq 0$, which proves monotonicity in correctness. For the final claim, $\sum_c q_c^2$ is symmetric and convex, so the same transfer strictly decreases it whenever the two coordinates differ; PCMD is one minus that sum and no term in it involves P . The point mass and the uniform vector again give its extremes, by the same succession of transfers that gave the bounds on $\bar{D}_K$. $\square$

These are promptwise identities, before averaging over prompts. In particular, applying the nonlinear formula to a mean correctness generally changes the result. They explain the shared measurement in the training and hosted studies: success depends on total correct mass; sampled diversity also depends on its allocation.

Raw $\text{distinct}@K$ measures sampled coverage, not the total number of positive-probability modes. For an uninformed reference, suppose a finite space has N actions and N_c actions map to valid key c . Uniform independent action sampling then gives $\sum_c [1 - (1 - N_c/N)^K]$ expected verified keys. The simpler expression $m[1 - (1 - 1/N)^K]$ applies when the m valid actions have distinct keys, as in our reported finite-action references. Open-ended program and derivation spaces have no canonical nominal “uniform” generator. Table 20 applies these references to the trained policies.

A.2 ENDPOINTS AND THE REPORTED DIVERSITY AXIS

Two terminal endpoints are registered as co-primary, meaning neither is subordinate to the other and both are carried for every model–domain–level block: $\text{pass}@K$, the probability of finding any correct output, and raw $\text{distinct}@K$, the number of verified keys found. $\text{pass}@K$ is printed in the tables here and in App. F; $\text{distinct}@8$ is retained beside each of them in the machine-readable records, which is where it is read, because it rises with correctness and so restates accuracy wherever it is printed as diversity.

A separate question is which statistic should carry the diversity axis in the figures and the prose. Raw `distinct@K` is coupled to the number of successful draws: by (6) it grows with P at fixed q (Lemma A.1), and empirically it correlates .936 with `pass@8` across the 375 untrained measurements, while `PCMD` correlates -.332 on the 225 of them with sufficient support. The two coefficients are not read on the same population: definedness is itself accuracy-dependent, as the support paragraph below sets out. Reported as diversity, raw `distinct@K` therefore largely restates correctness, and the comparison it invites between a high-`distinct@8` domain and a low-`distinct@8` one is mostly a comparison of accuracy. We consequently report `PCMD` (2) as the diversity axis, which is a functional of q alone (Lemma A.1). Fig. 15 shows what that separation buys. From 7B up, Python is answered correctly on at least .988 of held-out prompts at Levels 1–3 while its `PCMD` never exceeds .044: near-perfect accuracy reached by returning the same answer almost every time. Read on the `distinct@8` axis that same block would look like the broadest domain in the table.

Support and explicit gaps. `PCMD` is undefined for a prompt returning fewer than two verified responses, and definedness is itself accuracy-dependent: the fraction of prompts defining `PCMD` correlates .988 with `pass@8`. This is an information limit rather than an artifact, since a policy that almost never succeeds exhibits no distribution of successes to measure. We mark it everywhere it applies. Every reported `PCMD` carries the number of prompts that define it, and measurements resting on fewer than 30 such prompts sit below the support bar. Figures draw those hollow rather than leaving them empty, from five defining prompts upward, so a weak measurement shows up as a thin estimate carrying its own error bar rather than as an absence. Below five nothing is drawn: a standard error of zero on two or three prompts records those prompts agreeing, not a precise estimate, and a mark there would read as the firmer measurement it is not. Of the 375 measurements of frozen base models across all four families, 225 clear the 30-prompt bar, and the gaps concentrate where success is rarest: 62 of the 150 are Countdown, whose frozen success rate is too low to support any diversity statement. A hollow or absent entry means diversity was not reliably measurable there; it does not mean diversity was low.

A paired difference carries a separate bar of 20, because both of its sides are measured on the same prompts and only their difference is read. The distinction is not a relaxation for convenience: the side that falls short is almost always the RLVR-only control, whose definedness is depressed by the very concentration under study, so holding a difference to the stricter bar would censor exactly the comparisons where replay separates most. A cell with either side under 20 remains a gap.

`PCMD` is not a new measurement of the saved responses. It is exactly one minus the conditional-collision U-statistic of App. H.1; we report the diversity orientation because a metric that increases with diversity reads more directly alongside `pass@8`. Every `PCMD` value is recomputed from canonical keys already stored with the saved responses, without resampling or regrading.

The derived contrast `distinct@K` – `pass@K` = $\mathbb{E}[D_K - 1[D_K > 0]]$ estimates the number of verified modes beyond the first. It is computable without knowing the full valid set $\mathcal{C}_x$ but remains coupled to the number of successful draws, so we keep it as an interpretive decomposition rather than as a diversity measure. Because every headline evaluation fixes $K = 8$, reporting `distinct@8/8` would add no information beyond a linear rescaling; we retain the raw count and label its units explicitly.

A.3 WHY BALANCE WITHIN THE REPLAY BANK

Uniform sampling over keys prevents discovery frequency from determining replay frequency. To explain its relationship to the evaluation metric we analyze an equal-weight categorical surrogate, without length normalization (Fig. 6 draws what the two terms preserve); App. D states the objective as implemented. A stored response’s probability need not equal its key’s total probability. For $k = |\mathcal{B}_x|$ banked keys, write $p(c)$ for their probabilities and P_B for their total mass. Then

$$-\frac{1}{k} \sum_{c \in \mathcal{B}_x} \log p(c) = -\log P_B + \log k + \text{KL}(U_k \parallel p(c) \mid c \in \mathcal{B}_x). \quad (8)$$

The objective rewards greater banked verified mass and a more balanced conditional distribution, aligning with `pass@K` and `distinct@K`. The combined update need not improve both at every step: banked-key coverage contributes $\sum_{c \in \mathcal{B}_x} [1 - (1 - p(c))^K]$ to expected `distinct@K` and, at fixed P_B and k , concavity makes it largest under uniform allocation, while $1 - (1 - P_B)^K$, the probability of any banked key, depends only on total mass (Fig. 13). Uniformity also maximizes the smallest conditional banked-key mass, making equal retention explicit.

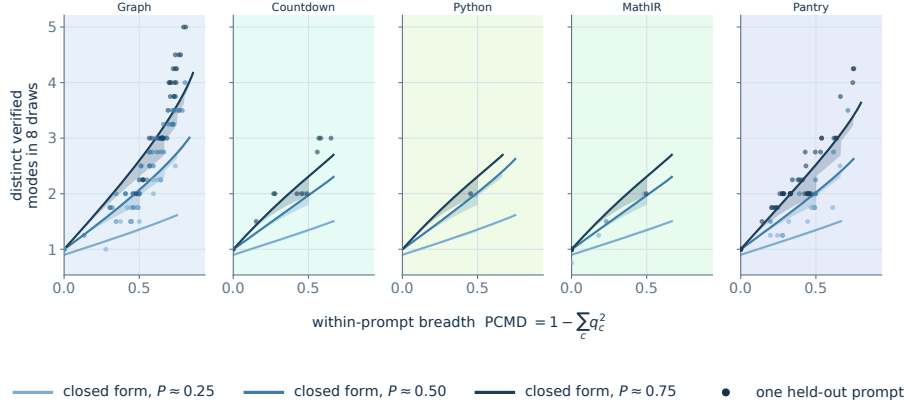


Figure 13: **The decomposition against measured prompts, by domain.** Bands are the coverage sum above with the banked restriction dropped, over all of a prompt’s correct keys: $\sum_c [1 - (1 - Pq_c)^8]$, where $p(c) = Pq_c$. They are evaluated over allocations from a point mass to uniform; at each diversity a band spans only the key counts that can attain it, over the range those prompts realize. Points are 1,092 held-out prompts returning at least two verified responses, at the terminal checkpoint of the Qwen2.5-0.5B replay arms, binned by measured verified mass P . Panels are the five domains and carry the tints of their cards in Fig. 2, because a mode means something different in each and the number of keys a prompt can occupy is a property of the domain. Graph and Pantry occupy the full range; Python and MathIR sit at the left edge, MathIR almost entirely so — nearly every one of its prompts returns a single key, where two correct answers are the same answer, because its certified modes are derivations of one answer rather than alternatives. These prompts are held out and so carry no bank; the keys a policy occupies here stand in for what a training prompt’s bank would have to hold. That count stays far below the capacity $M = 16$: what bounds these policies is how many modes they find, not how many the bank can hold, so raising M alone would not relieve what binds them.

That alignment is deliberate, which is a reason for caution rather than a result: the bank objective raises $\text{distinct}@K$ by construction, so $\text{distinct}@K$ cannot also serve as the evidence that replay worked. The objective is aligned with $\text{pass}@K$ as well, and what makes that endpoint evidence rather than recitation is where it is measured: banks hold only training-prompt responses, and evaluation responses never enter training. PCMD carries the diversity axis because the objective does not target it at all.

Part II. Benchmark and method

B BENCHMARK AND EVALUATION PROTOCOL

MODEBENCH takes correctness and identity from one executable validator, the map $V(\cdot, s_x)$ of App. A: invalid outputs receive reward zero and no key, valid outputs reward one and a canonical execution key. What matters for the protocol is who may see the catalogue, and the answer is nobody: neither its members nor its size reach training, replay, scheduling, or stopping. Table 3 specifies the five domains and which surface forms merge. The public dataset preserves each domain’s split roles and construction identity. The training comparisons reported here use Levels 1 to 3.

Table 3: **MODEBENCH at Level 1**. Correctness and identity come from one executable path; training starts without the mode catalogue. *Valid modes* is the mean number of correct execution modes per evaluation prompt with the per-prompt range beside it, exact over the released Level-1 multi-answer split. *Reached* is what one frontier deployment actually produces on the same construction — distinct verified modes per prompt at 512 draws, over the 32 problems per domain of Fig. 5, an outcome-independent subset of the same evaluation split — so the two columns separate the modes a domain admits from the modes anything has been observed to give on the same prompts. MathIR is the domain where that gap is structural rather than a sampling limit, because its modes are derivations of one answer and most of the certified ones are longer routes. Each higher level is a separate construction with its own support distribution (App. B.3). The exact prompt each policy receives is reproduced verbatim in App. C for a direct, byte-level audit of the evaluation interface, chat-template wrapper included.

Domain	Generated answer / check	Executed key	Aliases that merge	Valid modes	Reached
Graph coloring	Complete assignment; check fixed colors and edges	Color vector	Spacing only	6.27	2.53
				4–12	
Countdown	Parse operands; execute exactly	Normalized AST	Commutative order	4.52	3.47
				2–8	
Python factors	Execute restricted lambda externally	Return vector	Source programs with same behavior	229.44	3.75
				16–1,512	
MathIR	Execute bounded action program	State trajectory	Programs with the same states	5.00	1.91
				5	
PantryPlan	Exact ingredient allocation; check all bounds	Ingredient support	Quantity and ordering aliases	18.19	6.91
				8–40	

B.1 SPLIT IDENTITIES AND EXECUTABLE VALIDATION

Every domain’s training and evaluation splits are disjoint, at the sizes Table 6 records. PantryPlan and the Level-2 constructions carry development splits besides, which no reported result reads (App. B.3). Python materialization independently executes at least two functions with different valid keys for every row; MathIR executes every certified route through the same interpreter used for model outputs.

Table 4: **Deterministic materialization identities**. The two domains whose materialization is itself executable, as App. B.1 describes: Python runs functions, MathIR runs certified routes. Hashes bind the released rows to those checks, and the full hashes and construction records accompany the data. PantryPlan’s row hashes follow below. Exact modes per prompt are the training split here; Table 3 reports the evaluation split, which is disjoint from this one.

	Python factors	MathIR action menu
Training rows	384	384
Evaluation rows	128	128
Exact modes per prompt (train)	16–3,600	5
Training-row SHA-256	bcfa9acc...37cf7	bfc98362...5d3835
Evaluation-row SHA-256	be0f621c...94d183	df5783dd...328b96

PantryPlan. The deterministic v2 release has 384/64/128 training/development/evaluation prompts across four formulation families. Exact support ranges from 8–45 over the training split and 8–40 over the 128 evaluation prompts, which average 18.19 feasible supports. Training and evaluation

row hashes are `a4883c5e...a757f` and `5d81ba8d...5923`. The validator checks exact inventory, serving steps, mass, nutrient bounds, and dietary tags; identity is ingredient support. A verified allocation is not a claim of palatability, cookability, or safety.

B.2 PYTHON EXECUTION CONTRACT

Candidates are stripped, limited to 240 characters and 64 AST nodes, and parsed in expression mode. The only accepted form is `lambda n: EXPR` with exactly one ordinary argument named `n`: no defaults, positional-only, keyword, or variadic arguments. Integer literals have absolute value at most 10,000. The expression grammar permits `n`, `+`, `-`, `*`, floor division, remainder, integer comparisons, Boolean `and/or`, unary `plus/minus/not`, and conditionals. Boolean and non-integer literals, calls, imports, attributes, comprehensions, containers, subscripts, assignment, loops, and other identifiers are rejected. The specification format admits J distinct integers in $[4, 1000]$, with J between two and eight. Every released Level-1 row uses 4 of them, drawn from $[6, 96]$, and carries at least two valid behavior vectors.

External execution. The trainer never executes candidate code. A locked parent client launches an isolated Python `-I` process in a fresh session, with a persistent JSON-lines worker accepting only restricted source and trusted task JSON. The worker repeats the AST checks, compiles the expression with empty `__builtins__`, and executes the frozen inputs under a 0.25-second real-time alarm; the parent enforces a separate one-second response deadline. A valid response contains the integer vector and key. Timeouts, empty or malformed responses, broken pipes, non-integer outputs, and failed divisor checks yield `⊥` and zero reward. Broken pipes, operating-system errors, and malformed responses permit one clean worker restart; a timed-out request remains failed. Worker stdout is never parsed as a candidate answer, and exceptions never produce a key.

Behavioral identity. For each of the J frozen inputs n_j , the returned d_j must be an integer but not a Boolean, satisfy $1 < d_j < n_j$, and divide n_j exactly. All checks must pass. The key is the ordered vector `python_factor: d1, ..., dJ`; programs merge exactly when their vectors agree. The exact support count $\prod_j |\{d : 1 < d < n_j, d \mid n_j\}|$ is used only for dataset validation.

B.3 MATCHED LEVEL CONSTRUCTIONS

Levels 2 to 5 are separate constructions, each calibrated against a fixed Level-1 reference at a larger model, and each admitted on its own terms. What they share is the verifier, the canonicalizer and the notion of a mode; what grows is the structure to be searched.

Level 2. Level 2 changes problem structure while preserving the verifier and canonicalizer. Its 384/128/128 training/development/evaluation rows have disjoint identities across the Level-1 and Level-2 splits. Training histograms match the native Level-1 training sets. Development and evaluation histograms match designated Level-1 development and confirmation reserves in Graph, Countdown, Python, and MathIR; Pantry uses native references, with the counts of its 64-row Level-1 development histogram scaled by two to give the 128-row target that its Level-2 development split is built to match.

These construction reserves are not the native Level-1 terminal test sets. The displayed terminal evaluations have different valid-key histograms between levels in Graph, Countdown, and Python. Cross-level changes therefore combine task structure and test population; they do not isolate difficulty at fixed test-set support. Replay comparisons remain paired within each level on the same prompts, seeds, fresh objective, and training budget.

Before treatment training, frozen Qwen2.5-0.5B development evaluation admitted a Level-2 domain only when `pass@8` lay in $[\text{.10}, \text{.90}]$ and was strictly below its construction-reference Level-1 value under identical decoding. The matched-level identity manifest records support histograms, row hashes, construction seeds, disjointness checks, and these admission receipts.

Scale-calibrated Level 3. Level 3 uses frozen Qwen2.5-3B against the same fixed Qwen2.5-0.5B Level-1 reference. Recipes are selected on development problems before fresh held-out confirmation, requiring absolute `pass@1` and `pass@8` differences of at most `.04` and `.08` in every domain, with

Table 5: **What Level 2 changes.** The verifier, the canonicalizer and the notion of a mode are the same at both levels; only the structure to be searched grows.

Domain	Level 1	Level 2 structural change
Graph coloring	Three hidden vertices	Four hidden vertices
Countdown	Three operands	Four operands, with values up to 14
Python factors	Four executed cases, all at most 96	Four cases, including a value above 96
MathIR	One-sided linear families	Variable-on-both-sides families
PantryPlan	Base dietary/composition constraints	More active dietary exclusions and tighter composition

four independent groups of eight samples per prompt. `distinct@8` is reported but is not a fitting objective. The interface uses native chat, temperature 1, top- p 1, and 192 generated tokens: boxed-direct answers for Graph and the hybrid solver with legal-syntax guidance for the other domains. This matches fixed measured targets empirically; it is not a two-sided equivalence test.

The Python Level-3 interface used in the hint ablation removes the strategy hints (App. I), so its difficulty is not covered by that match. All reported results use the original datasets and prompts.

Levels 4 and 5. Both continue the level sequence against the same fixed Qwen2.5-0.5B Level-1 reference and the same .04 and .08 tolerances on `pass@1` and `pass@8`, calibrated at 7B and 14B. Level 5 is difficulty-matched in all five domains and carries a composite record naming each domain’s source and protocol digests. Level 4 is admitted with a recorded exception. Four of its domains — Graph, Countdown, Python and PantryPlan — are difficulty-matched; its MathIR construction is not, for a reason that belongs to the target rather than to the search for it. Matching there demands a `pass@8/pass@1` ratio of 5.50; the best ratio any construction in this family attains at 7B is 4.16, against a ceiling of 8 for a homogeneous population, and a ratio of averages cannot exceed its best component. No mixture of these constructions reaches the target, so the target sits outside the achievable set: MathIR at 7B is far more bimodal than its Level-1 reference, with measured heterogeneity gaps of .169–.398 against the reference’s .060, and three further difficulty mechanisms measured inert. All five Level-4 datasets are frozen at 384/128/128, verified, and disjoint from every other level, and the level is admitted on that basis: a target proved unreachable is a property of the construction, not a gap in it. What admission does not extend to is a matched-difficulty claim for MathIR, which its entries carry as an exception rather than a match. Figs. 3 and 15 plot both levels without marking that exception, and should be read against this paragraph.

B.4 FROZEN BASE MODELS ACROSS ALL FIVE LEVELS

Fig. 3 reports the frozen Qwen2.5-Instruct scales across all five levels and five domains; Fig. 17 adds the other three families, for 375 completed measurements in all. Every model uses the same 128 held-out prompts within a domain and level, with four independent groups of eight draws per prompt, so each plotted point rests on 4,096 responses. Each group contributes an indicator of any verified output to `pass@8` and the number of distinct verified canonical keys to the registered `distinct@8`; we average groups within prompts and then average prompts. PCMD instead pools a prompt’s four groups before scoring it, under the rule and for the reason given in App. A.1. For `pass@8` and `distinct@8` the four groups stay separate $K = 8$ observations throughout; only PCMD pools them.

These measurements use native chat, $T = 1$, top- $p = 1$, and a 192-token output limit, with boxed-direct Graph answers and the registered hybrid interface elsewhere. Every constructed level is measured on the prompts it was admitted with, unchanged. That interface is not the one the training runs evaluate through, so this grid and the untrained endpoint of a training comparison are two measurements of the same weights rather than one number reported twice. They can differ by a lot: on Python at Level 1 this grid answers most prompts, where the same weights under the training template and the resampling’s wider draw budget clear two verified answers on 73 of 128 and sit at PCMD .168, the figure Section 5.1 quotes. Read a grid row against other grid rows, and a training contrast against its own untrained arm; the two are not interchangeable. These held-out measurements are separate from development fitting and do not imply that success decreases monotonically with level for every model and domain: `pass@8` ends below its Level-1 value in 33 of the 35 model–domain series the levels support, not in all of them. Moving along the two axes also costs diversity by different amounts. Taking the median range of PCMD along a row of

the grid, scale moves it .251 at fixed level against .104 for level at fixed scale, so the level axis and the parameter count are not interchangeable ways of narrowing how a model answers.

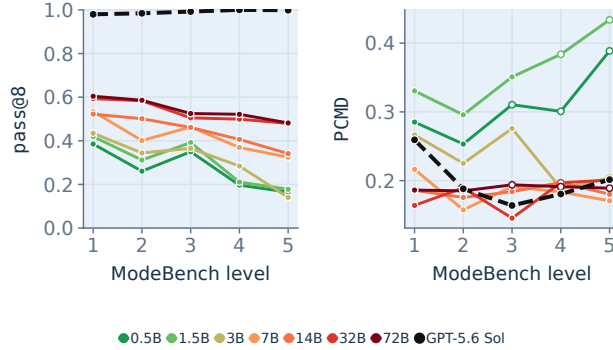


Figure 14: **The ladder read along one axis, for one family.** Both axes averaged equally over the five domains, one line per frozen Qwen2.5 scale in the colours of Fig. 3, with GPT-5.6 Sol dashed in black. Left: `pass@8` falls with level at every scale while the hosted deployment stays near one. Right: PCMD over the same cells, pooling the held-out prompts with the training split where that split has been measured. The two panels must be read together rather than separately. PCMD exists only where a cell clears the 30-prompt support bar, and of the 40 scale–level points on the right only 28 rest on five domains that all clear it; the rest average whichever domains remain, and are drawn hollow. Those thinned points put the smallest scales highest at the hard levels, which follows the support loss that comes with falling accuracy on the left rather than any gain in diversity. Diversity per domain, where it is measurable without that averaging, is in Figs. 3 and 15.

B.5 OTHER MODEL FAMILIES AT MATCHED SCALE

The main text holds the training recipe fixed and reads the scale axis within Qwen2.5. Fig. 17 instead places SmoLM2, Falcon3 and OLMo-2 beside those scales on the same axes, under the identical protocol: the same 128 held-out prompts per domain and level, the same four groups of eight draws, the same verifier and the same support requirement. The comparison is taken on the 4 levels every family covers, since only Qwen2.5 reaches Level 5 and a family average over more levels would put the level axis inside a contrast about recipes. Parameter count sets colour, so two families at the same size take the same colour and are read against each other directly.

At matched parameter count the families do not agree on diversity. Near 7B, where Qwen2.5, Falcon3 and OLMo-2 each place a model and SmoLM2 does not, the three separate on PCMD while their `pass@8` stays close; the ordering holds into the 10–15B band, though Falcon3 and OLMo-2 sit close enough there to read as a pair rather than as two ranks. Size therefore does not fix how many ways a model answers: at matched scale a change of recipe moves PCMD by a median .166, against .198 for a family’s entire scale range — a same-size change of recipe is worth about as much diversity as the whole parameter range within one. Both medians are taken within a family over the 4 shared levels, so they are a matched pair; the .251 quoted above is a different statistic, read across the Qwen grid’s seven scales at every level. We report this as a measured property of the released checkpoints, not as a claim about which pretraining or post-training choice is responsible, which these frozen measurements cannot separate.

B.6 TRAINING AND EVALUATION SETTINGS

Responses are independent within each evaluation draw. For each training seed, metrics average over all 128 prompts and four draws; raw responses and draw-level records are retained. The co-primary terminal outcomes registered in App. A.2 are realized here as `pass@8` and raw `distinct@8`; `mean@8` is diagnostic. Evaluation never supplies training data, bank entries, checkpoint selection, or a stopping rule. Complete paired five-seed blocks receive nominal point-wise 95% Student-*t* intervals; partial primary intersections report exact counts and descriptive means without intervals. Intervals condition on the frozen prompts and evaluation requests. Domain averages are explicitly labeled, and no missing terminal result is replaced by a shallower

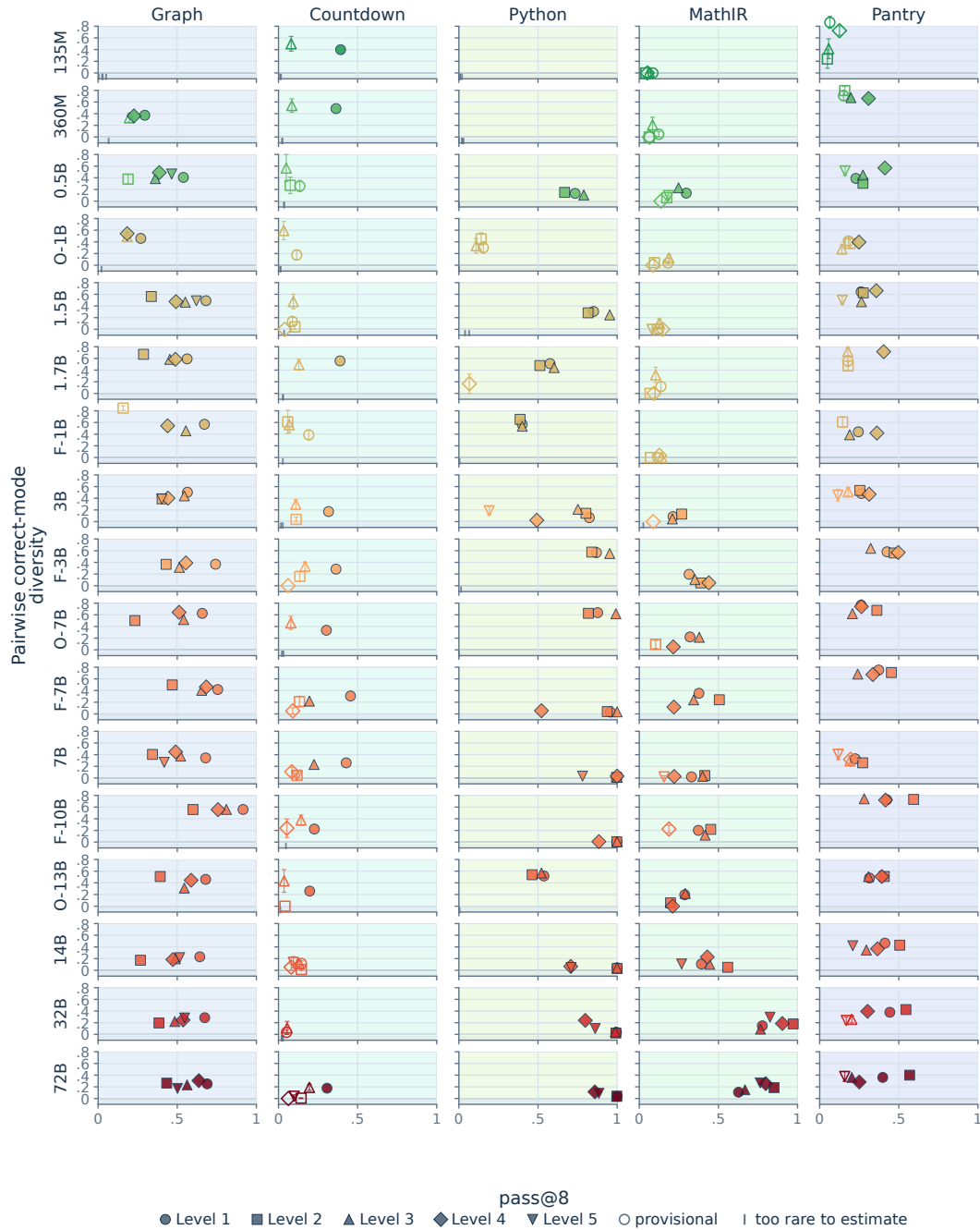


Figure 15: **Every frozen scale on its own.** Each row is one scale drawn without the others over it, across every family and every hosted deployment of Fig. 17; columns are the five domains and marker shape identifies the level. Each point pools four independent eight-draw groups on 128 held-out prompts at $T = 1$. Reading across a row gives one model over all five domains; reading down a column gives one domain over every scale. The two axes separate cleanly at 7B: Python answers almost every prompt correctly and almost always with the same mode (pass@8 .781–1.000, PCMD .010–.033), while Graph reaches pass@8 .344–.680 with PCMD .268–.447. A filled mark clears the 30-prompt bar; a hollow one carries its estimate and standard error from five to twenty-nine prompts, which is thin rather than absent and is drawn so it can be read as thin. Ticks below an axis mark measurements resting on fewer than five prompts, where a standard error of zero would record two or three prompts agreeing rather than a precise estimate; those are given no height. The smallest scales are mostly such: the measurable entries number 1 at 135M and 5 at 360M, so their rows are largely empty by measurement rather than by omission. The hosted frontier deployments carry no parameter count and so belong to no row; they appear in Figs. 3 and 17, where diversity is read against level rather than against scale.

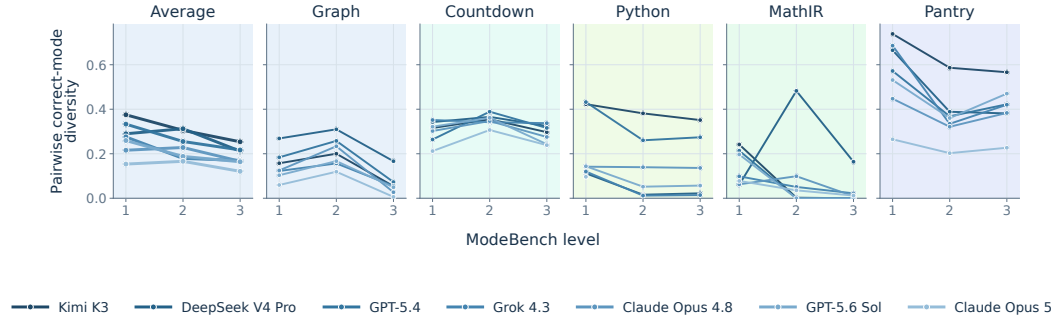


Figure 16: **Harder levels narrow the frontier.** Success-conditional diversity against the level axis, one line per hosted deployment, over 128 held-out prompts and eight stateless requests per prompt per cell. Colour runs with collapse, darkest most concentrated. Only the rungs every deployment covers are drawn: Levels 4 and 5 exist for one deployment alone, and a line running past its neighbours would read as that deployment holding up where the others fall rather than as the only one measured there (App. N.1 reports those two probes separately). Correctness is not drawn, because there is nothing to see — every hosted cell sits at $\text{pass}@8 \geq .898$ or above, so the levels cost these deployments accuracy they do not have to give and diversity they do. The Average panel weights domains equally over a set held fixed for each deployment, so the domain mix cannot move with the level; on it every one of the 7 falls from Level 1 to Level 3, by .033 to .121. The domains do not agree on how: MathIR runs to zero, Python and Graph fall, and Countdown does not narrow at all. Open marks sit below the support bar and stay off the line, as in Fig. 26. The higher levels also add strategy guidance to the MathIR, Python and PantryPlan prompts that Level 1 does not carry, so part of the fall on those three is instruction-following rather than difficulty; Graph and Countdown add none, and every deployment falls on those two alone (Apps. I and N.1).

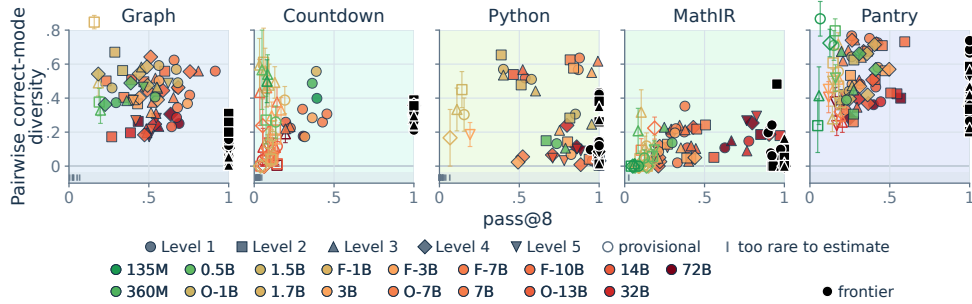


Figure 17: **Four model families on the axes of Fig. 3.** Frozen instruction-tuned models from SmoILM2, Qwen2.5, Falcon3 and OLMo-2, plotted as $\text{pass}@8$ against PCMD; colour runs green to red with parameter count on a log scale, and marker shape identifies the level. This figure carries every model family and every level collected, where Fig. 3 shows Qwen2.5 alone. Grey markers are all 7 hosted deployments of Section 2.3, of which the main-body figure draws only GPT-5.6 Sol; they have no parameter count and so sit off that ramp. Fig. 15 separates the same data one scale per row. Ticks below each axis mark measurements whose success is too rare for PCMD to be estimated on enough prompts; those are given no height. Because colour encodes size rather than family, models of different families at the same scale share a colour and can be compared directly.

Table 6: **Matched run contract.** Settings and systems changes are shared within each model–domain–seed comparison, including the zero-derivative replay control. Both arms use the same optimizer-update and fresh-rollout budgets and retain the replay scoring/backward path. Realized bank workloads, total FLOPs, and wall-clock time are not matched. The control does not reallocate replay work to additional useful baseline training (App. Q.1).

Component	Setting
Data	384 training and 128 disjoint evaluation prompts per domain and level; no evaluation row enters training or replay.
Fresh rollouts	$G = 16$, temperature 1, top- $p = 1$; one prompt group per update.
Optimization	Eight passes (3,072 updates), one PPO epoch, $\beta_{KL} = 0$ except in the reference-KL cohort of App. P.12, which moves that coefficient alone; paired initialization, prompt stream, optimizer, hardware class, and checkpoint cadence.
Evaluation	Passes 0, .5, . . . , 8; four fixed temperature-1, top- $p = 1$, $K = 8$ draws per prompt, plus a separate temperature-0 greedy trace. No best-checkpoint selection.
Replay	Capacity $M = 16$ per prompt; one round-robin bank per update; $\lambda_{\text{replay}} = .10$. Controls execute the same code with an exact-zero replay derivative; realized banks and workloads can differ.
Training seeds	Qwen2.5-0.5B: 43–47; Falcon3-1B: 55–59; Qwen2.5-3B: 70–74.
Learning rate	Qwen-0.5B and Falcon-1B: constant 2×10^{-7} , with optimizer-state offload for Falcon. Qwen-3B: cosine decay with 10^{-7} peak, 10^{-8} floor, and 10% warmup.
Optimizer	Fused Adam in <code>bfl16</code> with decoupled weight decay, $\beta_1 = .9$, $\beta_2 = .95$, weight decay 0, gradient-norm clip 1.0, gradient checkpointing on. Offload swaps the fused kernel for its CPU twin and changes no hyperparameter.
Policy update	One PPO epoch per batch with parameters synchronized every update, so each rollout is used once and on-policy. The surrogate keeps the standard symmetric clip at $\varepsilon = .2$; at one epoch the importance ratio is 1 when the clip is evaluated, so it binds on no reported update and is listed to pin the loss, not because it acts.
Response tokens	Graph and Countdown: 192; Python: 192/512/192 at 0.5B/1B/3B; MathIR: 64/128/64; PantryPlan: 8.
Evaluation seeds	Base 610100/610200/610300/610400/76299 for Graph/Countdown/Python/MathIR/PantryPlan; draw d uses base plus d .

Table 7: **Pinned base-model revisions.** Every run in this paper starts from these exact commits; a different revision of the same model name is a different initialization and is not admissible as a matched arm, whatever name the checkpoint carries.

Base model	Pinned revision
Qwen2.5-0.5B-Instruct	7ae557604adf67be50417f59c2c2f167def9a775
Falcon3-1B-Instruct	28ba2251970a01dd1edc7ba7dad2eb71216ccfdcf
Qwen2.5-3B-Instruct	aa8e72537993ba99e69dfaafa59ed015b17504d1

checkpoint. Source admission and curve gaps follow App. S.1; comparison-specific intersections and the replay-weighting bootstrap are specified with their results.

C EXACT DOMAIN PROMPTS

Each domain uses one system message and one user message, with no few-shot examples. The same template renders training and evaluation for every method. What it does not do is impose one model’s chat surface on another. The listings below are the Qwen rendering, and a Falcon3 run trains and is evaluated on a paired template that carries the identical system and user text onto Falcon’s own `<|system|>`/`<|user|>` markers; the two are validated against one shared role contract, so the surface differs and the task does not. The frozen base-model grid goes further and renders every scale through its own tokenizer’s chat template, refusing to run a model whose native template is absent, which is how SmolLM2 and OLMo-2 are measured (App. B.4). No model is asked to read a foreign chat format. These are the first rows of the frozen Level-1 evaluation splits, including the exported chat-template wrapper; Levels 2 to 5 are separate constructions on the same interface (App. B.3). The checked generator `ops/check_paper_domain_prompts.py` reproduces them from the datasets and fails closed when the typeset appendix has drifted from them. A two-space continuation indent marks display wrapping: rejoining such a line to the one above it with a single space recovers the original string, and in the PantryPlan listing a break without that indent is a real newline in the prompt.

C.1 GRAPH COLORING

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/graph_coloring_modebench_v2.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Color this graph with vertices 1 through 6. The undirected edges are: (1,2), (1,3),
(3,5), (3,6), (4,6), (5,6). Assign each vertex one color from {1, 2, 3} so that no
edge has the same color on both endpoints. A partial coloring is 2??1?2, where ?
means uncolored. Keep every shown digit fixed. There are exactly 3 question marks.
Return exactly one final answer and no explanation: exactly 3 digits, each one 1,
2, or 3, for the missing positions from left to right, inside \boxed{}.<|im_end|>
<|im_start|>assistant
```

C.2 COUNTDOWN

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/exact_countdown_easy3_probe.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Using the numbers [3, 6, 9], create an arithmetic expression that equals 18. Use
each given number exactly once. You may use +, -, *, /, and parentheses. Return
exactly one final answer and no explanation: the expression inside
\boxed{}.<|im_end|>
<|im_start|>assistant
```

C.3 PYTHON FACTORS

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/python_factor_modebench_v1.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Write one pure Python function with the exact form lambda n: EXPR. An external
Python tool will call it once for each n in [18, 82, 91, 93]. For every call,
return an integer d satisfying 1 < d < n and n % d == 0. Different valid return
vectors are different solution modes. You may use integer literals, n, +, -, *,
//, %, comparisons, Boolean operators, and conditional expressions; calls,
imports, attributes, containers, and other names are forbidden. Return exactly the
one-line lambda inside \boxed{} and no explanation.<|im_end|>
<|im_start|>assistant
```

C.4 MATHIR ACTION MENU

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/mathir_action_menu_v1.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
MathIR linear-menu-v1. Solve for x by choosing an executable action sequence. Every
selected action is applied exactly to BOTH sides of the current equation, followed
by exact simplification. Bindings: a=2, b=-9, c=-1. Initial equation: x/a + b = c.
Action menu: A means div(a); B means add(b); C means sub(b); D means sub(c); E
means sub(mul(a,b)); F means mul(a). Choose 1-4 action IDs. Illegal operations,
repeated states, and paths that do not finish with x isolated are rejected. Return
only the IDs separated by semicolons inside the answer box; for example formatting
only: \boxed{A;C}. Do not return x's numeric value.<|im_end|>
<|im_start|>assistant
```


C.5 PANTRYPLAN

Template qwen_pantry_support_mask; 384 training and 128 evaluation rows in var/data/pantry_plan_modebench_v2.

```
<|im_start|>system
Return only six binary digits and no other text. Each digit maps to the
corresponding Pantry row in printed order. A trusted environment will choose exact
quantities on precisely the selected rows.<|im_end|>
<|im_start|>user
Create one feasible high fiber snack from this frozen pantry.
Nutrition values are per 100 g. Quantities are exact integer grams.

Pantry:
- almonds: available=100g, step=25g, minimum-if-used=50g; energy=584kcal,
  protein=21.5g, fiber=10.8g, sodium=0.0mg; tags=tree_nut
- banana: available=125g, step=25g, minimum-if-used=50g; energy=97.0kcal,
  protein=0.74g, fiber=4.62g, sodium=0.0mg; tags=none
- carrots: available=150g, step=25g, minimum-if-used=50g; energy=45.0kcal,
  protein=0.941g, fiber=3.1g, sodium=86.6mg; tags=none
- grape_tomatoes: available=100g, step=25g, minimum-if-used=50g; energy=27.0kcal,
  protein=0.83g, fiber=2.1g, sodium=6.0mg; tags=none
- navel_orange: available=150g, step=25g, minimum-if-used=50g; energy=47.0kcal,
  protein=0.91g, fiber=2.0g, sodium=9.0mg; tags=none
- sunflower_seeds: available=125g, step=25g, minimum-if-used=50g; energy=571kcal,
  protein=18.9g, fiber=7.22g, sodium=0.0mg; tags=none

Use 2 to 4 ingredients.
Forbidden tags: none
Targets:
- mass_g: minimum 125, maximum 200
- energy_kcal: minimum 239.4, maximum 432.25
- protein_g: minimum 6.423
- fiber_g: minimum 3.478
- sodium_mg: maximum 37.6

Choose the ingredient support as a six-bit mask in the exact Pantry row order. Bit 1
includes that row and bit 0 excludes it.<|im_end|>
<|im_start|>assistant
```

D REPLAY ALGORITHM

Table 2 lists the symbols used here. This section states the objective as implemented; App. A.3 reads the same objective against the evaluation metric, which is why that analysis sits with the measurement rather than here. For each prompt x , $\mathcal{B}_x$ stores at most M verified policy exemplars, one per canonical key; the serialized cursor σ selects banks for replay. The same update supports either fresh objective.

D.1 FRESH TASK ADVANTAGES

For a group of G fresh responses with verified binary rewards R_i and $R = \sum_i R_i$, Dr.GRPO uses $A_i = R_i - R/G$, while MaxRL uses

$$A_i^{\text{MaxRL}} = \begin{cases} GR_i/R - 1, & R > 0, \\ 0, & R = 0. \end{cases}$$

The centered MaxRL estimator (Tajwar et al., 2026) gives failures advantage -1 in mixed groups and weights each success more strongly when successes are scarce. All-correct and all-failure groups have zero task advantage. Lemma P.3 gives its harmonic pass@ k mean objective. These advantages apply only to fresh responses; bank exemplars enter the separate verified-likelihood loss.

D.2 STATE AND OPTIMIZER UPDATE

Algorithm 1 One verified-replay optimizer update: $o = \text{Dr.GRPO}$ gives Re:Dr, $o = \text{MaxRL}$ gives Re:Max

Require: Policy π_θ ; prompt x and verifier specification s_x ; group size G ; fresh objective $o \in \{\text{MaxRL}, \text{Dr.GRPO}\}$; replay weight λ_{replay} ; states $(\mathcal{B}, \sigma)$

Ensure: Atomically updated $(\theta', \mathcal{B}', \sigma')$

- 1: Draw the on-policy group $\mathcal{G} = \{y_i \sim \pi_\theta(\cdot | x)\}_{i=1}^G$
- 2: **for** $i = 1, \dots, G$ **do**
- 3: Execute $\tilde{c}_i \leftarrow V(y_i, s_x)$ and observe task reward R_i
- 4: Set $c_i \leftarrow \tilde{c}_i$ iff $R_i > 0 \wedge \tilde{c}_i \neq \perp$; otherwise set $c_i \leftarrow \perp$
- 5: **end for**
- 6: Compute fresh task advantages A_i^o from $\{R_j\}_{j=1}^G$; for $o = \text{MaxRL}$ use the formula in App. D.1
- 7: After the full group is validated, deterministically retain one policy exemplar for each new $c_i \neq \perp$ in $\mathcal{B}_x$, up to capacity M
- 8: $(z, \sigma') \leftarrow \text{NEXTBANK}(\{\mathcal{B}_u\}_u, \sigma)$ by a fixed round-robin cursor over prompts; the replayed prompt z need not be the prompt x that supplied the fresh group
- 9: **if** a replay bank $\mathcal{B}_z$ exists **then**
- 10: For each exemplar $e_c \in \mathcal{B}_z$, compute $\ell_c = |e_c|^{-1} \log \pi_\theta(e_c | z)$
- 11: $L_{\text{replay}} \leftarrow -|\mathcal{B}_z|^{-1} \sum_{c \in \mathcal{B}_z} \ell_c$
- 12: **else**
- 13: $L_{\text{replay}} \leftarrow 0$
- 14: **end if**
- 15: $w_{\text{rep}} \leftarrow (G - 1)/G^2$ for one replay pseudo-group per G on-policy rows
- 16: $\theta' \leftarrow \text{UPDATE}(\theta, L_{\text{PPO}}(A^o) + w_{\text{rep}} \lambda_{\text{replay}} L_{\text{replay}})$
- 17: Checkpoint $(\theta', \mathcal{B}', \sigma')$ as one atomic state

D.3 ADMISSION, CAPACITY, AND DETERMINISTIC SCHEDULING

A bank is keyed by the SHA-256 digest of the exact prompt-token sequence. Only loss-active rows with positive reward and a nonempty validator key qualify. A row is loss-active when it carries weight in the policy-gradient loss, which the framework withholds from a response that reaches the generation cap without emitting an end-of-sequence token. That exclusion is disabled in every run reported here — MaxEnt length control refuses to run with it on, since it needs the truncated rows — so the qualifier admits every rollout row and excludes none; it is stated because a deployment that enables it would silently change what the bank can hold. The full group is validated against the pre-group bank before any new key is committed. For each new key, the lexicographically smallest normalized response-token tuple is retained; new keys are admitted in sorted-key order. Admission is therefore invariant to rollout-row order. The bank keeps one exemplar per key, up to $M = 16$, with no eviction or replacement. Later keys may enter diagnostics but not replay support when capacity is full. Prompt-hash collisions or changed tokens under an existing hash are hard errors.

At each update, the scheduler sorts nonempty prompt-bank hashes and selects one by a fixed round-robin cursor; singleton banks are eligible. The cursor is independent of reward, evaluation, desired support, or entropy, and is serialized with the bank. Stored prompt and response tokens are materialized unchanged into a right-padded teacher-forcing batch. Prompt and padding positions are masked. Each exemplar contributes its mean response-token log probability, averaged uniformly across keys in the selected bank before applying λ_{replay} and the exact $(G - 1)/G^2$ rollout scaling in Algorithm 1. That constant is the weight one lucky success already carries in a centered fresh group, so a replay pseudo-group enters at the same size as the event it stands in for: the proof of Lemma P.2 gives a lone correct row the centered reward $(G - 1)/G$, and the outer $1/G$ that averages the group supplies the second factor. Neither exhaustive support nor evaluation outputs enter this loss.

D.4 MATCHED CONTROLS AND RECOVERABLE STATE

Controls share validation, bank updates, scheduling, materialization, and the score pass, with the replay score-gradient vector replaced by exact zeros. Treatment activates only that derivative. Policies can subsequently differ in samples, bank contents, selected banks, exemplar lengths, and realized computation. Logs retain bank/key counts, replayed token counts, raw and applied losses, score-gradient summaries, cursor phase, and the exact-zero switch. Model and optimizer state, bank counts and exemplar tokens, replay and data cursors, and request-stream state are checkpointed

atomically; the loader validates schema, capacity, scheduler and estimator constants, token types, prompt hashes, key/exemplar consistency, and cursor bounds before accepting a resume.

The zero-derivative switch changes the learning signal, not the configured replay traversal: live score computation and backward still execute with zero score derivatives. Fresh-task and replay gradients accumulate before the same optimizer step. This is a control for the replay signal; it neither equalizes realized work nor allocates that work to additional baseline learning (App. Q.1).

D.5 WHAT THE BANK NEEDS FROM A VERIFIER

Replay reads one thing from the task that ordinary RLVR does not: an equivalence on outputs. Everything else in Algorithm 1 is objective-agnostic. That equivalence has to be decidable while the rollout is in hand, cheap enough to run on every row of every group, a function — the same output always yields the same key — and consistent with the reward, so that “still correct” and “a different answer” cannot disagree. MODEBENCH supplies it from the executable check that already grants credit, which is why no judge or embedding threshold stands between the data and the claim.

The widely used mathematical reasoning sets do not supply it. A grader for MATH, GSM8K or AIME returns correctness on a final answer, and the final answer is unique, so an identity keyed on what the grader returns gives every correct response to a prompt the same key. A bank under that identity holds one exemplar per prompt, the replay term reduces to rehearsing the single success the policy already found, and the mechanism this paper measures has nothing to rebalance. Getting more than one mode out of such a task means keying identity on the derivation instead of the answer, and MathIR is the cautionary case: it keys on the executed state trajectory, its certified alternatives are mostly longer routes, and across 27,450 verified draws no model in this paper produced any of those longer routes. The shorter certified routes are not unreachable — 67% of the certified two-state modes and 52% of the three-state modes appear somewhere in those draws — but they are not reached within a prompt, which is where diversity is counted: MathIR’s terminal PCMD averages .006, or 1.01 effective verified modes per prompt. Our own evidence therefore places open-ended derivations at the hard end of this method’s range, not the easy end.

MathIR is nonetheless retained in every five-domain mean this paper reports, and retaining it lowers them. Its gain sits below every other domain’s, so in all 6 cells where the five domains are complete the five-domain mean is under the four-domain mean over the same cells, by between .010 and .074 PCMD; at Qwen2.5-0.5B the Re:Dr gain is +.294 over five domains against +.364 over the four whose modes are alternative answers. The reported figure is the more conservative of the two. A reader who holds that a derivation-keyed domain does not test the same mechanism can read the four-domain figure instead, as the hosted comparison reports one (App. N.1); it is the larger number in every cell, so no claim here turns on which of the two is read.

A model judging whether a candidate derivation is new relative to the bank would restore an equivalence where execution cannot supply one, and it is the natural adaptation. It is also a trade rather than a free extension, and the terms are worth stating. It gives up the property that identity and reward come from one check, so the diversity being optimized and the diversity being reported would no longer be the same object. It gives up determinism unless the judge is pinned to a fixed model, prompt and temperature and frozen for the run; admission here is invariant to rollout-row order (App. D.3), and a judge that answers the same pair two ways forfeits that. It also puts the identity inside the optimization: replay reinforces whatever the judge calls new, so the judge’s blind spots become a target, which is the reason the comparators in this paper that require an in-loop judge are cited rather than run.

Those costs are measurable rather than hypothetical, and MODEBENCH is the instrument for measuring them. A judge can be scored against ground-truth identity on the domains where an executable key exists, before it is trusted on domains where none does, and the disagreement rate on that calibration set bounds what it would contribute as training signal. Reporting PCMD under the executable key while training under the judge key would keep measurement independent of the signal on exactly the cells that permit both. This paper takes neither step; the design is set out because the conditions it would have to meet are specific, and the MathIR result is a reason to expect a smaller gain where a mode is a derivation rather than an answer.

D.6 WHAT THE BANK DOES NOT RANK

Uniform rehearsal over discovered keys says nothing about whether a discovered key deserved to be kept. A verified output can be long-winded, or can reach a correct answer by reasoning that would not survive inspection, and the verifier will still grant it credit: a canonical key describes an executed output, not the thinking behind it. Re:Max preserves modes; it does not appraise them.

Two parts of the update already blunt the specific worry that verbosity is what gets reinforced. An exemplar is scored by its *mean* response-token log probability, $\ell_c = |e_c|^{-1} \log \pi_\theta(e_c | z)$, so its contribution does not grow with its length, and the bank holds one exemplar per key, so a verbose mode cannot occupy more of a bank than a terse one. The idealized account is sharper still: under the mean-token exemplar loss the effective per-key weights go as L_b^{-1} , so the limit tilts toward shorter exemplars rather than longer ones, and is uniform only when lengths are equal. Length inflation is therefore not a direction this objective rewards.

What remains true is that the bank applies no quality order at all, and that is a choice rather than an oversight. Equal weighting over verified keys is the ingredient these experiments isolate; mixing a brevity or quality prior into the same term would make a measured gain ambiguous between keeping more modes and preferring better ones, which is the one distinction the design exists to make.

The seam for anyone who wants both is admission. A row already enters a bank only when it is loss-active, carries positive reward and parses to a canonical key (App. D.3); a token budget, a process-reward score or a trace-quality classifier is a fourth conjunct on that rule, and the scheduler, the pseudo-group scaling and the matched control are untouched by it. Exemplar selection within a key and eviction at capacity are the other two places such a signal naturally attaches. Nothing in this paper’s claim competes with that literature: the claim is that when a verified output is a *new* mode, keeping it is worth doing, and worth tracking beside whatever quality signal is already being optimized.

The composition is not free, and this benchmark shows where it binds. Where a domain’s alternative modes are intrinsically longer than its preferred one, a brevity filter removes exactly the modes that make its support non-trivial: MathIR’s alternatives are detours through extra states, and Python’s rarer return vectors need longer lambdas than the modal one. Diversity and terseness therefore pull against each other hardest in the domains where diversity is already hardest to hold, which is a reason to read PCMD after such a filter rather than before it.

E FIXED SEMANTIC MAXENT COMPARATOR

Fixed Semantic MaxEnt is a fresh-rollout comparator, not a replay method: it adds no bank, reuses no verified exemplar, and touches none of the state App. D specifies. It is set out here because it is the one alternative objective this paper implements rather than cites, and because App. F.4 reads its measured effects against that specification. It is entropy-inspired: for validator-positive canonical outcomes, an empirical predictor combines successful policy history and successful leave-one-out peers. Scoring row i of a group of G on prompt x , write $n_x(a)$ for the historical count of key a at that prompt, $m_{-i}(a)$ for its count among that row’s $G - 1$ peers, $N_x = \sum_a n_x(a)$, and $\mathcal{K}_i$ for the union of the historical and peer-observed keys. Unit pseudocounts and one reserved unseen bucket give

$$\hat{p}_i(a) = \frac{n_x(a) + m_{-i}(a) + 1}{N_x + G - 1 + |\mathcal{K}_i| + 1}, \quad \hat{p}_i(\text{UNSEEN}) = \frac{1}{N_x + G - 1 + |\mathcal{K}_i| + 1}. \quad (9)$$

What the counts accumulate. The estimator keeps one count table per prompt, addressed by the SHA-256 digest of the unpadded prompt-token sequence, and keeps nothing global: two prompts never share a count, and a key’s count at one prompt says nothing about how often that key was produced at another. A row reaches the table only when it is loss-active, carries positive task reward, and parses to a canonical key — the three conditions the replay bank admits on (App. D.3) — so invalid, unsuccessful and unparseable rows leave it unchanged. The table is strictly cumulative over every update the run has taken on that prompt: counts only rise, and there is no window, decay, eviction or reset. It is serialized into the checkpoint’s client state beside the optimizer, so a resumed run continues the history rather than restarting it. Within a group the order is score first, then commit: every candidate is scored against the history that earlier groups left plus its own $G - 1$ peers, and the group is added only once all of its candidates have been scored, so no row ever contributes to

the count that scores it. The added detached advantage on an eligible successful row is

$$A_{i,t}^{\text{sem}} = \eta \frac{\min\{-\log \hat{p}_i(a_i), \kappa\} - \mathbb{E}_{\hat{p}_i}[\min\{-\log \hat{p}_i(a), \kappa\}]}{\kappa}, \quad \eta = .10, \kappa = 5. \quad (10)$$

The score does not depend on the token index, so row i carries the same $A_{i,t}^{\text{sem}}$ at every t . Invalid and unsuccessful rows receive zero, and the centered score is bounded by η in magnitude. It can favor an underrepresented verified outcome present in a fresh rollout, but supplies no direct reinforcement to an absent remembered exemplar. Its heterogeneous empirical effects therefore remain a comparison with exploration rather than a retention guarantee.

For the true conditional distribution $q_\theta = p_\theta(\cdot \mid \mathcal{C})$,

$$\nabla_\theta H(q_\theta) = \mathbb{E}_{a \sim q_\theta} [(-\log q_\theta(a)) \nabla_\theta \log q_\theta(a)]. \quad (11)$$

This motivates the direction of redistribution, not an unbiased-gradient claim for the empirical estimator. Hold a detached predictor score vector s and baseline b fixed independently of the scored action, for example conditional on its leave-one-out peers. Averaging successful rows under the unconditional policy then gives

$$\mathbb{E}_{Z \sim p} [\mathbf{1}\{Z \in \mathcal{C}\} (s_Z - b) \nabla_\theta \log p_Z] = P \mathbb{E}_{c \sim q} [s_c \nabla_\theta \log q_c] + (\mathbb{E}_q[s] - b) \nabla_\theta P.$$

A historical baseline need not equal the current conditional mean, so this bonus can change correctness as well as diversity. Even with the exact unclipped score $s_c = -\log q_c$ and baseline $b = H(q)$, averaging over unconditional rollout rows gives $P \nabla_\theta H(q)$. For a single correct key, conditional entropy is zero, yet the implemented unseen bucket can give that key negative semantic advantage. The comparator thus carries neither an exact entropy-gradient interpretation nor an individual-mode preservation guarantee.

Part III. Training results

F EMPIRICAL RESULTS AND COMPARISONS

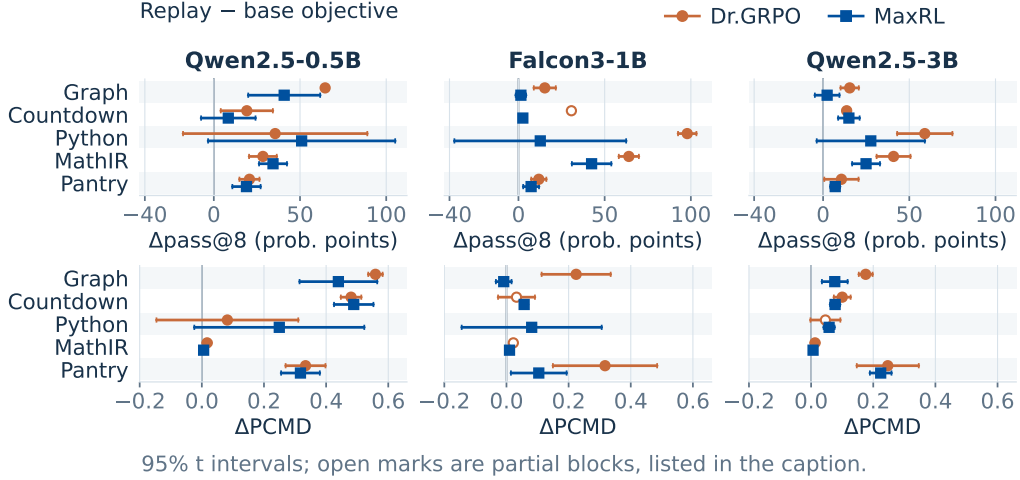


Figure 18: **Canonical replay complements both fresh objectives.** Each point compares replay with its matched no-replay control at pass eight, within domain and scale, on the paper’s two axes: correctness in probability points, and PCMD on the successes alone. The rows do not share a seed population. For `pass@8` all 75 MaxRL pairs are complete and Falcon Countdown uses four Dr.GRPO pairs. For PCMD a seed counts only where both arms define it: on the Dr.GRPO side that leaves Falcon Countdown at four, Falcon MathIR at one, Qwen2.5-3B Python at two, and Falcon Python blank rather than zero. Nominal intervals accompany complete blocks only.

F.1 TERMINAL COMPARISONS ACROSS SCALES

Within each model–domain–seed comparison, methods share initialization, data identities, the eight-pass horizon, evaluation requests, verifier, and replay scheduling code. The replay control runs the same bank and scoring path with zero replay derivative. Resulting policies can build different banks. The admissible intersections are defined in App. S.1.

Dr.GRPO retention contrasts. Table 9 gives the same Re:Dr contrast per domain and scale that Table 8 gives as levels, and reading the two together is what separates a gain from a ceiling: a large difference on a domain whose control already sits high is a different result from the same difference on one that starts at zero. Both co-primary block means are positive in all 14 complete five-seed comparisons, and the registered `distinct@8` endpoint increases in all 74 admissible pairs; its per-cell values are reported in the machine-readable record rather than here, because it rises with correctness and cannot separate the two. The paired PCMD effects can. At Qwen2.5-0.5B, Countdown `pass@8` rises from .480 to .672 while PCMD rises +.480 (paired 95% interval [+ .447, + .513]). Python correctness rises from .172 to .528, with a PCMD effect of only +.082 ([−.146, +.310]); MathIR rises from .512 to .796 with PCMD +.017 ([+.003, +.031]). These intervals are nominal, unadjusted Student-*t* estimates over five paired training seeds. Correctness and diversity separate cleanly here, so the same Countdown comparison is also read on the success-conditional axis, where Dr.GRPO ends at PCMD .007 and Re:Dr at .487: the replayed policy is not merely succeeding more often, its successes are spread across modes where the control’s are not. Countdown keys identify computation routes, but these held-out endpoints do not identify which previously discovered routes survived training.

Fig. 19 resolves the domain-specific results behind the main cross-domain display. All five 3B MaxRL domain comparisons are complete: both arms have seeds 70–74 at the registered terminal endpoint (50/50 runs). The Dr.GRPO pair uses five seeds except Falcon Countdown, which uses four. These populations remain separate: a factorial interaction requires the common four-arm intersection, not differences of aggregates with different seeds.

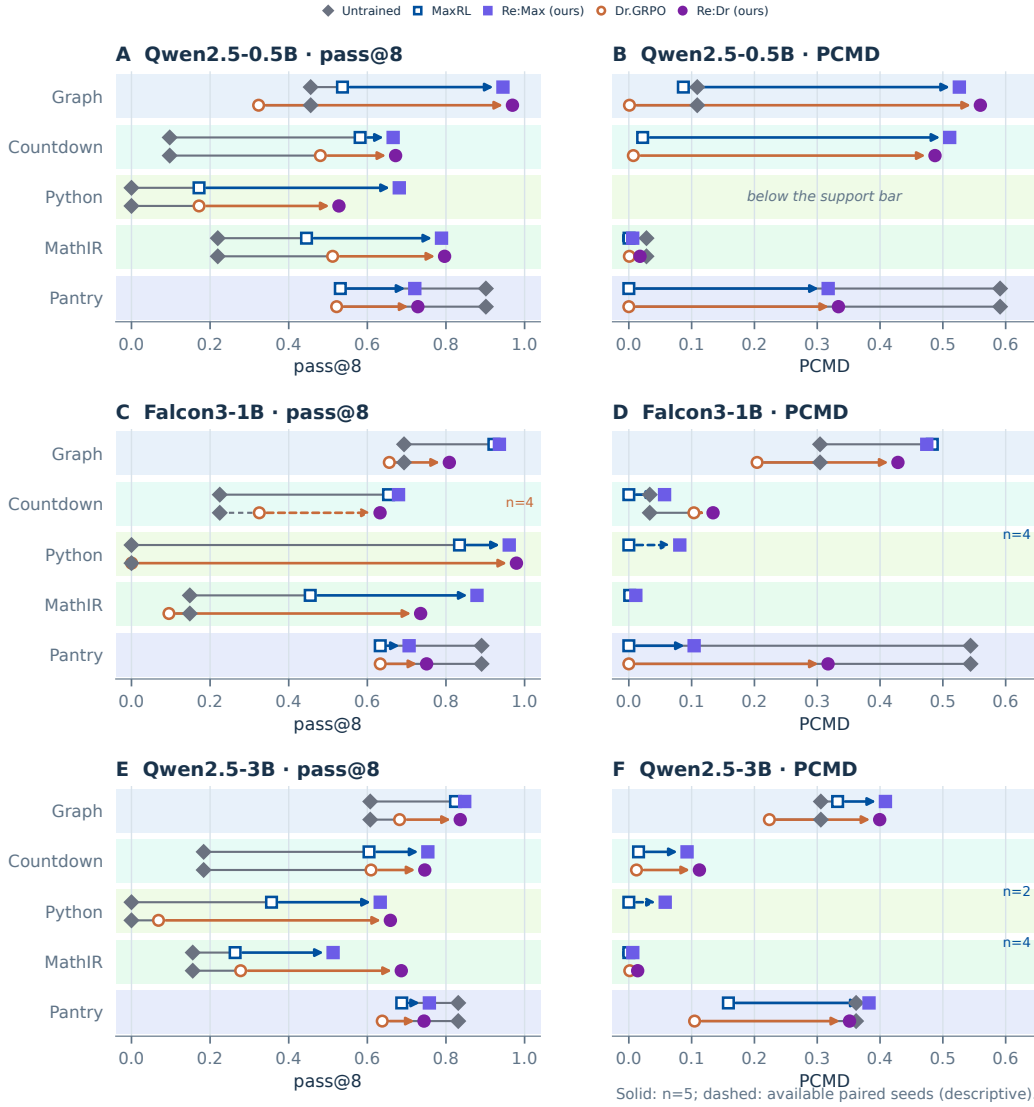


Figure 19: Per-domain results behind the cross-domain averages in Fig. 9. Rows show Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns show correctness and PCMD, the same two axes the main results use. On $\text{pass}@8$ every MaxRL–Re:Max pair is complete at five terminal seeds across all three models, and Dr.GRPO–Re:Dr is complete except for Falcon Countdown at four. Solid connectors identify the complete comparisons and dashed ones carry their count. The two columns do not share a seed population: PCMD is undefined for a seed whose policy succeeds on too few prompts, so the right column carries its own counts, and a row reading *below the support bar* is one where no seed on one side of either pair defines PCMD — diversity unmeasured there, not measured as zero. This panel applies the 30-prompt bar of App. A.1; Table 9 pairs seeds at the lower 20-prompt bar and so reports its own, larger counts for the same cells. Domain tints match Fig. 2. Replay arrows, trained-arm markers, and untrained references use the same encoding as Fig. 9.

Table 8: **Where each arm ends, after eight training passes.** The levels behind the differences in Table 9: both arms on correctness and on diversity over the successes alone, as paired-seed means at pass 8. Blocks are five seeds unless a superscript on the domain says otherwise (App. S.1); a superscript on a single PCMD value is that cell’s own seed count, which can fall below its block’s when the support bar thins one arm. An em dash is a cell where no seed defines PCMD on 30 prompts, so diversity is unmeasured rather than zero; every one is a Dr.GRPO control. Table 9 still differences three of the four, because a paired difference carries the lower bar of 20 defined prompts rather than this table’s 30 (App. A.2); the one cell where even that bar fails is the uncoloured entry there. Shading runs red through yellow to green from zero up to the largest level of that metric, on one scale shared by both arms, so the two columns of a pair can be read against each other.

Model	Domain	Dr.GRPO		Re:Dr	
		pass@8	PCMD	pass@8	PCMD
✂ Qwen2.5-0.5B	Graph coloring	.323	.001	.969	.560
	Countdown	.480	.007	.672	.487
	Python factors	.172	—	.528	.000 ²
	MathIR	.512	.001	.796	.018
	PantryPlan	.522	.000	.729	.334
Falcon3-1B	Graph coloring	.656	.204	.809	.428
	Countdown ⁴	.325	.102	.632	.134
	Python factors	.000	—	.979	.060
	MathIR	.095	—	.736	.022
	PantryPlan	.633	.000	.751	.317
✂ Qwen2.5-3B	Graph coloring	.682	.224	.836	.399
	Countdown	.609	.012	.746	.112
	Python factors	.069	—	.659	.098
	MathIR	.278	.001	.686	.014
	PantryPlan	.638	.105	.744	.351

Table 9: **Replay improves correctness and verified diversity at every scale.** Re:Dr minus matched Dr.GRPO at the terminal checkpoint, per domain and scale behind the cross-domain averages of Fig. 9. Columns, the equally weighted Avg, the superscripted paired seed counts and the em dashes for a block with too few verified pairs on one side all read as they do in Table 1. The two Avg columns are equally weighted within seed over a basis fixed per row: for pass@8 that is all five domains, on the seeds all five share; for PCMD it is the domains every seed of that row defines, which drops Python at both larger scales and Countdown and MathIR at Falcon3-1B, so the PCMD average is never taken over a domain set that changes from seed to seed.

	Δ pass@8						Δ PCMD					
	Graph	Countdown	Python	MathIR	PantryPlan	Avg	Graph	Countdown	Python	MathIR	PantryPlan	Avg
✂ Qwen2.5-0.5B	+.646	+.191	+.356	+.285	+.207	+.337	+.559	+.480	+.082	+.017	+.334	+.294
Falcon3-1B	+.153	+.307 ⁴	+.979	+.640	+.118	+.442 ⁴	+.224	+.032 ⁴	—	+.022 ¹	+.317	+.271
✂ Qwen2.5-3B	+.154	+.137	+.590	+.409	+.106	+.279	+.176	+.100	+.046 ²	+.013	+.247	+.134

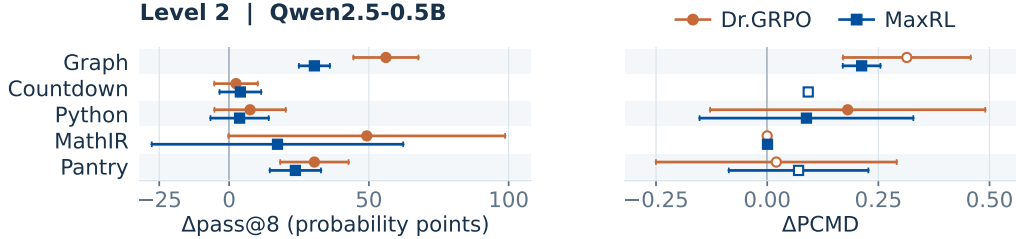
Write $P = \text{pass@8}$ (Table 2). The endpoint tables pair it with PCMD, the paper’s diversity axis, and report exact paired terminal effects on both. The registered distinct@8 endpoint and the derived count $B = \text{distinct@8} - P$ of verified modes beyond the first correct outcome are not tabulated here: both rise with correctness, so neither distinguishes succeeding more often from succeeding in more ways, and both are reported in full in the adjacent machine-readable records. The one exception is Table 10, where an interaction needs both of its differences on the same seed and too few Level-2 seeds define PCMD on all four arms to carry one. The two axes also carry different seed populations, listed separately: a seed enters the PCMD column only where both arms define it. A dash denotes no observed pair; partial means retain $n/5$ without an interval. Complete-block intervals are nominal 95% estimates, without multiple-comparison or repeated-look adjustment. All underlying arm means and seed effects remain in the adjacent machine-readable records.

At Qwen2.5-0.5B, Re:Max raises PCMD by +.439, +.489, and +.317 on Graph, Countdown, and PantryPlan, respectively; each five-seed interval excludes zero. Falcon Graph instead has a negative mean diversity effect (−.009, interval [−.034, +.016]), and its correctness effect is inconclusive (+.014, [−.016, +.044]). Positive cross-domain effects therefore do not imply a diversity gain in every domain. All five Qwen2.5-3B blocks are complete, with PCMD effects of +.076, +.077, +.058, +.006, and +.224 for Graph, Countdown, Python, MathIR, and PantryPlan; every nominal interval excludes zero, although MathIR’s gain is small enough to be of no practical size; that domain’s support leaves almost no reachable alternative to gain. Correctness intervals

exclude zero for Countdown, MathIR, and PantryPlan; Graph and Python remain inconclusive. Read across the cohort, Re:Max ends above MaxRL in 14 of the 15 comparisons that clear the paired support bar, and Re:Dr above Dr.GRPO in 14 of 14 (App. A.2).

Cross-domain factorial summaries. Averaging five domains equally within each paired seed, Re:Max minus MaxRL gives $+0.307$ on $\text{pass}@8$ at Qwen2.5-0.5B (unadjusted 95% interval $[+0.208, +0.406]$) and $+0.133$ at Falcon3-1B ($[+0.043, +0.223]$). Both use five paired seeds. The Dr.GRPO replay effects on correctness are $+0.337$ at Qwen2.5-0.5B, $+0.442$ at Falcon3-1B, and $+0.279$ at Qwen2.5-3B. Diversity is not averaged across domains here: PCMD admits a seed only where both arms define it, so a mean over all five would rest on a population that changes with the domain. Where Table 9 does report a PCMD average it fixes the basis first, to the domains every seed of that row defines. The per-domain PCMD effects are drawn in Fig. 19. The Falcon Dr.GRPO average uses four common admissible seeds without an interval; the other two use five. At Qwen2.5-3B, MaxRL/Re:Max arm means are $.547/.701$ on correctness and $.744/1.107$ on raw $\text{distinct}@8$. Equal domain averages within each of seeds 70–74 give replay effects of $+0.154$ ($[+0.086, +0.222]$) and $+0.363$ ($[+0.285, +0.441]$), respectively. The common four-arm intersection contains fourteen complete five-seed blocks and Falcon Countdown at four seeds.

F.2 TERMINAL COMPARISON ON THE HARDER LEVEL



Replay – base; 95% t intervals. Open marks are partial blocks; the panels do not share a cohort.

Figure 20: Replay effects within Level 2. Replay and control are compared on the same prompts and training seeds, with separate Dr.GRPO and MaxRL effects, on correctness and on PCMD over the successes alone. All five domains have complete five-seed factorials on $\text{pass}@8$, including the twenty PantryPlan endpoints. The diversity panel does not share that cohort: PCMD counts a seed only where both arms define it, which at Level 2 leaves Dr.GRPO at three seeds on Graph, four on MathIR, three on PantryPlan and none at all on Countdown, and MaxRL at one on Countdown and four on PantryPlan. A domain with no drawn mark is one where the comparison cannot be made rather than one where it came out at zero. MathIR’s flat diversity is the domain rather than the method: every Level-2 problem it poses admits a single shortest derivation, so an arm that solves it the short way has no second mode to hold. Construction and admission are described in Section 2.2; terminal absolute values appear in Fig. 11. Differences between levels combine task, population, and protocol changes, so they do not identify a causal effect of difficulty alone.

Fig. 11 spans three levels, so its across-domain $\text{pass}@8$ mean is carried by the four domains — Graph coloring, Countdown, Python factors and PantryPlan — that hold all five seeds at step 3,072 in all four methods at every level. Its PCMD mean is carried by the three — Graph coloring, MathIR and PantryPlan — that additionally clear the support bar in all four methods at every level, and MathIR enters that mean on the seeds its arms support, which the figure’s record gives per arm. Both bases are the same for every arm and level, so no arm rests on an easier subset than the one it is being read against. Means average domains equally within each seed, then the seed means. All 100 Level-2 endpoints are admitted; each within-domain contrast uses seeds 43–47, including all four arms. The levels have separate held-out prompts, with population differences in App. B.3. Each level is a separate construction rather than a point on a scale, so the figure plots what replay adds within a level rather than four absolute positions per level. Every bar is a difference of two means reported above, both sides sharing the level, the prompts, the seeds and the domain basis; the differences are descriptive and carry no interval.

The untrained checkpoint each level’s training starts from is recorded beside the gains rather than drawn with them: it is a position, not a gain. The figure’s machine-readable record carries it per level,

averaged over every admitted step-0 cell on the domains the bars use, with its support. Levels 1 and 3 carry one on both axes. At Level 2 only PantryPlan clears the support bar before training, so no untrained diversity is reported there; a value resting on one of the three matched domains would not be comparable to the rest, and an unmeasured diversity is a different statement from a diversity of zero.

Level-2 Graph provides the clearest replay effects on both axes under both objectives: PCMD rises $+ .212$ ($[+.170, +.255]$, five seeds) under Re:Max and $+ .314$ ($[+.170, +.458]$, three) under Re:Dr. Python’s diversity effects are positive in mean with intervals spanning zero ($+ .088$ and $+ .181$), and MathIR’s are indistinguishable from zero on both arms. Pantry’s complete five-seed Dr.GRPO replay contrast is $+ .305$ on P ; its MaxRL contrast is $+ .237$. The harder construction also costs diversity coverage rather than correctness coverage: PCMD admits a seed only where both arms define it, and at Level 2 that leaves Pantry at four seeds under Re:Max and three under Re:Dr, Graph at three under Re:Dr, and Countdown with one seed under Re:Max and none under Re:Dr. Those cells are gaps in what can be measured, not measurements of no effect.

For each common seed and endpoint Y , the replay interaction is

$$I = (Y^{\text{Re:Max}} - Y^{\text{MaxRL}}) - (Y^{\text{Re:Dr}} - Y^{\text{Dr.GRPO}}).$$

Replay can add value under both fresh objectives without a positive interaction. On Level-2 Graph, MaxRL alone raises P relative to Dr.GRPO by $+ .233$ ($[+.140, +.325]$), while the replay interaction on P is $- .256$ ($[-.329, -.183]$). This is an additional replay benefit after stronger fresh optimization, without a superadditive interaction.

Table 10: Fresh-objective effects and replay interactions on Level 2. All three contrasts within a domain use the same four-arm seed intersection. The interaction is $(\text{Re : Max} - \text{MaxRL}) - (\text{Re : Dr} - \text{Dr.GRPO})$. Each entry gives its mean and, only for a complete five-seed block, a paired unadjusted 95% Student- t interval. This is the one table that keeps the `distinct@8` axis: an interaction needs both differences on the same seed, and at Level 2 the seeds where all four arms define PCMD number five only on Python, four on MathIR, three on Graph, two on PantryPlan and none on Countdown — too thin to carry the estimand.

Domain	n	Contrast	ΔP	ΔD	ΔB
Graph	5	MaxRL – Dr.GRPO	$+ .233$ $[+.140, +.325]$	$+ .322$ $[+.162, +.482]$	$+ .089$ $[+.019, +.159]$
Graph	5	Re:Max – Re:Dr	$- .023$ $[-.062, +.016]$	$- .029$ $[-.198, +.140]$	$- .006$ $[-.142, +.130]$
Graph	5	Replay $\times$ MaxRL interaction	$- .256$ $[-.329, -.183]$	$- .351$ $[-.446, -.255]$	$- .095$ $[-.186, -.003]$
Countdown	5	MaxRL – Dr.GRPO	$- .014$ $[-.073, +.045]$	$- .014$ $[-.073, +.045]$	$+ .000$ $[+.000, +.000]$
Countdown	5	Re:Max – Re:Dr	$+ .002$ $[-.008, +.012]$	$- .002$ $[-.047, +.044]$	$- .004$ $[-.042, +.035]$
Countdown	5	Replay $\times$ MaxRL interaction	$+ .016$ $[-.047, +.078]$	$+ .012$ $[-.068, +.092]$	$- .004$ $[-.042, +.035]$
Python	5	MaxRL – Dr.GRPO	$+ .000$ $[+.000, +.000]$	$+ .000$ $[+.000, +.000]$	$+ .000$ $[+.000, +.000]$
Python	5	Re:Max – Re:Dr	$- .037$ $[-.232, +.157]$	$- .188$ $[-.988, +.613]$	$- .150$ $[-1.014, +.713]$
Python	5	Replay $\times$ MaxRL interaction	$- .037$ $[-.232, +.157]$	$- .188$ $[-.988, +.613]$	$- .150$ $[-1.014, +.713]$
MathIR	5	MaxRL – Dr.GRPO	$+ .283$ $[-.193, +.759]$	$+ .283$ $[-.193, +.759]$	$+ .000$ $[+.000, +.000]$
MathIR	5	Re:Max – Re:Dr	$- .036$ $[-.158, +.085]$	$- .035$ $[-.158, +.088]$	$+ .002$ $[-.003, +.006]$
MathIR	5	Replay $\times$ MaxRL interaction	$- .320$ $[-.864, +.225]$	$- .318$ $[-.860, +.224]$	$+ .002$ $[-.003, +.006]$
PantryPlan	5	MaxRL – Dr.GRPO	$+ .019$ $[-.067, +.105]$	$+ .019$ $[-.081, +.119]$	$+ .000$ $[-.021, +.021]$
PantryPlan	5	Re:Max – Re:Dr	$- .049$ $[-.113, +.014]$	$- .061$ $[-.249, +.128]$	$- .012$ $[-.145, +.122]$
PantryPlan	5	Replay $\times$ MaxRL interaction	$- .068$ $[-.168, +.032]$	$- .080$ $[-.279, +.119]$	$- .012$ $[-.134, +.110]$

Level-2 Graph endpoint detail. The development admission comparison lowers frozen-model `pass@8` from .438 to .148. At pass 8, Dr.GRPO/MaxRL reach .202/.434, versus .763/.739 with their replay variants. The paired correctness gains are $+ .561$ and $+ .305$ (unadjusted 95% Student- t intervals $[+.445, +.677]$ and $[+.250, +.360]$, five seeds each). The weakest replay run (.711) exceeds the strongest non-replay run (.477). Replay adds 1.019/.668 raw `distinct@8`, yielding 1.243/1.214 under the two fresh objectives. Correctness rises steeply here, so that raw gain is partly a correctness gain; PCMD separates the two. On the same block Dr.GRPO ends at .111 and MaxRL at .112, against .346 and .325 for their replay variants — roughly a threefold difference in the chance that two verified colorings differ, at matched conditioning on being correct. These observations show improved correctness and sampled support on harder Graph; they do not establish that broader support causes the correctness gain. Cross-level prompts are distinct, and the Level-2 aggregate covers Graph, Countdown, Python and MathIR.

F.3 ACCURACY AND VERIFIED MODES THROUGHOUT TRAINING

The following curves include every primary method and model scale used in the main paper. Each objective pair uses its domain-specific terminal seed intersection as one fixed cohort throughout training. A checkpoint is plotted only when all members have complete, valid evaluations for both methods. Missing or conflicting checkpoints leave gaps; seed counts never shrink to extend a mean. Four fixed-seed draws of eight samples on the same 128 held-out prompts define each evaluation. Accuracy and mode curves use identical seeds and checkpoints, without smoothing or outcome-based selection.

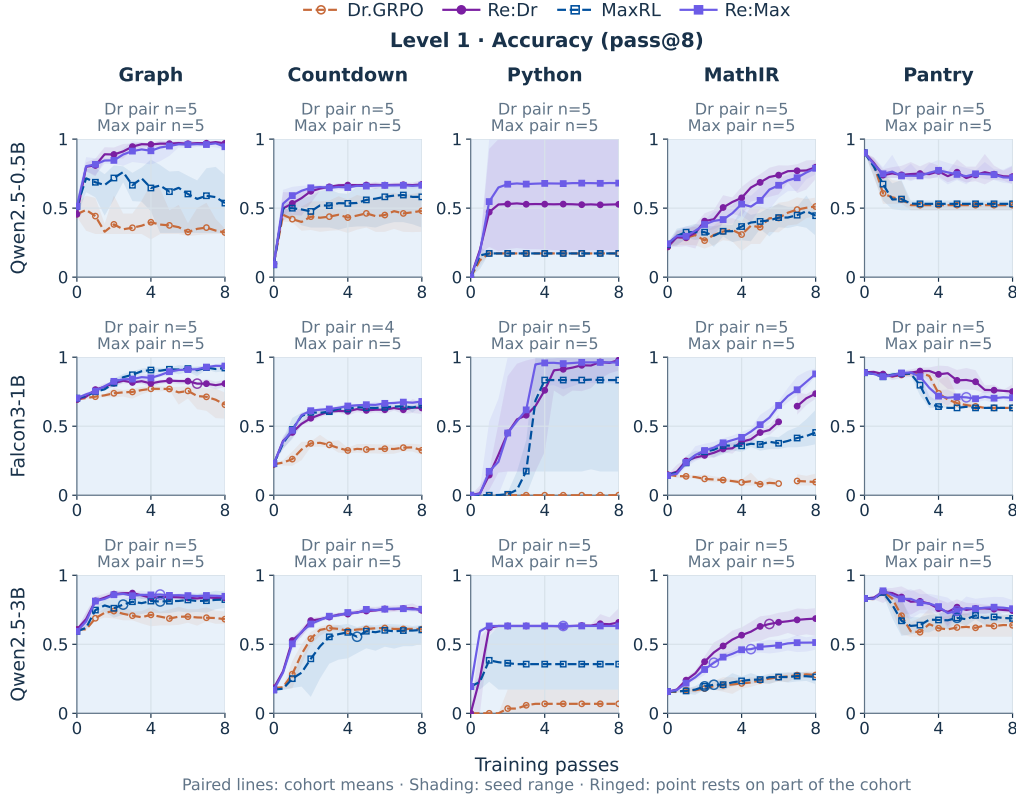


Figure 21: **Accuracy throughout training at all three model scales.** Rows are Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns are the five domains. Dr.GRPO, Re:Dr, MaxRL, and Re:Max use the same method colors as Fig. 9. Lines are fixed-cohort means of pass@8; bands are seed ranges, not confidence intervals. Counts identify each objective’s terminal paired seeds. Qwen2.5-3B MaxRL uses all five seeds in every domain; Falcon Countdown Dr.GRPO/replay uses four. Conflicted checkpoints are admitted by attempt provenance wherever exactly one attempt’s trajectory continues past them (App. S.3), and a checkpoint the admission leaves open is still drawn from the cohort seeds that report it, ringed to show that it rests on part of the cohort. One checkpoint, Falcon3-1B MathIR at pass 6.5, is reported by no seed of either Dr.GRPO arm and is the only break in the figure. It is left unjoined rather than interpolated across, so that break is an absence of evidence rather than a measured value of zero.

F.4 SUPPORTING COMPARATORS

We ran four objectives that act on this same pressure as controls: UCPO’s on-policy advantage redistribution, RLEP’s reuse of successful trajectories, a fixed semantic-entropy bonus, and replay weighted by how often fresh rollouts rediscovered each mode. Their gains are local rather than general, and they fall on correctness rather than on diversity — UCPO helps Countdown correctness while its PCMD there is $+0.002$ ($[-0.018, +0.023]$) and its MathIR effects are inconclusive, and RLEP raises MathIR correctness with PCMD exactly flat. The weighting control is the most direct of the four, because it changes one thing: keep the same bank and the same reuse, and weight modes by rediscovery frequency instead of equally. Equal weighting adds $+0.318$ extra modes per prompt over

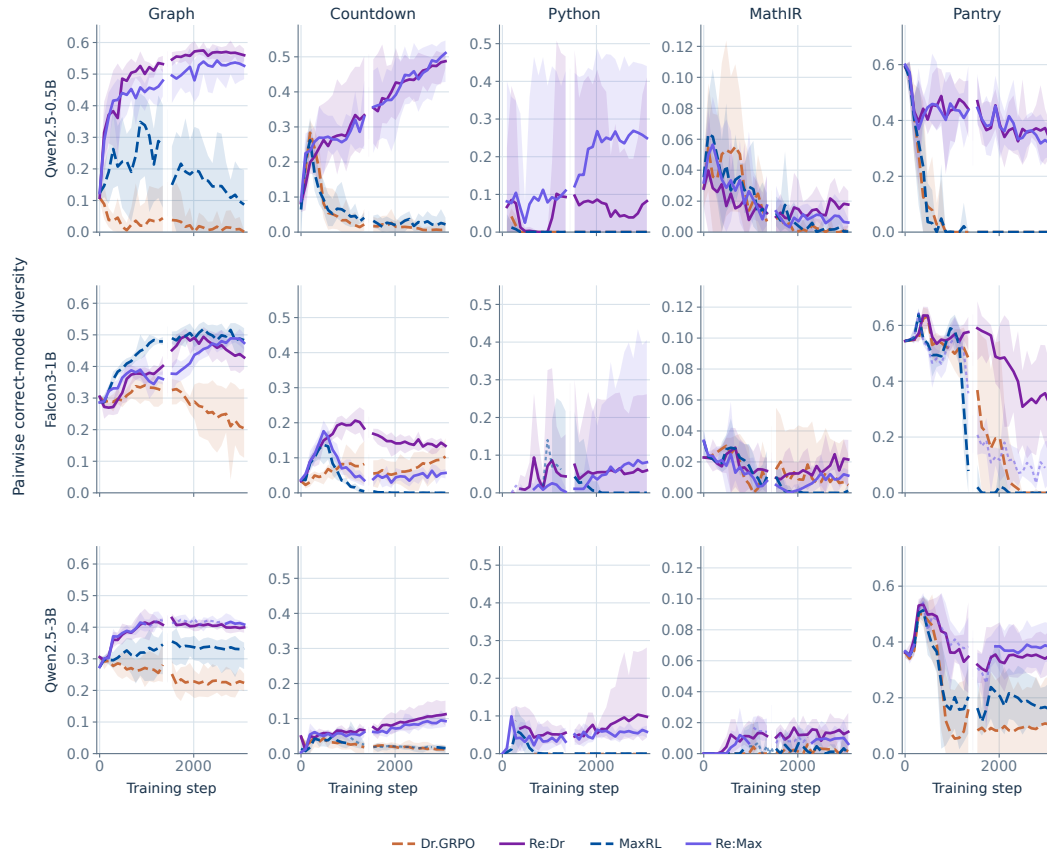


Figure 22: Verified-mode diversity throughout training at all three model scales. The same model rows and methods as Fig. 21, now measuring PCMD. The population is not the same one. PCMD is reconstructed from the saved responses on the evaluation grid those responses were written on, so this figure is drawn against training step over a finer grid than the registered 192-step grid of Fig. 21, and each series carries its own seed cohort, of one to five seeds, rather than that figure’s paired intersection. Counts are in the machine-readable record beside the plate. Because the metric conditions on the verified draws, a falling line reports that successes concentrated, not that they became rarer — the distinction the accuracy curves cannot draw. The two fresh objectives fall in Graph, Countdown, and Pantry at every scale, reaching PCMD .000 in Pantry at 0.5B and 1B, while the replay arms rise or hold. Lines are descriptive means over seeds and bands show seed ranges. A count of one has no band. Domain-specific vertical scales are shared across model rows; absolute heights across different domains should be read against their labeled axes. A step is plotted where at least five prompts return two verified responses. That is below the 30-prompt bar the tables report against, and deliberately: suppressing every step under 30 left panels blank where the measurement exists, and a blank reads as an absence rather than as the thin estimate it is. Under five prompts nothing is plotted, because a standard error of zero on two or three prompts records agreement rather than precision. Segments are never joined across a suppressed step, so a gap marks a step whose diversity could not be measured rather than one measured and found to be zero. Where the cohort is incomplete the mean rests on part of it: a solid stretch is one where every seed reports, a finely dotted stretch one where only some do. The distinction matters because a seed qualifies by succeeding often enough to define PCMD, so a mean over whoever qualifies can move with the qualifying rather than with the metric. Python is the domain to read this way, at every one of the three model scales drawn here.

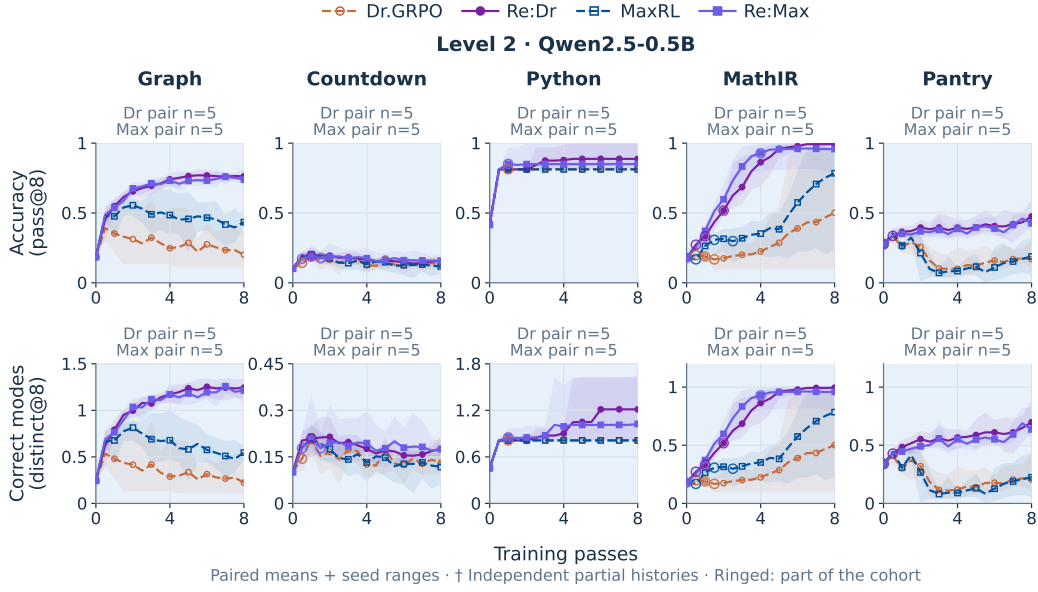


Figure 23: **Accuracy and registered verified-mode counts during Level-2 training.** Qwen2.5-0.5B is the model tested at this level. Rows show pass@8 and the registered distinct@8 for all four primary methods; Fig. 24 gives the success-conditional reading. All five domains contain five terminal paired seeds per objective, as the per-panel counts record. Both metrics follow the same evaluation rules and the provenance admission of App. S.3. Every checkpoint at this level is reported by at least one cohort seed, so no curve breaks; the ringed points are those resting on part of the cohort, and they fall early, where a requeue duplicated the first evaluations.

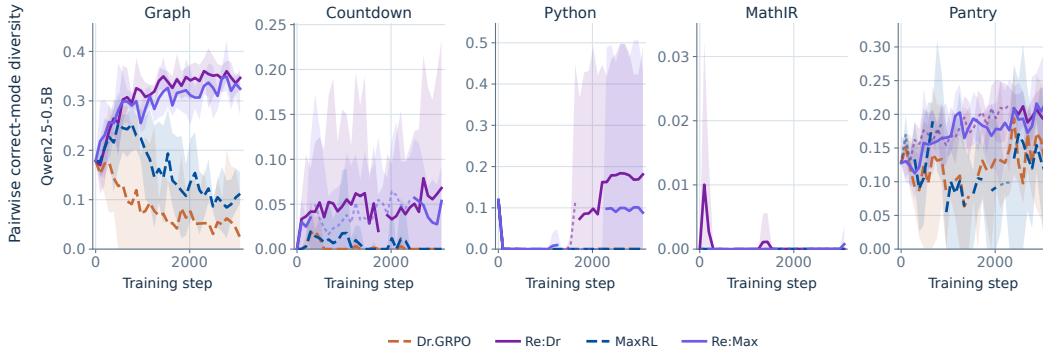


Figure 24: **Verified-mode diversity during Level-2 training.** The same model and methods as Fig. 23, measuring PCMD on the grid the saved responses were written on rather than on that figure’s registered paired grid, so the seed cohort is per series here and not the paired intersection there. Graph separates the arms most clearly: both replay arms rise to roughly .34 while Dr.GRPO and MaxRL fall to near .11, so the replayed policies end with about three times the chance that two verified colorings differ. MathIR collapses to PCMD .000 under every arm at this level. A step is drawn from five defining prompts upward, as in Fig. 22 and for the same reason; Countdown and Python are the panels that rest on that band rather than on the 30-prompt bar the tables use, and are read as thin estimates rather than as trajectories measured to the same precision as Graph. Segments are not joined across a suppressed step.

it $([+.272, +.358])$ with no aggregate correctness difference (App. F.6). Reusing successes is not the active ingredient; refusing to let popularity decide how often each one is rehearsed is.

GRPO changes group-reward normalization; UCPO redistributes fresh on-policy advantage; sparse RLEP-Dr replays verified successes without uniform canonical-key weighting. Fixed Semantic MaxEnt supplies a fresh-rollout exploration incentive (App. E). These are controls for alternative explanations of the retention results, and they are read at the coverage they have rather than at the coverage the primary comparisons have: GRPO runs at all three scales, UCPO and sparse RLEP-Dr only at Qwen2.5-0.5B and Falcon3-1B with 50/50 and 47/50 registered runs, Fixed Semantic MaxEnt completes 10/15 registered blocks, and the weighting ablation completes all nine of its.

The alternatives give local rather than uniform benefits, and the benefits are in correctness. Qwen-0.5B Countdown UCPO raises P by $+.133$ $([+.005, +.262])$ while its PCMD there is $+.002$ $([-.018, +.023])$, and its MathIR effects are inconclusive. MathIR RLEP-Dr raises P by $+.126$ $([+.030, +.222])$ with PCMD exactly flat; its Countdown and Falcon Pantry effects are inconclusive. Seven five-seed blocks raise P by an interval clearing zero, and exactly one moves PCMD that far: Falcon Countdown GRPO, by $-.105$ $([-.147, -.063])$, a narrowing rather than a gain. The full five-seed 3B plain-GRPO blocks have intervals including zero on both axes. Each comparator is read against matched Dr.GRPO on the seeds where both define PCMD, which leaves Qwen-0.5B Graph at one UCPO seed and four for RLEP-Dr, and no Python cell at any scale. RLEP’s frozen offline pool was collected at temperature .7, top- $p = .95$; all reported evaluations use the common temperature-1, top- $p = 1$ contract. An offline pool versus an online bank remains an algorithmic difference.

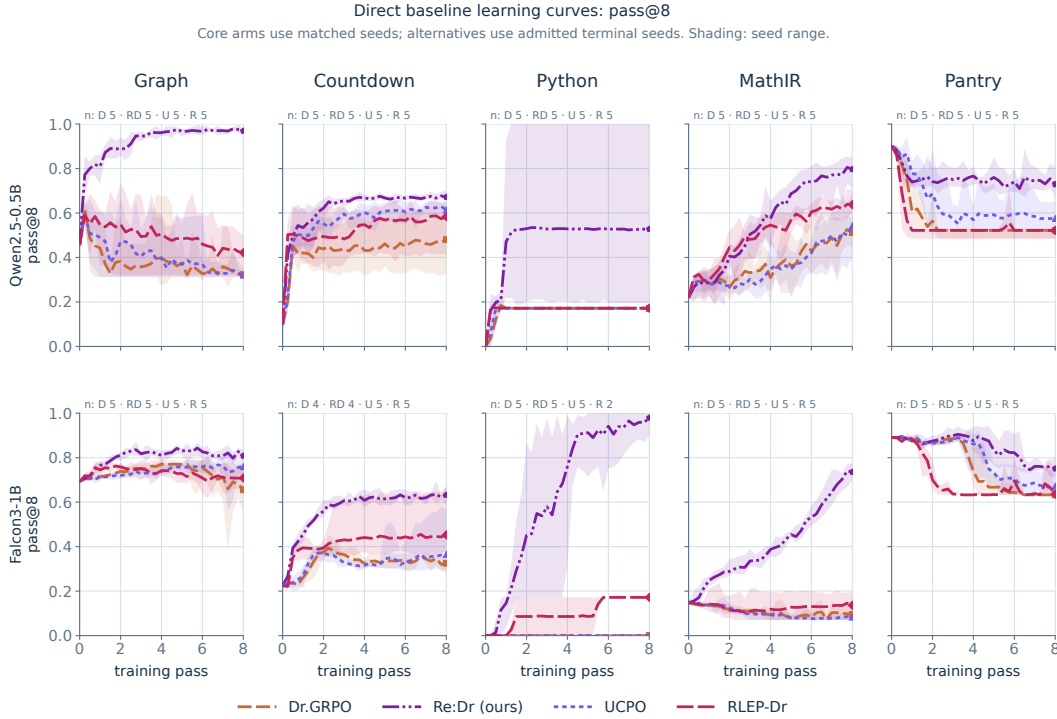


Figure 25: **Accuracy trajectories for the supporting comparators.** UCPO and sparse RLEP-Dr are compared with Dr.GRPO and Re:Dr at the two smaller scales, on pass@8. Rows show Qwen2.5-0.5B and Falcon3-1B; columns are the five domains, and labels give exact method-specific seed counts. A checkpoint is drawn from the cohort seeds that report it. Bands show the seed range and the vertical axes span $[0, 1]$ throughout. MaxRL and Re:Max at all three scales are compared on both metrics in Figs. 21 and 22.

F.5 DIVERSITY-PRESERVING COMPARATORS

Two recent objectives target output diversity directly, and both are run here on the Level-1 Qwen2.5-0.5B panel at five domains and five seeds. GAPO (Anschel et al., 2025) replaces the task reward with a group frequency-aware reward, $1 - (f_i - 1/L)$ over verifier-positive rollouts, where L

is the prompt’s valid-answer count. MODEBENCH certifies L , so the method can be run on its own assumption rather than on an estimate. SetPO (Li et al., 2026a) adds each rollout’s leave-one-out marginal contribution to its group’s kernelized set diversity, over frozen sentence embeddings of the generated text; its coefficient was fixed before any scientific cell ran, at the value placing the shaping term near the ratio our adaptive controller targets.

Both give local gains. GAPO moves PCMD on PantryPlan and Countdown and is inert on Python-Factors, where its uniform target is unreachable: L exceeds the sixteen-rollout group on 95% of those prompts and reaches 3600, leaving the $1/L$ term numerically dead and the reward a plain duplicate penalty. MathIR is a different case and not a GAPO failure — no method moves it, the largest effect on that column being $+0.017$, because its certified support is dominated by derivations the policy never produces. SetPO’s one clear diversity gain is PantryPlan. Neither reaches Re:Max’s diversity across the panel, and the contrast is where each acts: GAPO flattens frequency inside the fresh group, while replay equalizes rehearsal across a bank of verified modes over time.

These two rows are read against a re-run Dr.GRPO control rather than the one the other rows use. Their runtime could not be inherited, they sit on different GPUs, and they take the corrected disjoint-draw evaluator. The substitution is measurable and small: the re-run control’s terminal PCMD sits $+0.003$ from the published control over twenty paired cells, per-cell -0.014 to $+0.034$, against the effects of $+0.225$ and $+0.320$ the table reports.

F.6 UNIFORM VERSUS FRESH-FREQUENCY REPLAY

Fig. 10 presents the main comparison; this section gives the full protocol and model–domain contrasts.

The weighting ablation retains the same bank construction and update code, changing only the within-bank target: uniform weights over retained keys versus weights proportional to validator-positive fresh-rollout counts. Policy-dependent bank contents can subsequently differ. Positive contrasts below favor uniform weighting. The co-primary contrasts are ΔB and ΔP ; the five-domain Qwen2.5-0.5B aggregate first averages domain effects within each matched seed, giving equal domain weight.

Complete-block intervals enumerate all $5^5 = 3,125$ ordered bootstrap resamples of the five paired training seeds and use linearly interpolated 2.5th and 97.5th percentiles. A resampled seed carries its complete domain vector when estimating the five-domain aggregate. These nominal intervals quantify seed variability within the fixed benchmark, without resampling prompts or domains or adjusting across comparisons.

The Qwen2.5-0.5B five-domain effect is $+0.318$ extra modes per prompt ($[+0.272, +0.358]$), with positive within-seed domain averages in all five seeds. Graph, Countdown, and PantryPlan drive this result; Python and MathIR remain inconclusive. The correctness effect is $+0.010$ ($[-0.119, +0.139]$), establishing neither a gain nor equivalence. Python’s correctness effects range from -0.781 to $+0.828$ across seeds despite small extra-mode effects in four seeds.

Falcon Graph also favors uniform weighting on extra modes ($+0.233$, $[+0.182, +0.320]$). Its two correctness readings point different ways, and neither establishes a loss: its registered ΔP is $+0.045$ on an interval $[-0.0004, +0.0910]$ that includes zero, while mean per-response correctness, which the table does not carry, falls by -0.0152 ($[-0.0214, -0.0090]$). Falcon Pantry is inconclusive: $\Delta B = -0.146$ ($[-0.564, +0.414]$) and $\Delta P = -0.021$ ($[-0.063, +0.026]$). Qwen2.5-3B Graph is complete at five pairs, with an extra-mode effect of $+0.079$ ($[+0.011, +0.135]$); its correctness interval includes zero. Pantry also has five pairs, with $\Delta B = +0.207$ ($[+0.038, +0.377]$) and inconclusive correctness ($+0.011$, $[-0.020, +0.043]$). The evidence supports a contribution from key balancing in the stated domains, not a uniform accuracy or diversity advantage across every model–domain block.

This ablation is also what separates the method’s two gains, and they do not have the same source. The replay term is a length-normalized log-likelihood on the policy’s own verified successes, which is rejection-sampling fine-tuning (Dong et al., 2023) with a deduplication rule; raising correctness by rehearsing self-generated successes is an established effect of that family and is not attributable to how the rehearsal is weighted. What the weighting changes is the diversity: uniform against fresh-frequency weighting moves extra modes by $+0.318$ on an interval excluding zero, while its correctness effect of $+0.010$ spans zero in both directions. Reuse buys accuracy; weighting modes equally buys diversity. Claims in this paper that credit the memory for both should be read with that split in mind.

G FIXED SEMANTIC-ENTROPY FACTORIALS

A fixed semantic-entropy bonus can be applied with or without verified replay. Pantry semantic arms use the registered E85 repairs: original E81/E82/E83 Pantry runs received zero semantic advantage because of an outcome-key defect and remain excluded. The historical Falcon factorial display had not applied that repair; no value from its superseded Pantry column is reused here. The matched four-arm studies compare its effect on Dr.GRPO, its effect on Re:Dr, and the difference of these effects. This restores the full factorial detail beyond the main semantic-only comparator. We reconstruct exact terminal four-draw endpoints under the current source-admission policy: identical repeated metrics are deduplicated, unapproved conflicting retries abort, and the previously excluded Falcon Countdown replay seed 59 remains excluded. All three contrasts in that cell therefore use the common four seeds 55–58; other 0.5B/1B cells use five. No old five-seed Falcon Countdown interval is reused. Tables report paired seed means and pointwise Student-*t* 95% intervals, without pooling models or domains.

The Qwen2.5-3B extension has only fixed seed 70 and no semantic-only arm. Its on-replay comparison is descriptive, has no interval and cannot identify a factorial interaction. These are fixed-coefficient experiments, not a sweep over entropy strengths. Their endpoints do not establish that semantic entropy reproduces the mechanism of a bank that revisits verified keys. `results/semantic_current_summary_20260912.json` retains per-seed end-Table 11: Fixed semantic-entropy effects at every trained scale. Without replay is semantic-only minus Dr.GRPO; on replay is semantic-plus-replay minus Re:Dr; interaction subtracts the first effect from the second. Extra modes equal `distinct@8` minus `pass@8`.

Model	Domain	<i>n</i>	Contrast	Δ pass@8	Δ distinct@8	Δ extra modes
Qwen2.5-0.5B	Graph	5	Without replay	0.002 [−0.028, 0.031]	0.004 [−0.034, 0.041]	0.002 [−0.010, 0.014]
	Graph	5	On replay	−0.002 [−0.027, 0.023]	−0.022 [−0.179, 0.135]	−0.020 [−0.161, 0.120]
	Graph	5	Interaction	−0.004 [−0.035, 0.028]	−0.026 [−0.193, 0.141]	−0.022 [−0.161, 0.116]
	Countdown	5	Without replay	0.141 [−0.047, 0.330]	0.184 [0.011, 0.358]	0.043 [0.001, 0.085]
	Countdown	5	On replay	−0.003 [−0.033, 0.028]	0.115 [0.022, 0.209]	0.118 [0.004, 0.232]
	Countdown	5	Interaction	−0.144 [−0.326, 0.037]	−0.069 [−0.233, 0.095]	0.075 [−0.051, 0.201]
	Python	5	Without replay	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
	Python	5	On replay	−0.002 [−0.717, 0.714]	0.016 [−0.747, 0.780]	0.018 [−0.164, 0.200]
	Python	5	Interaction	−0.002 [−0.717, 0.714]	0.016 [−0.747, 0.780]	0.018 [−0.164, 0.200]
	MathIR	5	Without replay	−0.121 [−0.349, 0.106]	−0.121 [−0.347, 0.105]	0.001 [−0.006, 0.008]
	MathIR	5	On replay	0.031 [−0.035, 0.097]	0.035 [−0.044, 0.114]	0.004 [−0.047, 0.055]
	MathIR	5	Interaction	0.153 [−0.018, 0.324]	0.156 [−0.050, 0.362]	0.003 [−0.052, 0.058]
	Pantry	5	Without replay	0.201 [0.119, 0.283]	1.120 [0.888, 1.352]	0.919 [0.713, 1.124]
	Pantry	5	On replay	0.141 [0.044, 0.238]	1.095 [0.368, 1.823]	0.955 [0.321, 1.589]
	Pantry	5	Interaction	−0.061 [−0.161, 0.040]	−0.025 [−0.826, 0.777]	0.036 [−0.682, 0.754]
Falcon3-1B	Graph	5	Without replay	0.036 [−0.192, 0.264]	0.205 [−0.483, 0.893]	0.169 [−0.292, 0.630]
	Graph	5	On replay	0.001 [−0.040, 0.043]	−0.006 [−0.113, 0.101]	−0.007 [−0.091, 0.077]
	Graph	5	Interaction	−0.035 [−0.237, 0.168]	−0.211 [−0.886, 0.464]	−0.176 [−0.649, 0.297]
	Countdown	4	Without replay	0.004 [−0.092, 0.100]	−0.042 [−0.258, 0.173]	−0.047 [−0.168, 0.074]
	Countdown	4	On replay	0.036 [−0.028, 0.101]	0.047 [−0.082, 0.177]	0.011 [−0.064, 0.087]
	Countdown	4	Interaction	0.032 [−0.098, 0.161]	0.090 [−0.188, 0.367]	0.058 [−0.099, 0.216]
	Python	5	Without replay	0.002 [−0.003, 0.008]	0.002 [−0.003, 0.008]	0.000 [0.000, 0.000]
	Python	5	On replay	−0.133 [−0.308, 0.043]	−0.091 [−0.503, 0.321]	0.042 [−0.296, 0.380]
	Python	5	Interaction	−0.135 [−0.310, 0.040]	−0.093 [−0.507, 0.321]	0.042 [−0.296, 0.380]
	MathIR	5	Without replay	−0.002 [−0.063, 0.060]	−0.002 [−0.063, 0.060]	0.000 [0.000, 0.000]
	MathIR	5	On replay	0.086 [0.004, 0.168]	0.108 [0.003, 0.214]	0.022 [−0.009, 0.054]
	MathIR	5	Interaction	0.087 [0.029, 0.146]	0.110 [0.043, 0.176]	0.022 [−0.009, 0.054]
	Pantry	5	Without replay	0.011 [−0.019, 0.041]	0.091 [−0.161, 0.342]	0.080 [−0.142, 0.301]
	Pantry	5	On replay	0.025 [−0.044, 0.094]	0.382 [−0.702, 1.465]	0.356 [−0.670, 1.382]
	Pantry	5	Interaction	0.014 [−0.052, 0.081]	0.291 [−0.646, 1.228]	0.277 [−0.615, 1.168]
Qwen2.5-3B	Graph	1	On replay	−0.016	−0.010	0.006
	Countdown	1	On replay	0.039	0.146	0.107
	Python	1	On replay	−0.590	−1.215	−0.625
	MathIR	1	On replay	0.033	0.010	−0.023
	Pantry	1	On replay	0.035	0.393	0.357

points, source hashes, exclusion evidence and all three metrics. `ops/build_paper_semantic_current_summary.py` reconstructs these contrasts with the same strict endpoint reader used by the current core comparison, including all documented exclusions.

H COLLAPSE AND REPLAY MECHANISM CHECKS

H.1 CONDITIONAL CONCENTRATION FROM SAVED TRAINING OUTPUTS

This retrospective analysis distinguishes the distribution of correct keys from the probability of producing a correct output. If $P = 0$, C is undefined and $\bar{D}_2 = 0$. For a fixed prompt with correct mass $P > 0$, write $q_c = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp]$ and define

$$C(q) = \sum_c q_c^2, \quad \bar{D}_2 = 2P - P^2 C(q). \quad (12)$$

The first quantity is the probability that two independent correct outputs share a key. The second is expected distinct verified keys at $K = 2$, the quantity the notation table writes $\bar{D}_K$, and not the extra-breadth $B = \text{distinct@}K - \text{pass@}K$ reported elsewhere. That identity follows by expanding $\bar{D}_2 = \sum_c [1 - (1 - Pq_c)^2]$. It makes explicit why sampled diversity can change through correctness or conditional concentration. A decrease in C alone does not imply majorization, larger total support, or recovery of any particular mode. In particular, observed collisions cannot establish that an unobserved key has exactly zero probability under the policy.

Connection to the conditional theory. Under the assumptions of Theorem P.4, the time change in its proof gives $dq_c/d\tau = q_c(q_c - C)$. Consequently,

$$\frac{dC}{d\tau} = 2 \left(\sum_c q_c^3 - C^2 \right) = 2 \text{Var}_{c \sim q}(q_c) \geq 0. \quad (13)$$

The variance vanishes exactly when q is uniform on its positive support. Thus concentration is monotone along this ideal categorical flow, with strict increase whenever the positive coordinates are unequal. The identity provides a theoretical basis for the diagnostic; it does not establish monotonicity for the implemented neural optimizer or a corresponding monotonic decrease under finite-bank replay.

Estimator and identifiable population. The pair statistic is the one defined in App. A.1, read here in the collision orientation, $\hat{C} = 1 - \widehat{\text{PCMD}}$, over a prompt’s $R \geq 2$ correct samples: the contrast this analysis reports is about concentration, so the orientation that rises with it is the one to print. Its promptwise unbiasedness, and the limits of that identity under joint selection, are established there. We leave $R < 2$ undefined, rather than assigning a concentration value to insufficient correct observations.

For each paired training seed s , let E_s contain exactly the prompts with a defined estimate in both conditions. The reported seed contrast is $|E_s|^{-1} \sum_{x \in E_s} (\hat{C}_{B,x} - \hat{C}_{A,x})$ when E_s is nonempty. We then weight defined training seeds equally, preserving all registered blocks and their missingness. Before/after contrasts use each method’s own initial checkpoint and step 3,072; replay contrasts compare matched terminal conditions. No other method substitutes for a missing initial evaluation. Five defined seeds receive nominal Student- t intervals; partial blocks are descriptive. These intervals describe the measured seed means and are not simultaneous guarantees or uncertainty over unseen prompts.

The hosted analysis uses the same promptwise collision concept but pools correct pairs. At a common sampling budget, the ratio of expected colliding and correct pair counts is $\sum_x P_x^2 C(q_x) / \sum_x P_x^2$. Its finite-sample ratio is not generally unbiased for that target. The training contrast here weights eligible prompts equally. Changing correctness can change either eligibility or hosted pair weights, so the two aggregate values are not directly interchangeable.

Retrospective protocol and source coverage. The estimator and comparisons were recorded before the initial concentration contrasts, with the sampling-stream amendment also preceding those effects. Two retrospective extensions then followed the paper’s completed source census. The first included thirteen newly available terminal checkpoints and withdrew one conflicted initial checkpoint. The September 12 extension adds all ten remaining Level-2 Pantry terminal checkpoints, reusing 752 unchanged checkpoint identities and changing no estimator or eligibility rule. All prior results, caches, and source snapshots remain preserved. The final collection contains 475 registered runs and 902 available checkpoint evaluations, with 74 Dr.GRPO and 75 MaxRL Level-1 replay pairs and 25 pairs per objective at Level 2 before conditional eligibility. Complete terminal coverage neither supplies missing initial outputs nor makes every conditional contrast identifiable. Exact source hashes and both retrospective extensions’ timing remain bound to the result artifact.

Sampling-stream audit. The saved draws carry fewer distinct sampling streams than their nominal count, and the comparisons below are built so that nothing turns on the difference. Four saved eight-output groups do not provide 32 independently seeded responses. The recorded parent seeds increment by one; the audited vLLM V0 sampler increments each child’s seed by its output index. The resulting nominal stream ranges overlap, giving eleven distinct identifiers. For each prompt and checkpoint we retain the earliest draw/output position for every identifier, independently of correctness or key identity. Repeated positions sometimes disagree, so identical seed integers are not treated as a proof of identical outputs. Archived actor source is available for 150 of the 400 factorial cells and none of the 75 additional GRPO cells; historical dependency identity is not cryptographically established for every run. The stream mapping rests on the audited implementation and recorded neutral request contract, with the historical-runtime assumption stated explicitly.

No reading in this appendix rests on those shared identifiers. Every comparison is computed a second and third time in orientations that compare the lower five identifiers in condition A with the upper six in B, and then reverse the assignment, so the two sides of the contrast draw from disjoint streams. That disjointness is verified prompt by prompt rather than assumed: it holds in every prompt of both orientations for all 627 block–seed records the analysis contains. The orientations agree in direction with the primary reading in eleven of the twelve before/after blocks that carry Section 5.1, and in all six replay-minus-control blocks. Each orientation retains its own eligible prompts and seed counts.

What remains an assumption is narrower than the overlap, and is worth stating exactly because the two are easy to conflate. Disjoint streams remove the shared-identifier concern; they do not by themselves establish that draws are independent, and shared identifiers also recur across prompts, so neither their separation across conditions nor these selected-population means establishes independent prompt observations. Independent fixed-law sampling is an explicit assumption of the estimator identity, not a conclusion of the source audit.

Original `mean@8`, `pass@8`, and `distinct@8` remain averages over the four intact groups, reconstructed from all saved verified keys. Cross-group overlap affects dependence, not linearity of those marginal averages. The complete result artifact also retains each intact-group collision comparison, a fixed-first-group check, fixed-across-seed prompt intersections, and four-condition change differences. A naive concatenated 32-position collision is retained solely as a reuse diagnostic. These views are not treated as independent replications or substituted for missing cells.

Findings and their limits. In Graph and PantryPlan, all twelve Dr.GRPO/GRPO before/after comparisons across the three scales show increased conditional concentration, with nominal intervals above zero. They are the Graph and Pantry rows of the three Level-1 initial-to-final tables below, two objectives at each scale. Correctness also increases on those same eligible prompts. Graph’s increases range from .098 to .274; PantryPlan’s range from .447 to .886. The two disjoint orientations agree in direction in eleven of these twelve blocks; Falcon Graph with GRPO is the exception. The evidence therefore supports observed training-induced concentration in these tasks, with a visible sampling sensitivity, rather than a universal claim about every RLVR run or exact disappearance of modes.

Re:Dr lowers concentration relative to Dr.GRPO in all six Graph and PantryPlan comparisons: changes range from $-.756$ to $-.273$ in Graph and $-.433$ to $-.300$ in PantryPlan. These are the Level-1 replay-minus-control tables rather than the initial-to-final ones, and the Dr.GRPO row of a replay-minus-control table is Re:Dr minus Dr.GRPO, not Dr.GRPO’s own contrast. Both disjoint orientations agree in direction in all six, and their primary nominal intervals lie below zero. These comparisons also have lower per-sample correctness on their jointly eligible prompts. They establish a change in the conditional correct-key distribution on that observable population; they do not hold correctness fixed or imply that every accuracy measure improves on the same prompts. Full-test `pass@8` and sampled diversity remain the separately reported primary outcomes of the replay experiment.

The other domains constrain the interpretation. Dr.GRPO/GRPO Python initial outputs provide no jointly eligible prompts at this budget, so their longitudinal collision contrast is unidentified. Some Countdown initial comparisons use only one or two eligible prompts per seed. Several MathIR conditions already have observed collision one, leaving little detectable change. MaxRL comparisons, incomplete saved-output cohorts, and the two sampling orientations are reported explicitly in the following tables; we do not infer a universal concentration benefit from raw diversity gains.

In the completed 3B MaxRL replay cohort, Countdown, Python, and PantryPlan show lower concentration with nominal intervals below zero; MathIR’s measured contrast is zero. Graph’s primary contrast is negative, but its two disjoint orientations disagree in direction, as they do for Falcon Graph with MaxRL. At Level 2, both Graph replay contrasts are negative with agreement across orientations. Complete PantryPlan training still leaves sparse conditional populations: Re:Dr has a defined collision contrast for three seeds (2–16 eligible prompts), with mean $+ .495$; Re:Max has four defined seeds (1–19 eligible prompts), with mean $+ .439$. Their disjoint-orientation means are also positive but retain fewer defined seeds. Neither contrast receives a five-seed interval. Thus the full endpoint diversity gains coexist with increased concentration on these small jointly correct subsets; they do not establish uniformly reduced conditional concentration.

The tables below share one reading. Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds, and each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons, and every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.242	[+0.110, +0.373]	17–40	+0.257 (5)	+0.284 (5)
Graph	GRPO	5	+0.221	[+0.097, +0.344]	15–28	+0.268 (5)	+0.287 (5)
Graph	MaxRL	5	+0.145	[−0.033, +0.324]	18–40	+0.115 (5)	+0.174 (5)
Graph	Re:Dr	5	−0.507	[−0.558, −0.456]	45–49	−0.519 (5)	−0.465 (5)
Graph	Re:Max	5	−0.399	[−0.458, −0.339]	40–51	−0.436 (5)	−0.348 (5)
Countdown	Dr.GRPO	5	+1.000	[+1.000, +1.000]	1–2	− (0)	+1.000 (5)
Countdown	GRPO	5	+0.964	[+0.863, +1.065]	1–2	− (0)	+0.920 (5)
Countdown	MaxRL	4	+0.977	—	0–2	− (0)	+1.000 (4)
Countdown	Re:Dr	5	+0.467	[+0.117, +0.816]	1–3	− (0)	+0.713 (4)
Countdown	Re:Max	5	+0.420	[+0.359, +0.480]	1–2	− (0)	+0.493 (5)
Python	Dr.GRPO	0	—	—	0	− (0)	− (0)
Python	GRPO	0	—	—	0	− (0)	− (0)
Python	MaxRL	0	—	—	0	− (0)	− (0)
Python	Re:Dr	0	—	—	0	− (0)	− (0)
Python	Re:Max	0	—	—	0	− (0)	− (0)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	5–8	+0.000 (5)	+0.000 (1)
MathIR	GRPO	5	+0.040	[−0.071, +0.151]	4–10	+0.000 (5)	+0.250 (2)
MathIR	MaxRL	5	+0.022	[−0.039, +0.084]	5–9	+0.000 (5)	+0.067 (5)
MathIR	Re:Dr	5	+0.059	[−0.046, +0.163]	8–11	+0.000 (5)	+0.367 (2)
MathIR	Re:Max	5	+0.063	[−0.009, +0.134]	9–11	+0.000 (5)	+0.250 (5)
Pantry	Dr.GRPO	5	+0.833	[+0.819, +0.847]	54–64	+1.000 (5)	+1.000 (5)
Pantry	GRPO	5	+0.849	[+0.845, +0.853]	64–65	+1.000 (5)	+1.000 (5)
Pantry	MaxRL	5	+0.859	[+0.836, +0.881]	54–65	+1.000 (5)	+1.000 (5)
Pantry	Re:Dr	5	+0.413	[+0.345, +0.481]	78–89	+0.543 (5)	+0.538 (5)
Pantry	Re:Max	5	+0.446	[+0.324, +0.567]	80–88	+0.593 (5)	+0.550 (5)

Table 12: Initial to final: Qwen2.5-0.5B, Level 1.

H.2 DIVERSITY BEFORE AND AFTER RLVR-ONLY TRAINING

The problem precheck evaluates unmodified Dr.GRPO and GRPO at passes 0 and 8 on all three models and five domains: 150 complete runs, each with four fixed $K = 8$ draws. The five-domain macro-average of extra verified modes falls by 98.2%–99.4% at Qwen2.5-0.5B and 41.5%–71.6% at larger scales. Correctness falls in 10 of the 30 domain–method–scale comparisons, while macro raw distinct@8 ends above its initial value at Qwen2.5-3B. This distinguishes loss of multiple correct outcomes from the broader question of whether any correct outcome is sampled. These are measured changes in the registered runs: they do not establish zero probability for the lost keys, nor a universal decrease in raw diversity across domains.

The same decomposition on the success-conditional axis gives close to the same numbers, which is the check that matters here: extra verified modes could in principle have fallen only because correctness fell. Macro PCMD declines by 98.5% and 99.8% at Qwen2.5-0.5B under GRPO and Dr.GRPO, and by 50.8%–65.1% at the larger scales, against the 98.2%–99.4% and 41.5%–71.6% measured on extra modes. Terminal macro PCMD reaches .000 for Dr.GRPO at Qwen2.5-0.5B. The collapse is therefore a property of how verified probability is allocated, not an artifact of how much of it there is. Falcon3-1B Countdown is the one arm where PCMD rises under Dr.GRPO, from a near-degenerate .033; it does not lift the macro average above zero decline.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.233	[+0.124, +0.343]	41–54	+0.059 (5)	+0.428 (5)
Graph	GRPO	5	+0.098	[+0.026, +0.170]	48–56	-0.186 (5)	+0.273 (5)
Graph	MaxRL	3	-0.113	–	48–52	-0.083 (3)	+0.039 (3)
Graph	Re:Dr	5	-0.007	[-0.065, +0.050]	37–45	+0.048 (5)	+0.045 (5)
Graph	Re:Max	2	-0.062	–	51–52	-0.300 (2)	-0.015 (2)
Countdown	Dr.GRPO	5	+0.022	[-0.038, +0.082]	6–8	+0.000 (5)	– (0)
Countdown	GRPO	5	+0.180	[+0.128, +0.231]	6–11	+0.126 (5)	– (0)
Countdown	MaxRL	2	+0.182	–	11	+0.143 (2)	– (0)
Countdown	Re:Dr	4	+0.035	–	10–11	+0.158 (4)	– (0)
Countdown	Re:Max	1	+0.135	–	11	+0.143 (1)	– (0)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	0	–	–	0	– (0)	– (0)
Python	Re:Dr	0	–	–	0	– (0)	– (0)
Python	Re:Max	0	–	–	–	– (0)	– (0)
MathIR	Dr.GRPO	5	+0.105	[+0.088, +0.122]	6	– (0)	+0.098 (5)
MathIR	GRPO	5	+0.054	[-0.099, +0.206]	3–7	– (0)	+0.233 (5)
MathIR	MaxRL	1	+0.278	–	6	+1.000 (1)	+0.133 (1)
MathIR	Re:Dr	5	+0.086	[+0.026, +0.146]	6–7	– (0)	+0.111 (5)
MathIR	Re:Max	2	+0.167	–	5–6	+1.000 (2)	+0.000 (2)
Pantry	Dr.GRPO	5	+0.886	[+0.886, +0.886]	76	+1.000 (5)	+1.000 (5)
Pantry	GRPO	5	+0.863	[+0.798, +0.928]	76	+0.945 (5)	+1.000 (5)
Pantry	MaxRL	1	+0.886	–	76	+1.000 (1)	+1.000 (1)
Pantry	Re:Dr	4	+0.437	–	75–83	+0.451 (4)	+0.618 (4)
Pantry	Re:Max	2	+0.746	–	76–78	+0.740 (2)	+0.978 (2)

Table 13: Initial to final: Falcon3-1B, Level 1.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.274	[+0.232, +0.316]	44–47	+0.030 (5)	+0.339 (5)
Graph	GRPO	5	+0.274	[+0.207, +0.341]	46–49	+0.046 (5)	+0.364 (5)
Graph	MaxRL	5	+0.072	[-0.002, +0.145]	45–55	-0.208 (5)	+0.215 (5)
Graph	Re:Dr	5	-0.038	[-0.060, -0.016]	48–53	-0.227 (5)	+0.121 (5)
Graph	Re:Max	5	-0.145	[-0.198, -0.093]	46–52	-0.220 (5)	+0.165 (5)
Countdown	Dr.GRPO	5	+0.160	[+0.109, +0.211]	11–12	+0.200 (5)	-0.016 (5)
Countdown	GRPO	5	+0.182	[+0.182, +0.182]	11	+0.200 (5)	+0.000 (5)
Countdown	MaxRL	5	+0.056	[-0.046, +0.159]	9–11	+0.050 (5)	+0.000 (5)
Countdown	Re:Dr	5	+0.020	[-0.060, +0.099]	11–15	+0.169 (5)	-0.071 (5)
Countdown	Re:Max	5	-0.100	[-0.145, -0.055]	10–12	+0.000 (5)	-0.050 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	5	+0.000	[+0.000, +0.000]	2–9	+0.000 (2)	+0.000 (5)
Python	Re:Dr	0	–	–	0	– (0)	– (0)
Python	Re:Max	5	-0.023	[-0.050, +0.003]	9	+0.000 (5)	-0.040 (5)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	11–14	+0.000 (5)	+0.000 (5)
MathIR	GRPO	5	+0.000	[+0.000, +0.000]	9–11	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	+0.000	[+0.000, +0.000]	7–14	+0.000 (5)	+0.000 (5)
MathIR	Re:Dr	4	-0.012	–	14	+0.000 (4)	+0.000 (4)
MathIR	Re:Max	5	+0.000	[+0.000, +0.000]	12–15	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	+0.447	[+0.266, +0.627]	49–67	+0.478 (5)	+0.245 (5)
Pantry	GRPO	5	+0.469	[+0.411, +0.526]	49–62	+0.514 (5)	+0.271 (5)
Pantry	MaxRL	5	+0.431	[+0.297, +0.565]	59–69	+0.394 (5)	+0.246 (5)
Pantry	Re:Dr	5	+0.184	[+0.081, +0.288]	64–71	+0.087 (5)	+0.037 (5)
Pantry	Re:Max	5	+0.142	[+0.018, +0.266]	68–75	+0.046 (5)	-0.015 (5)

Table 14: Initial to final: Qwen2.5-3B, Level 1.

Initial evaluations are measured within each method. Qwen2.5-3B MathIR and Pantry replay-arm seeds 71–74 carry slightly different initial requests from plain GRPO (.154 versus .162, and .832 versus .828 on P), so those cells have separate initial references.

Graph and PantryPlan contain substantial initial diversity at every scale; the other three domains begin with at most .011 extra modes per prompt. A retained-diversity percentage is therefore reported only when the initial extra-mode count exceeds .05. Raw `distinct@8` ends below its initial value in six of the 15 Dr.GRPO domains. Re:Dr does so in two available domain summaries, both PantryPlan; Falcon Countdown uses four admissible seeds. On the success-conditional axis the contrast is sharper: averaged over the domains that remain measurable at the terminal step, Dr.GRPO ends at P_{CMD} .017, .103, and .086 at 0.5B, 1B, and 3B against Re:Dr’s .233, .201, and .194. These observations concern sampled held-out support, not the survival histories of particular stored modes.

Uniform-action references are meaningful for the enumerable Graph and Pantry representations, and App. A.1 derives them. They are not defined for open-ended programs or derivations. Pantry’s bit-uniform reference exceeds every final model, so this domain is a support-concentration stress test rather than evidence of complete support coverage.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.625	[+0.295, +0.955]	2–10	+0.633 (2)	+0.860 (5)
Graph	MaxRL	5	+0.452	[+0.387, +0.518]	6–10	+0.767 (4)	+0.484 (5)
Graph	Re:Dr	5	-0.204	[-0.282, -0.126]	9–11	+0.033 (5)	-0.372 (5)
Graph	Re:Max	5	-0.128	[-0.262, +0.006]	9–11	+0.333 (5)	-0.267 (5)
Countdown	Dr.GRPO	1	+0.000	–	0–1	– (0)	– (0)
Countdown	MaxRL	1	+0.000	–	0–1	– (0)	– (0)
Countdown	Re:Dr	5	+0.000	[+0.000, +0.000]	1	– (0)	– (0)
Countdown	Re:Max	3	+0.000	–	0–1	– (0)	– (0)
Python	Dr.GRPO	5	+0.302	[+0.302, +0.302]	32	+0.000 (5)	+0.378 (5)
Python	MaxRL	5	+0.302	[+0.302, +0.302]	32	+0.000 (5)	+0.378 (5)
Python	Re:Dr	5	+0.094	[-0.259, +0.448]	32	-0.224 (5)	+0.146 (5)
Python	Re:Max	5	+0.199	[-0.088, +0.485]	32	-0.070 (5)	+0.258 (5)
MathIR	Dr.GRPO	3	+0.000	–	0–5	– (0)	+0.000 (3)
MathIR	MaxRL	4	+0.000	–	0–6	– (0)	+0.000 (4)
MathIR	Re:Dr	5	+0.000	[+0.000, +0.000]	6	– (0)	+0.000 (5)
MathIR	Re:Max	4	+0.000	–	6	– (0)	+0.000 (4)
Pantry	Dr.GRPO	3	-0.578	–	0–3	-0.500 (1)	-1.000 (1)
Pantry	MaxRL	3	-0.238	–	1–5	-1.000 (1)	-1.000 (1)
Pantry	Re:Dr	2	-0.049	–	15–16	-0.075 (2)	-0.050 (2)
Pantry	Re:Max	3	+0.020	–	18–20	+0.092 (3)	-0.174 (3)

Table 15: Initial to final: Qwen2.5-0.5B, Level 2.

Domain	Control	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.756	[-0.831, -0.680]	37–40	-0.853 (5)	-0.807 (5)
Graph	MaxRL	5	-0.587	[-0.759, -0.414]	41–73	-0.625 (5)	-0.605 (5)
Countdown	Dr.GRPO	5	-0.529	[-0.605, -0.452]	42–78	-0.529 (5)	-0.535 (5)
Countdown	MaxRL	5	-0.547	[-0.635, -0.459]	45–84	-0.578 (5)	-0.530 (5)
Python	Dr.GRPO	5	-0.090	[-0.341, +0.160]	22	-0.115 (5)	-0.062 (5)
Python	MaxRL	5	-0.266	[-0.577, +0.044]	22	-0.298 (5)	-0.272 (5)
MathIR	Dr.GRPO	5	-0.008	[-0.017, +0.001]	44–68	-0.006 (5)	-0.008 (5)
MathIR	MaxRL	5	-0.005	[-0.013, +0.003]	32–56	-0.007 (5)	-0.002 (5)
Pantry	Dr.GRPO	5	-0.433	[-0.518, -0.347]	54–66	-0.442 (5)	-0.494 (5)
Pantry	MaxRL	5	-0.417	[-0.547, -0.288]	53–71	-0.363 (5)	-0.508 (5)

Table 16: Replay minus control at the final checkpoint: Qwen2.5-0.5B, Level 1. Each row is the replay arm of the named control minus that control, so a negative value is diversity the replay arm keeps and the control loses.

Domain	Control	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.309	[-0.523, -0.094]	31–64	-0.129 (5)	-0.452 (5)
Graph	MaxRL	5	-0.021	[-0.040, -0.002]	95–100	+0.100 (5)	-0.013 (5)
Countdown	Dr.GRPO	4	-0.168	–	33–42	-0.139 (4)	-0.233 (4)
Countdown	MaxRL	5	-0.063	[-0.079, -0.047]	78–81	-0.062 (5)	-0.063 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	5	-0.093	[-0.350, +0.165]	22–128	-0.039 (5)	-0.090 (5)
MathIR	Dr.GRPO	5	+0.004	[-0.007, +0.015]	9–14	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	-0.001	[-0.010, +0.009]	40–72	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	-0.404	[-0.600, -0.209]	74–81	-0.435 (5)	-0.352 (5)
Pantry	MaxRL	5	-0.129	[-0.223, -0.035]	81	-0.203 (5)	-0.043 (5)

Table 17: Replay minus control at the final checkpoint: Falcon3-1B, Level 1. Each row is the replay arm of the named control minus that control, so a negative value is diversity the replay arm keeps and the control loses.

Domain	Control	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.273	[-0.299, -0.247]	54–71	-0.363 (5)	-0.151 (5)
Graph	MaxRL	5	-0.162	[-0.261, -0.062]	68–83	-0.169 (5)	+0.059 (5)
Countdown	Dr.GRPO	5	-0.137	[-0.183, -0.090]	70–75	-0.153 (5)	-0.139 (5)
Countdown	MaxRL	5	-0.105	[-0.126, -0.084]	56–76	-0.113 (5)	-0.095 (5)
Python	Dr.GRPO	2	-0.190	–	0–22	-0.221 (2)	-0.141 (2)
Python	MaxRL	5	-0.153	[-0.265, -0.041]	22–81	-0.203 (5)	-0.083 (5)
MathIR	Dr.GRPO	5	-0.010	[-0.022, +0.003]	23–34	-0.017 (5)	+0.003 (5)
MathIR	MaxRL	5	+0.000	[+0.000, +0.000]	21–27	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	-0.300	[-0.420, -0.179]	71–83	-0.337 (5)	-0.305 (5)
Pantry	MaxRL	5	-0.292	[-0.319, -0.265]	81–89	-0.355 (5)	-0.237 (5)

Table 18: Replay minus control at the final checkpoint: Qwen2.5-3B, Level 1. Each row is the replay arm of the named control minus that control, so a negative value is diversity the replay arm keeps and the control loses.

Domain	Control	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.577	[-0.736, -0.418]	9–25	-0.599 (5)	-0.544 (5)
Graph	MaxRL	5	-0.405	[-0.472, -0.339]	28–42	-0.262 (5)	-0.416 (5)
Countdown	Dr.GRPO	4	-0.230	–	0–9	-0.281 (4)	-0.292 (4)
Countdown	MaxRL	4	-0.073	–	0–10	+0.000 (3)	-0.118 (4)
Python	Dr.GRPO	5	-0.209	[-0.563, +0.146]	104	-0.219 (5)	-0.234 (5)
Python	MaxRL	5	-0.106	[-0.397, +0.184]	104	-0.076 (5)	-0.121 (5)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	12–122	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	-0.001	[-0.004, +0.002]	29–125	+0.000 (5)	-0.001 (5)
Pantry	Dr.GRPO	3	+0.495	–	0–16	+0.656 (1)	+0.554 (2)
Pantry	MaxRL	4	+0.439	–	0–19	+0.267 (3)	+0.208 (2)

Table 19: **Replay minus control at the final checkpoint: Qwen2.5-0.5B, Level 2.** Each row is the replay arm of the named control minus that control, so a negative value is diversity the replay arm keeps and the control loses.

Table 20: **Absolute references for the registered raw `distinct@8`.** *Uniform* samples the exposed action representation uniformly and is defined only for Graph (27 color strings) and PantryPlan (64 bit masks). Frozen is the matched pass-0 model; the final two columns are pass-8 means. Dashes avoid pretending that an open-ended program or derivation space has a canonical uniform distribution.

Model	Domain	Uniform	Frozen	Dr.GRPO	Re:Dr
Qwen2.5-0.5B	Graph coloring	1.633	.561	.325	2.438
	PantryPlan	2.153	2.426	.522	1.539
Falcon3-1B	Graph coloring	1.633	1.070	1.034	1.717
	PantryPlan	2.153	2.162	.633	1.576
Qwen2.5-3B	Graph coloring	1.633	1.012	1.060	1.637
	PantryPlan	2.153	1.445	.841	1.514

H.3 FRESH-GRADIENT DEGENERACY AND OBSERVED OUTPUT COLLAPSE

A Dr.GRPO group with identical rewards has exactly zero fresh task gradient. Only mixed groups containing both correct and incorrect responses can supply fresh reward information; optimizer momentum can still move a policy when that task gradient is zero. The fractions of mixed groups over all 3,072 Qwen2.5-0.5B updates, averaged over five seeds, are:

Domain	Dr.GRPO	Re:Dr
Graph coloring	.144	.954
Countdown	.140	.456
Python factors	.015	.185
MathIR	.364	.453
PantryPlan	.123	.789

Thus 98.5% of Python control updates carry zero fresh task gradient. Replay supplies a verified-likelihood gradient directly and can also change the policy so that later fresh groups become mixed.

A raw-response audit of all five Qwen2.5-0.5B Python control seeds establishes observed output collapse directly. At pass 0, a draw has 7.99 unique raw completions per prompt on average. By steps 288–480, every seed produces 32 byte-identical temperature-1 samples per prompt, also identical to the logged greedy completion, on all 128 prompts. This persists through step 3,072. Terminal draws contain 1.00 unique completion per prompt in the control and 1.97 under replay. A collapsed completion can be incorrect: this is one observed completion per prompt, not one correct key everywhere. Aggregate endpoint equality or zero draw spread alone would not establish that conclusion, nor does this finite audit rule out low-probability tails.

These checks explain a vulnerability of the registered sparse binary-reward recipe. They do not establish dominance over methods that keep fresh on-policy signal active. Dynamic mixed-group sampling, larger groups, reference KL, token-entropy regularization, and wider-decoding evaluations remain relevant controls beyond the registered factorial.

H.4 WHETHER THE CONTROLS TRAINED, AND WHAT A LEARNING RATE CAN REACH

The controls run at a constant 2×10^{-7} at Qwen2.5-0.5B and Falcon3-1B and a 10^{-7} peak cosine at Qwen2.5-3B (Table 6), with no sweep over that coefficient; several terminate near zero, and the untrained model outscores the terminal control in two domains (Table 1). A reader is entitled

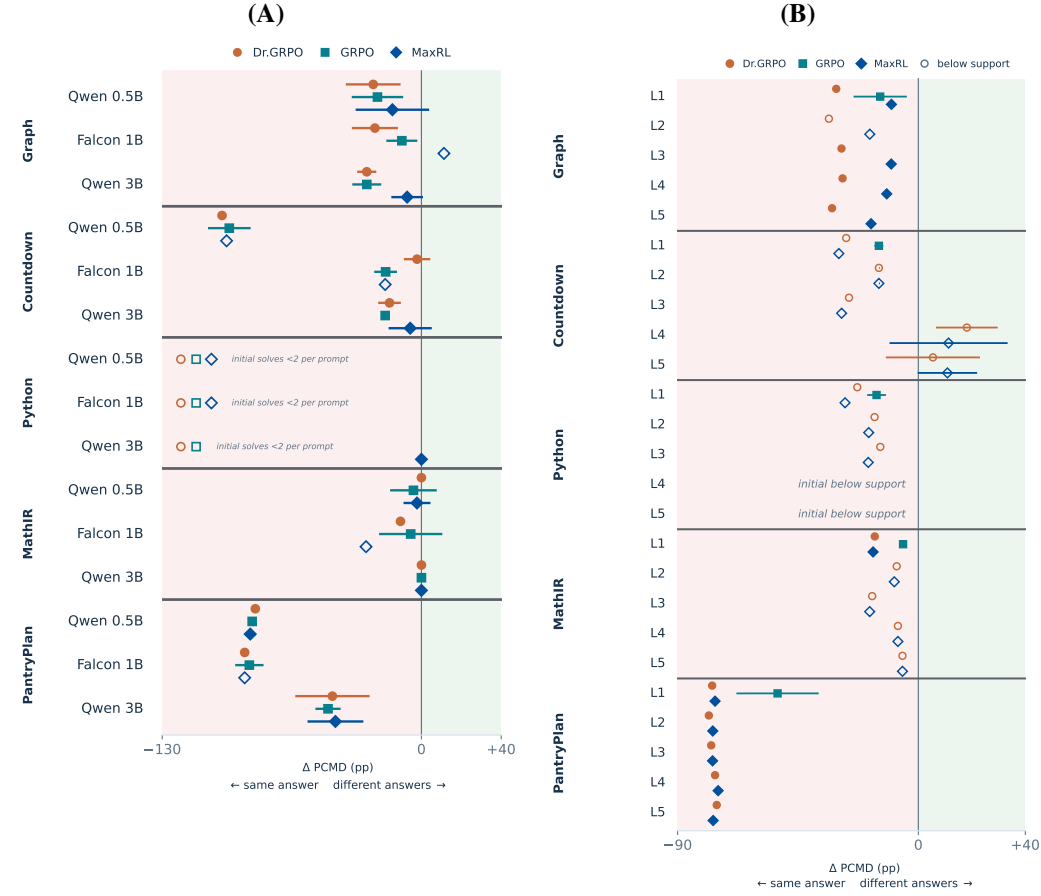


Figure 26: Concentration along both axes. (A) Concentration at every scale. Fig. 8 for all three scales rather than Qwen2.5-3B alone: final minus initial PCMD under Dr.GRPO, GRPO and MaxRL, five domains by three scales. Marks are means over five seeds with nominal 95% seed intervals; open marks carry fewer than five seeds and no interval, or name an objective PCMD cannot score because the initial checkpoint returns two correct responses to no prompt. The scale ordering is the same one Fig. 3 uses. **(B) Training narrows diversity on constructions it never trained on.** Final minus initial PCMD for Qwen2.5-3B policies trained at Level 1 and read on each level’s held-out split, so every row above L1 is transfer rather than training at that level. Marks are means over training seeds against the one untrained checkpoint per cell, with nominal 95% seed intervals; the spread is across seeds, not a paired design. Open marks fall below the support bar, where the estimate stands but its precision does not. Dr.GRPO narrows at every level while MaxRL stays near zero, and Countdown crosses to the broadening side by Levels 4 and 5 — the direction is not a property of training alone but of training and construction together. Rows labeled *initial below support* have no untrained PCMD to subtract, because the initial checkpoint never returns two correct responses to one prompt on that split; the difference is undefined there rather than small. GRPO is read at Level 1 only, so its marks appear on the L1 rows alone and Levels 2–5 carry Dr.GRPO and MaxRL.

to ask whether the accuracy deficit we attribute to concentration is instead a control that never optimised, in which case a gain measured against it would say nothing about how correct mass is divided. What has to hold is that these controls learn, not that they learn as well as another rate would let them, and Table 21 settles that from the runs already reported.

The controls move. Of 15 control cells, 8 end above the frozen model they started from and 6 below it, with a mean of +0.039. The largest movement in each direction is +0.426 (Countdown at Qwen2.5-3B) and -0.380 (Pantry at Qwen2.5-0.5B). A policy whose step size is too small to act does not move four tenths of `pass@8` in either direction. These are policies being optimised, which is all this subsection needs them to be; how much further a different coefficient would carry them is a separate question from whether the runs they are compared against function.

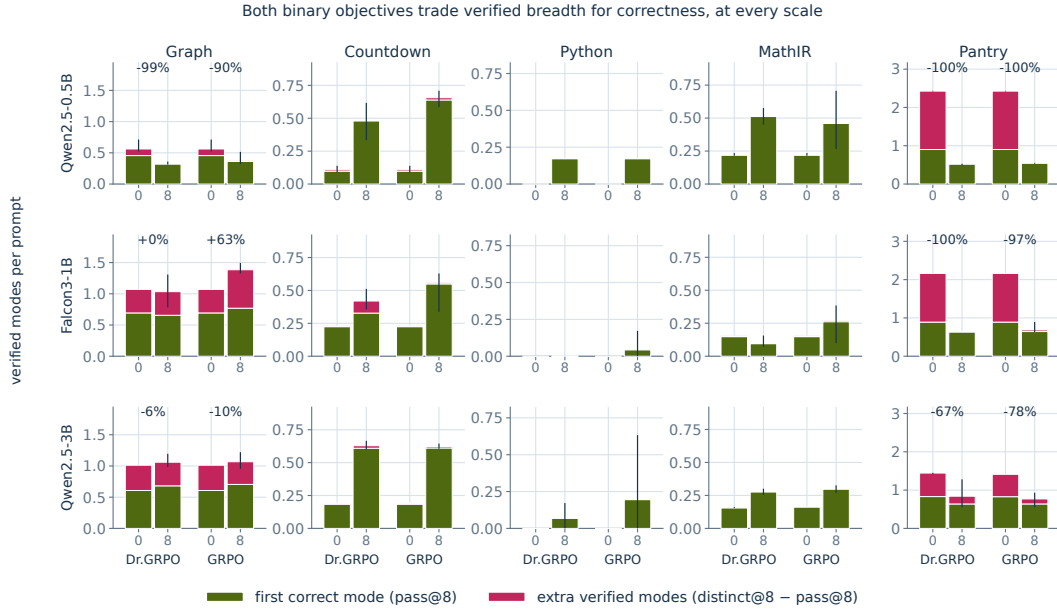


Figure 27: **RLVR-only training reduces average extra verified modes at each tested scale.** Bars split the registered `distinct@8` into the first correct mode and every verified mode beyond it, at training pass 0 and pass 8, for unmodified Dr.GRPO then GRPO in each panel. All 150 evaluations are complete at five seeds and four fixed $K = 8$ draws; whiskers are the seed range and pass 0 is measured per method. Badges give the change in extra verified modes, printed only where pass-0 diversity exceeds .05 of them; the other three domains start at .011 or less.

Table 21: **What each control did with its budget.** `pass@8` of the frozen model against the Dr.GRPO control at step 3,072 with the same update and fresh-rollout budgets, five seeds per cell, with the per-sample accuracy over those same draws, the share of updates whose rollout group carries a task gradient at all, and the paired replay arm for reference. One prompt group is drawn per update, and Dr.GRPO’s group advantage is identically zero when every response in a group earns the same reward, so “mixed” is the fraction of updates in which the learning rate multiplies something other than an exact zero. Computed this way the Qwen2.5-0.5B column reproduces App. H.3 to three decimals.

Model	Domain	control <code>pass@8</code>			control <code>mean@8</code>		mixed groups	Re:Dr <code>pass@8</code>
		frozen	terminal	Δ	frozen	terminal		
Qwen2.5-0.5B	Graph	.456	.323	−.132	.178	.323	.144	.969
	Countdown	.097	.480	+.383	.014	.465	.140	.672
	Python	.000	.172	+.172	.000	.172	.015	.528
	MathIR	.220	.512	+.292	.037	.464	.364	.796
	Pantry	.902	.522	−.380	.344	.522	.123	.729
Falcon3-1B	Graph	.693	.656	−.037	.178	.300	.777	.809
	Countdown	.225	.328	+.103	.037	.268	.400	.636
	Python	.000	.000	+.000	.000	.000	.009	.979
	MathIR	.148	.095	−.053	.023	.068	.158	.736
	Pantry	.891	.633	−.258	.318	.633	.523	.751
Qwen2.5-3B	Graph	.607	.682	+.075	.215	.360	.706	.836
	Countdown	.184	.609	+.426	.049	.569	.281	.746
	Python	.000	.069	+.069	.000	.069	.011	.659
	MathIR	.156	.278	+.122	.061	.168	.152	.686
	Pantry	.831	.638	−.193	.267	.558	.482	.744

Where a control ends near zero, a coefficient has almost nothing to multiply. The lowest terminal cells are the ones that began there: the frozen model scores `pass@8` of zero on Python at all three scales, 3 cells in all. A group whose responses all earn the same reward contributes an advantage of exactly zero, and in Python at Falcon3-1B that describes 99.1 percent of updates, so scaling the step size scales an exact zero. The remaining 0.009 of updates carries the whole

of the fresh signal available in that cell, and it is not nothing: the control does meet verified successes there, and ends at zero on held-out prompts regardless. The question such a cell poses is therefore not how large a step to take on a rare signal but whether what it finds survives, which is the discovery-and-retention split of App. P.6 rather than a tuning failure.

Where a control falls below its start, it is demonstrably updating. The declines are the cells the objection reads as failure to learn, and the raw-response audit above rules that reading out directly. At pass 0 a Qwen2.5-0.5B Python control draw carries 7.99 unique raw completions per prompt; by steps 288–480 every seed emits 32 byte-identical temperature-1 samples per prompt on all 128 prompts, matching its own greedy trace, and this persists to step 3,072. A policy that is barely being updated stays near its initialisation and keeps its sampling entropy. It does not converge onto a single string within a tenth of the budget. The decline is movement, and its direction is the one this paper reports.

Per-sample accuracy, beside the union it is usually contrasted with. `mean@8` is the per-sample accuracy over the same draws as `pass@8`, and the table carries it for every control so the familiar RLVR signature can be checked rather than assumed. It rises in 15 of 15 cells, which is every one of them, and in 7 it rises while `pass@8` does not — the pattern under which RLVR is normally reported to work, per-sample accuracy improving as the k -sample union fails to (Yue et al., 2025). The plainest case is Pantry at Falcon3-1B, where per-sample accuracy climbs from 0.318 to 0.633 while `pass@8` falls from 0.891 to 0.633. A run that had failed to optimise would not move the first number.

The distance between the two columns is a second read on the same thing. A policy that still samples several ways scores below its own `pass@8`, because some of its eight draws are wrong; a policy that emits one completion scores level with it. 12 of the frozen cells carry a gap of at least .02, and 7 of the terminal controls have closed it to less than that, several to exactly zero. That is the byte-identical collapse of the audit above read off an aggregate rather than off raw completions, and it is what a narrowed `pass@8` beside a risen `mean@8` means here.

The coefficient is common to both sides. Every arm shares its control’s learning rate, schedule, initialisation, prompt stream, optimiser, rollout count and budget, and differs in one knob (Table 6). So the coefficient sets the operating point both arms are read at; it is not free to set the sign of the difference between them. This is why the comparison survives a choice of rate that a reader would not have made, and it is also why it cannot answer what a different rate would have shown.

The largest gap is retention, not balance. The widest single margin, Re:Dr’s `pass@8` of .979 against a control at .000 on Python at Falcon3-1B, is not a claim about how correct mass is divided, and we do not offer it as one. Both arms start from the same frozen policy at zero and draw rollouts under the same recipe. The control’s own rollouts carry a verified success in 0.009 of its updates, and it still finishes at zero; what separates the arms is what becomes of those rare successes, which is retention of discovery and belongs to the boundary a memory addresses rather than to mode balance. The balance results of Section 5.2 rest on PCMD, which is conditional on success and is not defined at all where a policy has none.

What this does not establish. The audit is not a sweep and says nothing about what a larger coefficient reaches. What it does rule out is the reading that a different coefficient would leave the diversity result standing while removing the accuracy one, since both are read from the same checkpoints of the same runs.

H.5 FIXED-BANK EXEMPLAR SCORE TRAJECTORIES

The fixed-bank study tracks Qwen2.5-0.5B Re:Dr on Graph for seeds 43–47. Before step 384’s fresh group, bank membership and fresh counts are frozen; admission then stops while uniform replay continues through 3,072 updates under the same training recipe. Every frozen exemplar is identified by prompt and canonical-key fingerprints, without selection by its score trajectory. Each scheduled replay visit records mean token log probability s and sequence log probability ℓ . Changes compare an identity’s first and final post-freeze visits; observation steps can differ across identities.

All 3,406 frozen identities across 1,575 seed–prompt pairs have finite, aligned scores at every scheduled visit: 8–9 visits per identity and 29,041 observations in total. The pooled me-

dian change is $+0.038$ nat/token (95% interval $[+.025, +.048]$), but the 10th percentile is $-.256$ and 2.61% of identities lose more than $.5$ nat/token ($[1.54, 3.77]\%$). The worst intermediate change is -1.328 nat/token. On the sequence scale, the median is $+1.154$ nat/sequence ($[+.104, +.198]$), while 21.99% lose more than $.5$ nat/sequence. Lengths range from 4 to 100 tokens, so the two threshold percentages have different meanings.

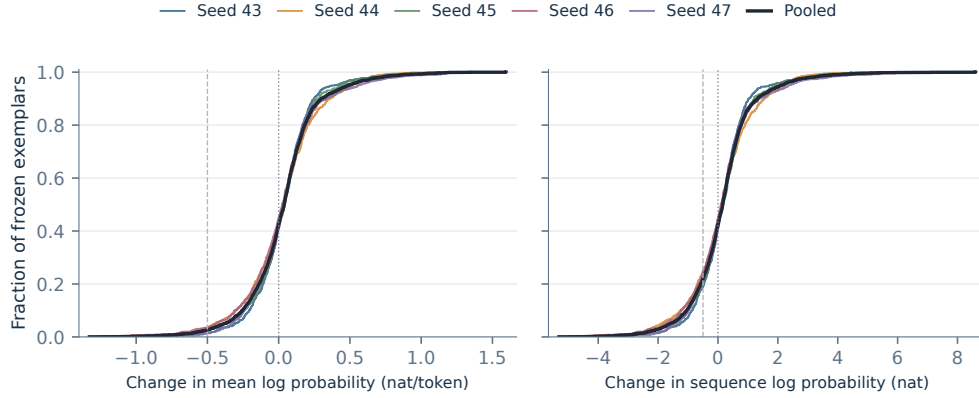


Figure 28: **Fixed-exemplar scores improve centrally but retain a declining tail.** Empirical cumulative distributions show first-to-final post-freeze score changes for every frozen identity, with one curve per seed and the pooled identity distribution. Mean token and sequence scores are separate panels; positive values indicate increased teacher-forced likelihood. Neither panel measures the exact probability of generating a canonical mode during policy sampling.

Pooled summaries weight identities equally. Descriptive 95% intervals use 10,000 hierarchical bootstrap draws (random seed 121), resampling prompts within each original seed and then five seeds with replacement; repeated seeds reuse their within-draw prompt sample. Linear empirical quantiles and percentile intervals are analysis choices. The artifact `paper/results/e121_fixed_bank_survival.json` retains all seed summaries, identity trajectories, source hashes, and audit diagnostics.

These are teacher-forced score surrogates, not exact probabilities of sampling canonical modes, and visits do not cover every intervening update; the declining tail excludes a claim that every exemplar improves. Without a matched no-replay fixed-bank arm, the study does not identify a causal replay effect, and it does not numerically validate Theorem P.20’s floor. Its contribution is longitudinal measurement of fixed exemplars alongside the held-out evidence.

Part IV. What diversity represents

I MATCHED PROMPT-HINT ABLATION

I.1 REMOVING STRATEGY HINTS FROM UNCHANGED PROBLEMS

We freeze a paired follow-up on Python factors, MathIR, and Pantry at Levels 2 and 3. Within each of these six cells, the 32 smallest SHA-256 hashes of (20260911, level, domain, row index) select problems from the 128-row evaluation split, without consulting outcomes. Both arms generate eight fresh draws per identical problem. The original arm retains the published prompt. The neutral arm removes the Python suggestions to test small divisors with nested conditionals or dispatch on listed values, the MathIR ordered algebraic-isolation suggestion, and the Pantry preference for high-energy/protein, low-sodium ingredients such as seeds or oats. MathIR changes “those operations” to “the operations” to preserve the menu-ID instruction. The user problem, answer specification, executable verifier, and remaining format requirements are unchanged.

We report the paired neutral-minus-original effects on empirical $P_8 = \text{pass}@8$ and $D_8 = \text{distinct}@8$, with $B_8 = D_8 - P_8$ as additional verified breadth. All returned answers, including incorrect, empty, and truncated outputs, remain in their eight-draw groups; only verifier-accepted outputs create a verified key. Strict verification is primary; one unchanged, previously frozen formatting normalizer is a sensitivity analysis and preserves every strict success and canonical key. Python outputs in both arms undergo the same serial, warmed-verifier recheck, with original grades and any corrections retained in separate receipts.

Intervals use 20,000 whole-prompt bootstrap resamples, paired across arms and stratified by domain and level (seed 20260911). Cells contribute equally to reported macros. These are point-wise exploratory intervals for 32 problems per cell; crossing zero does not establish equivalence or absence of a prompt effect. A zero-width bootstrap interval records zero observed resampled variation, including all-tied P_8 outcomes; it is not precise evidence of population equivalence. Correct-pair collision and the certified-support uniform reference are secondary and retain their correct-pair denominators; the machine-readable report includes joint eligibility counts and undefined cells. Eight sampled draws do not identify total unseen support.

This appendix reports the complete local panel only. The separately registered frontier panel is not included here, and the overall local-plus-frontier experiment is incomplete.

For local models, we evaluate the initial Qwen2.5-0.5B model and fixed Level-2-trained DrGRPO and Re:Dr checkpoints. Python and MathIR use matched training seeds 43–47; Pantry uses seeds 43 and 46. Level 3 is transfer evaluation. Seed means retain the same selected problem set across checkpoints. Five-seed intervals resample whole training seeds and paired prompts; Pantry reports both seed effects and intervals conditional on the two checkpoints. The native local syntax constraints and 192-token budget differ from hosted inference, so effects are interpreted within each model and interface.

The PCMD columns are sparse by construction, and the sparsity is the result. PCMD needs two verified responses to a prompt in both arms, and on this cohort only five of the thirteen checkpoint–domain–level cells reach thirty such prompts. Three of those five are MathIR, where every verified draw returns the same key in both arms, so PCMD is .000 under either wording: the construction admits one canonical answer often enough that breadth cannot move. That leaves the two Re:Dr cells on Python factors as the only places this cohort can separate a change in concentration from a change in accuracy, and there removing the hint moves PCMD from .231 to .173 at Level 2 and from .256 to .323 at Level 3, on 66 and 57 paired prompts, while correctness also falls. The two disagree in sign, so this cohort shows no common direction for concentration under the intervention, and the fall in D_8 beside them cannot be read as one. One checkpoint leaves the Level 3 cell rather than entering it as a smaller number: with the hint removed it returns 1 verified draw in 256, against 256 of 256 under the original wording, so no prompt of its reaches two verified draws in both arms and PCMD is undefined for it. That cell therefore averages four checkpoints where P_8 and D_8 average five. App. K reads the same six cells at 64 draws per prompt, the budget at which this checkpoint becomes estimable again. Read the dashes as absent measurements rather than as null effects.

Table 22: Local Qwen2.5-0.5B checkpoints under strict verification. Each arrow gives original $\rightarrow$ neutral. P_8 is empirical `pass@8`, D_8 is verified `distinct@8`, and $B_8 = D_8 - P_8$. Effects are neutral minus original; brackets are pointwise 95% intervals. Local rows average the displayed number of training seeds. Five-seed intervals include seed and paired-prompt resampling; two-seed Pantry intervals condition on the two fixed checkpoints. PCMD is the success-conditional axis: the mean probability that two verified draws on a prompt differ, over the prompts that return two verified draws under both wordings, with Δ PCMD the neutral-minus-original contrast on that same paired population. Unlike P_8 and D_8 , it cannot fall merely because the model succeeds less often, since a prompt it stops answering twice leaves the estimate instead of entering it as a smaller number. A dash is a cell where fewer than thirty prompts clear that bar, which 5 of the 18 cells here do: the sparsity is itself the finding, and the PCMD columns are read as a statement about how rarely this cohort can carry the measurement rather than as a survey of it.

Model	Cell	P_8 : O $\rightarrow$ N	D_8 : O $\rightarrow$ N	ΔP_8 [95%]	ΔD_8 [95%]	ΔB_8	PCMD: O $\rightarrow$ N	Δ PCMD
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.656 $\rightarrow$ 0.125	0.69 $\rightarrow$ 0.12	-0.531	[-0.719, -0.344]	-0.56 [-0.75, -0.38]	-0.03	-
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.250 $\rightarrow$ 0.281	0.25 $\rightarrow$ 0.28	+0.031	[+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00	-
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.156 $\rightarrow$ 0.188	0.19 $\rightarrow$ 0.22	+0.031	[+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00	-
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.719 $\rightarrow$ 0.031	0.81 $\rightarrow$ 0.03	-0.688	[-0.844, -0.531]	-0.78 [-0.97, -0.59]	-0.09	-
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.219 $\rightarrow$ 0.219	0.22 $\rightarrow$ 0.22	+0.000	[-0.125, +0.125]	+0.00 [-0.12, +0.12]	+0.00	-
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.281 $\rightarrow$ 0.312	0.66 $\rightarrow$ 0.69	+0.031	[+0.000, +0.094]	+0.03 [-0.06, +0.12]	+0.00	-
DrGRPO ($s = 5$)	MathIR L2	0.531 $\rightarrow$ 0.506	0.53 $\rightarrow$ 0.51	-0.025	[-0.100, +0.031]	-0.03 [-0.10, +0.03]	+0.00	.000 $\rightarrow$.000
DrGRPO ($s = 5$)	MathIR L3	0.194 $\rightarrow$ 0.156	0.19 $\rightarrow$ 0.16	-0.037	[-0.113, +0.019]	-0.04 [-0.11, +0.02]	+0.00	-
DrGRPO ($s = 2$)	Pantry L2	0.219 $\rightarrow$ 0.203	0.28 $\rightarrow$ 0.25	-0.016	[-0.156, +0.125]	-0.03 [-0.19, +0.12]	-0.02	-
DrGRPO ($s = 2$)	Pantry L3	0.172 $\rightarrow$ 0.172	0.27 $\rightarrow$ 0.25	+0.000	[-0.078, +0.078]	-0.02 [-0.16, +0.11]	-0.02	-
DrGRPO ($s = 5$)	Python factors L2	0.812 $\rightarrow$ 0.000	0.81 $\rightarrow$ 0.00	-0.812	[-0.938, -0.656]	-0.81 [-0.94, -0.66]	+0.00	-
DrGRPO ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.000	1.00 $\rightarrow$ 0.00	-1.000	[-1.000, -1.000]	-1.00 [-1.00, -1.00]	+0.00	-
Re:Dr ($s = 5$)	MathIR L2	0.994 $\rightarrow$ 0.975	0.99 $\rightarrow$ 0.97	-0.019	[-0.075, +0.000]	-0.02 [-0.07, +0.00]	+0.00	.000 $\rightarrow$.000
Re:Dr ($s = 5$)	MathIR L3	0.312 $\rightarrow$ 0.275	0.31 $\rightarrow$ 0.28	-0.037	[-0.138, +0.044]	-0.04 [-0.14, +0.04]	+0.00	.000 $\rightarrow$.000
Re:Dr ($s = 2$)	Pantry L2	0.516 $\rightarrow$ 0.500	0.75 $\rightarrow$ 0.73	-0.016	[-0.109, +0.078]	-0.02 [-0.20, +0.17]	+0.00	-
Re:Dr ($s = 2$)	Pantry L3	0.234 $\rightarrow$ 0.281	0.39 $\rightarrow$ 0.50	+0.047	[+0.000, +0.109]	+0.11 [+0.05, +0.19]	+0.06	-
Re:Dr ($s = 5$)	Python factors L2	0.887 $\rightarrow$ 0.556	1.24 $\rightarrow$ 0.68	-0.331	[-0.525, -0.150]	-0.57 [-0.85, -0.32]	-0.24	.231 $\rightarrow$.173
Re:Dr ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.494	1.42 $\rightarrow$ 0.66	-0.506	[-0.825, -0.194]	-0.76 [-1.07, -0.38]	-0.26	.256 $\rightarrow$.323

The frozen formatting normalizer is a registered secondary grading for these cells. It rescued no additional response for any local checkpoint here, so every strict and normalized figure in the table above is identical and the secondary table is not repeated.

Two-seed Pantry sensitivity. Two checkpoints cannot support a stable estimate of training-seed variability. The following lists both strict prompt effects; the intervals in the local table condition on these two checkpoints. DrGRPO L2 (seed 43: $\Delta P_8 = +0.000$, $\Delta D_8 = +0.03$; seed 46: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.09$); DrGRPO L3 (seed 43: $\Delta P_8 = +0.031$, $\Delta D_8 = +0.06$; seed 46: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.09$); Re:Dr L2 (seed 43: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.06$; seed 46: $\Delta P_8 = +0.000$, $\Delta D_8 = +0.03$); Re:Dr L3 (seed 43: $\Delta P_8 = +0.031$, $\Delta D_8 = +0.12$; seed 46: $\Delta P_8 = +0.062$, $\Delta D_8 = +0.09$).

Interpreting the intervention. The Python intervention removes an executable nested-conditional example along with divisor-testing and dispatch guidance; it also shortens the prompt. Its paired effect concerns this combined removal and does not isolate a preference cue from example or length effects. A reduction in `distinct@8` accompanied by lower `pass@8` is not by itself evidence of greater concentration among correct outputs: both endpoints fall when the model simply succeeds less often. This is precisely the ambiguity PCMD removes, since it is estimated from the verified draws alone, and it is the reading to consult when the two registered endpoints move together; the two rightmost columns of Table 22 report it for every cell that can carry it. Here the initial model is the single fixed Qwen2.5-0.5B-Instruct checkpoint before the reported RL training; it is already instruction-tuned and has no fine-tuning-seed replication in this ablation.

Observed prompt dependence. Python shows a large loss of correctness after guidance removal. Across the five DrGRPO seeds, mean P_8 falls from .8125 to zero at Level 2 and from 1.0 to zero at Level 3; D_8 falls identically and $B_8 = 0$ in both arms. With no correct neutral DrGRPO outputs, its conditional-correct concentration is undefined. Re:Dr retains neutral $P_8 = .55625$ and $.49375$ and $D_8 = .675$ and $.65625$ at Levels 2 and 3, respectively, while losing additional diversity: $\Delta B_8 = -.2375$ and $-.25625$. The initial checkpoint also loses Python correctness (Table 22). These observations identify substantial dependence on the combined Python wording intervention; a raw mode-count decline cannot be interpreted solely as a redistribution among correct answers.

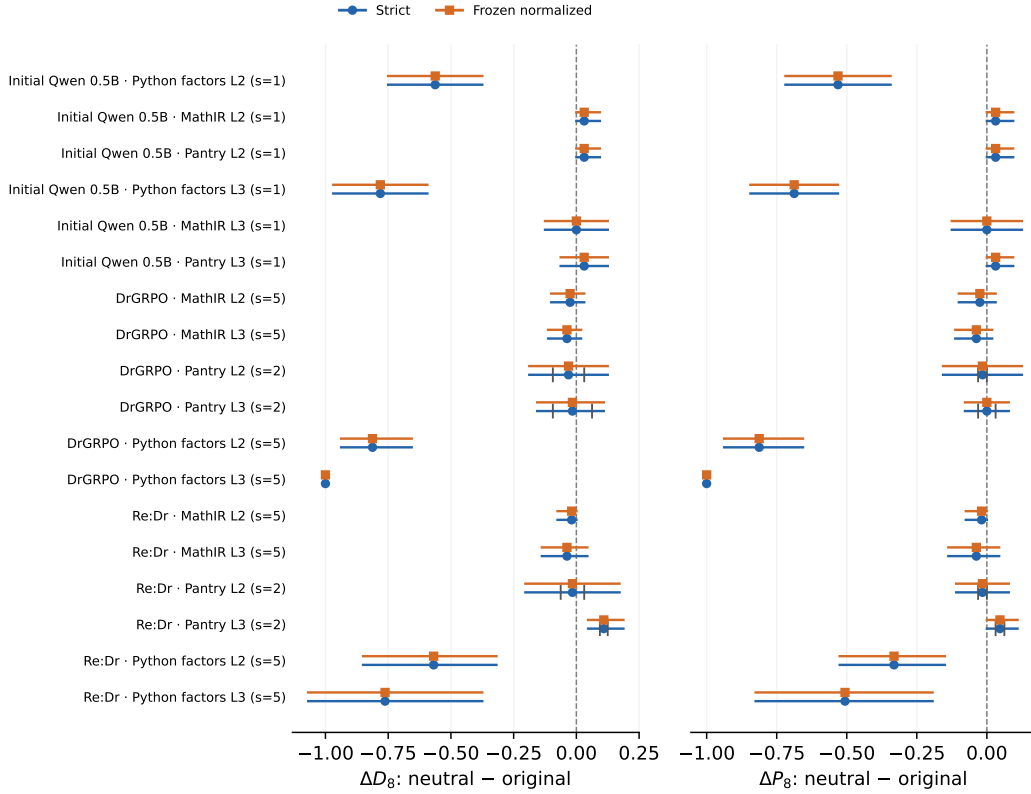


Figure 29: Prompt-hint removal effects for local checkpoints, preserving every registered domain-level cell. Blue circles use strict verification; orange squares use the same frozen formatting normalizer. Horizontal bars show the intervals described in the tables; zero is the vertical reference. Gray endpoint marks show the two individual Pantry seed effects.

Post-hoc Python failure audit. A descriptive audit of all 5,120 saved DrGRPO Python draws (five seeds, two levels, both wordings) finds that all 2,560 original outputs extract the same lambda, whose body matches the conditional example in the prompt:

```
lambda n: 2 if n % 2 == 0 else 3 if n % 3 == 0 else 5
```

Of the 2,560 neutral draws, 1,816 (70.94%) are invalid Python expressions, 59 violate other restricted-language rules, 376 return non-integers, 276 fail the proper-divisor condition, and 33 lack an extracted candidate after reaching the token limit. Thus most neutral failures involve syntax; truncation accounts for only 1.29%. The unchanged extractor, parser, and executor reproduce every stored grade and successful canonical key. This taxonomy was developed after the paired results were known and does not identify an internal training mechanism. Counts, deterministic paired examples, and source hashes are retained in `results/modebench_prompt_ablation_python_failure_diagnostic_20260911.json`.

Concentration after hint removal. Every trained MathIR checkpoint has $B_8 = 0$ under both wordings at both levels. Within each problem’s eight-draw group, every observed pair of correct outputs shares a canonical key, in every trained checkpoint-level cell: correct-pair collision is 1.0, versus the certified-support uniform reference of .2 — a reference this domain cannot reach. Table 22 carries the same fact on the other axis, as PCMD .000 under both wordings in every reportable MathIR cell: the hint is not what holds those answers together. For example, at Level 2, Replay DrGRPO retains mean $P_8 = .975$ without the order hint, compared with .99375 originally. Across its five checkpoints, the neutral arm has 4,179 colliding pairs out of 4,179 correct pairs, from 154 eligible checkpoint-problem groups; the original arm has 4,382 out of 4,382 pairs from 157 such groups. Both arms evaluate the same 32 problems at each checkpoint. This preserves the observed concentration despite removing the specified order hint; it does not measure all unseen modes or remove the other interface constraints. Pantry effects are mixed. Re:Dr at Level 3 gains .109375 in

D_8 , comprising .046875 in P_8 and .0625 in B_8 , but this comparison has only two training seeds. The separate completed 64-draw hosted follow-up tests both wordings on sixteen problems per cell in App. K; its population and draw budget remain distinct from this local eight-draw experiment.

J THE DECODING GRID BEHIND THE OBJECTION

Does a different decoding setting reopen a collapsed policy? The cheapest remedy would be to turn the temperature up. We put every frozen terminal checkpoint of the cohort the rest of the paper reports back through the ordinary evaluation path at six temperatures from 0.5 to 2.0 — five domains, two arms, five seeds, 300 measured checkpoint cells (Fig. 30A). A RLVR-only policy stays where it collapsed at every one of them. In Graph its PCMD never exceeds .021 across the entire sweep: two correct colourings are almost never two different colourings, at any setting we tried, while its paired Re:Dr arm sits at .562 at ordinary settings. The largest value any temperature buys any control is Graph’s .021, against replay arms between .330 and .562 in the three domains where diversity is available at all. Decoding is not where the missing modes went.

What one cell is. Each cell loads one terminal checkpoint, evaluates it once through the ordinary training evaluation path at a chosen $(T, \text{top-}p, K)$, and exits; no optimizer step, rollout or export occurs, so a cell is a pure function of the checkpoint and its decoding settings. Every non-decoding argument — prompt template, split, response length, draw seeds — is inherited from the configuration the source run itself recorded rather than retyped. Pooling and the support bar follow App. A.1 unchanged, and every value is recomputed from the retained per-draw records, whose canonical keys were written by the validator that graded them; nothing is resampled or regraded.

Evaluation hardware, and what it costs. GPU model changes floating-point reduction order and therefore the sampled tokens, so a sweep that mixed hardware would fold it into the trade-off the sweep exists to isolate. This grid is measured on one GPU model throughout, which is the stronger guarantee for reading across the table: no comparison within it carries a hardware difference. It is not the same as measuring each checkpoint on the machine that trained it, and the difference is bounded rather than assumed. Of 200 reproduction checks against the published terminal endpoints, the 72 on checkpoints whose training model matches the evaluation model reproduce pass@8 , mean@8 , distinct@8 and the greedy trace exactly, to the last representable unit; of the 128 measured across models, 125 fall inside the same tolerance, the three exceptions being the greedy trace twice and mean@8 once, which are the statistics most sensitive to reduction order. A separate control re-measured a100-trained checkpoints on the evaluation model at $T = 1$: $\text{mean } |\Delta \text{PCMD}|$ of .003 over twelve paired comparisons, maximum .013, against replay-minus-control gaps near .5. Changing GPU model moves these measurements by thousandths where the effect under study is halves.

A wider budget and a truncated nucleus. Temperature is the version of the objection that gets asked, but it is not the only knob. An earlier Qwen2.5-0.5B cohort — twelve passes to step 4,609, with a replay arm predating the Re:Dr objective contract — was swept over a wider grid on the same benchmark, splits, verifier and evaluation path: the same six temperatures, a quadrupled budget of $K = 32 \times 2$ draws at $T \in \{1.0, 1.3, 1.6\}$, and nucleus truncation to $\text{top-}p .95$ at $T \in \{1.0, 1.6\}$, for 110 measured cells (Fig. 30B). Its control behaves as the current cohort’s does, and neither the larger budget nor the truncated nucleus reopens it either. Because that cohort is not the one the rest of the paper reports, its control range is read here as the wider sweep of the same question rather than as a matched comparison, and its replay marks are a scale reference only.

What the grids do and do not establish. Both measure decoding of frozen terminal policies. Neither shows that a control *trained* differently, or trained at another temperature, would not close the gap; temperature at evaluation is not temperature during rollout. They cover one model scale. One reading to avoid: temperature 2.0 degrades correctness in Countdown and Python, so a PCMD that fails to rise there says the policy has stopped solving the task, not that its diversity was unreachable. The records `paper/results/decoding_objection_e125.json` and `paper/results/decoding_objection_e72.json` retain every cell, its seeds, its defined-prompt count and its source path; `ops/build_paper_decoding_objection_e125.py` and `ops/build_paper_decoding_objection.py` regenerate them and the displayed tables.

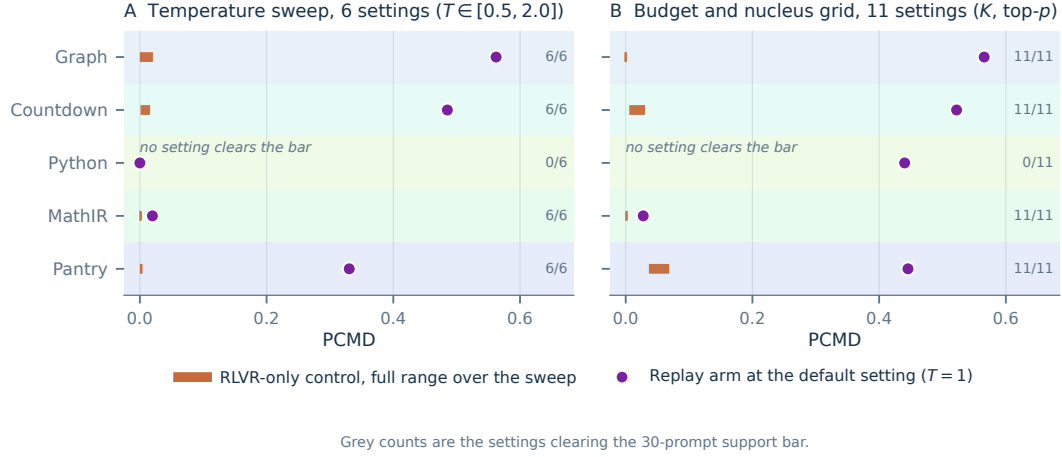


Figure 30: **No decoding setting reopens a collapsed policy.** Both sweeps are Qwen2.5-0.5B terminal checkpoints, five seeds, evaluated through the ordinary evaluation path. Each bar is the Dr.GRPO control’s PCMD range over every setting in that sweep which clears the support bar; each marker is its paired replay arm at ordinary settings ($T = 1$, $K = 8$), for scale. **A** is the cohort reported throughout, at six temperatures from 0.5 to 2.0, 300 measured checkpoint cells. **B** is an earlier twelve-pass cohort swept over eleven settings on the same benchmark, splits, verifier and evaluation path — the same temperatures, a quadrupled budget of $K = 32 \times 2$ draws at $T \in \{1.0, 1.3, 1.6\}$, and nucleus truncation to top- p .95 at $T \in \{1.0, 1.6\}$, for 110 measured cells. That cohort’s replay arm predates the Re:Dr objective contract, so its markers are a scale reference rather than a matched contrast, and the control range is what the wider grid contributes. Python never clears the bar under either control, which is itself the collapse reported in Section 5 rather than a gap in the sweeps. A bar drawn narrower than .004 is padded to stay visible.

K SAMPLING-BUDGET ABLATION

K.1 PROTOCOL AND ESTIMANDS

Large-budget $\text{pass}@k$ evaluation can reveal differences hidden at small sampling budgets (Yue et al., 2025). We therefore freeze a second follow-up after observing the eight-draw prompt control: within every domain-level cell we take ranks 1–16 of its original outcome-independent SHA-256 selection. The six cells retain both prompt wordings, the same mathematical problems, and the same fixed initial and trained checkpoints. Every arm receives 64 fresh draws; none of the earlier eight-draw responses enters this analysis. The initial Qwen2.5-0.5B-Instruct checkpoint has no training-seed replication. DrGRPO and Re:Dr retain five matched seeds in Python and MathIR and two in Pantry. Trained Level-3 evaluation is transfer from Level-2 training.

For a complete 64-draw pool let c be its number of correct outputs and n_j its counts of distinct verified canonical keys. At $k \in \{1, 2, 4, 8, 16, 32, 64\}$, primary curves average all size- k subsets without replacement from the complete pool:

$$P_k = 1 - \frac{\binom{64-c}{k}}{\binom{64}{k}}, \quad D_k = \sum_j \left[1 - \frac{\binom{64-n_j}{k}}{\binom{64}{k}} \right], \quad B_k = D_k - P_k.$$

An infeasible numerator combination is zero. Every incorrect, empty, refused, or truncated response remains in the pool; only verifier-accepted responses contribute keys. These correlated curve points describe the retained pool, not complete unseen support. Ordered prefixes by preassigned draw index, rather than response arrival time, are a separately labeled sensitivity in the source report. $P_{64} - P_8$, $D_{64} - D_8$, and $B_{64} - B_8$ separate first-success gains from additional observed modes.

Strict verification is primary. The unchanged frozen formatter is a sensitivity, preserving every strict success and canonical key. All Python texts undergo the same serial warmed-verifier recheck; original grades and any corrections are retained. Intervals use 20,000 whole-problem bootstrap replicates (seed 20260911), paired across wordings and stratified by domain and level. Five-seed intervals additionally resample whole matched training seeds with shared prompt indices; Pantry intervals

condition on its two checkpoints. Intervals are pointwise and exploratory for 16 problems per cell. Zero-width intervals reflect zero observed resampled variation, not precise evidence of equivalence.

To compare breadth at matched observed correctness, rarefy m correct draws alone:

$$R_m = \sum_j \left[1 - \frac{\binom{c-n_j}{m}}{\binom{c}{m}} \right].$$

The primary paired conditional comparison keeps only prompts with $c \geq m$ under both wordings; eligibility changes with m . Every eligible denominator is retained, and an undefined registered seed is never silently omitted from an equal-seed mean. Two full-64 summaries follow, and they weight prompts differently. Pooled collision C counts equal-key correct pairs across the prompts of one checkpoint before dividing, so a prompt enters in proportion to its $\binom{c}{2}$ and the prompts a model solves most often dominate. Promptwise PCMD averages $1 - \sum_k \binom{n_k}{2} / \binom{c}{2}$ over the prompts with $c \geq 2$, weighting each equally; that is the aggregation App. A.1 requires, and the one every other PCMD in this paper uses. Both then take equal-seed means.

All support counts here are certified lower bounds $L \leq M$, not exhaustive support sizes: Python uses its two externally certified witnesses, MathIR five certified modes, and Pantry its frozen certificate count. Accordingly $1/L$ is an upper reference for uniform collision over full support; $L[1 - (1 - 1/L)^m]$ is a lower reference for uniform expected breadth, not a lower bound on model breadth. Observed distinct counts may exceed L . The source report also averages this uniform breadth over $J \sim \text{Hypergeom}(64, c, k)$ to match unconditional correctness at each k .

Throughout this section $G_P = P_{64} - P_8$, $G_D = D_{64} - D_8$, and $G_B = B_{64} - B_8$; each O/N entry gives original/neutral wording, and ΔG_D is neutral minus original with its pointwise 95% interval. In every figure, colour identifies the checkpoint method or hosted deployment and solid against dashed lines the original against neutral wording; lines and shaded intervals use strict grading and crosses show the frozen-normalization sensitivity. Trained local rows average checkpoint values equally across seeds. Discovery curves put P_k and D_k on separate axes for each level and domain, with pass probability always on a 0–1 scale; correct-draw panels add dotted method-specific uniform lower references over the same eligible prompts, and an undefined mean is a gap. In the full-64 breadth tables U is the certified-support uniform reference under the matching weighting, E the eligible checkpoint–problem groups per wording and Q the correct-pair counts, both summed descriptively across checkpoints and neither defining the equal-seed mean; ΔPCMD is neutral minus original with its pointwise 95% interval on the jointly eligible prompts, undefined where no prompt is jointly eligible. Table 23: Eight-to-64 discovery gains under strict verification. Each row represents a fixed hosted deployment; its intervals resample paired prompts and contain no variation across independently trained model seeds.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
GPT-5.4	Python factors L2	0.014/0.000	2.017/3.051	2.003/3.051	+1.033 [−0.044, +2.371]
GPT-5.6 Sol	Python factors L2	0.000/0.405	0.000/0.866	0.000/0.462	+0.866 [+0.545, +1.231]
Grok 4.3	Python factors L2	0.000/0.000	0.667/1.013	0.667/1.013	+0.346 [−0.583, +1.128]
GPT-5.4	Python factors L3	0.025/0.000	2.100/2.340	2.076/2.340	+0.240 [−0.379, +0.930]
GPT-5.6 Sol	Python factors L3	0.000/0.586	0.000/0.759	0.000/0.174	+0.759 [+0.581, +0.952]
Grok 4.3	Python factors L3	0.000/0.000	0.424/1.576	0.424/1.576	+1.152 [+0.640, +1.689]
GPT-5.4	MathIR L2	0.043/0.042	0.043/0.909	0.000/0.868	+0.867 [+0.587, +1.111]
GPT-5.6 Sol	MathIR L2	0.027/0.004	0.027/0.686	0.000/0.682	+0.659 [+0.416, +0.888]
Grok 4.3	MathIR L2	0.000/0.001	0.223/0.322	0.223/0.321	+0.099 [−0.226, +0.438]
GPT-5.4	MathIR L3	0.000/0.000	0.055/0.663	0.055/0.663	+0.609 [+0.342, +0.888]
GPT-5.6 Sol	MathIR L3	0.000/0.000	0.000/0.515	0.000/0.515	+0.515 [+0.318, +0.714]
Grok 4.3	MathIR L3	0.000/0.000	0.167/0.805	0.167/0.805	+0.638 [+0.348, +0.935]
GPT-5.4	Pantry L2	0.001/0.015	1.158/2.583	1.156/2.568	+1.426 [+0.423, +2.641]
GPT-5.6 Sol	Pantry L2	0.086/0.101	0.917/1.789	0.831/1.688	+0.872 [+0.245, +1.575]
Grok 4.3	Pantry L2	0.000/0.000	1.022/3.377	1.022/3.377	+2.354 [+1.592, +3.123]
GPT-5.4	Pantry L3	0.000/0.002	1.666/3.219	1.666/3.218	+1.553 [+0.776, +2.367]
GPT-5.6 Sol	Pantry L3	0.032/0.129	1.625/1.842	1.592/1.713	+0.217 [−0.489, +0.921]
Grok 4.3	Pantry L3	0.000/0.000	1.810/4.466	1.810/4.466	+2.656 [+1.698, +3.640]

The frozen formatting normalizer is a registered secondary grading for these cells. It rescued no additional response for any local checkpoint here, so every strict and normalized figure in the table above is identical and the secondary table is not repeated.

Table 24: Eight-to-64 discovery gains under frozen normalization sensitivity. Each row represents a fixed hosted deployment; its intervals resample paired prompts and contain no variation across independently trained model seeds.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
GPT-5.4	Python factors L2	0.000/0.000	2.156/3.216	2.156/3.216	+1.060 [-0.071, +2.386]
GPT-5.6 Sol	Python factors L2	0.008/0.010	0.113/0.794	0.105/0.784	+0.681 [+0.351, +1.066]
Grok 4.3	Python factors L2	0.000/0.000	0.667/1.013	0.667/1.013	+0.346 [-0.583, +1.128]
GPT-5.4	Python factors L3	0.000/0.000	2.431/2.604	2.431/2.604	+0.173 [-0.657, +1.075]
GPT-5.6 Sol	Python factors L3	0.011/0.007	0.120/0.490	0.109/0.483	+0.370 [+0.112, +0.674]
Grok 4.3	Python factors L3	0.000/0.000	0.424/1.576	0.424/1.576	+1.152 [+0.640, +1.689]
GPT-5.4	MathIR L2	0.043/0.042	0.043/0.909	0.000/0.868	+0.867 [+0.587, +1.111]
GPT-5.6 Sol	MathIR L2	0.027/0.004	0.027/0.686	0.000/0.682	+0.659 [+0.416, +0.888]
Grok 4.3	MathIR L2	0.000/0.001	0.223/0.322	0.223/0.321	+0.099 [-0.226, +0.438]
GPT-5.4	MathIR L3	0.000/0.000	0.055/0.663	0.055/0.663	+0.609 [+0.342, +0.888]
GPT-5.6 Sol	MathIR L3	0.000/0.000	0.000/0.515	0.000/0.515	+0.515 [+0.318, +0.714]
Grok 4.3	MathIR L3	0.000/0.000	0.167/0.805	0.167/0.805	+0.638 [+0.348, +0.935]
GPT-5.4	Pantry L2	0.001/0.015	1.158/2.583	1.156/2.568	+1.426 [+0.423, +2.641]
GPT-5.6 Sol	Pantry L2	0.004/0.004	0.954/1.834	0.950/1.830	+0.880 [+0.237, +1.547]
Grok 4.3	Pantry L2	0.000/0.000	1.022/3.377	1.022/3.377	+2.354 [+1.592, +3.123]
GPT-5.4	Pantry L3	0.000/0.002	1.666/3.219	1.666/3.218	+1.553 [+0.776, +2.367]
GPT-5.6 Sol	Pantry L3	0.001/0.020	1.586/2.462	1.585/2.443	+0.877 [+0.272, +1.436]
Grok 4.3	Pantry L3	0.000/0.000	1.810/4.466	1.810/4.466	+2.656 [+1.698, +3.640]

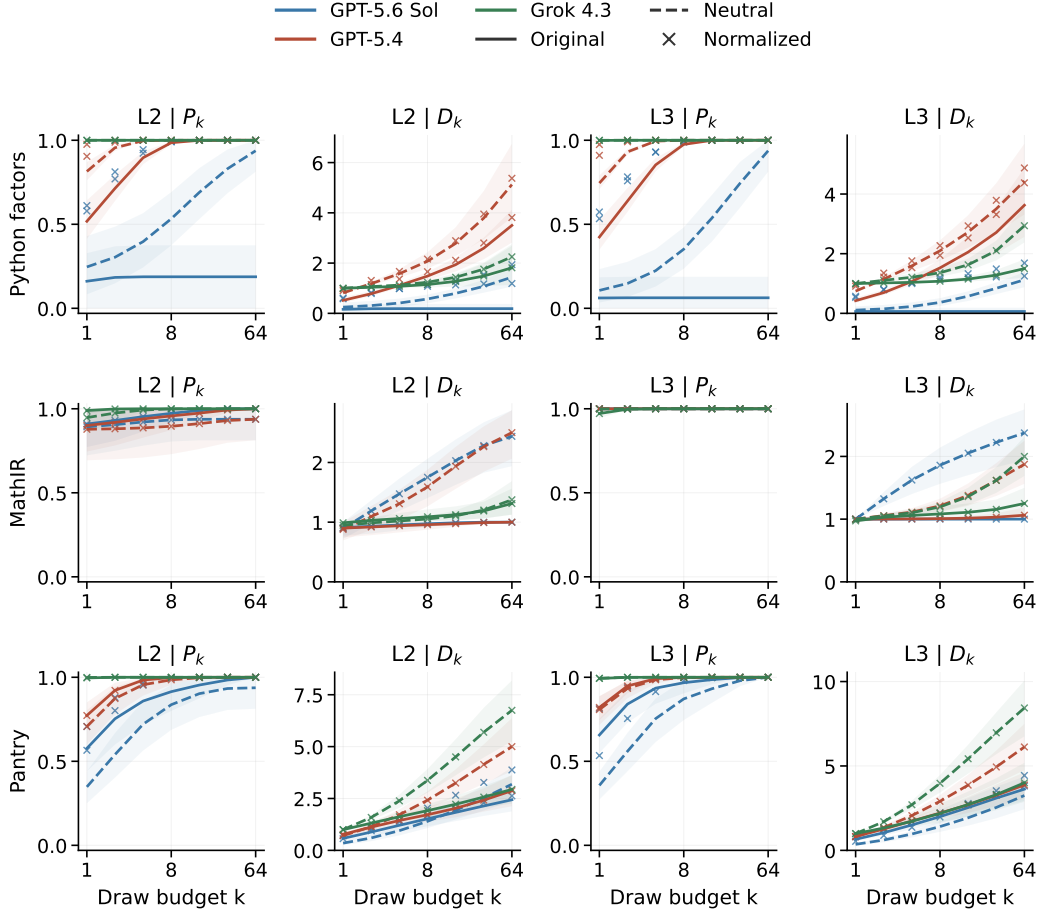


Figure 31: Fresh 64-draw discovery curves. Prefix estimates are separately retained in the report with pointwise intervals for every model and cell.

Table 25: Full-64 strict breadth under both pair weightings. C pools correct pairs within a fixed checkpoint or deployment, so a prompt enters in proportion to its $\binom{c}{2}$; promptwise PCMD averages $1 - \sum_k \binom{n_k}{2} / \binom{c}{2}$ over prompts with $c \geq 2$, weighting each equally, which is the aggregation App. A.1 uses for every other PCMD in this paper. A seed with no eligible pair leaves its all-seed mean undefined, with every individual denominator, the per-budget joint eligibility counts and the separate pooled rates retained in the report.

Model	Cell	C_O	C_N	PCMD _O	PCMD _N	U_O^C	U_O^{PCMD}	E_O/E_N	Q_O/Q_N	$\Delta \text{PCMD [95\%]}$
GPT-5.4	Python factors L2	0.790	0.665	0.257	0.341	0.500	0.500	16/16	10374/21720	+0.084 [-0.006, +0.186]
GPT-5.6 Sol	Python factors L2	1.000	0.974	0.000	0.184	0.500	0.500	3/13	4467/6074	+0.021 [+0.000, +0.032]
Grok 4.3	Python factors L2	0.963	0.938	0.037	0.062	0.500	0.500	16/16	32256/32256	+0.025 [+0.002, +0.048]
GPT-5.4	Python factors L3	0.732	0.620	0.334	0.384	0.500	0.500	16/16	6747/18043	+0.050 [-0.026, +0.128]
GPT-5.6 Sol	Python factors L3	1.000	0.966	0.000	0.130	0.500	0.500	1/10	1953/2097	+0.031 [+0.031, +0.031]
Grok 4.3	Python factors L3	0.981	0.893	0.019	0.107	0.500	0.500	16/16	32193/32256	+0.087 [+0.041, +0.154]
GPT-5.4	MathIR L2	1.000	0.756	0.000	0.228	0.200	0.800	16/15	28440/28227	+0.228 [+0.161, +0.302]
GPT-5.6 Sol	MathIR L2	1.000	0.678	0.000	0.302	0.200	0.800	16/15	28645/28377	+0.302 [+0.200, +0.402]
Grok 4.3	MathIR L2	0.968	0.985	0.033	0.014	0.200	0.800	16/16	31600/29677	-0.020 [-0.076, +0.017]
GPT-5.4	MathIR L3	0.998	0.944	0.002	0.056	0.200	0.800	16/16	32256/32256	+0.054 [+0.027, +0.087]
GPT-5.6 Sol	MathIR L3	1.000	0.676	0.000	0.324	0.200	0.800	16/16	32256/32256	+0.324 [+0.208, +0.432]
Grok 4.3	MathIR L3	0.966	0.949	0.033	0.051	0.200	0.800	16/16	30520/32193	+0.019 [-0.046, +0.064]
GPT-5.4	Pantry L2	0.680	0.499	0.252	0.483	0.069	0.934	16/16	20097/17436	+0.231 [+0.069, +0.395]
GPT-5.6 Sol	Pantry L2	0.674	0.624	0.275	0.474	0.076	0.934	16/15	12941/5001	+0.180 [+0.058, +0.327]
Grok 4.3	Pantry L2	0.684	0.412	0.315	0.589	0.066	0.934	16/16	32005/32193	+0.274 [+0.143, +0.411]
GPT-5.4	Pantry L3	0.546	0.392	0.431	0.610	0.065	0.937	16/16	22332/21975	+0.179 [+0.103, +0.257]
GPT-5.6 Sol	Pantry L3	0.566	0.652	0.422	0.441	0.061	0.937	16/16	15229/4946	+0.019 [-0.136, +0.172]
Grok 4.3	Pantry L3	0.673	0.294	0.330	0.703	0.063	0.937	16/16	31830/31881	+0.373 [+0.224, +0.523]

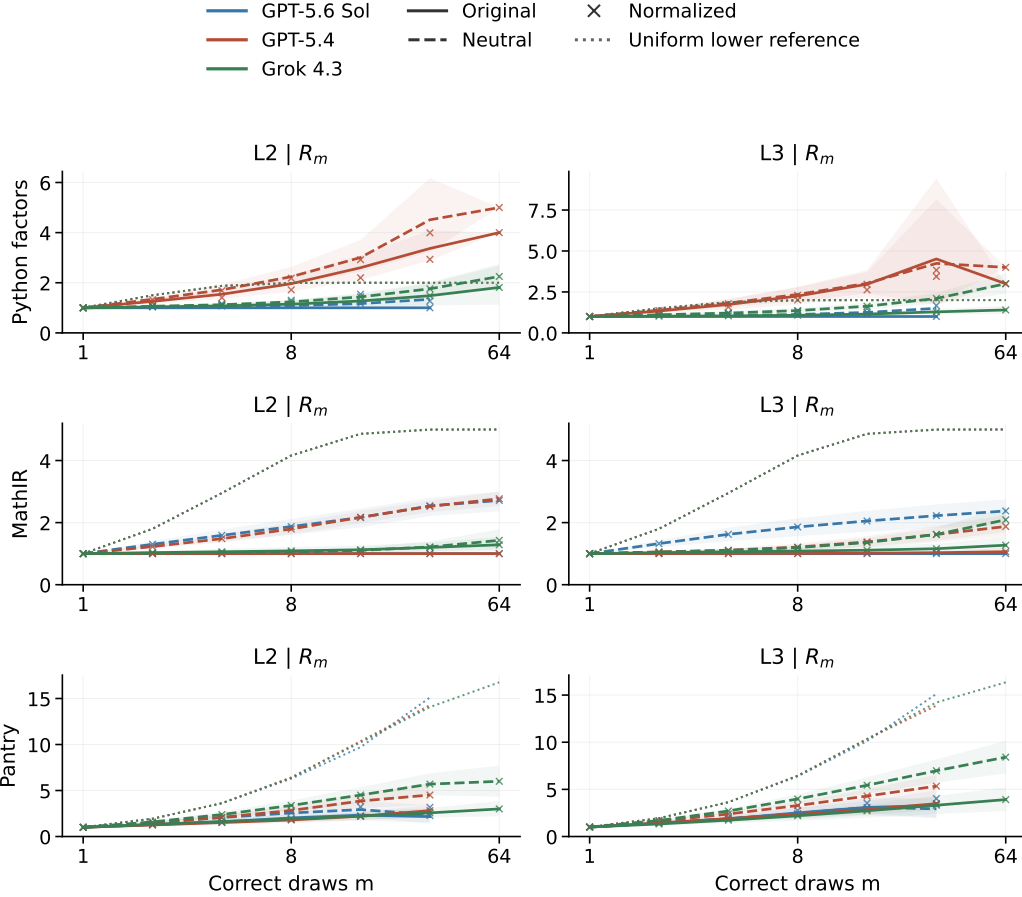


Figure 32: Breadth after exactly m correct draws, on prompts jointly eligible under both wordings. Eligibility varies with m , so curves do not hold the eligible population fixed across the horizontal axis; the strict joint counts for every method, level and correct-draw budget, including groups with zero eligible prompts, are retained in the report rather than printed, and the $m = 2$ counts the breadth contrast rests on are the E column of Table 25.

Table 26: Eight-to-64 discovery gains under strict verification. $s = 1$ denotes one fixed initial checkpoint, not training-seed replication. Five-seed intervals resample seeds and paired prompts; two-seed Pantry intervals condition on the observed checkpoints and retain all paired prompt outcomes.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
DrGRPO ($s = 5$)	Python factors L2	0.000/0.000	0.000/0.000	0.000/0.000	+0.000 [+0.000, +0.000]
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.147/0.105	0.303/0.165	0.156/0.060	-0.138 [-0.217, -0.061]
Re:Dr ($s = 5$)	Python factors L2	0.000/0.231	0.058/0.463	0.058/0.232	+0.405 [+0.185, +0.620]
DrGRPO ($s = 5$)	Python factors L3	0.000/0.033	0.000/0.033	0.000/0.000	+0.033 [+0.000, +0.120]
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.218/0.096	0.593/0.096	0.376/0.000	-0.497 [-0.766, -0.270]
Re:Dr ($s = 5$)	Python factors L3	0.000/0.239	0.153/0.531	0.153/0.292	+0.379 [+0.111, +0.678]
DrGRPO ($s = 5$)	MathIR L2	0.021/0.017	0.021/0.017	0.000/0.000	-0.004 [-0.068, +0.045]
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.344/0.235	0.344/0.235	0.000/0.000	-0.109 [-0.383, +0.164]
Re:Dr ($s = 5$)	MathIR L2	0.010/0.011	0.010/0.011	0.000/0.000	+0.001 [-0.036, +0.050]
DrGRPO ($s = 5$)	MathIR L3	0.023/0.018	0.026/0.018	0.004/0.000	-0.008 [-0.065, +0.038]
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.254/0.246	0.359/0.360	0.105/0.114	+0.001 [-0.123, +0.095]
Re:Dr ($s = 5$)	MathIR L3	0.064/0.056	0.064/0.056	0.000/0.000	-0.008 [-0.099, +0.059]
DrGRPO ($s = 2$)	Pantry L2	0.332/0.323	0.718/0.736	0.386/0.414	+0.018 [-0.422, +0.433]
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.440/0.419	0.857/0.955	0.416/0.537	+0.099 [-0.201, +0.392]
Re:Dr ($s = 2$)	Pantry L2	0.141/0.147	0.415/0.429	0.274/0.282	+0.014 [-0.123, +0.152]
DrGRPO ($s = 2$)	Pantry L3	0.268/0.258	0.949/1.025	0.681/0.766	+0.076 [-0.237, +0.465]
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.259/0.384	0.927/1.078	0.668/0.694	+0.151 [-0.109, +0.399]
Re:Dr ($s = 2$)	Pantry L3	0.204/0.149	0.568/0.461	0.364/0.312	-0.107 [-0.321, +0.098]

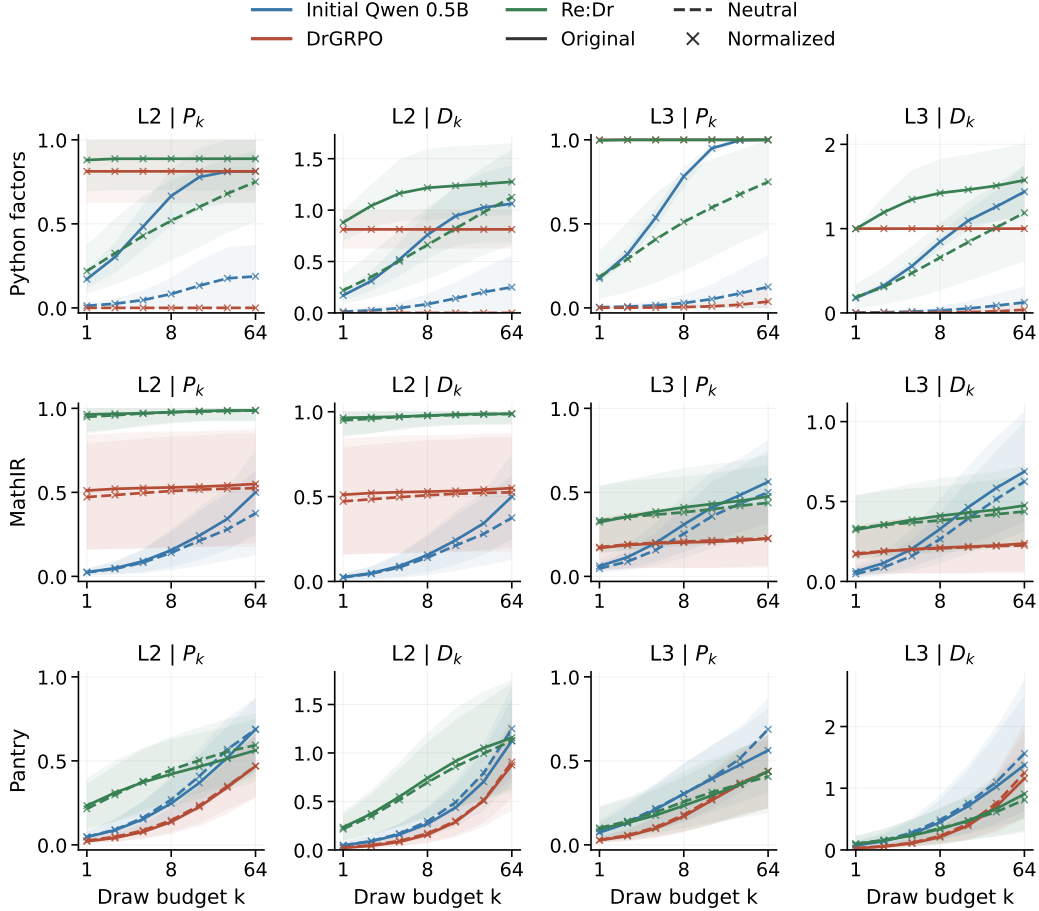


Figure 33: Fresh 64-draw discovery curves. Prefix estimates are separately retained in the report with pointwise intervals for every model and cell.

Table 27: Full-64 strict breadth under both pair weightings. C pools correct pairs within a fixed checkpoint or deployment, so a prompt enters in proportion to its $\binom{c}{2}$; promptwise PCMD averages $1 - \sum_k \binom{n_k}{2} / \binom{c}{2}$ over prompts with $c \geq 2$, weighting each equally, which is the aggregation App. A.1 uses for every other PCMD in this paper. A seed with no eligible pair leaves its all-seed mean undefined, with every individual denominator, the per-budget joint eligibility counts and the separate pooled rates retained in the report.

Model	Cell	C_O	C_N	PCMD _O	PCMD _N	U_O^C	U_O^{PCMD}	E_O/E_N	Q_O/Q_N	$\Delta_{\text{PCMD}} [95\%]$
DrGRPO	Python factors L2	1.000	--	0.000	--	0.500	0.500	65/0	130977/0	--
Initial Qwen 0.5B	Python factors L2	0.791	0.875	0.097	0.167	0.500	0.500	13/3	1270/24	-0.252 [-0.395, +0.019]
Re:Dr	Python factors L2	0.809	0.800	0.191	0.156	0.500	0.500	71/53	140722/18289	-0.037 [-0.139, +0.037]
DrGRPO	Python factors L3	1.000	--	0.000	--	0.500	0.500	80/0	161280/0	--
Initial Qwen 0.5B	Python factors L3	0.875	1.000	0.101	0.000	0.500	0.500	16/1	1010/3	-0.513 [-0.513, -0.513]
Re:Dr	Python factors L3	0.805	0.797	0.195	0.227	0.500	0.500	80/52	160275/12893	+0.030 [-0.023, +0.117]
DrGRPO	MathIR L2	1.000	1.000	0.000	0.000	0.200	0.800	43/42	80766/74119	+0.000 [+0.000, +0.000]
Initial Qwen 0.5B	MathIR L2	1.000	1.000	0.000	0.000	0.200	0.800	3/3	64/64	+0.000 [+0.000, +0.000]
Re:Dr	MathIR L2	1.000	1.000	0.000	0.000	0.200	0.800	79/79	155016/151485	+0.000 [+0.000, +0.000]
DrGRPO	MathIR L3	0.980	1.000	0.038	0.000	0.200	0.800	16/18	24457/25393	-0.038 [-0.202, +0.000]
Initial Qwen 0.5B	MathIR L3	0.883	0.912	0.128	0.162	0.200	0.800	7/6	282/181	+0.013 [-0.014, +0.052]
Re:Dr	MathIR L3	1.000	1.000	0.000	0.000	0.200	0.800	34/34	46891/50301	+0.000 [+0.000, +0.000]
DrGRPO	Pantry L2	0.374	0.276	0.733	0.780	0.047	0.953	9/10	101/193	+0.220 [-0.030, +0.470]
Initial Qwen 0.5B	Pantry L2	0.849	0.787	0.587	0.623	0.052	0.945	7/10	212/178	-0.078 [-0.300, +0.064]
Re:Dr	Pantry L2	0.600	0.613	0.343	0.218	0.051	0.942	16/18	10144/8510	-0.093 [-0.220, +0.007]
DrGRPO	Pantry L3	0.203	0.228	0.808	0.680	0.047	0.950	10/12	134/165	-0.156 [-0.359, +0.081]
Initial Qwen 0.5B	Pantry L3	0.375	0.313	0.307	0.341	0.049	0.945	7/6	531/705	-0.017 [-0.109, +0.091]
Re:Dr	Pantry L3	0.636	0.666	0.337	0.198	0.051	0.941	10/11	4006/3946	-0.112 [-0.319, +0.008]

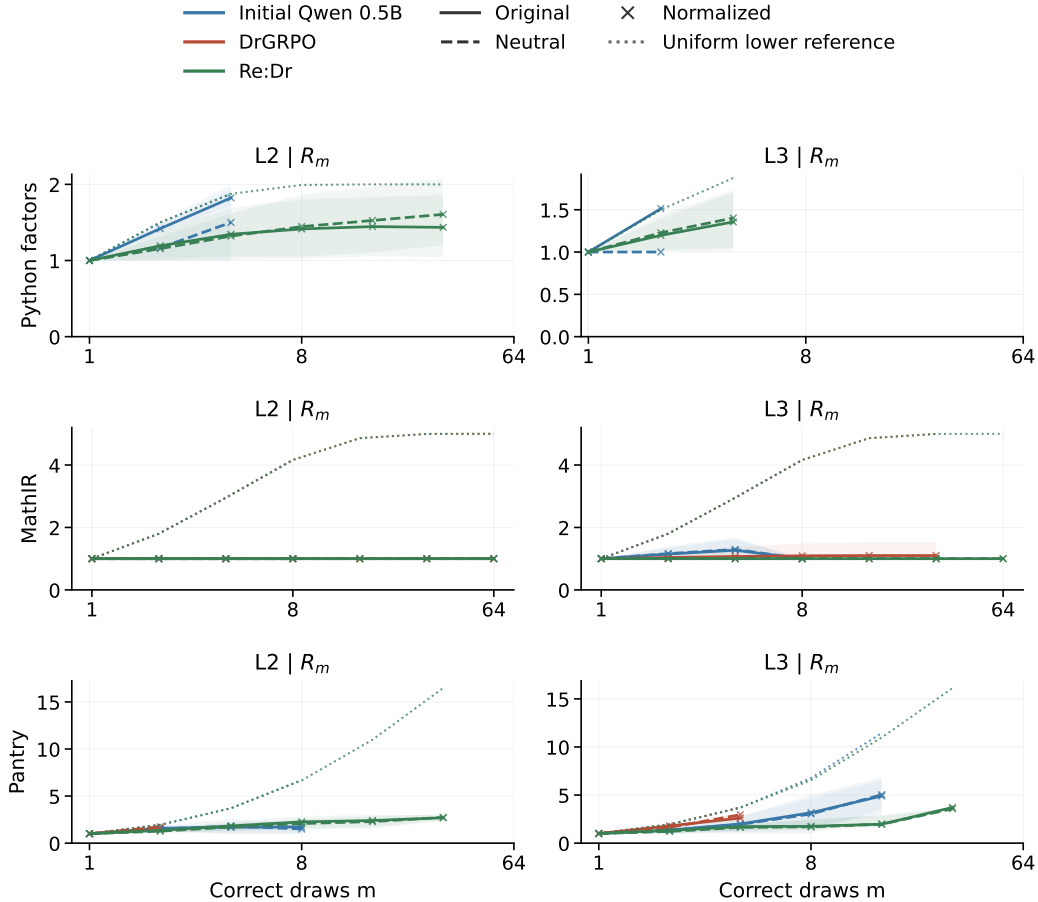


Figure 34: Breadth after exactly m correct draws, on prompts jointly eligible under both wordings. Eligibility varies with m , so curves do not hold the eligible population fixed across the horizontal axis; the strict joint counts for every method, level and correct-draw budget, including groups with zero eligible prompts, are retained in the report rather than printed, and the $m = 2$ counts the breadth contrast rests on are the E column of Table 27.

K.2 HOSTED DISCOVERY: GUIDANCE, FORMATTING, AND LATE MODES

The completed hosted panel contains 36,864 fresh responses from GPT-5.4, GPT-5.6 Sol, and Grok 4.3: sixteen fixed problems in each of six Python/MathIR/Pantry level cells, two wordings, and 64 draws per condition. The full report retains both grading conventions, all observed failures, ordered-prefix sensitivities, and every conditional-eligibility denominator. These fixed deployment configurations form a separate experiment from the reasoning-disabled main table. The historical Level-3 problems also precede the newly admitted neutral-Python dataset; this follow-up evaluates no checkpoint trained on that successor dataset.

MathIR guidance changes GPT’s correct-solution breadth. At Level 2, both GPT deployments produce at most one verified mode per problem under original wording even with 64 draws. At matched budgets of eight correct draws, neutral wording increases mean breadth by .794 ([.623, .990]) for GPT-5.4 and .872 ([.606, 1.123]) for GPT-5.6 Sol. The Level-3 effects are .204 ([.106, .320]) and .860 ([.563, 1.141]). Grok’s corresponding intervals include zero at both levels. The GPT results therefore survive conditioning on observed correctness; Grok’s intervals leave its corresponding effect unresolved. These pointwise exploratory intervals resample the sixteen paired problems, without replication across hosted model training seeds.

Pantry continues discovering modes after first success. Under neutral wording, GPT-5.4 gains 2.583 and 3.219 verified modes between eight and 64 draws at Levels 2 and 3; Grok gains 3.377 and 4.466. Grok’s empirical pass probability is already 1.0 at eight draws in both cells, so these gains consist entirely of additional modes. At Level 3 its distinct count rises from 3.972 to 8.438. At eight correct draws, neutral minus original breadth is 1.781 ([1.226, 2.338]) for Grok and .934 ([.670, 1.184]) for GPT-5.4; GPT-5.6 Sol’s Level-3 contrast is inconclusive. These observed counts measure finite-sample discovery rather than exhaustive solution coverage.

Python requires the formatting sensitivity. GPT-5.6 Sol’s original-wording strict pass probability remains .1875 at Level 2 and .0625 at Level 3 from eight through 64 draws. The frozen formatting normalizer instead gives $P_{64} = 1.0$ under both wordings at both levels, with all four P_8 values above .989. Thus its low strict scores cannot be interpreted as an absence of valid mathematical solutions. The normalized neutral-minus-original increase in late mode discovery is .681 ([.351, 1.066]) at Level 2 and .370 ([.112, .674]) at Level 3. Both the strict failure pattern and the normalized sensitivity remain visible; formatting rescue is not a change to the verifier or mode key.

K.3 ALL FIVE DOMAINS AND ALL THREE LEVELS AT 512 DRAWS

This original-wording follow-up retains thirty-two fixed problems in each of five domains at Levels 1, 2, and 3. Every problem has 512 responses: 480 problems and 245,760 responses in total. Every request uses GPT-5.6 Sol with medium reasoning, an 8,192-token output limit, and omitted temperature and top- p . Native receipt headers identify the provider-reported snapshot `gpt-5.6-sol-2026-07-09`. Collection occurred in stages; common settings and this header do not establish identical stochastic behavior across collection times.

The main figure applies the frozen formatting normalizer. The strict frozen verifier remains primary and grades the identical complete pools. In fixed draw-index order, the first 64 responses solve 474 of 480 problems after normalization and 418 under strict grading. At 512 responses these counts are 477 and 436, respectively. These ordered prefixes differ from rarefaction at the same budget inside the full pool. Invalid, refused, and truncated answers remain in every sampling denominator; transport retries do not count as extra samples.

For each complete pool of size $N = 512$, with c correct responses and verified-mode multiplicities n_j , we compute

$$P_N(k) = 1 - \frac{\binom{N-c}{k}}{\binom{N}{k}}, \quad D_N(k) = \sum_j \left[1 - \frac{\binom{N-n_j}{k}}{\binom{N}{k}} \right],$$

then average all thirty-two problems equally within each domain and level. All curves use the same complete 512-response pools and budgets $k \in \{1, 2, 4, 8, 16, 32, 64, 128, 256, 512\}$. An

earlier 64-response endpoint need not equal the rarefied $k = 64$ estimate from this expanded pool. The last-doubling contrast is $D_{512}(512) - D_{512}(256)$. Pointwise 95% intervals use 20,000 whole-problem bootstrap replicates within domain and level (seed 20260911); the same resampled problems form both sides of each contrast. Curves and ordered-prefix sensitivities under both grading conventions remain in the accompanying source record.

Domain	Level	Known support (range)	D/P at 512		ΔD , 256 to 512 [95% CI]	
			Normalized	Strict	Normalized	Strict
Graph	1	6.97 (4–12)	2.531/100.0	2.531/100.0	0.222 [0.142, 0.305]	0.222 [0.142, 0.305]
Graph	2	6.03 (4–12)	2.625/100.0	2.625/100.0	0.268 [0.170, 0.373]	0.268 [0.170, 0.373]
Graph	3	6.66 (4–12)	1.719/100.0	1.719/100.0	0.137 [0.059, 0.228]	0.137 [0.059, 0.228]
Countdown	1	≥ 4.53 (2–6)	3.469/100.0	3.375/100.0	0.148 [0.070, 0.243]	0.164 [0.083, 0.259]
Countdown	2	≥ 4.40 (3–7)	3.156/100.0	3.094/100.0	0.218 [0.123, 0.320]	0.235 [0.136, 0.344]
Countdown	3	≥ 4.25 (2–8)	2.812/100.0	2.562/100.0	0.181 [0.090, 0.289]	0.191 [0.097, 0.297]
Python	1	≥ 2.00 (2)	3.750/100.0	2.875/100.0	0.592 [0.438, 0.750]	0.492 [0.334, 0.656]
Python	2	≥ 2.00 (2)	1.438/100.0	0.562/43.8	0.073 [0.020, 0.135]	0.148 [0.078, 0.227]
Python	3	≥ 2.00 (2)	1.344/100.0	0.281/28.1	0.013 [0.000, 0.032]	0.086 [0.031, 0.156]
MathIR	1	≥ 5.00 (5)	1.906/93.8	1.906/93.8	0.071 [0.012, 0.153]	0.071 [0.012, 0.153]
MathIR	2	≥ 5.00 (5)	1.000/100.0	1.000/100.0	0.002 [0.000, 0.006]	0.002 [0.000, 0.006]
MathIR	3	≥ 5.00 (5)	0.969/96.9	0.969/96.9	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
PantryPlan	1	≥ 16.90 (8–39)	6.906/100.0	6.906/100.0	0.622 [0.458, 0.799]	0.622 [0.458, 0.799]
PantryPlan	2	≥ 16.50 (8–39)	4.875/100.0	4.625/100.0	0.702 [0.476, 0.951]	0.702 [0.472, 0.962]
PantryPlan	3	≥ 16.87 (8–40)	5.562/100.0	5.250/100.0	0.719 [0.527, 0.914]	0.678 [0.489, 0.875]

Table 28: All fifteen domain–level cells at 512 responses per problem, and what the final doubling added. D/P gives mean distinct verified modes and the percentage of problems solved at least once in the complete 512-response pool. Support gives each cell’s mean and range across thirty-two problems: exact for Graph, a certified lower bound elsewhere. Both grading conventions retain all 480 problems and 245,760 responses. The paired contrast $\Delta D = D_{512}(512) - D_{512}(256)$ compares rarefaction means within the final pool, rather than novel keys in the last chronological block. Intervals are pointwise; they neither provide simultaneous coverage across budgets and cells nor establish asymptotic saturation.

Graph support exhausts all fixed-color-compatible assignments. Countdown references enumerate binary expression trees; the validator also accepts unary negation and retains it in canonical keys, so binary counts are lower bounds. Python, MathIR, and Pantry references retain their certified interpretation. Discovered modes may exceed a lower bound without establishing exhaustive coverage of the available modes.

The expansion from sixteen to thirty-two problems per cell followed earlier observations. It retains all 240 original problems and adds the next sixteen per cell in the same outcome-independent SHA256 ranking (seed 20260911). The 122,880 new responses use 512 fresh draws per added problem; earlier eight-draw evaluations of those problems are excluded. Each launched cell retains its complete fixed cohort and all response slots. These results describe finite-budget discovery: small or zero late gains do not rule out rare unseen modes. Source bindings, full curves, strict grades, support references, and ordered-prefix sensitivities remain in the accompanying JSON.

K.4 OBSERVED SAMPLING-BUDGET EFFECTS

The full experiment is complete: 36,864 hosted and 110,592 local responses, 147,456 in total. Each of the six hosted model–wording cohorts has all 6,144 planned draws; strict and normalized analyses retain every response.

The completed local panel contains 110,592 fresh responses from all 25 registered checkpoints. Each trained checkpoint is evaluated in its own domain at both levels; the initial checkpoint covers all six cells. Serial regrading corrects one original Python grade from failure to success, confirmed once in an independent warmed worker using the unchanged verifier and timeouts. The saved receipt does not identify the cause of the original discrepancy. No response is rescued by formatting normalization, so strict and normalized analyses coincide throughout this panel. The original responses and grades remain available alongside the correction audit and source identities.

Two weightings of the same correct pairs. Tables 25 and 27 carry the full-64 pooled collision beside the promptwise weighting this paper uses everywhere else (App. A.1): every prompt with

at least two verified draws contributes one $\widehat{PCMD}$, and eligible prompts count equally rather than in proportion to $\binom{c}{2}$. Across the 70 defined model–cell–wording entries the two readings agree closely, with a median absolute difference of .003 between promptwise $PCMD$ and pooled $1 - C$, so no reading in this appendix turns on the choice. They do separate where one prompt supplies most of a cell’s correct pairs, and in both directions: under original wording the initial checkpoint has pooled $1 - C = .151$ against promptwise .587 on Level-2 PantryPlan, and pooled .625 against promptwise .307 on Level 3. Nineteen entries differ by at least .05. The promptwise column is the one to read as diversity at matched correctness; the pooled column remains a descriptive rate over a cell’s retained pairs, in which the prompts a model solves most often weigh most. What no weighting removes is the selection: a prompt enters either column only when it clears two verified draws, so the eligible population still tracks correctness. It is thinnest where the two readings diverge most — both initial-checkpoint PantryPlan entries above rest on 7 of 16 problems — and it can differ sharply between the wordings, as at initial-checkpoint Level-3 Python, eligible on 16 problems under the original wording and on one under the neutral. Promptwise $PCMD$ equalizes what each eligible prompt contributes, not which prompts are eligible, which is the joint-selection caveat of App. A.1.

MathIR: concentration largely persists through 64 draws. At Level 2, both trained methods have $B_k = 0$ at every measured budget under both wordings. Re:Dr has original $P_8 = .9774$ and $P_{64} = .9875$; neutral wording gives .9767 and .9875. Its D_k equals P_k throughout, so the small late gain consists entirely of first successes. At exactly eight correct draws, both wordings have $R_8 = 1$ on the same 76 eligible checkpoint–problem groups, versus the certified-support uniform lower reference $5[1 - (4/5)^8] = 4.1611$. These groups reuse the same 16 problems across five checkpoints. At Level 3, all neutral trained groups and all original Re:Dr groups also contain at most one observed correct key. There is one exception in original DrGRPO: a single problem at seed 46 contains two keys with counts 28 and 9, giving mean $B_{64} = .0125$ and $B_{64} - B_8 = .0035$. Its equal-seed correct-pair collision is .9803; the other trained MathIR cells have collision 1.0, against the uniform upper reference .2. Thus the larger budget reveals a rare additional mode without supporting an exact single-mode claim for every trained condition.

Python: prompt dependence and late discovery coexist. Original DrGRPO has unchanged $P_k = D_k = .8125$ at Level 2 and 1.0 at Level 3 from eight through 64 draws. Neutral DrGRPO has zero correct responses out of 5,120 at Level 2 and three out of 5,120 at Level 3. Each of those three successes occurs alone in its checkpoint–problem group, so Level 3 mean $P_{64} = D_{64} = .0375$ while correct-pair collision remains undefined. Re:Dr behaves differently. Under original wording its pass probability is already unchanged from eight to 64, but D_k gains .0582 at Level 2 and .1529 at Level 3. Under neutral wording, P_k rises from .5186 to .7500 and from .5109 to .7500, respectively; D_k rises from .6618 to 1.1250 and from .6561 to 1.1875. These final mode counts remain below the original 1.2750 and 1.5750. The corresponding additional-mode gains are .2318 [.0574, .4565] and .2923 [.0659, .5445], with the registered seed-and-prompt intervals. Larger budgets therefore recover both first successes and additional verified alternatives in the neutral replay condition. These unconditional gains do not identify a change in the conditional distribution of correct solutions by themselves.

Pantry: eight draws understate observed discovery. Across the trained Pantry cells and both wordings, $D_{64} - D_8$ ranges from .4151 to 1.0247, with additional-mode gains ranging from .2739 to .7664. The initial checkpoint also gains .8565–1.0778 modes. In the two observed training seeds, Level 3 point estimates even change method ordering: original DrGRPO/Re:Dr have $D_8 = .2075/.3381$ but $D_{64} = 1.1563/.9063$; neutral wording gives .2253/.3515 at eight and 1.2500/.8125 at 64. This is a descriptive ranking of these checkpoints, not evidence of a population-level method advantage. Pantry has only two training seeds, and fixed-correctness comparisons at larger m often have few or no jointly eligible problems. The eligibility table and undefined means remain essential to interpreting the conditional curves.

These observations support persistent concentration in several trained conditions and substantial late discovery in others, which is why no single saturation claim covers all domains: a flat finite-sample curve still cannot exclude rarer unseen modes. The $k = 8$ points rarefy the fresh 64-draw pools on 16 problems per cell rather than restating the previous experiment’s 32-problem scores. The completed hosted panel below adds a separate comparison of fixed deployments on these historical tasks; it is not evidence from newly trained Level-3 checkpoints.

L INFERENCE FOLLOW-UPS: ADAPTATION, MODEL MIXTURES, AND COARSER KEYS

These three analyses require no additional training. Their frozen protocols precede the reported comparisons. Pointwise 95% intervals use 20,000 paired whole-problem bootstrap replicates (seed 20260911); all perturbations, model arms and fixed training seeds move together within a problem. Intervals do not estimate variation over a population of training seeds, and the 21 model pairs are not independent replications. Machine-readable results retain every registered domain, level, pair, wording and grading sensitivity.

L.1 PANTRY PORTFOLIOS UNDER CHANGED REQUIREMENTS

We evaluate the initial Qwen2.5-0.5B checkpoint and Level-2 Dr.GRPO and Re:Dr checkpoints at fixed seeds 43 and 46 on 32 held-out problems. All 192 single-ingredient outages come from task specifications independently of generated answers; exhaustive search of the original quantity grid finds 181 feasible revised tasks. Verified feasibility witnesses are withheld from the model. The five feasible additional dietary restrictions are reported separately because only five problem clusters qualify.

Each portfolio contains eight plans. We revalidate saved plans unchanged, then allow at most eight sequential recovery calls if none survives, stopping at the first valid plan. Unresolved portfolios consume all eight calls. The common recovery interface shows the revised task, initial portfolio, previous recovery outputs and deterministic verifier feedback. Cumulative success includes portfolios that needed no recovery. We average perturbations within each source problem, then weight problems equally.

Controls retain each checkpoint, verifier and token cap. Temperature is chosen from $\{0.7, 1.0, 1.3\}$ by outage survival on 16 separate development problems (384 responses per checkpoint). Diversity prompting makes eight sequential calls, requesting a new support while retaining the complete output history, including failures. These strategies match initial response counts; their context lengths and total token costs differ.

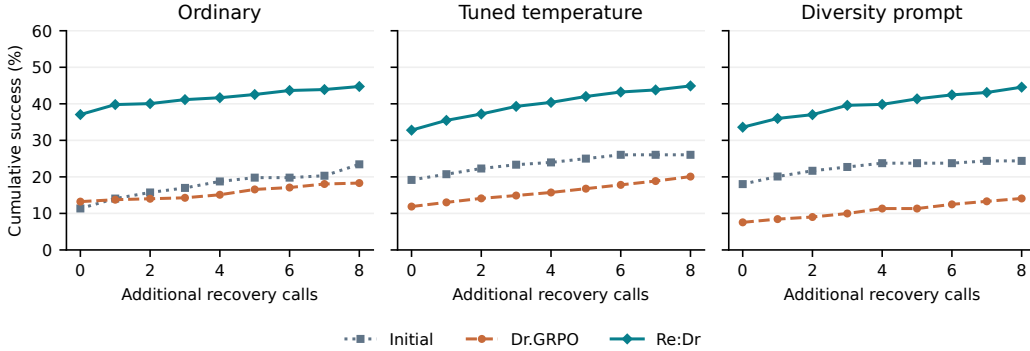


Figure 35: **Recovery after independently chosen feasible Pantry outages.** Zero additional calls measures survival of the eight saved plans. Subsequent points include successful recovery within the displayed call budget. Trained curves average fixed seeds 43 and 46; initial-model curves use one checkpoint. All panels use the same 32 problems; paired uncertainty appears in Table 30.

Ordinary sampling improves saved survival by 23.83 percentage points [12.16, 35.89], reduces capped recovery burden by 2.08 calls [1.14, 3.03], and improves success within eight extra calls by 26.43 points [14.69, 38.18]. Mean input and output token totals fall by 2,539.27 [1, 618.01, 3, 468.86] and 427.40 [386.83, 466.30], respectively. However, original correctness also increases by 1.234 correct responses out of eight [.578, 1.984]. Conditioning descriptively on equal observed correct counts leaves a 1.05-point survival gain [0.00, 3.53] on 19 eligible problems. This demonstrates adaptation gains under the tested intervention; it does not isolate diversity as their causal mechanism.

Logical input tokens include reused prefixes and do not measure cached compute. Historical ordinary-sampling generation time is unavailable. Per-perturbation totals represent deploying a portfolio once and recovering from one update; they differ from actual collection totals, which count each new response once. Training cost is outside this inference accounting.

Table 29: Eight initial plans and up to eight recovery calls under ingredient outages (32 problem clusters) and additional dietary restrictions (five problem clusters). Success columns are percentages. Calls include unresolved cases; token totals charge the initial portfolio once per revised task.

Perturbation	Checkpoint	Strategy	Saved	By 8 calls	Calls	Input tokens	Output tokens
Ingredient outages	Initial	Ordinary	11.35	23.44	6.63	11301.75	376.49
	Initial	Tuned temperature	19.17	26.04	6.13	10864.88	370.17
	Initial	Diversity prompt	18.02	24.38	6.22	11794.81	352.87
	Dr.GRPO 43	Ordinary	9.79	14.01	7.09	12439.22	823.70
	Dr.GRPO 43	Tuned temperature	13.44	16.77	6.79	12017.57	775.44
	Dr.GRPO 43	Diversity prompt	5.21	8.39	7.44	14103.23	848.09
	Dr.GRPO 46	Ordinary	16.67	22.60	6.47	11265.39	637.59
	Dr.GRPO 46	Tuned temperature	10.31	23.39	6.75	11531.40	644.94
	Dr.GRPO 46	Diversity prompt	9.90	19.79	6.90	13124.10	731.00
	Re:Dr 43	Ordinary	39.38	47.97	4.47	9078.92	297.98
	Re:Dr 43	Tuned temperature	28.23	40.94	5.19	9718.01	310.50
	Re:Dr 43	Diversity prompt	36.93	47.19	4.68	10465.22	344.02
	Re:Dr 46	Ordinary	34.74	41.51	4.94	9547.16	308.52
	Re:Dr 46	Tuned temperature	37.34	48.85	4.53	9148.77	300.05
	Re:Dr 46	Diversity prompt	30.26	41.93	5.06	10584.02	313.50
Additional dietary restrictions	Initial	Ordinary	0.00	20.00	6.80	11392.80	371.20
	Initial	Tuned temperature	20.00	40.00	6.40	11175.20	387.20
	Initial	Diversity prompt	0.00	0.00	8.00	13630.20	405.00
	Dr.GRPO 43	Ordinary	0.00	0.00	8.00	13310.20	849.20
	Dr.GRPO 43	Tuned temperature	0.00	0.00	8.00	13394.60	856.00
	Dr.GRPO 43	Diversity prompt	0.00	0.00	8.00	15045.40	939.60
	Dr.GRPO 46	Ordinary	0.00	20.00	7.40	12063.00	650.40
	Dr.GRPO 46	Tuned temperature	0.00	0.00	8.00	12544.20	615.40
	Dr.GRPO 46	Diversity prompt	0.00	0.00	8.00	14132.40	725.60
	Re:Dr 43	Ordinary	20.00	60.00	5.20	9732.60	326.00
	Re:Dr 43	Tuned temperature	40.00	40.00	4.80	9402.80	307.60
	Re:Dr 43	Diversity prompt	40.00	40.00	4.80	10693.60	355.60
	Re:Dr 46	Ordinary	40.00	80.00	3.20	7853.20	271.20
	Re:Dr 46	Tuned temperature	40.00	80.00	3.60	8127.80	265.20
	Re:Dr 46	Diversity prompt	20.00	40.00	5.00	10738.20	341.00

Table 30: Re:Dr minus Dr.GRPO under outages, averaging fixed seeds 43 and 46. Survival and final-success changes are percentage points. Intervals resample 32 paired problems.

Strategy	Δ saved	Δ by 8 calls	Δ calls
Ordinary	23.83 [12.16, 35.89]	26.43 [14.69, 38.18]	-2.08 [-3.03, -1.14]
Tuned temperature	20.91 [9.19, 32.89]	24.82 [14.04, 35.52]	-1.91 [-2.78, -1.05]
Diversity prompt	26.04 [16.93, 35.65]	30.47 [21.51, 39.27]	-2.30 [-3.04, -1.57]

Table 31: Selected temperatures and actual collection cost. Calibration is shown separately and is included in the new-collection totals; saved ordinary portfolios require no new initial generation. Seconds measure generation, not training or hardware-normalized compute.

Checkpoint	Scope	T	Responses	Input tokens	Output tokens	Seconds
Dr.GRPO 43	Calibration	1.3	384	229392	19871	136.3
Dr.GRPO 43	New collection	1.3	4874	4870884	272484	3302.0
Dr.GRPO 46	Calibration	1.3	384	229392	15544	126.5
Dr.GRPO 46	New collection	1.3	4634	4365524	220458	2792.9
Re:Dr 43	Calibration	0.7	384	229392	8552	111.1
Re:Dr 43	New collection	0.7	3559	3160284	92227	1527.7
Re:Dr 46	Calibration	1.0	384	229392	8585	112.5
Re:Dr 46	New collection	1.0	3592	3169337	88300	1510.2
Initial	Calibration	1.3	384	229392	9521	123.5
Initial	New collection	1.3	4416	4035069	116619	1957.4

L.2 COMPLEMENTARITY ACROSS HOSTED MODELS

All seven original-protocol cohorts contribute 107,520 responses on the same 1,920 prompts (five domains, three levels, 128 prompts per cell). This primary analysis uses strict grading and the original Opus 5 Python interface; it is a different condition from the revised-Opus, normalized hosted display. Every one of the 21 model pairs is included. We compare random 4+4 portfolios with eight responses from each constituent, taking exact expectations over subsets of the saved draws. An independent audit matches the formula against all 4,900 subsets for each of 252 deterministic pair/prompt examples.

All pairs improve distinct outcomes over the average constituent by .115–.266. Ten improve over both constituents in point estimates; nine have positive pointwise intervals against both. At a fixed successful-response budget, 2+2 correct draws beat the average four-correct-draw constituent for all pairs (.073–.177 additional outcomes); eligibility varies by pair. These conditional comparisons separate some correctness effects without identifying unobserved support or a causal training mechanism. Shared preferences remain substantial: on joint prompts with at least two correct draws per model, cross-model collision is .624–.740, compared with .704–.819 for mean within-model collision. These are equal-prompt averages, not pooled pair counts. The full record stratifies by domain and level and provides exact-support references or bounds from verified support witnesses. Absence of a shared observed key does not establish disjoint true supports. All average-constituent gains remain positive under coarser keys and the frozen normalization sensitivity. Equal response counts do not match provider tokens, compute, monetary cost or reasoning interfaces.

L.3 SENSITIVITY TO TASK-RELEVANT EQUIVALENCES

Graph groups global color renamings that fix every anchored color; vertex identities remain fixed. Python groups opposite members d and n/d of the same unordered proper factor pair, retaining input-vector order. Countdown flattens associative additions and multiplications and sorts their children, preserving operands, multiplicity, and subtraction/division boundaries. MathIR keeps its simplified state trajectory. Pantry keeps ingredient supports, whose differences can matter under outages. Every map is frozen from task semantics and applied consistently to authenticated correct keys. The independent audit verifies all 6,574 unique successful hosted prompt/key combinations. Correctness is unchanged; deterministic merging cannot increase observed support or reduce correct-pair collision.

Uniform fine-key mass generally induces nonuniform coarse-group mass. Graph/Python references are recomputed from exact groups or coarsened verified witnesses; no unsupported numerical Countdown reference is retained. A uniform distribution over coarse groups is a separate hypothetical policy. The original-wording Python Level-3 gain falls from .419 to zero; Level 2 falls from .431 to .075. Neutral-wording coarse gains remain .556 at Level 2 and .500 at Level 3. Thus the original Python effect can reflect opposite members of the same factor pair, and the conclusion depends on wording and the meaning of output identity. Neither divisor vectors nor factor-pair vectors identify different algorithms.

Reproduction. `ops/build_paper_inference_followups.py` checks frozen analysis hashes, reconstructs Pantry means and stopping identities from per-problem records, and renders these tables and curves. The accompanying `results/inference_followups_20260912.json` binds the protocols, complete analysis records, paper values and figure. Full domain/level, per-seed, normalization, overlap and cost details remain in the bound analysis files.

M PORTFOLIO SURVIVAL UNDER WITHDRAWN OPTIONS

A portfolio of verified answers is worth more than the one answer a task needs only if it still holds a valid answer when the task changes. This appendix tests that in all five domains, and finds it in the four whose portfolios are measurably broader once correctness is held fixed — with PCMD, the measure the paper reports, predicting which those are (.57 across 38 cells). Nothing is retrained and nothing is resampled: the analysis reads the verified outcome keys the reported factorial already produced, at the terminal endpoints the PCMD curves report, so all 474 cells carry the paper’s own endpoint, with 128 evaluation prompts and four independent blocks of eight draws each. The protocol was frozen before any arm was compared; the bound record states which later estimator choices were not, and reports every variant they range over.

Table 32: Every 4+4 mixture against its average eight-response constituent and against each one separately. Fine and coarse columns use all 1,920 prompts; correctness matching compares 2+2 correct responses with four from each constituent on the eligible subset, whose size is n . A and B are the constituents in the displayed order. Entries are distinct-mode differences with pointwise 95% intervals, except the final column, which reports pass@8 differences from the average constituent in percentage points.

A							
Pair		Fine		Coarse		Correctness matched	n
DeepSeek V4 Pro + Kimi K3	0.148	[0.136, 0.160]	0.142	[0.131, 0.154]	0.107	[0.098, 0.115]	1858
DeepSeek V4 Pro + Opus 4.8	0.198	[0.185, 0.211]	0.183	[0.171, 0.197]	0.153	[0.142, 0.164]	1847
DeepSeek V4 Pro + Opus 5	0.262	[0.249, 0.275]	0.252	[0.240, 0.265]	0.177	[0.165, 0.190]	1510
DeepSeek V4 Pro + GPT-5.4	0.133	[0.122, 0.143]	0.127	[0.116, 0.138]	0.098	[0.090, 0.107]	1695
DeepSeek V4 Pro + GPT-5.6 Sol	0.235	[0.223, 0.247]	0.224	[0.212, 0.237]	0.154	[0.142, 0.166]	1396
DeepSeek V4 Pro + Grok 4.3	0.123	[0.112, 0.134]	0.117	[0.106, 0.127]	0.091	[0.083, 0.099]	1859
Kimi K3 + Opus 4.8	0.187	[0.174, 0.200]	0.172	[0.159, 0.185]	0.138	[0.127, 0.148]	1885
Kimi K3 + Opus 5	0.266	[0.253, 0.279]	0.255	[0.242, 0.268]	0.162	[0.150, 0.175]	1543
Kimi K3 + GPT-5.4	0.127	[0.115, 0.139]	0.112	[0.101, 0.124]	0.083	[0.075, 0.092]	1720
Kimi K3 + GPT-5.6 Sol	0.202	[0.190, 0.214]	0.195	[0.183, 0.207]	0.093	[0.083, 0.104]	1417
Kimi K3 + Grok 4.3	0.115	[0.104, 0.126]	0.110	[0.099, 0.121]	0.073	[0.066, 0.080]	1900
Opus 4.8 + Opus 5	0.187	[0.176, 0.199]	0.182	[0.171, 0.193]	0.098	[0.087, 0.109]	1534
Opus 4.8 + GPT-5.4	0.201	[0.186, 0.215]	0.185	[0.172, 0.199]	0.156	[0.144, 0.168]	1713
Opus 4.8 + GPT-5.6 Sol	0.247	[0.234, 0.261]	0.237	[0.223, 0.250]	0.151	[0.138, 0.164]	1409
Opus 4.8 + Grok 4.3	0.180	[0.167, 0.194]	0.171	[0.158, 0.184]	0.141	[0.130, 0.153]	1894
Opus 5 + GPT-5.4	0.228	[0.215, 0.241]	0.217	[0.204, 0.230]	0.158	[0.145, 0.171]	1493
Opus 5 + GPT-5.6 Sol	0.169	[0.157, 0.182]	0.161	[0.149, 0.174]	0.143	[0.130, 0.156]	1370
Opus 5 + Grok 4.3	0.231	[0.219, 0.244]	0.225	[0.213, 0.237]	0.145	[0.133, 0.157]	1547
GPT-5.4 + GPT-5.6 Sol	0.156	[0.145, 0.167]	0.147	[0.136, 0.158]	0.094	[0.084, 0.104]	1400
GPT-5.4 + Grok 4.3	0.130	[0.119, 0.142]	0.121	[0.111, 0.132]	0.095	[0.086, 0.104]	1722
GPT-5.6 Sol + Grok 4.3	0.211	[0.199, 0.224]	0.204	[0.192, 0.216]	0.118	[0.107, 0.130]	1418

B							
Pair		ΔD vs A		ΔD vs B		Δ pass@8 vs mean	
DeepSeek V4 Pro + Kimi K3	0.279	[0.254, 0.304]	0.017	[-0.008, 0.043]	0.22	[0.08, 0.39]	
DeepSeek V4 Pro + Opus 4.8	0.089	[0.064, 0.114]	0.307	[0.282, 0.332]	0.31	[0.15, 0.49]	
DeepSeek V4 Pro + Opus 5	-0.061	[-0.082, -0.039]	0.585	[0.560, 0.610]	7.48	[7.19, 7.78]	
DeepSeek V4 Pro + GPT-5.4	0.097	[0.074, 0.121]	0.168	[0.145, 0.192]	0.64	[0.41, 0.90]	
DeepSeek V4 Pro + GPT-5.6 Sol	-0.003	[-0.025, 0.019]	0.473	[0.447, 0.500]	7.16	[6.71, 7.62]	
DeepSeek V4 Pro + Grok 4.3	0.059	[0.037, 0.082]	0.186	[0.163, 0.210]	0.34	[0.17, 0.52]	
Kimi K3 + Opus 4.8	-0.053	[-0.080, -0.027]	0.427	[0.401, 0.452]	0.19	[0.07, 0.32]	
Kimi K3 + Opus 5	-0.188	[-0.212, -0.165]	0.719	[0.692, 0.747]	7.34	[7.08, 7.61]	
Kimi K3 + GPT-5.4	-0.040	[-0.064, -0.015]	0.293	[0.267, 0.320]	0.73	[0.48, 1.00]	
Kimi K3 + GPT-5.6 Sol	-0.167	[-0.191, -0.144]	0.571	[0.543, 0.600]	7.38	[6.92, 7.85]	
Kimi K3 + Grok 4.3	-0.080	[-0.104, -0.056]	0.309	[0.284, 0.335]	0.12	[0.03, 0.23]	
Opus 4.8 + Opus 5	-0.027	[-0.045, -0.009]	0.401	[0.379, 0.423]	7.42	[7.14, 7.71]	
Opus 4.8 + GPT-5.4	0.274	[0.248, 0.300]	0.127	[0.101, 0.153]	0.93	[0.65, 1.24]	
Opus 4.8 + GPT-5.6 Sol	0.118	[0.096, 0.141]	0.377	[0.350, 0.403]	7.51	[7.03, 7.99]	
Opus 4.8 + Grok 4.3	0.226	[0.202, 0.250]	0.135	[0.110, 0.160]	0.11	[0.02, 0.23]	
Opus 5 + GPT-5.4	0.515	[0.489, 0.541]	-0.060	[-0.082, -0.037]	6.30	[5.90, 6.71]	
Opus 5 + GPT-5.6 Sol	0.254	[0.232, 0.277]	0.084	[0.063, 0.106]	2.11	[1.66, 2.57]	
Opus 5 + Grok 4.3	0.490	[0.467, 0.514]	-0.028	[-0.048, -0.007]	7.41	[7.13, 7.69]	
GPT-5.4 + GPT-5.6 Sol	-0.046	[-0.067, -0.025]	0.359	[0.335, 0.383]	5.61	[5.15, 6.08]	
GPT-5.4 + Grok 4.3	0.102	[0.078, 0.127]	0.159	[0.135, 0.182]	0.93	[0.65, 1.23]	
GPT-5.6 Sol + Grok 4.3	0.386	[0.360, 0.412]	0.037	[0.015, 0.058]	7.55	[7.08, 8.04]	

Table 33: Hosted coarse-key sensitivity under strict original-protocol grading. Distinct counts include all 1,920 prompts; collision conditions on at least two correct responses. Intervals are pointwise.

Model	Fine D@8		Coarse D@8		Fine collision		Coarse collision	
DeepSeek V4 Pro	1.826	[1.792, 1.859]	1.762	[1.729, 1.795]	0.720	[0.710, 0.731]	0.739	[0.729, 0.750]
Kimi K3	2.087	[2.047, 2.129]	1.952	[1.913, 1.992]	0.688	[0.678, 0.699]	0.723	[0.712, 0.733]
Opus 4.8	1.608	[1.575, 1.641]	1.556	[1.525, 1.588]	0.795	[0.785, 0.805]	0.811	[0.801, 0.821]
Opus 5	1.180	[1.155, 1.206]	1.152	[1.128, 1.177]	0.856	[0.845, 0.866]	0.867	[0.857, 0.877]
GPT-5.4	1.755	[1.718, 1.792]	1.679	[1.644, 1.715]	0.729	[0.717, 0.741]	0.750	[0.739, 0.762]
GPT-5.6 Sol	1.349	[1.317, 1.382]	1.312	[1.281, 1.345]	0.778	[0.765, 0.790]	0.794	[0.782, 0.806]
Grok 4.3	1.698	[1.665, 1.732]	1.645	[1.613, 1.679]	0.791	[0.782, 0.800]	0.807	[0.798, 0.816]

Table 34: Local Re:Dr minus Dr.GRPO endpoint probes: 32 problems per domain/level and fixed seed averages. Level-2 trained checkpoints are evaluated at Levels 2 and 3; this is not a reanalysis of every main-paper trajectory. MathIR/Pantry definitions are unchanged.

Domain	Level	Wording	Seeds	Fine $\Delta D@8$	Coarse $\Delta D@8$
Python	2	original	5	0.431 [0.406, 0.456]	0.075 [0.025, 0.138]
MathIR	2	original	5	0.463 [0.387, 0.531]	0.463 [0.387, 0.531]
Pantry	2	original	2	0.469 [0.203, 0.750]	0.469 [0.203, 0.750]
Python	3	original	5	0.419 [0.400, 0.444]	0.000 [0.000, 0.000]
MathIR	3	original	5	0.119 [0.019, 0.225]	0.119 [0.019, 0.225]
Pantry	3	original	2	0.125 [-0.109, 0.375]	0.125 [-0.109, 0.375]
Python	2	neutral	5	0.675 [0.519, 0.838]	0.556 [0.438, 0.675]
MathIR	2	neutral	5	0.469 [0.375, 0.562]	0.469 [0.375, 0.562]
Pantry	2	neutral	2	0.484 [0.234, 0.766]	0.484 [0.234, 0.766]
Python	3	neutral	5	0.656 [0.537, 0.775]	0.500 [0.419, 0.575]
MathIR	3	neutral	5	0.119 [0.025, 0.213]	0.119 [0.025, 0.213]
Pantry	3	neutral	2	0.250 [-0.031, 0.562]	0.250 [-0.031, 0.562]

One withdrawn option. After a portfolio exists, its task loses exactly one option: a colour at one vertex (Graph), one arithmetic operation (Countdown), one returned divisor value (Python), one menu action (MathIR), one ingredient (Pantry, the encoding App. L.1 already uses). Options come from the task specification in a fixed order, never from a generated answer. A withdrawal can only remove answers from the original verified support, so that support certifies the revised task: it is feasible exactly when some original answer survives. Enumerating it reproduces the published certified mode count of all 1,280 prompt rows. Of 9,864 options, 9,118 leave the task feasible and 8,308 of those remove at least one answer; both sets are reported.

What counts as surviving. A portfolio’s saved answers are the verified draws among its eight, and it survives when one of them is still valid. Graph, Countdown and Python decide that from the outcome key itself, so a verified answer outside the generator’s census is still scored — the Countdown census is built from an expression generator that never emits unary negation while the reward verifier accepts it, and the cohort draws 2 such outcomes 50 times among its 846,398 verified draws. MathIR’s key is an executed state path and Pantry’s feasibility lives in its witness plan, so those two are decided by the census; no verified outcome of theirs falls outside it.

Separating diversity from correctness. Raw survival moves with correctness, because a policy that verifies more often has more saved answers to survive with. Holding the number of verified draws fixed at 1, 2 or 4 removes that, as an exact expectation over subsets of the saved correct draws — the estimator App. L.2 already uses. A gain that is absent at one draw and grows with the budget is a property of how a portfolio spreads, not of how often it is right. Expected distinct outcomes at the same budget measure that spread directly. Raw survival improves in 39 of Table 35: Withdrawn options per domain and level. Support is the certified mode count of a prompt, reproduced by enumeration. A binding option leaves the task feasible and removes at least one answer; the Kept columns are the share of the original support that survives one and two binding withdrawals. The Pairs column counts binding pairs.

Domain	Level	Prompts	Support	Options	Binding	Kept	Pairs	Kept ₂
Graph	1	128	6.3	1152	818	0.57	1957	0.34
Graph	2	128	6.1	1536	1068	0.57	3457	0.35
Countdown	1	128	4.5	512	147	0.48	21	0.44
Countdown	2	128	4.4	512	185	0.49	40	0.37
Python	1	128	229.4	1433	1433	0.68	7716	0.50
Python	2	128	251.7	1647	1647	0.71	10381	0.55
MathIR	1	128	5.0	768	448	0.40	384	0.23
MathIR	2	128	5.0	768	512	0.45	512	0.25
Pantry	1	128	18.2	768	651	0.44	1231	0.18
Pantry	2	128	18.2	768	653	0.45	1228	0.19

40 domain, cohort and arm comparisons, and tracks the correctness gain closely enough that it cannot carry the claim alone. With the verified-draw count held fixed the picture separates: 18 comparisons improve at one draw, 29 at two and 28 of 39 at four. Graph, Countdown, Python and Pantry are the domains whose portfolios are measurably broader at a fixed budget, and they are the ones whose survival gains are material, reaching 19.6 points. MathIR’s interval clears zero by a tenth of a point on portfolios that are not broader at all (0.004 distinct outcomes), which is the near-zero PCMD gain Table 9 reports for that domain, read on a downstream task.

Table 36: Each replay arm minus the fresh objective it was added to, pooled over every matched seed, scale and level. Survival columns are percentage points and ΔD_4 is expected distinct outcomes at four verified draws. Binding options are used except in the all-feasible column, whose interval, like the raw column’s, is in the bound record. Pointwise 95% intervals use 20,000 paired whole-prompt bootstrap replicates (seed 20260917), resampling prompts with a prompt’s blocks, seeds, scales and levels moving together, so they carry sampling uncertainty inside this cohort and not the spread a fresh set of training seeds would add. The bound record lists every cell.

Domain	Arm	Raw	Budget 1	Budget 2	Budget 4	Budget 4 (all)	ΔD_4
Graph	Re:Dr	34.78	0.00 [0.00, 0.00]	10.51 [8.76, 12.31]	19.65 [15.61, 23.82]	15.66	1.290
Graph	Re:Max	18.98	0.00 [−0.00, 0.00]	6.62 [5.23, 7.98]	10.99 [8.25, 13.81]	8.31	0.646
Countdown	Re:Dr	5.32	3.51 [1.88, 5.30]	6.52 [4.27, 8.94]	9.58 [6.81, 12.63]	2.53	0.626
Countdown	Re:Max	1.95	−2.40 [−4.62, −0.21]	−1.25 [−3.70, 1.14]	0.10 [−2.62, 2.78]	−0.05	0.490
Python	Re:Dr	38.99	−0.22 [−1.25, 0.91]	4.09 [3.17, 5.08]	7.67 [6.82, 8.58]	7.67	0.317
Python	Re:Max	22.84	1.53 [0.87, 2.24]	3.85 [3.04, 4.72]	5.68 [4.80, 6.64]	5.68	0.198
MathIR	Re:Dr	33.61	0.01 [0.00, 0.05]	0.12 [0.03, 0.23]	0.14 [0.00, 0.30]	0.08	0.004
MathIR	Re:Max	21.64	0.00 [−0.04, 0.05]	0.03 [−0.03, 0.10]	0.09 [−0.01, 0.21]	0.06	0.003
Pantry	Re:Dr	16.55	−1.11 [−1.65, −0.54]	4.36 [3.82, 4.87]	9.78 [8.90, 10.65]	9.20	0.810
Pantry	Re:Max	13.04	−0.64 [−1.04, −0.21]	3.12 [2.72, 3.53]	6.61 [5.83, 7.39]	6.20	0.547

Two columns are structural rather than empirical. Graph’s one-draw column is exactly zero because each hidden vertex has exactly one colour that conflicts with a given colouring: a single saved answer survives the same share of withdrawals whichever answer it is, so no policy can differ there. And the all-feasible column is diluted wherever most withdrawals remove nothing — Countdown withdraws an operation that 63% of its feasible options never used — which is why binding options carry the comparison and the unrestricted set is shown beside it.

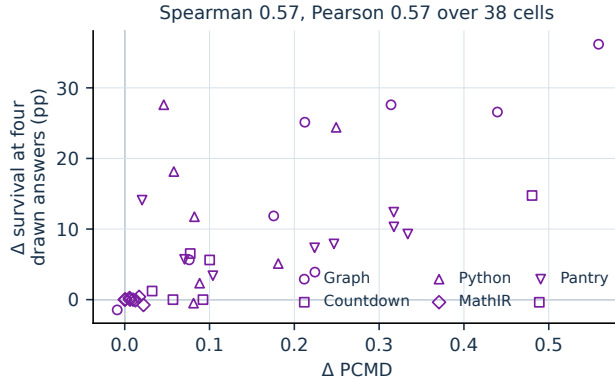


Figure 36: Per-cell agreement between diversity and downstream worth. Each mark is one replay arm against its fresh objective in one domain, scale and level: its PCMD gain from the bound retention record against its survival gain at four verified draws, in percentage points. Marker shape identifies the domain. Per-cell values are retained in `results/portfolio_withdrawals_20260917.json`.

M.1 TWO WITHDRAWALS AT ONCE

A single exclusion is the mildest change a solved task can undergo, so every pair of binding withdrawals is scored as well: 26,927 of them, each re-derived for the joint task rather than assumed. In four domains the joint survivors have to be exactly the outcomes both single withdrawals keep, because each withdrawal excludes outcomes on its own terms. MathIR need not behave that way — dropping two menu actions can in principle remove a state path that either action alone still realizes — but re-enumerating its reduced menus shows that it does, on all 1,344 of its pairs and all 29,213 overall. Two withdrawals are the harsher test: they leave a smaller share of the original support (Table 35), and the gain does not simply follow. It grows in Graph, Python and MathIR and compresses in Countdown and Pantry. Those are the two domains a double withdrawal leaves least room in: Countdown has the narrowest supports here, 4.5 outcomes on average, and a double Pantry withdrawal keeps the smallest share of any domain’s support, 18%. Across every comparison 25 of 39 improve at four verified draws, against 28 for a single withdrawal. The effect is not an artefact of the mildest possible change, and its size is a property of how much room a revised task leaves.

Table 37: One binding withdrawal against two, at two and four verified draws. Entries are percentage-point gains of each replay arm over the fresh objective it was added to, pooled as in Table 36.

Domain	Arm	One @2	One @4	Two @2	Two @4
Graph	Re:Dr	10.51	19.65 [15.61, 23.82]	10.02	23.65 [19.14, 28.25]
Graph	Re:Max	6.62	10.99 [8.25, 13.81]	5.93	12.51 [9.37, 15.69]
Countdown	Re:Dr	6.52	9.58 [6.81, 12.63]	3.29	6.11 [0.87, 12.22]
Countdown	Re:Max	-1.25	0.10 [-2.62, 2.78]	-5.53	-4.87 [-11.80, 0.76]
Python	Re:Dr	4.09	7.67 [6.82, 8.58]	5.37	10.10 [8.77, 11.55]
Python	Re:Max	3.85	5.68 [4.80, 6.64]	5.64	8.16 [6.79, 9.64]
MathIR	Re:Dr	0.12	0.14 [0.00, 0.30]	0.18	0.20 [0.01, 0.45]
MathIR	Re:Max	0.03	0.09 [-0.01, 0.21]	0.04	0.12 [-0.01, 0.29]
Pantry	Re:Dr	4.36	9.78 [8.90, 10.65]	2.85	7.23 [6.51, 7.95]
Pantry	Re:Max	3.12	6.61 [5.83, 7.39]	2.03	4.94 [4.35, 5.57]

M.2 DECODING SETTINGS AS A SUBSTITUTE

A portfolio can also be widened at inference time, which App. L.1 tests in PantryPlan with a tuned temperature and a diversity prompt. The decoding grid answers the temperature and sampling-budget half of that question in all five domains without new generation: it holds the same 128 evaluation prompts, hash-identical to the frozen ones, sampled from a control and a replay arm at five seeds each under 11 settings — six temperatures from 0.5 to 2.0, a four-fold wider sampling budget and two nucleus settings (App. J). The comparison favours the baseline: the control is credited with the best setting in the whole grid, chosen on its own survival, while the replay arm stays at the reference setting. No decoding setting recovers what the control’s portfolio lacks. Its survival moves by at Table 38: Survival at four verified draws across the decoding grid, as percentages. The control columns give its reference setting, its range over the whole grid, and the temperature of its best setting. The last column is the replay arm at the reference setting minus the control at its best, with a paired whole-prompt bootstrap interval. These are E72 terminal checkpoints, a separate cohort from Table 36, whose replay arm predates the Re:Dr objective contract.

Domain	Control	Control grid	Best	Replay	Δ vs best
Graph	58.61	58.59–58.85	T2	95.91	36.97 [35.53, 38.42]
Countdown	15.49	12.57–16.16	T1.6	40.09	28.87 [18.98, 38.87]
Python	70.54	70.36–70.54	T0.5	77.56	8.33 [7.15, 9.69]
MathIR	72.23	72.18–72.55	T2	73.60	1.37 [0.61, 2.30]
Pantry	50.65	50.02–51.79	T2	64.09	12.73 [10.22, 15.31]

most 3.6 points across the entire grid, while the replay arm at one fixed setting leads it by 1.4–37.0 points, every interval excluding zero. The control’s best setting is a temperature one in every domain, so neither the wider sampling budget nor nucleus truncation ever won it more. That is the collapse mechanism read on a downstream task: raising the temperature, widening the budget or truncating the tail resamples the modes a policy still has, and a policy that has lost its alternatives has none to resample. The diversity-prompt control needs new generation and stays PantryPlan’s alone.

M.3 RECOVERY AFTER THE CHANGE

Survival asks whether a saved answer is still valid. What the change costs when none of them is — extra calls, and whether cheaper inference-time measures would have done as well — is the question App. L.1 answers in PantryPlan alone, because answering it needs generation. That same protocol runs here in all five domains over 20 cells — the matched control and replay checkpoints of the reported factorial at two seeds each — on 533 withdrawals drawn from the frozen table. A portfolio is eight answers; when none survives, the policy gets at most eight sequential recovery calls, stopping at the first valid one, and is shown the revised task, its own saved answers and the verifier’s rejections. Two inference-time strategies stand beside ordinary sampling: a temperature chosen per cell from $\{0.7, 1.0, 1.3\}$ on 16 problems held back from the test set, and a diversity prompt that makes eight sequential calls while carrying the whole history. The ordinary portfolio is the cell’s retained terminal draw block, so it generates nothing. PantryPlan’s answers are support masks that a trusted environment projects onto quantities (App. C.5); a fresh generation is graded through that projection and a retained one is not, because the retained draws already store it. Against ordinary sampling and against a tuned temperature, replay resolves more withdrawals with fewer calls in every domain, all ten intervals excluding zero, saving between 0.55 and 5.37 calls of the eight. The whole cohort spends 9,872 recovery responses on the replay arm against 18,335 on the control, so the saving is in generated responses, not only in call counts.

Table 39: Recovery in all five domains. Saved is the share of withdrawals some saved answer already survives, By 8 the share resolved within eight extra calls, and Calls the mean number spent, counting unresolved cases at eight. Percentages; two matched seeds per domain. The first three columns are the fresh objective, the last three the replay arm added to it.

Domain	Strategy	Fresh saved	Fresh by 8	Fresh calls	Replay saved	Replay by 8	Replay calls
Graph	Ordinary	15.62	22.66	6.38	83.20	91.02	1.01
Graph	Tuned temperature	15.62	20.31	6.55	77.34	89.06	1.36
Graph	Diversity prompt	29.69	30.47	5.58	62.50	74.61	2.44
Countdown	Ordinary	2.70	2.70	7.78	9.46	10.81	7.23
Countdown	Tuned temperature	2.70	2.70	7.78	9.46	13.51	7.05
Countdown	Diversity prompt	2.70	2.70	7.78	5.41	9.46	7.41
Python	Ordinary	14.06	14.06	6.88	69.92	77.73	2.21
Python	Tuned temperature	14.06	14.06	6.88	71.88	75.78	2.12
Python	Diversity prompt	14.06	14.06	6.88	62.50	67.97	2.79
MathIR	Ordinary	38.39	44.20	4.60	58.93	65.18	2.92
MathIR	Tuned temperature	37.05	45.09	4.61	63.84	67.86	2.67
MathIR	Diversity prompt	41.07	45.09	4.58	47.32	53.57	3.85
Pantry	Ordinary	31.25	31.64	5.48	47.66	52.34	4.04
Pantry	Tuned temperature	31.25	44.53	4.81	53.12	62.11	3.38
Pantry	Diversity prompt	56.64	58.20	3.39	33.98	45.70	4.71

Table 40: Replay minus the fresh objective it was added to, on the same withdrawals. Survival and resolution are percentage points; calls are calls. Intervals resample prompts as elsewhere in this appendix, so a prompt’s withdrawals and both of its seeds move together.

Domain	Strategy	Δ saved	Δ by 8	Δ calls
Graph	Ordinary	67.58 [57.81, 76.56]	68.36 [59.38, 76.95]	-5.37 [-6.05, -4.65]
Graph	Tuned temperature	61.72 [51.95, 71.09]	68.75 [61.33, 75.78]	-5.20 [-5.84, -4.52]
Graph	Diversity prompt	32.81 [21.09, 44.14]	44.14 [34.77, 53.12]	-3.14 [-3.95, -2.30]
Countdown	Ordinary	6.76 [1.35, 13.89]	8.11 [1.43, 16.18]	-0.55 [-1.12, -0.11]
Countdown	Tuned temperature	6.76 [1.35, 13.89]	10.81 [3.33, 19.32]	-0.73 [-1.35, -0.21]
Countdown	Diversity prompt	2.70 [0.00, 6.58]	6.76 [1.32, 13.64]	-0.38 [-0.76, -0.06]
Python	Ordinary	55.86 [44.92, 65.23]	63.67 [51.56, 74.22]	-4.67 [-5.45, -3.76]
Python	Tuned temperature	57.81 [46.88, 67.19]	61.72 [50.39, 71.48]	-4.76 [-5.54, -3.87]
Python	Diversity prompt	48.44 [37.11, 59.38]	53.91 [41.80, 65.62]	-4.09 [-4.99, -3.14]
MathIR	Ordinary	20.54 [11.16, 30.09]	20.98 [12.71, 29.82]	-1.68 [-2.37, -1.02]
MathIR	Tuned temperature	26.79 [18.26, 35.65]	22.77 [13.89, 32.27]	-1.94 [-2.66, -1.28]
MathIR	Diversity prompt	6.25 [-4.09, 16.67]	8.48 [-0.46, 18.26]	-0.73 [-1.50, 0.03]
Pantry	Ordinary	16.41 [8.20, 25.39]	20.70 [11.72, 30.47]	-1.44 [-2.18, -0.76]
Pantry	Tuned temperature	21.88 [13.28, 31.25]	17.58 [10.94, 24.61]	-1.43 [-2.00, -0.91]
Pantry	Diversity prompt	-22.66 [-31.25, -13.67]	-12.50 [-21.88, -2.73]	1.32 [0.55, 2.05]

The diversity prompt is the harder comparison, and it is not uniform. It is what most helps a collapsed policy, raising the control’s saved survival in Graph and Pantry, where a history of prior answers stands in for diversity the policy does not have. It *lowers* the replay arm wherever it moves it at all; the record shows that effect without separating its cause from the longer context the strategy carries. The two cross in Pantry, the one domain where the diversity prompt beats replay outright, and the one whose answers are six-bit support masks — a support set small enough to enumerate. Elsewhere replay still leads it, by 2.7–48.4 points of saved survival. A portfolio that already holds alternatives is therefore worth more than either decoding or prompting for them, except where the alternatives are few enough to enumerate.

What this establishes. A portfolio’s retained alternatives are worth having in four of the five domains once correctness is held fixed, and PCMD predicts which ones. Withdrawals are chosen from task specifications and certified by exhaustive enumeration, so no comparison depends on a model’s own account of feasibility, and neither a harsher change nor a better decoding setting accounts for the result (Apps. M.1, M.2). Three limits are explicit. A withdrawal is one exclusion, not an arbitrary revision. A diversity prompt, which is what most helps a collapsed policy, beats replay in the one domain whose answers are few enough to enumerate (App. M.3). And these intervals resample prompts rather than training seeds, so they carry sampling uncertainty within this cohort and not the spread a fresh set of training seeds would add.

Design record. These choices were fixed before any arm was compared: withdrawal family per domain; option enumeration order; exclusion-only feasibility certificate; terminal-endpoint cohort; survival definition. These were chosen with the raw contrast already visible: fixed verified-draw budget estimator; restriction to binding options. Raw, all-feasible and binding variants at budgets 1, 2 and 4 are all reported.

Reproduction. ops/prepare_portfolio_withdrawals.py freezes the prompts, certified supports and withdrawn options; ops/analyze_portfolio_withdrawals.py scores the saved outcomes against them; ops/audit_portfolio_withdrawals.py re-derives 60 supports by an independent route, matches the fixed-budget expectation against exhaustive subset enumeration on 4,084 comparisons, and checks that all 846,398 verified outcomes are scored against the support they belong to. The accompanying results/portfolio_withdrawals_20260917.json binds the frozen protocol, the complete per-cell record and every value printed here.

Part V. Fixed deployments

N INFERENCE-ONLY CONCENTRATION IN A HOSTED MODEL

This exploratory evaluation measures the current output distribution of GPT-5.6 Sol under the inherited MODEBENCH interface. It is separate from the matched training interventions: it adds no frontier-model training, replay comparison, or controlled model-size contrast. Only the completed, audited deployment enters these tables.

Protocol and provenance. The September 11, 2026 run uses the Azure deployment `gpt-5.6-sol`; saved provider headers identify snapshot `gpt-5.6-sol-2026-07-09`. Each of the five domains contributes 128 held-out prompts at each of three levels. Eight stateless requests per prompt yield 15,360 responses. Requests use medium reasoning, an 8,192-token output limit, no tools and no conversation state. The service returns temperature 1.0 and top- p .98; neither setting was overridden. These settings and the hosted natural-language interface differ from the local training evaluation’s token-level grammar constraints.

Level 1 uses the native E118 evaluation splits retained in E124. Level 2 uses the frozen matched r5 construction; Level 3 uses the matched v3 construction admitted by the completed canonical v3 confirmation. Level 3 is an exploratory test extension here, not a third level of the paper’s training intervention. All cells use the multi-answer split. The Level-2/Level-3 support histograms match a separate fixed-reference Level-1 confirmation reserve, which differs from the native Level-1 split in Graph, Countdown and Python. Prompts differ across levels and are not paired by row index. Calibration against small-model references does not guarantee increasing difficulty for this hosted model.

Exact prompts, dataset identities and hashes, requests, provider-exposed responses, grades, retry receipts, usage and verifier snapshots are retained in the experiment’s evidence directory. All 15,360 intended samples completed, with distinct response identifiers; 107 failed API attempts were retained and recovered by retries. Provider random-number independence and hidden reasoning contents cannot be audited from the exposed responses.

Table 41: **Hosted correctness and concentration under strict grading.** GPT-5.6 Sol, with medium reasoning. Each level weights five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is the percentage of within-prompt correct pairs sharing a mode, pair weighted within each domain. Brackets give pointwise 95% intervals from whole-prompt resampling.

Level	M	P	D	C (95% interval)
1	83.0	95.8	1.71	74.0 [71.9, 76.1]
2	62.9	77.5	1.16	82.3 [80.3, 84.4]
3	68.7	80.3	1.18	84.5 [82.8, 86.2]

Metrics and uncertainty. Per-response correctness is mean@8 in this paper (called pass@1 in the hosted report). Raw distinct@8 counts verified keys across eight draws, including zero for unsolved prompts. For prompt x , let c_x be its number of correct draws and n_{xc} its count for correct key c . The within-cell correct-pair collision estimate is

$$\hat{C} = \frac{\sum_x \sum_c \binom{n_{xc}}{2}}{\sum_x \binom{c_x}{2}}.$$

It describes concentration conditional on successful draws, with prompts weighted by their number of correct pairs. Level summaries average the five domain estimates equally. When the certified correct-key count m_x is available, a uniform-correct-key reference replaces the numerator by $\sum_x \binom{c_x}{2} / m_x$. This differs from a uniform-action baseline and is a descriptive reference, not a normative sampling requirement. Countdown’s verifier accepts modes beyond its enumerated binary-expression library, including unary negation, so its total support is unknown and this reference is omitted. Intervals use 2,000 whole-prompt bootstrap replicates, seed 20260911, resampling within each domain and level and keeping all eight draws together. They are pointwise descriptive intervals.

Table 42: **GPT-5.6 Sol under strict executable grading.** Every cell contains 128 prompts and 1,024 responses. M and P are mean@8 and pass@8 (%); D is distinct@8. C is correct-pair collision (%), with 95% prompt-bootstrap interval); U is the uniform-correct-key collision reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Formatting sensitivity. Strict grades remain primary. A secondary typography normalizer was introduced after inspecting the first 15 responses. It fixes a frozen set of presentation conventions, including escaped Python/LaTeX syntax, and reuses the original executable validators without repairing calculations or program logic. It recovers 2,580 additional verified outputs; strict successes and their keys are retained. This is a post hoc diagnostic, not a replacement benchmark. All 3,072 primary Python responses passed an independent serial recheck with no grade changes. One cold-start timeout in the secondary grading cache was reconciled with repeated serial verification; the original cache and correction audit remain preserved.

Table 43: **Post hoc formatting-normalized diagnostic.** Same samples and metric definitions as Table 42, with fixed presentation normalization before executable verification. The strict table remains primary; normalization was designed after the first 15 responses.

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Concentration and level. The strict five-domain collision average is 74.0%, 82.3% and 84.5% at Levels 1–3; the normalized diagnostic gives 74.1%, 81.2% and 83.6%. This aggregate pattern is not a monotonic law within domains. Graph collision is 89.7%, 83.3%, then 95.1%; normalized PantryPlan collision is 46.9%, 64.0%, then 53.0%. Normalized overall correctness also rebounds from 83.2% at Level 2 to 85.7% at Level 3. Thus a higher level label is not equivalent to a measured increase in difficulty for this model, and raw distinct-mode counts also change with correctness and support.

Level-3 Graph separates concentration from failed responses particularly clearly: all 1,024 outputs verify, eight draws produce 1.16 distinct keys on average, and 3,407 of 3,584 cor-

rect pairs share a key (95.1%). The uniform-correct-key reference is 19.0%; the excess is 76.1 percentage points (95% interval [73.5, 78.3]). Graph was selected after the GPT run for this illustration because its correctness is high, support is certified and formatting normalization changes no scores; both tables report all domains.

Interface and claim limits. Level-2/Level-3 MathIR instructions prescribe moving the right-side variable term left, removing the constant and then dividing. Python instructions suggest a nested small-divisor conditional chain, and PantryPlan instructions prefer particular ingredient families. These preferences can concentrate the very keys being measured. In addition, Level-1 PantryPlan requests a support mask whose quantities are supplied by a trusted projection; Levels 2–3 ask the model to provide quantities. MathIR verifies symbolic action compliance, which can reject a numerically valid shortcut that omits a required symbolic step. Neither these failures nor the cross-level differences isolate numerical reasoning ability. Canonical keys describe executed outputs, not hidden reasoning strategies. Eight draws cannot establish that an unseen mode has zero probability. The evidence supports concentration under the specified prompts and deployment configuration; it identifies neither a training cause nor a causal difficulty or model-size effect.

Reproduction record. The source record is `artifacts/frontier_modebench_gpt56sol_20260911/summary.json`. Its hash, complete cell statistics and run settings are copied into `results/frontier_hosted_20260911.json`; adjacent generated table bodies supply the displayed values. The evidence directory also retains `completion_audit.json`, `prompt_contract_audit.json`, the original requests and responses, and `interpretation_notes.md`. Hosted-model results are separate from the training-checkpoint archive and its matched-seed comparisons.

N.1 HOSTED CONCENTRATION ON THE SUCCESS-CONDITIONAL AXIS

Fig. 39 reports PCMD for the same audited responses, per domain and level and for every deployment in the cohort rather than this one alone. The hosted tables above report `correct_pair_collision`, which is one minus PCMD pooled over pairs; the values here instead average over prompts without weighting by how many verified pairs each contributed, matching the aggregation used everywhere else in this paper and stated in App. A.1. The two readings agree closely, and the pooled reading recomputed from the per-sample records reproduces the published collision estimate exactly, which checks the two pipelines against each other.

This deployment is highly accurate and, on three of five domains, close to deterministic in which verified answer it gives. MathIR reaches PCMD .000 at Levels 2 and 3: across every prompt with two or more verified draws, no two verified answers ever differed. Level-3 Graph sits at .049 against six available colorings. PantryPlan is the consistent exception. Python at Levels 2 and 3 is left blank because the model clears two verified draws on only 14 and 15 of 128 prompts there, which is too few to estimate diversity.

The Level-4 probe. Fig. 3 carries one Level-4 point per domain for GPT-5.6 Sol, at PCMD .181 over 5 domains. Two things separate it from every other hosted cell and it is read on its own account for both of them. It was served direct through OpenAI rather than the Azure deployment that served the other levels, which may be a different build; the served model string is recorded per response rather than assumed. And its Python cell rests entirely on the frozen formatting normalizer: strict grading rejects the LaTeX-escaped Python this deployment emits, verifying two of its 1,024 draws where the normalizer recovers 675. That is a presentation artifact rather than a capability measurement, and it affects the Levels 1–3 Python cells in the same direction.

Table 44 extends the measurement to the whole completed cohort. Every one of the seven deployments is concentrated on this axis, and none reaches 1.5 effective verified modes: macro PCMD runs from .142 for Claude Opus 5 to .320 for DeepSeek V4 Pro, or $N_{\text{eff}}^{(2)}$ between 1.17 and 1.47. Those macro averages are taken over the thirteen model–domain–level cells every deployment reports. That ceiling is partly MathIR’s doing, and the last two columns say by how much. MathIR keys a mode by its derivation rather than its answer, and most of the derivations it certifies are longer routes no measured policy produces, so it sits near zero for every deployment for reasons that are partly the construction’s. Averaged over the four domains whose modes are alternative answers rather than

alternative derivations of one answer, every deployment rises and the ceiling reaches 1.58 — still under one and three fifths effective verified modes, against domains admitting between four and several hundred. The concentration is not an artifact of that domain; the exact height of the bound is partly one.

Two of them, GPT-5.6 Sol and Claude Opus 5, return no measurable Python diversity at Levels 2 and 3 because the provider refused most of those requests, and averaging each deployment over whatever it can report would rank them against an easier set of cells than the other five: it puts DeepSeek V4 Pro at .280 rather than .320 and moves it below Kimi K3. The ordering does not follow correctness, and it is not stable across domains — GPT-5.4 and Kimi K3 lead on Python while DeepSeek V4 Pro leads on Graph — so this is a property of individual deployments rather than a single frontier-wide constant. What is common to all of them is the level: the most varied deployment measured still repeats its own verified mode on roughly seven of every ten pairs of correct answers. Every model also declines from Level 1 to Level 3. Fig. 16 draws this cohort one deployment per row, beside the open-scale grid it mirrors.

Table 44: **Verified-mode diversity across seven hosted deployments.** Macro PCMD averages the cells that clear the support bar, weighting the five domains and three levels equally, with $N_{\text{eff}}^{(2)} = 1/(1 - \text{PCMD})$ beside it and the per-level macro to its right. Each deployment contributes 128 held-out prompts per cell and eight stateless requests per prompt under its own default sampling. GPT-5.6 Sol and Claude Opus 5 clear the bar on 13 of 15 cells; the other five clear all 15.

Deployment	Macro PCMD	$N_{\text{eff}}^{(2)}$	Macro PCMD by level		
			L1	L2	L3
DeepSeek V4 Pro	.320	1.47	.289	.319	.232
Kimi K3	.302	1.43	.374	.307	.253
GPT-5.4	.264	1.36	.338	.247	.224
Grok 4.3	.239	1.31	.275	.179	.171
GPT-5.6 Sol	.226	1.29	.256	.229	.183
Claude Opus 4.8	.215	1.27	.217	.229	.168
Claude Opus 5	.142	1.17	.140	.164	.121

What the level axis carries besides difficulty. The Level-2 and Level-3 system messages add explicit strategy guidance to MathIR, Python and PantryPlan — an algebraic isolation order, a nested small-divisor chain, a preference over ingredient families — which the Level-1 messages do not carry; App. I gives the exact text and removes it. A deployment that follows an instruction concentrates on the mode that instruction names, so some of the decline above belongs to the prompt rather than to the task. How much is measured here rather than assumed. Averaged over the seven deployments, PCMD falls by .115 from Level 1 to Level 3 across the 20 guided deployment–domain pairs and by .048 across the 14 pairs in Graph and Countdown, which carry no strategy text at any level. Averaged over those two unguided domains alone, 7 of 7 deployments still decline. The guidance inflates the cross-level fall without accounting for it.

Two further readings bear on the same question, and they do not agree. Removing the MathIR guidance from local checkpoints leaves PCMD at .000 under both wordings, so at 0.5B the hint is not what holds those answers together (App. I). At 64 draws on hosted deployments, neutral MathIR wording instead raises diversity at matched correctness for both GPT deployments at Level 2, by intervals excluding zero, while Grok’s include it (App. K.2). The second is the one measured on deployments of this kind. Read the cross-level narrowing as concentration under the prompts as specified — which is what these deployments are asked, and what a user of them receives — rather than as an effect of difficulty alone.

O COMPARISON ACROSS HOSTED DEPLOYMENTS

Figure 39 compares verified output diversity across hosted deployments. Its main display applies the frozen formatting normalizer to every deployment and uses the complete revised-wording Opus 5 Python cohort at all three levels. Every response enters its accuracy and mode-count metrics; outputs are not selected by correctness or provider outcome. The other cells use their original prompts. This is a descriptive comparison across the stated task formulations, rather than a common-prompt model ranking.

The tables and Graph figure below preserve every original-protocol result, including failed draws and native refusals. The separate Python comparison reports the original and revised conditions under both grading rules. These records expose the effect of the prompt change without replacing the original evidence. Level 3 remains an exploratory test extension; no hosted-model training or replay intervention is performed. Original deployments enter the comparison only with a passing integrity audit and all 15,360 responses.

Completed hosted comparison. The admitted deployments are GPT-5.6 Sol, Claude Opus 5, GPT-5.4, Grok 4.3, Kimi K3, Claude Opus 4.8, DeepSeek V4 Pro: 7 complete runs, 107,520 responses, and the same 128 held-out prompts per domain and level with eight stateless samples per prompt. Only runs with a passing completion audit and the identical frozen prompt digest enter these tables. No training, model-side tools or conversation history is added.

Provider configurations. GPT-5.6 Sol: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 5: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. GPT-5.4: hosted API, requested reasoning setting medium, native output limit 8192. Grok 4.3: hosted API, requested reasoning setting medium, native output limit 8192. Kimi K3: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 4.8: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. DeepSeek V4 Pro: hosted API, requested reasoning setting medium, native output limit 8192.

Provider effort labels do not establish equal compute, and native token accounting differs. The saved native request and returned usage determine each configuration; provider defaults remain unspecified whenever native responses omit them.

The appendix retains strict executable grades alongside formatting-normalized scores. The main display uses the same frozen formatting rule for every cell and the complete separately collected revised Opus 5 Python condition; all other displayed cells retain the original request wording. The shared typography diagnostic was developed after the first 15 GPT-5.6 Sol responses and frozen before the prospective comparison runs. It retains its post hoc status for GPT-5.6 Sol. Inherited route preferences and the Level-1 PantryPlan interface change limit cross-level interpretation. Countdown has no certified finite-total-support uniform reference. These data establish neither a training cause nor a causal model-size or difficulty effect. All pair-collision intervals are pointwise; overlapping or nonoverlapping cell intervals are not a paired model-difference test.

Collection interruption: DeepSeek V4 Pro. 13 HTTP-200 bodies had no terminal finish reason or visible answer and were preserved as provider protocol failures. A later storage interruption left 192 logged physical request starts without a saved outcome or usage; their outcome and possible billing remain unknown. The audit registers these attempts separately from the 15,360 terminal draws. Only missing terminal draws were recollected; saved terminal responses, including truncations, were retained. Reported token totals are subtotals of known usage, not zero-cost claims for failed or interrupted attempts. Source-bound recovery sidecars preserve the original receipts and the exact interruption accounting.

O.1 A SEPARATE PYTHON PROMPT SENSITIVITY TEST

After the original Opus 5 Python refusals, synthetic tasks were used to select a direct arithmetic-expression formulation with no system message. We then collected eight independent draws for every original Python test task: all 384 prompts across three levels, with unchanged adaptive thinking, medium effort, 8,192 output tokens, allowed operators, public inputs and frozen graders. The revised user instruction asks for a one-line boxed Python lambda returning any proper divisor for each listed input. It does not prescribe a preferred divisor. Both task wording and system-message presence change; collection time also differs. This exploratory follow-up therefore supports a practical prompt repair without isolating its cause. Original refusals and all three residual refusals are retained. The original-protocol appendix tables retain all original Python responses; the main display uses this complete revised condition with an explicit caption note.

The new condition has no API failures or token-limit truncations. Strict grading accepts 2,282 responses; the frozen rule rescues 787 more by converting escaped percent signs (\%)

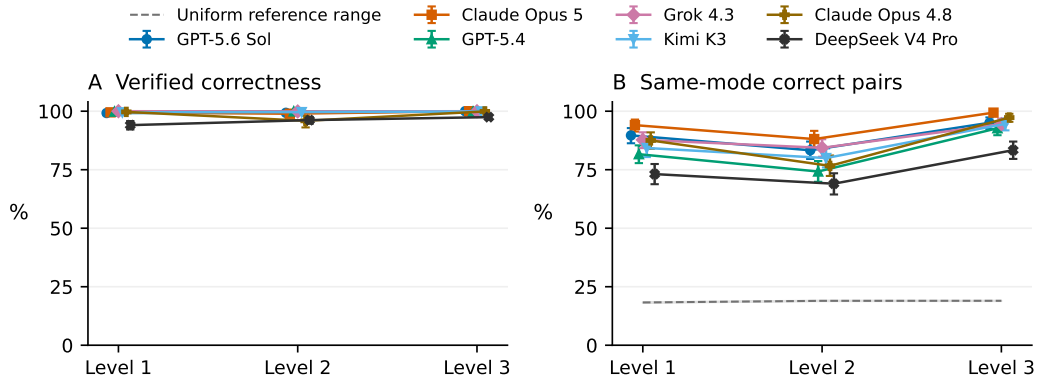


Figure 37: **Verified correctness and concentration in hosted Graph outputs.** Each deployment receives the same 128 prompts per level, with eight stateless samples per prompt. Intervals resample whole prompts. The gray band spans the models’ uniform-correct-key references, conditional on their observed correct draws. Graph was selected after the GPT-5.6 Sol run and before the prospective comparison runs. Level labels do not guarantee increasing difficulty for each model.

Table 45: **Opus 5 Python: original versus plain task wording.** Each row has 1,024 responses. R counts native refusals; A is per-response accuracy (%), D is distinct@8, and C is correct-pair collision (%). Strict and normalized results use the same frozen grader and typography rule. Original Level-2/3 collision is undefined because no prompt has two correct draws.

Condition	L	R	Strict			Normalized		
			A	D	C	A	D	C
Original	1	409	38.1	0.91	93.0	60.0	1.12	90.4
	2	1021	0.3	0.02	–	0.3	0.02	–
	3	1024	0.0	0.00	–	0.0	0.00	–
Plain user, no system	1	0	74.9	1.42	80.2	100.0	1.49	79.9
	2	0	71.8	1.31	84.7	100.0	1.38	85.7
	3	3	76.2	1.38	82.7	99.7	1.41	84.1

to Python modulo (%). Thus every non-refused answer passes after this formatting-only correction. Normalized collision is 79.9/85.7/84.1%, versus conditional uniform references of 1.43/1.15/1.15%. The residual three refusals occur on two prompts whose other identical requests are accepted. No retry-until-accepted selection is used.

The paired analysis resamples whole eight-draw prompt groups across conditions (20,000 replicates). Normalized accuracy gains at Levels 1/2/3 are 40.0 [34.7, 45.5], 99.7 [99.3, 100.0], and 99.7 [99.2, 100.0] percentage points (pointwise 95% intervals). Output draws themselves are independent, and the different correct-pair populations preclude interpreting a raw collision difference as a matched-population prompt effect.

The saved sensitivity record binds the original and revised native receipts, request payloads, strict regrade audit and frozen normalization outputs.

Separate first-valid retry diagnostic. After completing the fixed 3,072-response revised-wording cohort, a separate condition retries only its three remaining refused slots, retaining the first verified-valid response. All three succeed on their first additional request, and all reproduce a mode already seen for that prompt. This uses 3,075 requests to select 3,072 valid responses; its 100% accepted-set accuracy follows from selection and is not unconditional model accuracy. The selected set’s distinct@8 is unchanged at 1.492, 1.375, 1.414 at Levels 1–3. Level-3 correct-pair collision changes from 84.0909% to 84.1518%; the other levels are unchanged. All initial and retry responses remain preserved. The main figure continues to use the complete fixed eight-draw condition, including its three refusals; these selected-validity results are only a separate diagnostic with variable request effort.

O.2 A MATCHED REQUESTED-TEMPERATURE SENSITIVITY TEST

A separate fixed subset contains eight prompts from each of the five domains at each of three levels: 120 prompts, with eight independent draws per prompt and temperature. Grok 4.3 and Kimi K3 each receive requested temperatures $T = 1.0$ and $T = 1.5$, giving 960 draws per condition and 3,840 in total. The two conditions share task rows and all non-temperature request settings; both temperatures are collected concurrently within each deployment. Every response is retained. This subset is a separate experiment and does not replace any full-cohort result in the main hosted display.

Provider API contracts support the requested controls and the deployments accept the requests, but native responses do not report effective sampling temperature. DeepSeek is excluded from this contrast because its documented thinking-mode interface ignores the temperature setting. These contrasts therefore concern the tested requested settings, not arbitrary temperatures or all deployments.

Tables average all five domains equally within each level; All equally averages all 15 domain-level cells. Confidence intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, retaining each eight-draw group. They are pointwise and have no multiplicity adjustment. Correct-pair collision remains conditional on correct draws and can change its eligible prompt population; an undefined cell makes the corresponding macro undefined.

Observed sensitivity. For Grok, the observed diversity change is +0.050 [-0.058, +0.158] modes, and the collision change is -0.78 [-3.86, +2.12] percentage points. These intervals do not show a clear broadening at the tested setting. The per-response accuracy change is -2.29 [-3.85, -0.94] percentage points. This does not establish invariance to temperature in general.

Kimi returns 696/960 token-limited outputs at $T = 1.5$, compared with 0 at $T = 1.0$. Strict accuracy falls from 96.56% to 21.25%, while raw diversity falls from 1.933 to 0.983 modes. This is a severe token-limit and accuracy failure, not a clean increase in correct-output concentration. The five-domain collision mean is undefined at $T = 1.5$ because some cells have no eligible correct pairs. Neither model has provider-declared refusals in this experiment.

† marks an omitted interval when fewer than all 20,000 bootstrap replicates define the contrast. For Kimi collision, only 11,815 and 11,616 replicates are defined at Levels 1 and 2; the raw conditional percentile summaries remain in the source record. We do not present them as ordinary 95% intervals. Undefined cells are never replaced by zero or silently omitted.

Full domain-level estimates, counts, native outcomes and collision eligibility remain in the source-bound temperature analysis. Countdown has no finite-total-support uniform reference; finite-support sensitivity summaries use the other four domains. The paper record authenticates every included condition summary, native completion audit, frozen normalizer, paired analysis source and sampling-control review, with the complete fixed response cohort retained throughout.

O.3 GPT-5.6 SOL: TEMPERATURE, PASS@8 AND VERIFIED MODES

Figure 38 plots empirical `pass@8` against `distinct@8` for $T \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$, with reasoning `none`. Each condition retains the same 480 held-out prompts (32 per domain-level cell), with eight stateless responses per prompt: 3,840 responses per temperature and 19,200 in total. A prompt contributes one to `pass@8` if at least one of its eight answers is verified correct, and contributes its number of distinct correct canonical keys to `distinct@8`. Zero-success groups and all unsuccessful draws remain in the denominator. We compute success directly from the saved groups, not as $1 - (1 - p)^8$ from pooled per-response accuracy; task difficulty varies across prompts.

The expanded cohort retains all 120 original prompts and their 4,800 responses, adding 360 prompts and 14,400 responses. The extension follows the same outcome-independent row-hash ranking within each domain-level cell. Requests retain the frozen prompt bytes and verifier; only temperature varies in the sampling controls. New collection interleaves temperatures. Separate old-120 and new-360 analyses below assess cohort sensitivity; prompt composition and collection time differ between these cohorts.

The deployment rejected non-default temperatures with medium reasoning and rejected temperature 2.5. These curves use the supported reasoning `none` profile. Historical medium-reasoning measurements cover only the original 120 prompts and are omitted from the expanded-cohort

Table 46: **Requested-temperature sensitivity: Strict executable grades.** A is per-response accuracy (%), D is `distinct@8`, and C is correct-pair collision (%). D differences are mode counts; A/C differences are percentage points. Brackets give paired pointwise 95% intervals for the paired change in requested temperature. Every fixed draw remains in its original denominator.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Grok 4.3	1	A	98.12	95.62	-2.50 [-4.06, -0.94]
		D	1.900	2.000	+0.100 [-0.100, +0.300]
		C	72.89	73.16	+0.27 [-6.31, +6.37]
	2	A	99.06	99.38	+0.31 [-0.94, +1.56]
		D	1.425	1.425	+0.000 [-0.175, +0.200]
		C	86.61	85.20	-1.41 [-6.26, +3.30]
	3	A	98.75	94.06	-4.69 [-9.06, -0.94]
		D	1.500	1.550	+0.050 [-0.125, +0.225]
		C	84.04	82.85	-1.20 [-5.48, +2.79]
	All	A	98.65	96.35	-2.29 [-3.85, -0.94]
		D	1.608	1.658	+0.050 [-0.058, +0.158]
		C	81.18	80.40	-0.78 [-3.86, +2.12]
Kimi K3	1	A	92.50	22.50	-70.00 [-75.62, -63.75]
		D	2.200	1.075	-1.125 [-1.475, -0.775]
		C	66.23	53.12	-13.11 [-] [†]
	2	A	98.12	20.31	-77.81 [-82.50, -72.81]
		D	1.850	0.900	-0.950 [-1.300, -0.625]
		C	72.18	66.55	-5.63 [-] [†]
	3	A	99.06	20.94	-78.12 [-83.75, -72.81]
		D	1.750	0.975	-0.775 [-1.025, -0.525]
		C	78.33	—	—
	All	A	96.56	21.25	-75.31 [-78.44, -72.19]
		D	1.933	0.983	-0.950 [-1.133, -0.767]
		C	72.25	—	—

Table 47: **Requested-temperature sensitivity: Frozen formatting-normalized grades, the cells they change.** Only cells whose normalized grade differs from its strict grade appear; every cell absent here is identical under both gradings. A is per-response accuracy (%), D is `distinct@8`, and C is correct-pair collision (%). D differences are mode counts; A/C differences are percentage points. Brackets give paired pointwise 95% intervals for the paired change in requested temperature. Every fixed draw remains in its original denominator.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Kimi K3	1	A	92.50	22.81	-69.69 [-75.31, -63.44]
		C	66.23	53.61	-12.62 [-] [†]
	3	A	99.38	20.94	-78.44 [-84.06, -73.12]
		C	78.18	—	—
	All	A	96.67	21.35	-75.31 [-78.44, -72.19]
	All	C	72.20	—	—

tables and figure. This does not test temperature robustness of medium reasoning. A zero-temperature request does not establish deterministic outputs.

Each level averages five domains equally; All averages all 15 cells. Pointwise 95 percent intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, retaining all eight draws. There is no multiplicity adjustment. A zero-width empirical interval reflects constant observed groups, not certain success on unseen prompts.

Observed temperature frontier. At $T = 0.0$ and $T = 2.0$, normalized `pass@8` is 65.62% and 79.38%, respectively, while `distinct@8` is 0.673 and 1.146. Across these 5 measured temperatures, the highest observed aggregate `pass@8` occurs at $T = 1.5$ and the highest `distinct@8` occurs at $T = 2.0$. This finite exploratory sweep does not establish a population optimum or a continuous Pareto boundary, and level-specific curves differ.

Per-response accuracy changes from 62.11% to 55.94% between the endpoints. That diagnostic is distinct from `pass@8`: success within eight draws and the reliability of an individual draw need not change together as temperature varies.

Table 48: **GPT temperature frontier, both gradings.** P is solved prompts (% pass@8); D is distinct@8; A is per-response accuracy (%). G is the grading: S strict, N frozen formatting-normalized. Brackets give pointwise 95% intervals; all temperatures use reasoning none and eight draws per prompt.

T	Level	G	P [95% interval]	D [95% interval]	A [95% interval]
0.0	1	S	55.0 [48.8, 61.3]	0.569 [0.500, 0.637]	53.9 [47.7, 60.1]
	2		45.0 [39.4, 50.6]	0.456 [0.400, 0.512]	39.5 [34.4, 44.7]
	3		49.4 [42.5, 56.2]	0.519 [0.444, 0.594]	44.5 [38.3, 50.9]
	All		49.8 [46.2, 53.3]	0.515 [0.475, 0.554]	46.0 [42.6, 49.4]
0.5	1	S	70.6 [64.4, 76.9]	0.956 [0.850, 1.069]	57.5 [51.9, 63.0]
	2		50.0 [44.4, 55.6]	0.525 [0.463, 0.594]	38.6 [33.9, 43.4]
	3		54.4 [48.1, 60.6]	0.581 [0.506, 0.662]	43.0 [37.1, 48.9]
	All		58.3 [54.8, 61.9]	0.688 [0.640, 0.738]	46.4 [43.3, 49.5]
1.0	1	S	75.6 [69.4, 81.2]	1.181 [1.050, 1.312]	57.3 [52.0, 62.7]
	2		51.9 [45.6, 58.1]	0.581 [0.506, 0.662]	37.6 [33.2, 42.0]
	3		60.6 [54.4, 66.9]	0.713 [0.625, 0.806]	42.7 [37.3, 48.1]
	All		62.7 [59.2, 66.2]	0.825 [0.767, 0.885]	45.9 [42.9, 48.8]
1.5	1	S	80.6 [75.0, 86.2]	1.413 [1.269, 1.556]	56.6 [51.7, 61.4]
	2		58.1 [51.9, 64.4]	0.650 [0.569, 0.738]	38.4 [34.3, 42.7]
	3		65.6 [59.4, 71.2]	0.838 [0.731, 0.956]	43.0 [38.0, 48.2]
	All		68.1 [64.8, 71.5]	0.967 [0.900, 1.035]	46.0 [43.3, 48.8]
2.0	1	S	83.8 [78.1, 88.8]	1.519 [1.375, 1.663]	59.7 [54.8, 64.5]
	2		54.4 [48.1, 60.6]	0.600 [0.525, 0.675]	35.3 [31.4, 39.3]
	3		61.9 [56.2, 67.5]	0.819 [0.719, 0.925]	40.5 [35.8, 45.5]
	All		66.7 [63.3, 70.0]	0.979 [0.915, 1.044]	45.2 [42.6, 47.8]
0.0	1	N	70.6 [65.0, 76.2]	0.725 [0.662, 0.788]	70.5 [64.5, 76.2]
	2		60.6 [55.0, 66.2]	0.613 [0.556, 0.669]	54.5 [49.0, 59.9]
	3		65.6 [59.4, 71.9]	0.681 [0.606, 0.756]	61.4 [55.2, 67.6]
	All		65.6 [62.3, 69.0]	0.673 [0.635, 0.710]	62.1 [58.8, 65.5]
0.5	1	N	76.9 [71.2, 82.5]	1.038 [0.931, 1.150]	71.7 [66.4, 77.0]
	2		68.1 [62.5, 73.8]	0.706 [0.644, 0.769]	53.7 [48.8, 58.6]
	3		71.9 [65.6, 77.5]	0.775 [0.700, 0.850]	58.1 [52.3, 63.9]
	All		72.3 [69.2, 75.4]	0.840 [0.792, 0.890]	61.2 [58.2, 64.3]
1.0	1	N	79.4 [73.8, 84.4]	1.238 [1.106, 1.369]	70.4 [65.2, 75.5]
	2		69.4 [63.7, 75.0]	0.756 [0.681, 0.831]	51.6 [47.1, 56.2]
	3		76.9 [71.2, 82.5]	0.894 [0.806, 0.988]	55.2 [49.8, 60.7]
	All		75.2 [72.1, 78.3]	0.963 [0.904, 1.021]	59.1 [56.2, 62.0]
1.5	1	N	82.5 [77.5, 87.5]	1.494 [1.344, 1.644]	68.5 [63.7, 73.3]
	2		75.0 [69.4, 80.6]	0.844 [0.762, 0.931]	49.8 [45.5, 54.3]
	3		81.9 [76.2, 87.5]	1.038 [0.931, 1.156]	54.7 [49.6, 59.8]
	All		79.8 [76.7, 82.9]	1.125 [1.056, 1.196]	57.7 [54.9, 60.5]
2.0	1	N	85.0 [80.0, 90.0]	1.606 [1.456, 1.762]	69.6 [64.8, 74.3]
	2		72.5 [66.9, 78.1]	0.800 [0.731, 0.869]	45.7 [41.7, 49.8]
	3		80.6 [75.0, 86.2]	1.031 [0.931, 1.137]	52.5 [47.7, 57.5]
	All		79.4 [76.2, 82.5]	1.146 [1.079, 1.212]	55.9 [53.3, 58.6]

The paired $T = 2.0$ minus $T = 0.0$ contrast is 13.8 [10.8, 16.7] pass@8 percentage points and 0.473 [0.417, 0.529] distinct correct modes, with pointwise 95% intervals. All prompt-level records, domain estimates, original per-response and collision analyses, and paired comparisons remain in the retained reports for both strict and normalized grading.

Retained outcomes. At every temperature ($T = 0.0$, $T = 0.5$, $T = 1.0$, $T = 1.5$ and $T = 2.0$), 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. These outcome counts retain every registered draw and are separate from the verifier-based correctness and canonical-key measurements above.

Sensitivity to the prompt expansion. The original and additional prompt cohorts retain separate paired temperature comparisons. Differences between cohorts may reflect prompt composition or collection time; they are not causal time effects.

Across all 480 prompts, the paired $T = 2$ minus $T = 1.5$ contrast is -0.4 [-2.5, 1.7] pass@8 percentage points and 0.021 [-0.029, 0.071] distinct modes (95% intervals). Orig-

nal 120 prompts: the paired $T = 2$ minus $T = 1.5$ contrast is -1.7 [-5.0, 0.8] `pass@8` percentage points and -0.008 [-0.083, 0.067] distinct modes (95% intervals). Additional 360 prompts: the paired $T = 2$ minus $T = 1.5$ contrast is 0.0 [-2.8, 2.8] `pass@8` percentage points and 0.031 [-0.033, 0.094] distinct modes (95% intervals).

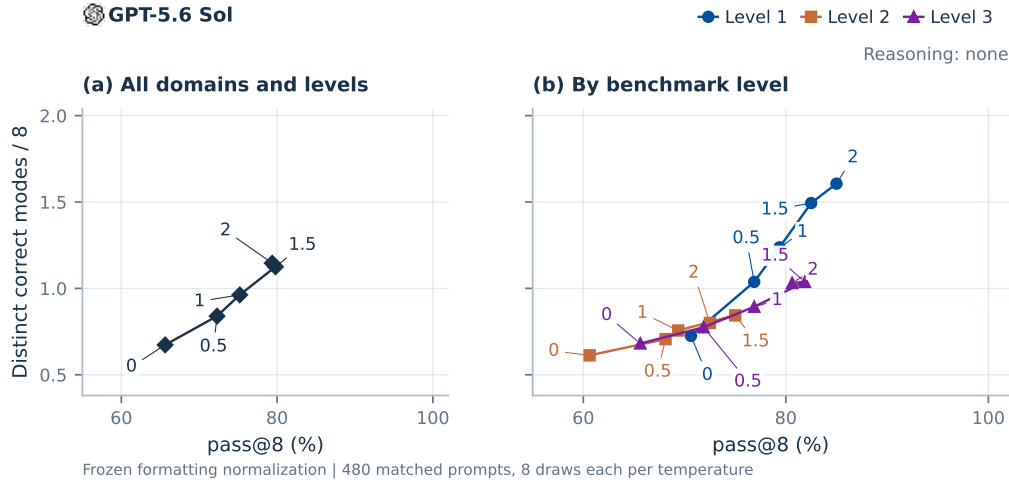


Figure 38: **Temperature traces `pass@8` and verified diversity.** GPT-5.6 Sol, with 480 fixed prompts and eight draws each at $T \in \{0, 0.5, 1, 1.5, 2\}$. Left: all domains and levels; right: separate levels, averaging five domains equally. Empirical `pass@8` is the fraction with at least one correct draw; the registered `distinct@8` averages unique correct modes, including zeroes, and so tracks `pass@8` across temperatures by construction. Connected points use reasoning `none` and the frozen formatting normalizer. App. O.3 gives pointwise uncertainty intervals for every benchmark level at each of the five measured temperatures, together with the per-domain values behind each plotted point.

O.4 NATIVE PROVIDER OUTCOMES AND UNDEFINED CONCENTRATION

Provider-declared refusals and filtering are deployment outcomes, not evidence that the underlying model cannot solve the mathematical task. Empty refusals remain failed draws under the frozen verifier; no requests or grades are overwritten. Classification uses native metadata, not a semantic detector of refusals written as ordinary answer text.

Correct-pair collision is undefined when no prompt supplies two correct draws. The five-domain mean is therefore undefined if any domain has no eligible pairs; such entries remain – rather than zero or a mean over fewer domains.

Native completion states, summed over every cell of each deployment. Empty visible answers are reported separately from provider refusals throughout.

- GPT-5.6 Sol — completed: 15,360
- Claude Opus 5 — end_turn: 12,838, refusal: 2,522
- GPT-5.4 — completed: 15,355, incomplete: 5
- Grok 4.3 — stop: 15,360
- Kimi K3 — length: 15, stop: 15,345
- Claude Opus 4.8 — end_turn: 15,355, max_tokens: 5
- DeepSeek V4 Pro — length: 632, stop: 14,728

Claude Opus 5 returns provider-declared refusals for 409, 1021, 1024 of the 1,024 Python requests at Levels 1, 2 and 3, respectively. Summed over those three levels, the native refusal categories on these Python draws are cyber: 2454. These are benign factor-selection tasks; the reported deployment policy must be distinguished from concentration among correct Python outputs.

Table 49: **Native service outcomes, every non-zero cell.** Each deployment answers 1,024 responses per level and domain. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades. Only cells with a non-zero counter are listed: the other 83 of the 105 are 0/0/0, including every cell for GPT-5.6 Sol, Grok 4.3, Kimi K3, whose service returned no refusal, filter or empty answer anywhere.

Deployment	L	Domain	R	F	E
Claude Opus 5	1	Python	409	0	409
	2	Python	1021	0	1021
	2	MathIR	35	0	35
	3	Python	1024	0	1024
GPT-5.4	3	MathIR	33	0	32
	1	PantryPlan	0	0	1
	2	Countdown	0	0	1
	3	Countdown	0	0	1
Claude Opus 4.8	3	PantryPlan	0	0	2
	3	Countdown	0	0	3
	3	PantryPlan	0	0	1
DeepSeek V4 Pro	1	Graph	0	0	59
	1	MathIR	0	0	4
	1	PantryPlan	0	0	13
	2	Graph	0	0	38
	2	Countdown	0	0	1
	2	MathIR	0	0	59
	2	PantryPlan	0	0	80
	3	Graph	0	0	24
	3	Countdown	0	0	7
	3	MathIR	0	0	100
	3	PantryPlan	0	0	246

Table 50: **Strict level summaries for completed hosted deployments.** Each level weights the five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is correct-pair collision (%; with pointwise 95% whole-prompt intervals), pair weighted within each domain. Provider settings differ; these are descriptive outputs.

Deployment	Level	M	P	D	C (95% interval)
GPT-5.6 Sol	1	83.0	95.8	1.71	74.0 [71.9, 76.1]
GPT-5.6 Sol	2	62.9	77.5	1.16	82.3 [80.3, 84.4]
GPT-5.6 Sol	3	68.7	80.3	1.18	84.5 [82.8, 86.2]
Claude Opus 5	1	85.3	94.5	1.32	86.3 [84.5, 88.0]
Claude Opus 5	2	76.6	80.2	1.16	—
Claude Opus 5	3	77.8	79.8	1.07	—
GPT-5.4	1	93.7	98.1	2.09	67.2 [65.1, 69.3]
GPT-5.4	2	80.6	97.7	1.61	74.7 [72.4, 76.8]
GPT-5.4	3	83.3	97.8	1.57	78.8 [76.9, 80.7]
Grok 4.3	1	98.3	99.5	1.97	72.4 [70.7, 74.0]
Grok 4.3	2	99.3	100.0	1.57	82.1 [80.6, 83.7]
Grok 4.3	3	98.8	100.0	1.55	83.0 [81.5, 84.6]
Kimi K3	1	96.9	98.4	2.31	62.5 [60.6, 64.4]
Kimi K3	2	96.8	100.0	2.05	69.4 [67.6, 71.2]
Kimi K3	3	97.8	100.0	1.90	74.6 [73.1, 76.2]
Claude Opus 4.8	1	96.4	98.8	1.64	78.4 [76.6, 80.3]
Claude Opus 4.8	2	98.5	100.0	1.67	77.2 [75.4, 79.1]
Claude Opus 4.8	3	98.9	99.8	1.51	83.4 [81.8, 84.9]
DeepSeek V4 Pro	1	96.4	98.4	2.01	71.0 [69.4, 72.7]
DeepSeek V4 Pro	2	93.7	99.1	1.84	68.7 [67.0, 70.3]
DeepSeek V4 Pro	3	91.0	99.7	1.62	78.8 [76.9, 80.9]

Table 51: **Strict executable grades, every hosted deployment.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Deployment	Level	Domain	M	P	D	C (95% interval)	U	
GPT-5.6 Sol	1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3	
	1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–	
	1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8	
	1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0	
	1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5	
	2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9	
	2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–	
	2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7	
	2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0	
	2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9	
	3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0	
	3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–	
	3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9	
	3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0	
	3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8	
	Claude Opus 5	1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
		1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
		1	Python	38.1	81.2	0.91	93.0 [88.7, 96.6]	0.7
		1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
		1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2		Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9	
2		Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–	
2		Python	0.3	2.3	0.02	–	–	
2		MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0	
2		PantryPlan	94.5	100.0	1.56	80.7 [76.0, 85.3]	6.4	
3		Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0	
3		Countdown	99.9	100.0	1.66	76.2 [71.7, 80.7]	–	
3		Python	0.0	0.0	0.00	–	–	
3		MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0	
3		PantryPlan	93.6	100.0	1.63	78.8 [74.2, 83.4]	6.5	
GPT-5.4		1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
	1	Countdown	91.8	100.0	1.73	76.6 [71.2, 81.3]	–	
	1	Python	87.6	100.0	2.52	56.4 [51.3, 61.5]	1.4	
	1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0	
	1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5	
	2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0	
	2	Countdown	91.1	100.0	1.98	63.0 [58.6, 67.5]	–	
	2	Python	47.9	96.1	1.47	72.9 [64.6, 80.2]	1.0	
	2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0	
	2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8	
	3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0	
	3	Countdown	82.2	93.8	1.72	69.1 [64.4, 73.5]	–	
	3	Python	50.1	97.7	1.58	74.4 [68.6, 79.8]	1.1	
	3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0	
	3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6	
	Grok 4.3	1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1		Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–	
1		Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4	
1		MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0	
1		PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5	
2		Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0	
2		Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–	
2		Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1	
2		MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0	
2		PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5	
3		Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0	
3		Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–	
3		Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2	
3		MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0	
3		PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5	
Kimi K3		1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
	1	Countdown	99.1	100.0	1.91	68.4 [63.8, 73.3]	–	
	1	Python	97.9	100.0	2.52	57.7 [53.5, 61.9]	1.4	
	1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0	
	1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5	
	2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0	
	2	Countdown	98.4	100.0	1.98	64.4 [59.9, 68.7]	–	
	2	Python	98.2	100.0	2.44	61.9 [57.6, 66.2]	1.1	
	2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0	
	2	PantryPlan	93.3	100.0	3.20	41.3 [36.6, 45.9]	6.6	
	3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0	
	3	Countdown	97.5	100.0	1.84	70.6 [66.1, 74.8]	–	
	3	Python	98.0	100.0	2.34	64.7 [60.1, 69.0]	1.2	
	3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0	

Strict executable grades, continued.

Deployment	Level	Domain	M	P	D	C (95% interval)	U
Claude Opus 4.8	3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5
	1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
	1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
	1	Python	98.0	100.0	1.40	85.7 [81.7, 89.4]	1.4
	1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
	1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
	2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
	2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
	2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
	2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
	2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
	3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
	3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
	3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
	3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
	3	PantryPlan	98.5	100.0	2.33	61.6 [56.7, 66.6]	6.5
	1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
	1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
DeepSeek V4 Pro	1	Python	99.3	100.0	1.41	88.8 [86.1, 91.4]	1.4
	1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
	1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
	2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
	2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
	2	Python	99.6	100.0	1.05	98.4 [97.1, 99.4]	1.2
	2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
	2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
	3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
	3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
	3	Python	99.6	100.0	1.08	97.8 [96.3, 99.0]	1.2
	3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
	3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

Table 52: **Post hoc formatting diagnostic: the cells it changes.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%). Only cells whose normalized grade differs from its strict grade appear; every cell absent here is identical under both gradings. The normalizer rescued no response for Grok 4.3.

Deployment	Level	Domain	M	P	D	C (95% interval)	U
GPT-5.6 Sol	1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
	1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
	2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
	2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
	2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
	3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
Claude Opus 5	3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
	3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7
	1	Python	60.0	94.5	1.12	90.4 [86.2, 94.3]	0.9
	2	PantryPlan	99.9	100.0	1.61	79.7 [75.0, 84.3]	6.5
	3	Countdown	100.0	100.0	1.66	76.1 [71.7, 80.7]	–
	3	PantryPlan	99.4	100.0	1.70	77.3 [72.6, 82.0]	6.5
GPT-5.4	1	Countdown	99.7	100.0	1.80	73.6 [68.5, 78.3]	–
	1	Python	98.7	100.0	2.57	56.8 [51.9, 61.6]	1.4
	2	Countdown	98.4	100.0	2.12	61.1 [57.0, 65.1]	–
	2	Python	91.3	100.0	1.87	74.0 [69.4, 78.1]	1.1
	3	Countdown	99.0	100.0	1.96	68.4 [64.2, 72.7]	–
	3	Python	91.6	100.0	1.91	72.6 [68.3, 77.1]	1.2
Kimi K3	1	Countdown	99.4	100.0	1.91	68.4 [63.9, 73.3]	–
	1	Python	98.2	100.0	2.53	57.7 [53.5, 61.9]	1.4
	2	Countdown	98.7	100.0	1.98	64.5 [60.1, 68.9]	–
	2	Python	98.3	100.0	2.44	61.9 [57.6, 66.2]	1.1
	2	PantryPlan	93.4	100.0	3.20	41.4 [36.7, 46.0]	6.6
	3	Countdown	98.1	100.0	1.84	70.3 [65.8, 74.5]	–
Claude Opus 4.8	3	Python	98.5	100.0	2.34	64.9 [60.3, 69.2]	1.2
	1	Python	98.6	100.0	1.40	85.8 [81.8, 89.5]	1.5
	3	PantryPlan	98.7	100.0	2.33	61.7 [56.7, 66.6]	6.5
DeepSeek V4 Pro	1	Python	99.4	100.0	1.41	88.8 [86.1, 91.4]	1.4
	2	Python	99.7	100.0	1.05	98.4 [97.1, 99.4]	1.2
	3	Python	99.7	100.0	1.08	97.8 [96.3, 99.0]	1.2

The result record `results/frontier_comparison_20260911.json` binds every source summary, completion audit, frozen prompt digest, raw-sample digest and primary graded sample digest. Pending deployments are omitted from both the figure and numerical comparisons. The figure was selected after the reference GPT run for its high strict correctness, certified support and lack of formatting corrections, and before collecting the prospective comparison runs. All five domains are retained in the tables, including disagreement across models.

Model	Level 1			Level 2			Level 3		
	pass@8 (%)	# modes	PCMD	pass@8 (%)	# modes	PCMD	pass@8 (%)	# modes	PCMD
GPT-5.6 Sol	98.0	1.80	.256	98.4	1.50	.229 [†]	99.2	1.45	.183 [†]
Claude Opus 5	98.3 [‡]	1.43 [‡]	.140	99.7 [‡]	1.44 [‡]	.164 [†]	99.8 [‡]	1.36 [‡]	.121 [†]
GPT-5.4	98.1	2.11	.338	98.4	1.72	.247	99.5	1.68	.224
Grok 4.3	99.5	1.97	.275	100.0	1.57	.179	100.0	1.55	.171
Kimi K3	98.4	2.31	.374	100.0	2.05	.307	100.0	1.90	.253
Claude Opus 4.8	98.8	1.64	.217	100.0	1.67	.229	99.8	1.51	.168
DeepSeek V4 Pro	98.4	2.01	.289	99.1	1.84	.319	99.7	1.62	.232

Table 53: **Hosted success and output diversity with medium reasoning, averaged by level.** Each level averages five domains equally, using 128 prompts per domain and eight responses per prompt. `pass@8` is the fraction with any verified response; # modes is mean `distinct@8` over all prompts, with zero modes for every prompt lacking a verified response. Both of those fall when a deployment simply succeeds less often, so PCMD carries the success-conditional axis beside them: the mean probability that two verified draws on the same prompt differ, averaged over prompts and then over domains. It is repeated here from Table 44, which reads the same draws, and a cell enters it only from the domains where at least thirty prompts return two verified draws: [†] marks the four cells that average fewer domains than the five, each of them missing Python to provider refusals. [‡] marks the three Claude Opus 5 cells whose Python column is the revised-wording cohort of App. O.1 rather than the original benchmark prompt the other deployments answer; the PCMD column is the original prompt throughout. Table 54 repeats every deployment over the four domains that share one prompt.

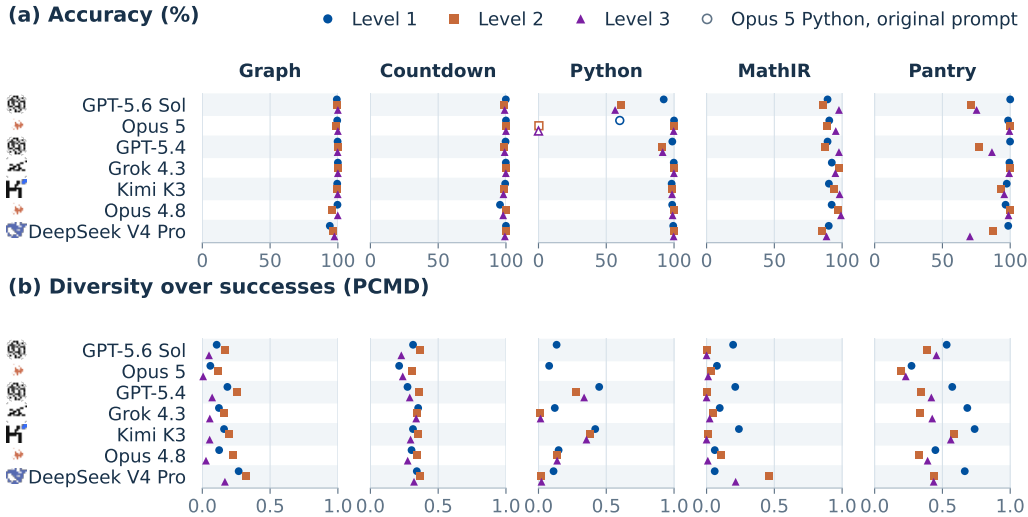


Figure 39: **Accurate hosted outputs can still cover few verified modes.** Columns show domains; markers distinguish Levels 1–3. Each point uses all eight responses to each of 128 prompts. Top: per-response accuracy. Bottom: PCMD, which conditions on success and so separates diversity among correct answers from accuracy itself. A fixed formatting normalizer applies to every deployment; Opus 5 Python uses revised task wording without a system message. Hollow marks in that one column repeat the original-prompt cells the revised ones replace, on the wording the other six deployments answer. This is the per-domain and per-level reading for every audited deployment. Table 53 reports the corresponding per-level averages of `pass@8` and `distinct@8` across all five domains, beside the PCMD of the same draws. Table 54 repeats the first two over the four domains that share one prompt.

Evaluation parity across deployments. One column of this cohort rests on wording the other deployments did not answer: Claude Opus 5’s Python cells in Fig. 39 and Table 53 use the revised condition of App. O.1. Every other reading in this paper holds one prompt across all seven deployments, including the PCMD column beside them, Table 44 and the strict and normalized cell tables above. Table 54 therefore repeats the comparison over Graph, Countdown, MathIR and Pantry alone, the four domains that share one prompt. Claude Opus 5 averages 1.42/1.45/1.35 verified modes there at Levels 1–3, against 1.43/1.44/1.36 over five domains with the revised Python cell and 1.36/1.17/1.08 with the original one. It is the least varied deployment at every level under all three readings, and the revised wording raises its Python diversity rather than lowering it, so the substituted cell moves the five-domain reading away from concentration rather than towards it. Which of the other six follows it varies with Python in or out, as their per-domain disagreement in Table 44 already reports; what survives every reading is the level, not the order. How far a Python wording change can carry this domain is itself measured, on three of the other six: App. K.4 answers Python under two wordings at 16 prompts and 64 draws, and pooled PCMD moves by between +.024 and +.131 across its six deployment-level cells, leaving every one of them concentrated under both wordings (highest PCMD .369, $N_{\text{eff}}^{(2)}$ 1.58). That bounds how far wording carries this domain; it does not identify what this particular revision would do to a deployment other than the one that answered it.

Model	Level 1		Level 2		Level 3	
	pass@8 (%)	# modes	pass@8 (%)	# modes	pass@8 (%)	# modes
GPT-5.6 Sol	97.5	1.90	98.2	1.59	99.4	1.54
Claude Opus 5	97.9	1.42	99.6	1.45	99.8	1.35
GPT-5.4	97.7	2.00	98.0	1.68	99.4	1.62
Grok 4.3	99.4	2.12	100.0	1.70	100.0	1.68
Kimi K3	98.0	2.26	100.0	1.95	100.0	1.79
Claude Opus 4.8	98.4	1.70	100.0	1.75	99.8	1.56
DeepSeek V4 Pro	98.0	2.16	98.8	2.04	99.6	1.75

Table 54: **Hosted success and output diversity with Python excluded.** The same draws as Table 53, averaged over Graph, Countdown, MathIR and Pantry alone: the four domains every deployment answers under one prompt, 128 prompts each and eight responses per prompt. Reading this table beside that one shows what the revised Python condition of App. O.1 carries, and what it does not.

O.5 THE REVISED PYTHON WORDING ON A SECOND DEPLOYMENT

App. O.1 substitutes a revised Python wording for one deployment. What that wording does to any *other* deployment is measured here, on the one we can reach directly. Both arms were collected together against `api.openai.com`: the published GPT-5.6 Sol cells were served by Azure, which may be a different build, so re-collecting only the revised arm would have confounded wording with deployment. The original arm reuses the published requests byte for byte, the revised arm reuses the wording of App. O.1, and both run over the same 384 frozen rows with eight draws each. This condition replaces no published cell and enters no cohort table or macro average.

The refusals are a property of the deployment. Across all 6,144 responses this deployment declares no refusal in either arm, including the original wording at Levels 2 and 3, where the Azure deployment refused most of the same frozen requests. The refusals the hosted appendix reports are therefore a property of the served deployment rather than of the task wording or the model, and this condition can compare the two wordings without one arm resting on responses the provider declined to give.

The revised wording costs this deployment on every axis. Table 55 reports both arms. Under the frozen formatting normalizer the revised wording lowers accuracy, `distinct@8` and PCMD at every level; the paired differences, resampling whole eight-draw prompt groups across arms (20,000 replicates), are accuracy $-.689 [-.745, -.631]$, $-.420 [-.467, -.374]$, $-.370 [-.418, -.320]$ at Levels 1–3; `distinct@8` $-.945 [-1.078, -.820]$, $-.703 [-.812, -.594]$, $-.656 [-.766, -.547]$ respectively. The revised arm clears the 30-prompt support bar on none of its three cells, so its PCMD is not reported: the wording does not merely move this deployment’s diversity, it removes enough verified draws to stop diversity being estimable at all.

Most of that cost is a surface the grader cannot parse. This deployment often wraps its conditional in the `\text{}` macro, which no rule in the frozen normalizer undoes, so such a response is rejected whatever expression it contains: 862 and 2,089 responses carry it in the two arms and not one of them verifies. The revised wording raises that rate from 28% to 68%. Among the responses that are parseable at all the two arms verify at 94.9% and 59.3%. That subset is not a random sample of either arm, so it bounds rather than identifies a content effect; what the table measures is scoreable diversity under a fixed grader, not demonstrated capability.

What this settles. The revised wording is a repair for a provider refusal, not an easier task. On a deployment that refuses neither wording it is worse on both registered endpoints, so the substituted cell in Fig. 39 and Table 53 does not flatter the deployment that answered it. It also shows what evaluating the whole cohort on the revised wording would have cost: on this deployment it removes most scoreable Python output and drops the domain below the support bar, which is why the parity reading in Table 54 sets Python aside rather than re-wording it.

Table 55: GPT-5.6 Sol Python under two wordings, one deployment. Both arms served by `api.openai.com` over the same 384 frozen rows, 128 prompts per level and eight draws per prompt. R counts provider-declared refusals; A is per-response accuracy (%), D is `distinct@8`, and PCMD is reported only where at least 30 prompts return two verified draws, as everywhere else in this paper. Strict and normalized columns share the frozen grader and typography rule. These cells replace no published GPT-5.6 Sol result and enter no cohort table, per-level average or macro average; every response here reports the served model string `gpt-5.6-sol`, which the Azure deployment reports with a dated snapshot suffix, so the two builds are compared nowhere in this paper.

Wording	L	R	Strict			Normalized		
			A	D	PCMD	A	D	PCMD
Original	1	0	37.0	1.02	.106	89.8	1.36	.114
	2	0	10.4	0.14	—	58.7	1.15	.053
	3	0	13.0	0.16	—	56.3	1.13	.048
Revised, no system	1	0	18.7	0.27	—	20.9	0.41	—
	2	0	13.4	0.26	—	16.7	0.45	—
	3	0	15.5	0.25	—	19.3	0.48	—

Model	Level 1			Level 2			Level 3		
	pass@8 (%)	# modes	PCMD	pass@8 (%)	# modes	PCMD	pass@8 (%)	# modes	PCMD
GPT-5.6 Sol	82.5	1.31	.226	66.9	0.76	.063	76.2	0.88	.095
GPT-5.4	82.5	1.67	.347	71.2	0.92	.134	70.0	0.94	.160
Grok 4.3	82.5	1.64	.493	86.9	1.41	.296	90.0	1.45	.314
Kimi K3	92.5	1.91	.595	92.5	1.37	.335	95.6	1.59	.361
Claude Opus 4.8	91.2	1.46	.216	100.0	1.62	.226	100.0	1.51	.185

Table 56: Hosted success and output diversity with reasoning disabled. Each level averages five domains equally: 32 prompts per domain and eight responses per prompt. `pass@8` is the fraction with any verified response; `# modes` is mean `distinct@8`, including failures. Both fall when the model simply succeeds less often, so PCMD carries the success-conditional axis beside them: it is the mean probability that two verified draws on the same prompt differ, over the prompts that return two verified draws in this condition and in the matched medium condition of Table 57, which is the population every PCMD reading in this appendix uses. The frozen formatting normalizer applies throughout. Appendix O.6 gives matched medium-reasoning results.

O.6 MATCHED REASONING-CONTROL COMPARISON

We selected original row indices 0–31 in each of five domains and three levels, without consulting outputs. Each model and condition therefore has 480 prompts and 3,840 responses. The medium baseline reuses the original audited responses for these exact prompts and eight sample slots; no new medium responses were collected. Both conditions preserve the original system and user prompt bytes, including the original Python wording. The 128-prompt medium results in Table 53 and Figure 39 are a separate, larger display; the comparison below uses only the matched 32-prompt subset.

Only the provider reasoning control changes, with an 8,192-token output budget retained in both conditions. GPT-5.6 Sol and GPT-5.4 request `reasoning.effort=none`; Grok 4.3 and Kimi K3

request reasoning_effort=none. Claude Opus 4.8 requests thinking.type=disabled while retaining output_config.effort=medium. These controls denote disabled explicit reasoning at the endpoint, without establishing the absence of hidden internal computation. Native response receipts are checked for control violations. The same frozen formatting normalizer and canonical mode grader apply to both conditions; the accompanying JSON retains strict grades, domain-level counts, controls, and source hashes.

Model	Level	Medium reasoning			Reasoning disabled		
		pass@8 (%)	# modes	PCMD	pass@8 (%)	# modes	PCMD
GPT-5.6 Sol	1	96.9	1.81	.311	82.5	1.31	.226
	2	98.8	1.54	.117	66.9	0.76	.063
	3	98.8	1.51	.195	76.2	0.88	.095
	All	98.1	1.62	.226	75.2	0.99	.145
GPT-5.4	1	96.9	2.06	.338	82.5	1.67	.347
	2	99.4	1.68	.162	71.2	0.92	.134
	3	99.4	1.82	.247	70.0	0.94	.160
	All	98.5	1.85	.255	74.6	1.18	.224
Grok 4.3	1	100.0	1.93	.211	82.5	1.64	.493
	2	100.0	1.62	.136	86.9	1.41	.296
	3	100.0	1.54	.121	90.0	1.45	.314
	All	100.0	1.70	.154	86.5	1.50	.364
Kimi K3	1	96.9	2.27	.343	92.5	1.91	.595
	2	100.0	2.02	.277	92.5	1.37	.335
	3	100.0	1.89	.264	95.6	1.59	.361
	All	99.0	2.06	.293	93.5	1.62	.426
Claude Opus 4.8	1	98.1	1.61	.202	91.2	1.46	.216
	2	100.0	1.67	.218	100.0	1.62	.226
	3	100.0	1.57	.185	100.0	1.51	.185
	All	99.4	1.62	.202	97.1	1.53	.209

Table 57: Matched medium and disabled reasoning, by level. Each row averages 160 prompts across five domains. All eight responses and zero-success prompts are retained. # modes denotes mean distinct@8; estimates use the frozen formatting normalizer. PCMD is paired within the level: it averages over the prompts of that level which return two verified draws in both conditions, between 65 and 159 of the 160 per cell, and every level cell here clears the thirty-prompt support bar. The pass@8 and # modes columns use all 160 prompts, so the two axes are read on the same draws but not on the same denominators, and the paired count behind every level cell is retained in the accompanying machine-readable record alongside the per-domain counts. The All row averages the same 480 prompts and 3,840 responses per condition with equal weight on every domain and level; its PCMD is the paired reading over the 245–458 prompts per deployment that return two verified draws in both conditions. That column separates concentration from lost accuracy, since a prompt one condition stopped answering twice leaves the estimate rather than entering it as a smaller number; the per-domain readings behind it are retained in the accompanying JSON, together with the prompts that only one of the two conditions answers twice and so enter neither average.

Incomplete deployments. Five of seven registered deployments passed complete-cohort admission, and neither partial cohort receives a score. They fall short in different ways, and that decides what can still be read from each.

DeepSeek V4 Pro’s draws are intact, so its measurement survives its exclusion and is reported here: read on the 437 prompts both of its conditions define, disabling reasoning moves PCMD from .282 to .369, a change of +.087. What that figure cannot do is join the table, for two reasons and neither is the recovery shortfall: both of its arms are graded strictly, because the frozen normalizer was only ever run on admitted cohorts, and one PantryPlan prompt holds seven draws rather than eight. A caveat cannot put a strictly graded number on the same scale as a normalized one. The shortfall itself is slight — 3,839 of 3,840 sample receipts were recovered, leaving one unresolved

request, which is not counted as a failure — and it is not what keeps the deployment out of the table. The figure is reported so the evidence is not lost, not so it can be compared.

Claude Opus 5 admits no such reading. A native response returned a thinking block and 40 thinking tokens despite an explicit disabled-thinking request, so its disabled condition did not disable reasoning. There is no reasoning-off measurement to preserve, rather than one withheld.

Scope of the comparison. Across all five complete deployments, disabling reasoning lowers both overall `pass@8` and observed `distinct@8`. Reduced success also reduces opportunities to observe distinct successful modes, so those two columns cannot separate concentration from lost accuracy. PCMD can, and the answer is not uniform: on the prompts both conditions define, diversity falls for GPT-5.6 Sol and GPT-5.4, is unchanged for Claude Opus 4.8, and rises for Kimi K3 and Grok 4.3, which more than doubles from .154 to .364 while its `distinct@8` falls. Disabling deliberation concentrates some deployments and spreads others, so the common fall in `distinct@8` tracks the accuracy drop rather than a shared collapse. Table 57 reads the same contrast at each level, and the direction belongs to the deployment rather than to the level: GPT-5.6 Sol falls at every level, Grok 4.3 and Kimi K3 rise at every level, GPT-5.4 is flat at Level 1 and falls at Levels 2 and 3, and Claude Opus 4.8 stays within .014 of zero throughout. No deployment reverses its own direction from one level to the next, and no level cell loses the measurement: every one clears the support bar on prompts both conditions answer twice. Medium and disabled outputs were collected at different times; matching sample slots are not shared random seeds. Omitted temperature and `top_p` controls retain provider defaults, which can vary over time. These are descriptive point estimates without uncertainty intervals, not causal claims about training.

Part VI. Idealized analysis

P MODE COLLAPSE AND VERIFIED REPLAY

We separate the per-prompt mean gradient from the geometry and noise of its updates. Outcome-binary score estimators share a correctness-gradient direction for differentiable neural policies, while their covariance can differ. The categorical specialization yields finite-step concentration and sampled symmetry breaking; fixed-bank replay has an explicit restoration threshold and recovery bound. Complete-response and local-update arguments state conditions for neural retention. These results do not certify the implemented AdamW/PPO trajectories. Table 2 lists the symbols used here.

P.1 CATEGORICAL ABSTRACTION AND MEAN UPDATES

The categorical results use the following model. The neural score identities, finite-step updates, and sampled-update results below state separately which assumptions they replace; none identifies a neural mode with an independently optimized logit.

Assumption P.1 (Categorical mean-flow model). Fix one isolated prompt and a finite category set $\mathcal{A} = \mathcal{C} \dot{\cup} \mathcal{N}$ with $m \geq 2$ correct execution modes and $n \geq 1$ incorrect categories, and set $p_a = \text{softmax}(z)_a$, $P = \sum_{c \in \mathcal{C}} p_c$, and $q_c = p_c/P$, so that P measures correctness while q describes how that correct probability is allocated among the individual correct modes.

- (A1) *Independent logits*. The policy carries one independently optimized logit for every category in $\mathcal{A}$, including one separate logit for each canonical correct mode.
- (A2) *Common length normalization*. Every response shares a single length scale, which is absorbed into the positive coefficient of the group estimator, rather than a scale that varies from one response to the next within the same rollout group.
- (A3) *Euclidean mean flow*. Updates are the infinitesimal expected updates in these logits, taken without optimizer preconditioning and without momentum carried between steps.
- (A4) *No competing regularizer*. Explicit reference KL is zero and no entropy term is present.
- (A5) *Nondegenerate start*. Logits are finite initially, $0 < P(0) < 1$, and the group size is $G \geq 2$.

We write “under Assumption P.1” for (A1)–(A5) together. Each limitation of the analysis can be traced to a specific item: AdamW and PPO violate (A3), variable-length GRPO violates (A2), and entropy regularization violates (A4).

A language model induces probabilities over canonical outcomes, but this identification of probabilities does not identify its update geometry. Aggregating several response strings into a mode does not generally commute with gradient flow in response logits. One independent logit per canonical outcome is therefore an explicit abstraction, not a reduction proved for the neural policy or its validator equivalence classes.

Draw a group $Z_1, \dots, Z_G \stackrel{\text{iid}}{\sim} p$ with $G \geq 2$, write $R_i = \mathbf{1}[Z_i \in \mathcal{C}]$, and let $R = \sum_i R_i$. At the on-policy point where the PPO ratio equals one, consider the common-length estimator

$$\hat{g}_G = \frac{1}{G} \sum_{i=1}^G w(R) \left(R_i - \frac{R}{G} \right) \nabla_z \log p_{Z_i}. \quad (14)$$

For the Dr.GRPO abstraction, $w(R) = 1$ after absorbing the shared positive length scale. Reward-standardized GRPO with equal or common response-length normalization takes $w(R)$ to be the reciprocal group reward standard deviation on mixed groups (with a fixed numerical stabilizer if used). Standard variable-length GRPO also weights each response by its own inverse length (Shao et al., 2024; Liu et al., 2025); that factor cannot generally be absorbed into $w(R)$ and is outside the equivalence proved here. In either included case, $0 < w(\ell) < \infty$ for $\ell \in \{1, \dots, G-1\}$. We define the entire update as zero on all-equal groups without evaluating an undefined reciprocal standard deviation; likewise, $w(R)R(G-R)$ below is defined as zero when $R = 0$ or $R = G$. All advantages and normalization weights are detached in the score estimator.

Lemma P.2 (A group baseline does not remove frequency amplification). *For $0 < P < 1$, the expected update in Equation (14) is*

$$\mathbb{E}[\hat{g}_G] = c_G(P) \nabla_z P, \quad c_G(P) = \frac{\mathbb{E}[w(R)R(G-R)]}{G^2 P(1-P)} > 0. \quad (15)$$

For the Dr.GRPO abstraction, $c_G(P) = (G-1)/G$. Group centering and reward standardization rescale the expected gradient without changing its direction under these assumptions.

Proof. Condition on the complete binary reward vector, whose sum is R . The score identities give

$$\mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 1] = \nabla_z \log P, \quad \mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 0] = \nabla_z \log(1-P).$$

There are R correct rows with centered reward $(G-R)/G$ and $G-R$ incorrect rows with centered reward $-R/G$. Therefore the conditional expectation of the unaveraged sum in (14) is

$$w(R) \frac{R(G-R)}{G} (\nabla_z \log P - \nabla_z \log(1-P)) = w(R) \frac{R(G-R)}{GP(1-P)} \nabla_z P.$$

The outer factor $1/G$ and expectation over $R \sim \text{Binomial}(G, P)$ yield (15). For $w \equiv 1$, $\mathbb{E}[R(G-R)] = G(G-1)P(1-P)$, proving the Dr.GRPO value. $\square$

The binomial identity also gives the denominator-free expression

$$c_G(P) = \frac{G-1}{G} \sum_{j=0}^{G-2} \binom{G-2}{j} w(j+1) P^j (1-P)^{G-2-j}.$$

The binomial weights sum to one, so c_G lies between $(G-1) \min_j w(j+1)/G$ and $(G-1) \max_j w(j+1)/G$. This establishes the smooth, positive extension to $P = 0, 1$ used in the subsequent flow and convergence arguments.

The next lemma specializes the practical-estimator analysis of Tajwar et al. (2026, Section 4.3 and App. D, Theorem 5) to our categorical notation; we include the derivation for completeness.

Lemma P.3 (Binary MaxRL has the same correctness-gradient direction). *Under Assumption P.1, let MaxRL assign $A_i^{\text{MaxRL}} = GR_i/R - 1$ when $R > 0$ and zero when $R = 0$, as in Section 3. For $\hat{g}_G^{\text{MaxRL}} = G^{-1} \sum_i A_i^{\text{MaxRL}} \nabla_z \log p_{Z_i}$ at the on-policy point, the expected update is*

$$\mathbb{E}[\hat{g}_G^{\text{MaxRL}}] = c_G^{\text{MaxRL}}(P) \nabla_z P, \quad c_G^{\text{MaxRL}}(P) = \frac{1 - (1-P)^{G-1}}{P} > 0, \quad (16)$$

with continuous endpoint values $c_G^{\text{MaxRL}}(0) = G-1$ and $c_G^{\text{MaxRL}}(1) = 1$.

Proof. Condition on $R = r > 0$. Each successful row has advantage $(G-r)/r$ and each failed row has advantage -1 . Using the same conditional score identities as above, the expected unaveraged sum is

$$(G-r)(\nabla_z \log P - \nabla_z \log(1-P)) = \frac{G-r}{P(1-P)} \nabla_z P.$$

After the outer factor $1/G$ and averaging over $R \sim \text{Binomial}(G, P)$,

$$\mathbb{E}[(G-R)\mathbf{1}\{R>0\}] = G(1-P) - G\Pr(R=0) = G[(1-P) - (1-P)^G].$$

Substitution gives Equation (16). Its endpoint values follow by continuity (or the finite geometric-series identity), which also gives the stated continuous extension at both boundaries. $\square$

The same finite-series identity gives the potential used below:

$$\Psi_{\text{MaxRL}}(P) := \int_0^P c_G^{\text{MaxRL}}(s) ds = \sum_{k=1}^{G-1} \frac{1 - (1-P)^k}{k}. \quad (17)$$

In particular, $\Psi_{\text{MaxRL}}(1) = \sum_{k=1}^{G-1} 1/k$. For binary rewards, $\text{best}@k$ equals $\text{pass}@k$, so this is a harmonic mixture of sampling objectives for $k = 1, \dots, G-1$. The upper limit is $G-1$, not G : the implemented centered estimator drops the entire update on all-failure groups. Subtracting an unconditional score baseline even on those groups instead gives the order- G estimator. This distinction concerns the exact finite-group expectation at the on-policy point; it does not assert unbiasedness of subsequent clipped or adaptive-optimizer updates.

P.2 WINNER-TAKE-ALL COLLAPSE IN THE CATEGORICAL MEAN FLOW

Consider $\dot{z} = c_G(P)\nabla_z P$, with any of the smooth positive coefficients established above: Dr.GRPO, common-length reward-standardized GRPO, or the specified centered binary MaxRL estimator. This infinitesimal expected-update model removes sampling noise and retains independently trained Euclidean logits. The conditional dynamics reduce to the standard frequency-dependent replicator system with fitness $f_c(q) = q_c$ (Hofbauer & Sigmund, 1998). The result is a specialization to the group estimators, not a new general theorem about replicator dynamics. Related correctness-indifferent collapse analyses include Sinha et al. (2026); Lochab et al. (2026).

Theorem P.4 (Standard replicator specialization for binary-reward RL). *Under Assumption P.1, if the conditional correct-mode distribution $q(0)$ has a unique largest coordinate c^* , then*

$$P(t) \longrightarrow 1, \quad q_{c^*}(t) \longrightarrow 1, \quad q_c(t) \longrightarrow 0 \quad (c \neq c^*).$$

Thus these categorical mean flows converge to a single correct execution mode for generic initial conditions, even though every mode in $\mathcal{C}$ has exactly the same reward.

Remark P.5. Theorem P.4 is asymptotic and internal to (A1)–(A3): it claims neither finite-time support loss nor convergence of the stochastic neural optimizers used in our experiments.

Proof. The coefficient c_G is smooth and bounded on $[0, 1]$, and the softmax reward gradient is bounded. Thus the logit flow exists for all finite times and cannot develop infinite logits at finite time. For a correct mode c and incorrect category a ,

$$\dot{z}_c = c_G(P)p_c(1 - P), \quad \dot{z}_a = -c_G(P)p_a P.$$

Also $\dot{P} = c_G(P)\|\nabla_z P\|_2^2 > 0$. If P converged to an interior value, continuity and positivity of c_G , together with

$$\|\nabla_z P\|_2^2 \geq P^2(1 - P)^2 \left(\frac{1}{m} + \frac{1}{n} \right),$$

would keep $\dot{P}$ bounded away from zero, a contradiction. Hence $P \rightarrow 1$.

Define the effective optimization time

$$\tau(t) = \int_0^t c_G(P(s))P(s)(1 - P(s)) ds.$$

If $\tau(\infty) < \infty$, every correct and incorrect logit would have finite total variation, because the absolute velocity of each is at most $\dot{\tau}$. All logits would then converge to finite values, contradicting $P \rightarrow 1$. Therefore $\tau(t) \rightarrow \infty$, leaving infinite effective time for the correct-simplex dynamics below.

On the correct simplex, changing from t to τ gives the replicator flow

$$\frac{dq_c}{d\tau} = q_c \left(q_c - \sum_{d \in \mathcal{C}} q_d^2 \right), \quad \frac{d}{d\tau} \log \frac{q_c}{q_d} = q_c - q_d. \quad (18)$$

The initially unique maximum remains the unique maximum. For $c \neq c^*$, set $\xi_c = q_c/q_{c^*}$, so $\xi_c(0) < 1$ and ξ_c is nonincreasing. Since $q_{c^*} \geq 1/m$,

$$\frac{d \log \xi_c}{d\tau} = -q_{c^*}(1 - \xi_c) \leq -\frac{1 - \xi_c(0)}{m}.$$

Hence $\xi_c(\tau) \leq \xi_c(0) \exp[-(1 - \xi_c(0))\tau/m] \rightarrow 0$. Every competing mode vanishes relative to c^* ; normalization then yields $q_{c^*} \rightarrow 1$, establishing the claimed single-mode limit. $\square$

If several modes tie for the largest initial probability, symmetry preserves the tie; the same ratio argument eliminates strictly smaller coordinates and leaves the tied maxima uniform. An exactly uniform correct distribution is therefore a special initial condition. The theorem concerns limiting concentration: finite softmax logits already give positive probabilities at every finite time, so finite-time positivity alone is not a retention result.

P.3 CONDITIONAL TRAJECTORIES OF OUTCOME-BINARY MEAN FLOWS

Theorem P.4 is stated for three estimators, which invites the question of which other estimators share its conclusion. Under Assumption P.1, outcome-binary estimators with positive correctness coefficients induce the same conditional trajectory, differing only in how fast it is traversed. The reason is one line of conditioning: a binary reward cannot tell the estimator which correct mode it just saw.

Proof of Theorem 3.1.

Proof. Condition on the binary reward vector. The score identities used in Lemma P.2 give $\mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 1] = \nabla_z \log P$ and $\mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 0] = \nabla_z \log(1 - P)$; neither depends on which correct mode or which incorrect category was drawn, which is the only place the binary reward is used. Since a is measurable with respect to that vector, summing the R successful and $G - R$ failed rows and taking expectations gives

$$\mathbb{E}[\hat{g}_a] = \frac{1}{G} \left(\mathbb{E}[R a(1, R)] \nabla_z \log P + \mathbb{E}[(G - R) a(0, R)] \nabla_z \log(1 - P) \right),$$

and substituting $\nabla_z \log P = \nabla_z P / P$ and $\nabla_z \log(1 - P) = -\nabla_z P / (1 - P)$ yields Equation (3). The flow $\dot{z} = c_a(P) \nabla_z P$ has correct-coordinate velocities $c_a(P) p_c(1 - P)$, so $\frac{d}{dt} \log(q_c/q_d) = c_a(P) P(1 - P)(q_c - q_d)$ and the stated time change removes c_a entirely, leaving Equation (18). $\square$

The same binomial regrouping that gave Lemma P.2 its denominator-free form applies to c_a . Using $R \binom{G}{R} = G \binom{G-1}{R-1}$ and $(G - R) \binom{G}{R} = G \binom{G-1}{R}$,

$$c_a(P) = \sum_{j=0}^{G-1} \binom{G-1}{j} P^j (1 - P)^{G-1-j} [a(1, j+1) - a(0, j)], \quad (19)$$

a polynomial in P of degree at most $G - 1$. So whenever the applied advantages are bounded, c_a is smooth and bounded on the closed interval $[0, 1]$, which is every regularity property that the proof of the collapse theorem drew on.

Equation (19) is a Bernstein expansion, and its control points have a reading that makes the sign of c_a checkable by inspection rather than assumed. Write

$$d_j := a(1, j+1) - a(0, j), \quad j = 0, \dots, G-1. \quad (20)$$

Holding the other $G - 1$ rows fixed at j successes, d_j is the change in the advantage a single row receives when that row flips from failure to success: the leave-one-out gap the objective puts between being right and being wrong. Since the Bernstein basis is positive and sums to one, $c_a(P)$ lies between $\min_j d_j$ and $\max_j d_j$ at every P . Nonnegative control points, with at least one positive, therefore certify positivity in the interior. This is a sufficient condition: mixed signs alone do not determine whether c_a is positive throughout the interval. For Dr.GRPO every d_j equals $(G-1)/G$, recovering the constant coefficient of Lemma P.2; for the centered MaxRL estimator $d_0 = G - 1$ and $d_j = G/(j+1)$ for $j \geq 1$, and summing the basis against them returns $[1 - (1 - P)^{G-1}]/P$ as in Lemma P.3. The consequence is that the collapse theorem was never about the three estimators it was stated for.

Corollary P.6 (Collapse for correctness-improving outcome-binary mean flows). *Under Assumption P.1, let a be bounded and reward measurable, and suppose it rewards correctness in the weak sense that its leave-one-out gaps satisfy $d_j \geq 0$ for every j with $d_j > 0$ for at least one, which by Equation (20) is a condition on $2G$ numbers that can be read off the estimator. Let $q(0)$ have a unique largest coordinate c^* . Then the conclusions of Theorem P.4 hold verbatim: $P \rightarrow 1$, $q_{c^*} \rightarrow 1$, and $q_c \rightarrow 0$ for every other correct mode. The surviving mode is whichever one led at initialization, and no admissible choice of the advantage a changes its identity — only the rate at which it takes the conditional distribution over.*

Proof. By Theorem 3.1 the flow is $\dot{z} = c_a(P) \nabla_z P$, which is the flow of Theorem P.4 with c_G replaced by c_a . That proof uses the coefficient only through its smoothness and boundedness on $[0, 1]$, which Equation (19) supplies, and its positivity on $(0, 1)$, which the hypothesis now supplies: the Bernstein basis functions are strictly positive on $(0, 1)$, so a nonnegative set of control points with one strictly positive member gives $c_a(P) \geq \binom{G-1}{j_0} P^{j_0} (1 - P)^{G-1-j_0} d_{j_0} > 0$ there. The identity of

c^* is fixed by $q(0)$ alone, because the replicator equation (18) reached after the time change contains neither the advantage coefficients nor the rollout group size. $\square$

The nonnegative-gap condition is sufficient rather than necessary. Some estimators with mixed-sign gaps still have $c_a(P) > 0$ throughout the interior and satisfy the same conclusion. If c_a has an interior zero, the mean flow can stop there; if it is negative, correctness decreases locally. Theorem 3.1 identifies the direction in all cases, but the collapse conclusion requires the stated positivity and initial imbalance. It does not describe shared-prompt or sampled neural updates.

Remark P.7 (Group size in the deterministic conditional flow). The group size enters the class only through c_a , which Equation (19) shows is a coefficient on $\nabla_z P$. The time change of Theorem 3.1 removes it, so for every $G \geq 2$ the conditional dynamics are the same replicator equation and Corollary P.6 applies unchanged: enlarging the rollout group rescales how fast the flow moves along the correctness axis and does not alter which correct mode survives it. What a larger group does change is the starvation bound of Lemma P.17 and the MaxRL coefficient of Corollary P.8, both of which concern discovery near $P = 0$ rather than retention near $p_b = 0$. These are mean-flow statements. Group size also changes gradient covariance and finite-sample symmetry breaking, as the sampled-update results below make explicit; the deterministic equivalence is not a stochastic invariance.

Corollary P.8 (What a stronger fresh objective does buy). *For the coefficients of Lemmas P.2 and P.3,*

$$\frac{c_G^{\text{MaxRL}}(P)}{c_G^{\text{Dr.GRPO}}} = \frac{G[1 - (1 - P)^{G-1}]}{(G - 1)P},$$

which decreases from G at $P \rightarrow 0$ to $G/(G-1)$ at $P = 1$. MaxRL therefore multiplies progress along the correctness axis by a factor approaching the group size in exactly the regime where Lemma P.17 bounds fresh signal to be scarce, with the factor approaching $G/(G-1)$ as correctness tends to one.

Proof. Divide $c_G^{\text{MaxRL}}(P) = [1 - (1 - P)^{G-1}]/P$ by $(G - 1)/G$. The quotient is c_G^{MaxRL} up to a positive constant and c_G^{MaxRL} is decreasing, with endpoint values $G - 1$ and 1 from Lemma P.3. $\square$

P.4 NEURAL SCORE IDENTITIES AND THE LIMITS OF CONDITIONAL INERTNESS

The distinction between correctness and its allocation among correct outputs is established in prior work: Sinha et al. (2026, Theorem 3.1, v1) derive the expected-return log-ratio dynamics for softmax outcome logits, and Lochab et al. (2026, Section 4.1, v1) identify binary reward’s indifference to allocation within the correct set. The extension here gives the mean and covariance of general outcome-binary group estimators in neural parameters, then states what survives finite steps, shared prompts, and response-length weighting. This subsection replaces the categorical assumptions with the differentiable-policy assumptions stated below.

Theorem P.9 (Per-prompt neural mean and covariance). *Fix a prompt and a differentiable policy π_θ on a finite or countable response space with common positive support, where θ lies in an open subset of $\mathbb{R}^d$. Let $\mathcal{C}$ be a fixed verifier-positive event, $P = \Pr_\theta(Y \in \mathcal{C}) \in (0, 1)$, and $s(Y) = \nabla_\theta \log \pi_\theta(Y)$. Assume differentiation may be interchanged with the probability sums over the full support and over $\mathcal{C}$, and that $\mathbb{E}\|s(Y)\|_2^2 < \infty$. For G independent responses, write $R_i = \mathbf{1}[Y_i \in \mathcal{C}]$, $R = \sum_i R_i$, and let $a : \{0, 1\} \times \{0, \dots, G\} \rightarrow \mathbb{R}$ be any finite, detached advantage. Define*

$$\hat{g}_a = \frac{1}{G} \sum_i a(R_i, R) s(Y_i), \quad v = \nabla_\theta P, \quad \Sigma_r = \text{Cov}(s(Y) \mid \mathbf{1}[Y \in \mathcal{C}] = r).$$

With

$$\beta_r = \frac{1}{G} \left(\frac{r a(1, r)}{P} - \frac{(G - r) a(0, r)}{1 - P} \right), \quad c_a(P) = \mathbb{E}[\beta_R],$$

where $R \sim \text{Binomial}(G, P)$, the exact identities are

$$\mathbb{E}[\hat{g}_a] = c_a(P)v, \tag{21}$$

$$\text{Cov}(\hat{g}_a) = \text{Var}(\beta_R)vv^\top + \frac{1}{G^2} \mathbb{E}[R a(1, R)^2 \Sigma_1 + (G - R) a(0, R)^2 \Sigma_0]. \tag{22}$$

In particular, c_a is the coefficient in Equation (3), without an independent-logit assumption.

Proof. Differentiating the correct and incorrect event probabilities gives

$$\mathbb{E}[s(Y) \mid Y \in \mathcal{C}] = v/P, \quad \mathbb{E}[s(Y) \mid Y \notin \mathcal{C}] = -v/(1 - P).$$

Conditioned on the complete reward vector, the response scores remain independent. The conditional mean of $\hat{g}_a$ is $\beta_R v$ and its conditional covariance is $G^{-2}[Ra(1, R)^2 \Sigma_1 + (G - R)a(0, R)^2 \Sigma_0]$. Taking expectations proves Equation (21); the law of total covariance proves Equation (22). $\square$

For one prompt, a common locally Lipschitz preconditioner $M(\theta)$ and positive coefficients $c_a(P)$ give deterministic mean fields $\dot{\theta} = c_a(P)M(\theta)\nabla_\theta P$ that trace the same orbit under a positive change of time, wherever the flows exist uniquely. This is a statement about the mean direction. Equation (22) shows that outcome-binary estimators can have different sampling covariance, including after their mean magnitudes are matched.

Corollary P.10 (A finite-step comparison in neural parameters). *Under Theorem P.9, suppose $c_a(P) > 0$ and take $\theta^+ = \theta + \eta \hat{g}_a / c_a(P)$ with $\eta > 0$. Let f be a twice differentiable scalar observable with $\|\nabla^2 f\|_{\text{op}} \leq L_f$ along every update segment, which is assumed to remain in its domain. Then, conditional on the current policy,*

$$|\mathbb{E}[f(\theta^+) - f(\theta)] - \eta \nabla f(\theta)^\top v| \leq \frac{L_f \eta^2}{2} \left(\|v\|_2^2 + \frac{\text{tr Cov}(\hat{g}_a)}{c_a(P)^2} \right). \quad (23)$$

Proof. Taylor’s theorem bounds the remainder after the linear term by $L_f \eta^2 \|\hat{g}_a\|_2^2 / (2c_a(P)^2)$. Take expectations and use Equation (21) together with $\mathbb{E}\|\hat{g}_a\|_2^2 = c_a(P)^2 \|v\|_2^2 + \text{tr Cov}(\hat{g}_a)$. $\square$

The normalization by $c_a(P)$ isolates direction for this comparison; it is not an additional step in the implemented algorithm. For two such estimators, their expected changes in f differ by at most the sum of the two second-order bounds. Conditional mode probabilities or PCMD may be used as f where the stated smoothness bounds hold. Thus mean-direction equivalence is a first-order statement for stochastic finite updates, and does not identify their trajectories or eventual surviving modes. Adam’s moments and subsequent PPO epochs require additional analysis.

Remark P.11 (Shared prompts need not follow the same trajectory). For prompt probabilities μ_x and shared parameters, the aggregate mean is

$$\mathbb{E}[\hat{g}_a] = \sum_x \mu_x c_a(P_x) \nabla_\theta P_x. \quad (24)$$

Different objectives generally assign different relative coefficients to the prompt gradients, so the per-prompt identity does not imply a common aggregate orbit. An explicit example also shows a difference in conditional diversity. Let $\theta = (u, v)$, $\sigma(t) = (1 + e^{-t})^{-1}$, and sample two prompts A, B equally. Set

$$P_A = \sigma(u + v), \quad P_B = \sigma(u - v), \quad q_A = (\sigma(v), 1 - \sigma(v)).$$

For prompt A , the probabilities of two correct responses and one incorrect response are $(P_A \sigma(v), P_A [1 - \sigma(v)], 1 - P_A)$; for B , use $(P_B/2, P_B/2, 1 - P_B)$. At $(u, v) = (0, \log 3)$, $P_A = 3/4$, $P_B = 1/4$, and $\nabla P_A = (3/16, 3/16)$, $\nabla P_B = (3/16, -3/16)$. For $G = 3$, Lemmas P.2 and P.3 give $c_{\text{Dr}} = 2/3$ and $c_{\text{Max}}(P) = 2 - P$, hence

$$\dot{\theta}_{\text{Dr}} = (1/8, 0), \quad \dot{\theta}_{\text{Max}} = (9/32, -3/64).$$

Prompt A has conditional disagreement $D_A = 1 - \sum_c q_{A,c}^2 = 2\sigma(v)[1 - \sigma(v)]$, so direct differentiation gives

$$\dot{D}_{A,\text{Dr}} = 0, \quad \dot{D}_{A,\text{Max}} = 9/1024 > 0.$$

Both prompts’ correctness increases under both mean fields, since $\dot{P}_A = (3/16)(\dot{u} + \dot{v}) > 0$ and $\dot{P}_B = (3/16)(\dot{u} - \dot{v}) > 0$ at this point. Changing a binary objective can therefore change conditional diversity through shared prompt gradients, despite the exact per-prompt identity.

Proposition P.12 (The residual from response-dependent normalization). *Under Theorem P.9, multiply each response’s score by a detached function $\omega(Y)$ with finite conditional second moments; inverse response length is the case $\omega(Y) = 1/L(Y)$. Define*

$$\hat{g}_{a,\omega} = \frac{1}{G} \sum_i a(R_i, R) \omega(Y_i) s(Y_i), \quad A_1 = \frac{\mathbb{E}[Ra(1, R)]}{G}, \quad A_0 = \frac{\mathbb{E}[(G - R)a(0, R)]}{G},$$

and, conditioning on $\mathbf{1}[Y \in \mathcal{C}] = r$, let $\bar{\omega}_r = \mathbb{E}[\omega(Y) \mid r]$, $K_r = \text{Cov}(\omega(Y), s(Y) \mid r)$, and $\sigma_{\omega,r}^2 = \text{Var}(\omega(Y) \mid r)$. Then

$$\mathbb{E}[\hat{g}_{a,\omega}] = \tilde{c}_a \nabla_{\theta} P + A_1 K_1 + A_0 K_0, \quad \tilde{c}_a = \frac{A_1 \bar{\omega}_1}{P} - \frac{A_0 \bar{\omega}_0}{1-P}, \quad (25)$$

$$\|A_1 K_1 + A_0 K_0\|_2 \leq \sum_{r=0}^1 |A_r| \sigma_{\omega,r} \sqrt{\text{tr } \Sigma_r}. \quad (26)$$

Proof. Conditioning on the reward vector gives $\mathbb{E}[\hat{g}_{a,\omega}] = A_1 \mathbb{E}[\omega s \mid 1] + A_0 \mathbb{E}[\omega s \mid 0]$. For each outcome, $\mathbb{E}[\omega s \mid r] = \bar{\omega}_r \mathbb{E}[s \mid r] + K_r$; substituting the conditional score means proves Equation (25). The vector Cauchy–Schwarz inequality gives $\|K_r\|_2 \leq \sigma_{\omega,r} \sqrt{\text{tr } \Sigma_r}$; the triangle inequality completes the bound. $\square$

The residual is a within-outcome covariance between response weighting and the policy score. A common length makes it zero; equal mean lengths for correct and incorrect responses need not. More generally, response-dependent information can leave the outcome-binary class. IPS uses outcome-frequency weights (Sinha et al., 2026, Section 4.4, v1), and UCPO adds a conditional-uniformity term (Lochab et al., 2026, Section 6, v1); neither intervention is excluded by the mean-direction identity proved here.

P.5 DIVERSITY AT FINITE STEPS AND SAMPLING BUDGETS

The softmax log-ratio identity of Sinha et al. (2026, Theorem 3.1 and App. A.1) and the amplification analysis of Lochab et al. (2026, Section 4.2) identify the underlying frequency effect. We now connect it to the reported PCMD metric, extend expected-gradient collapse to finite steps, and quantify disappearance within a finite inference budget. The categorical geometry remains explicit; the discrete and sampled results replace (A3) with their stated updates.

Proposition P.13 (Exact diversity drift and matched correctness). *Under Assumption P.1, consider $\dot{z} = c(P) \nabla_z P$ with $c(P) > 0$. Write $D(q) = 1 - \sum_c q_c^2$, $S_2 = \sum_c q_c^2$, and $V(q) = \sum_c q_c^3 - S_2^2 = \text{Var}_{C \sim q}(q_C)$. For incorrect conditional probabilities $r_a = p_a/(1-P)$, set $T_2 = \sum_a r_a^2$. In effective time $d\tau = c(P)P(1-P)dt$,*

$$\frac{dD}{d\tau} = -2V(q), \quad \frac{dD}{d \log[P/(1-P)]} = -\frac{2V(q)}{S_2 + T_2}. \quad (27)$$

Thus diversity strictly decreases at every nonuniform interior q . Positive-coefficient mean flows from the same initial logits follow the same D -versus- P curve, independently of their chosen outcome-binary advantage or group size.

Proof. Equation (18) gives $dS_2/d\tau = 2(\sum_c q_c^3 - S_2^2)$. The variance vanishes exactly when the positive coordinates of q are equal. Also,

$$\dot{P} = c(P) \|\nabla_z P\|_2^2 = c(P)P^2(1-P)^2(S_2 + T_2),$$

so $d \log[P/(1-P)]/d\tau = S_2 + T_2 > 0$, proving the second identity. Finally, the full logit equation in effective time is $dz_c/d\tau = q_c$ and $dz_a/d\tau = -r_a$; it contains no c . The same initial logits therefore give the same full trajectory after the time change, and hence the same curve at matched correctness for every positive-coefficient outcome-binary estimator considered here. $\square$

Theorem P.14 (Collapse under finite expected-gradient steps). *Retain the categorical model but replace (A3) by*

$$z_{k+1} = z_k + \eta_k c(P_k) \nabla_z P_k, \quad 0 < \eta_k < \infty, \quad \sum_{k=0}^{\infty} \eta_k = \infty, \quad (28)$$

where c is continuous and positive on $(0,1)$. Each step strictly decreases $D(q_k)$ unless q_k is uniform. If q_0 has a unique largest coordinate c^ , then $P_k \rightarrow 1$ and $q_{c^*,k} \rightarrow 1$. No upper bound on the finite step sizes is required.*

Proof. Let $h_k = \eta_k c(P_k)P_k(1 - P_k) > 0$. The exact updates are $z_c^+ = z_c + h_k q_c$ and $z_a^+ = z_a - h_k r_a$, so

$$q_c^+ = \frac{q_c e^{h_k q_c}}{\sum_d q_d e^{h_k q_d}}. \quad (29)$$

For fixed q , put $u_c(s) = q_c e^{s q_c} / \sum_d q_d e^{s q_d}$. Differentiation gives

$$\frac{d}{ds} \sum_c u_c(s)^2 = \sum_{c,d} u_c(s) u_d(s) (u_c(s) - u_d(s)) (q_c - q_d) \geq 0.$$

The ordering of $u(s)$ is that of q , and the inequality is strict whenever q is nonuniform. Integrating from 0 to h_k proves the diversity claim. Equation (29) also preserves the leader and makes its conditional probability nondecreasing.

Every correct logit increases and every incorrect logit decreases, hence P_k increases. Jensen's inequality gives

$$\log \frac{P_{k+1}}{1 - P_{k+1}} - \log \frac{P_k}{1 - P_k} = \log \sum_c q_c e^{h_k q_c} - \log \sum_a r_a e^{-h_k r_a} \geq h_k \sum_c q_c^2 \geq h_k / m.$$

If P_k had an interior limit, continuity and positivity of c would imply $\sum_k h_k = \infty$, contradicting bounded correctness odds. Thus $P_k \rightarrow 1$. Moreover, $\sum_k h_k = \infty$ is necessary: otherwise every logit would have finite total variation and converge to a finite value, contradicting $P_k \rightarrow 1$. Set $\alpha = q_{c^*,0}$ and $\xi_{c,k} = q_{c,k} / q_{c^*,k}$. For $c \neq c^*$,

$$\xi_{c,k+1} = \xi_{c,k} e^{-h_k (q_{c^*,k} - q_{c,k})}, \quad q_{c^*,k} - q_{c,k} \geq \alpha (1 - \xi_{c,0}) > 0.$$

The divergent sum of h_k forces every such ratio to zero, proving $q_{c^*,k} \rightarrow 1$. $\square$

This theorem concerns the exact expected gradient. Finite step sizes can change the trajectory across estimators; the exact matched-correctness identity in Proposition P.13 remains a mean-flow statement.

Lochab et al. (2026, Proposition A.1) establish that finite multinomial counts can break a uniform policy's symmetry. The next result quantifies the effect of those counts on PCMD after the centered group update, including its learning-rate and group-size dependence.

Proposition P.15 (Sampled updates break conditional symmetry). *Retain the categorical model, replacing (A3) by one sampled update $z^+ = z + \eta \hat{g}_G$ from Equation (14), with $\eta > 0$. Suppose $0 < P < 1$ and $q_c = 1/m$ for every correct mode. Then*

$$\mathbb{E}[D(q^+)] < 1 - 1/m, \quad \mathbb{E}[q_c^+] = 1/m. \quad (30)$$

As $\eta \rightarrow 0$, for the fixed initial policy and estimator,

$$\mathbb{E}[D(q^+)] = 1 - \frac{1}{m} - \eta^2 \frac{m-1}{m^3} \mathbb{E} \left[\frac{w(R)^2 R(G-R)^2}{G^4} \right] + O(\eta^3), \quad (31)$$

where the summand is zero on $R = 0, G$. For Dr.GRPO, $w = 1$ and

$$\mathbb{E} \left[\frac{R(G-R)^2}{G^4} \right] = \frac{G-1}{G^3} P(1-P) [1 + (G-2)(1-P)]. \quad (32)$$

Proof. Let N_c count occurrences of correct mode c . The centered advantages sum to zero, so the common softmax-score term cancels and $\hat{g}_c = \kappa_R N_c$, where $\kappa_R = w(R)(G-R)/G^2$ on mixed groups and $\kappa_R = 0$ otherwise. Conditional on R , the correct counts are $N \sim \text{Multinomial}(R; 1/m, \dots, 1/m)$, and

$$q_c^+ = \frac{e^{\eta \kappa_R N_c}}{\sum_d e^{\eta \kappa_R N_d}}.$$

Diversity is at most $1 - 1/m$, with equality only at uniform q^+ . The event $R = 1$ has positive probability; on this event one correct logit increases and the others do not, so q^+ is nonuniform. This proves the strict inequality for every $\eta > 0$. Permutation symmetry of the correct labels gives $\mathbb{E}[q_c^+] = 1/m$.

For a fixed vector v , Taylor expansion about uniform logits gives

$$\sum_c \text{softmax}(\eta v)_c^2 = \frac{1}{m} + \frac{\eta^2}{m^2} \sum_c (v_c - \bar{v})^2 + O(\eta^3), \quad \bar{v} = \frac{1}{m} \sum_c v_c.$$

Use $v_c = \kappa_R N_c$ and $\mathbb{E}[\sum_c (N_c - R/m)^2 \mid R] = R(1 - 1/m)$. There are finitely many possible groups and all mixed-group weights are finite, so the Taylor remainder is uniform over those groups. Averaging proves Equation (31). Finally,

$$\mathbb{E}[R(G - R)^2] = G(G - 1)(G - 2)P(1 - P)^2 + G(G - 1)P(1 - P),$$

by the binomial factorial moments, yielding Equation (32). $\square$

At fixed $P \in (0, 1)$, the Dr.GRPO coefficient in Equation (32) is $P(1 - P)^2/G + O(G^{-2})$. Thus larger groups reduce the leading expected one-step diversity loss from conditional symmetry at the same learning rate. This sampling effect is absent from the mean flow. The proposition does not assert monotone expected diversity from an arbitrary nonuniform policy or convergence of a stochastic optimizer.

Corollary P.16 (Finite-budget disappearance of competing modes). *In the mean flow of Theorem P.4, let c^* be the unique initial correct leader and define*

$$\alpha = q_{c^*}(0), \quad \Gamma_0 = \sum_{c \neq c^*} \frac{q_c(0)}{\alpha - q_c(0)}.$$

At effective time τ ,

$$1 - q_{c^*}(\tau) \leq \Gamma_0 e^{-\alpha\tau}, \quad D(q(\tau)) \leq 2\Gamma_0 e^{-\alpha\tau}. \quad (33)$$

For $K \geq 1$ independent full-policy draws at that checkpoint,

$$\Pr(\text{any correct mode other than } c^* \text{ is observed}) \leq K\Gamma_0 e^{-\alpha\tau}. \quad (34)$$

In particular, if $\tau \geq \max\{0, \alpha^{-1} \log(K\Gamma_0/\delta)\}$ for $0 < \delta < 1$, no competing correct mode is observed with probability at least $1 - \delta$, although every mode retains positive probability.

Proof. For $c \neq c^*$, write $\xi_c = q_c/q_{c^*} < 1$. The leader is nondecreasing, and Equation (18) gives

$$\frac{d}{d\tau} \log \frac{\xi_c}{1 - \xi_c} = -q_{c^*} \leq -\alpha.$$

Hence $\xi_c(\tau) \leq [q_c(0)/(\alpha - q_c(0))]e^{-\alpha\tau}$. Now $1 - q_{c^*} = \sum_{c \neq c^*} \xi_c / (1 + \sum_{c \neq c^*} \xi_c)$ and $D \leq 1 - q_{c^*}^2 \leq 2(1 - q_{c^*})$, proving the first bounds. A single full-policy draw hits a competing correct mode with probability $P(1 - q_{c^*}) \leq 1 - q_{c^*}$; a union bound over K draws completes the proof. $\square$

The bound gives a finite-budget interpretation of the rare-output pruning mechanism discussed by Lochab et al. (2026, Section 4.2.5). It can also be indexed by correctness: Proposition P.13 and $S_2 + T_2 \leq 2$ imply $\tau \geq \frac{1}{2} \log\{[P/(1 - P)]/[P(0)/(1 - P(0))]\}$. These are sufficient visibility bounds within the categorical model; they do not require literal support loss or identify an empirical collapse time for neural training.

P.6 A REPLAY SIGNAL WHEN FRESH GROUPS SUPPLY NONE

Lemma P.17 (Fresh-group starvation). *Under Assumption P.1 with correctness probability P , draw G fresh categories independently. The probability of a mixed-reward group is*

$$h_G(P) = 1 - P^G - (1 - P)^G \leq G \min\{P, 1 - P\}.$$

Every all-equal group has zero fresh task component for Equation (14) and for the specified centered MaxRL estimator, so a nonzero fresh task component requires a mixed group, an event whose probability is at most $h_G(P)$ at every correctness level.

Proof. The all-correct and all-incorrect events are disjoint and have probabilities P^G and $(1 - P)^G$. A mixed group requires both a correct and an incorrect sample, so the union bound supplies each of the two stated bounds, GP and $G(1 - P)$, and the minimum of the pair is therefore also a bound. $\square$

Lemma P.18 (Replay ascent persists as fresh signal vanishes). *Under Assumption P.1, let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed verified bank with normalized positive weights w_b , let $R_w = -\sum_b w_b \log p_b$ with $\rho_w > 0$, and extend w by zero outside the bank to obtain $\bar{w}$. The deterministic replay ascent vector is*

$$v^{\text{replay}} = -\rho_w \nabla R_w = \rho_w (\bar{w} - p),$$

which vanishes only at $p = \bar{w}$. If $p_b \leq w_b/2$ for a banked mode, then $v_b^{\text{replay}} \geq \rho_w w_b/2$, even when b is absent from the fresh group. For a uniform bank of $k \geq 2$ modes and fixed $\rho_w > 0$, as $p \rightarrow \mathbf{e}_b$ at one banked correct mode, $h_G(P) \rightarrow 0$ whereas $\|v^{\text{replay}}\|_2 \rightarrow \rho_w \sqrt{1 - 1/k} > 0$.

Proof. For $R_w = -\sum_b w_b \log p_b$, the softmax score identity gives $\partial R_w / \partial z_a = p_a - \bar{w}_a$, proving the replay formula and coordinate bound. Finally,

$$\|\bar{w} - \mathbf{e}_b\|_2^2 = (1 - 1/k)^2 + (k - 1)/k^2 = 1 - 1/k,$$

which gives the strictly positive limiting norm for $k \geq 2$. $\square$

Remark P.19. Together the two lemmas say only that a banked target can supply a verified signal when the fresh task gradient is unavailable. Lemma P.17 bounds an opportunity, not a gradient magnitude, and says nothing about other loss terms, clipping, or momentum carried across updates. The vector in Lemma P.18 averages the fixed bank, so a stochastic exemplar draw realizes it only in expectation, and a nonzero logit coordinate can still be annihilated by a degenerate neural Jacobian. Replay also need not increase the next mixed-group probability, since $h_G(P)$ is largest at $P = 1/2$.

P.7 FIXED-BANK REPLAY AND QUALITATIVE RETENTION

Let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed bank of k modes that the policy has produced and the validator has accepted. Uniform verified replay uses one exemplar per banked mode and, in the categorical reduction, minimizes

$$R_{\mathcal{B}}(z) = -\frac{1}{k} \sum_{b \in \mathcal{B}} \log p_b. \quad (35)$$

This is ordinary cross-entropy against the distribution that assigns mass $1/k$ to each banked mode and zero elsewhere. It is not conditioned on the bank: decreasing (35) can raise total bank mass as well as redistribute mass within the bank.

In an averaged continuous-time model, absorb a fixed round-robin replay frequency and loss scale into an effective dose $\rho > 0$. This assumes a fixed eligible-bank set and constant relative weighting; it does not identify finite alternating updates with simultaneous gradient flow. The prompt-isolated categorical mean flow for any of the three fresh binary-reward objectives above plus verified replay is

$$\dot{z} = c_G(P) \nabla_z P - \rho \nabla_z R_{\mathcal{B}}(z). \quad (36)$$

Let $\Psi(P) = \int_0^P c_G(s) ds$. For the finite-group estimators in Lemmas P.2 and P.3, c_G has a finite positive extension to $[0, 1]$; in particular, Dr.GRPO has $c_G = (G - 1)/G$ and MaxRL has $c_G = [1 - (1 - P)^{G-1}]/P$. Nothing below uses more than that. By Theorem 3.1 and Equation (19), every bounded reward-measurable objective contributes $c_a(P) \nabla_z P$ with c_a a polynomial, so it admits the same Ψ_a and the retention argument runs unchanged: what follows is a statement about verified replay added to any outcome-binary fresh objective, of which Re:Dr and Re:Max are the two this paper trains. Define

$$F_{\mathcal{B}}(z) = \rho R_{\mathcal{B}}(z) - \Psi(P(z)). \quad (37)$$

Then (36) is exactly gradient flow $\dot{z} = -\nabla_z F_{\mathcal{B}}$.

Theorem P.20 (Verified replay prevents banked-mode extinction). *Suppose logits are finite at time T and thereafter the nonempty verified bank $\mathcal{B}$ is fixed, fits within replay capacity, and receives a fixed positive effective replay dose ρ , the fresh objective being any bounded reward-measurable one in the sense of Theorem 3.1. Set $\Psi_{\max} = \max_{s \in [0, 1]} \Psi(s)$, which is finite, and under (36) define*

$$C_T = R_{\mathcal{B}}(z(T)) + \frac{\Psi_{\max} - \Psi(P(T))}{\rho}.$$

Then for every $b \in \mathcal{B}$ and every $t \geq T$,

$$p_b(t) \geq \exp(-kC_T) > 0. \quad (38)$$

Thus each banked probability is uniformly bounded away from zero for all $t \geq T$, including asymptotically, even when the bank does not cover every correct mode. This is stronger than finite-time positivity, which already follows from finite softmax logits without replay.

Proof. The smooth vector field has bounded velocity because c_G , $\nabla_z P$, and $\nabla_z R_B$ are bounded, so finite initial logits give a flow for every finite $t \geq T$. Along this flow,

$$\frac{d}{dt} F_B(z(t)) = -\|\nabla_z F_B(z(t))\|_2^2 \leq 0.$$

Since $\Psi(P(t)) \leq \Psi_{\max}$,

$$R_B(z(t)) \leq R_B(z(T)) + \frac{\Psi_{\max} - \Psi(P(T))}{\rho} = C_T.$$

Every $-\log p_b$ is nonnegative and their sum is kR_B . Therefore $-\log p_b(t) \leq kC_T$ for every banked mode, which is exactly Equation (38) for every time after T . $\square$

For the nonnegative coefficients of the named fresh estimators, $\Psi_{\max} = \Psi(1)$, recovering the simpler constant. Signed coefficients also satisfy the retention bound, but need not yield $P \rightarrow 1$. The floor is qualitative and can be numerically vacuous. For example, at $k = 16$ even a hypothetical $C_T = 160$ gives e^{-2560} , far below any useful evaluation probability. The argument establishes a uniform-in-time lower bound in the stated ideal flow, not practical visibility or a prediction for the experimental replay loss. The effective dose also depends on relative fresh/replay frequencies, not the replay coefficient alone.

A common nonnegative, locally integrable rate $\eta(t)$ multiplying both terms of (36) preserves the energy bound, since $\dot{F}_B = -\eta(t)\|\nabla F_B\|_2^2 \leq 0$. The convergence statements below additionally require $\int_T^\infty \eta(t) dt = \infty$ and then follow by reparameterizing optimization time. Infinitely many replay visits alone establish neither this condition nor a fixed replay-to-fresh weighting. Different time-varying relative weights are outside the fixed-potential argument.

Full coverage and the categorical target.

Corollary P.21 (Conditional full-coverage categorical limit). *Assume the conditions of Theorem P.20 and that by time T the bank covers the full correct support, $\mathcal{B} = \mathcal{C}$. Assume additionally $c_G(P) \geq 0$ on $[0, 1]$, as for the three named estimators. Let $u_c = 1/m$ on $\mathcal{C}$. Then the categorical mean flow converges to*

$$P(t) \rightarrow 1, \quad q(t) \rightarrow u.$$

Consequently

$$\min_{c \in \mathcal{C}} q_c(t) \rightarrow \frac{1}{m}, \quad \mathcal{H}(q(t)) \rightarrow \log m,$$

and conditional correct-mode collapse is impossible.

Proof. For full coverage,

$$R_C = \log m - \log P + \text{KL}(u||q). \quad (39)$$

The potential is bounded below by $-\Psi_{\max}$, so its descent gives $\int_T^\infty \|\nabla F_C\|_2^2 dt < \infty$. The finite-group c_G is smooth on $[0, 1]$, and softmax derivatives and the cross-entropy gradient are bounded. Thus the Hessian of F_C and the velocity are bounded; the gradient along the trajectory is uniformly continuous. Square integrability then implies $\nabla F_C \rightarrow 0$: recurring gradients of fixed positive size would otherwise contribute energy over intervals of a uniform minimum length.

Every limiting probability vector therefore satisfies the correct-coordinate equations

$$0 = \rho(p_c - 1/m) - c_G(P)p_c(1 - P).$$

Summing them gives $0 = -(1 - P)(\rho + c_G(P)P)$, hence $P = 1$. Each correct coordinate is then $1/m$ and every incorrect coordinate is zero. Compactness of the probability simplex and uniqueness of this limit establish convergence of the probability vector. $\square$

Positive weights and the implemented length normalization. Uniformity of the target is separate from protection against extinction. For fixed weights $w_b > 0$ with $\sum_{b \in \mathcal{B}} w_b = 1$, define $R_w = -\sum_b w_b \log p_b$ and use replay coefficient $\rho_w > 0$. The same potential argument gives, for all $t \geq T$,

$$p_b(t) \geq \exp(-C_{T,w}/w_b), \quad C_{T,w} = R_w(z(T)) + \frac{\Psi_{\max} - \Psi(P(T))}{\rho_w}.$$

For the following convergence statements assume $c_G \geq 0$. If $\mathcal{B} = \mathcal{C}$, then $R_w = H(w) - \log P + \text{KL}(w||q)$, and the independent-logit limit is $P \rightarrow 1$, $q \rightarrow w$. The preceding stationarity proof applies with $1/m$ replaced by w_c : summing the correct-coordinate equations forces $P = 1$, then each equation forces $p_c = w_c$.

A partial bank has a stronger limitation in this same ideal flow. For any fixed nonempty $\mathcal{B} \subseteq \mathcal{C}$ and fixed positive weights summing to one, extend w by zero to $\bar{w}$ on all categories. Then $p(t) \rightarrow \bar{w}$, so even correct modes outside the frozen bank vanish asymptotically. For $F = \rho_w R_w - \Psi(P)$, the preceding energy and bounded-Hessian argument still gives $\nabla F \rightarrow 0$; neither argument requires full bank coverage. At every subsequential probability limit, the correct-coordinate equations are

$$0 = \rho_w(p_c - \bar{w}_c) - c_G(P)p_c(1 - P).$$

Summing over correct categories gives $0 = -(1 - P)(\rho_w + c_G(P)P)$, hence $P = 1$; each equation then gives $p_c = \bar{w}_c$, and all incorrect mass is zero. Compactness and uniqueness of this limiting probability vector prove convergence. Thus protecting a fixed set of banked modes need not preserve unbanked correct alternatives. This conclusion concerns the fixed-bank categorical flow, not changing-bank or stochastic neural training.

This distinction matters for the implemented mean-token exemplar loss. Even if each key were represented by exactly one categorical exemplar of fixed length L_b , its loss would be AR_w , where

$$A = \frac{1}{k} \sum_{b \in \mathcal{B}} \frac{1}{L_b}, \quad w_b = \frac{L_b^{-1}}{\sum_{j \in \mathcal{B}} L_j^{-1}}.$$

Its effective categorical replay coefficient would be $\rho_w = \rho A$. The full-coverage limit would therefore weight shorter exemplars more, and would be uniform only for equal lengths. For an actual language model, $\pi_\theta(e_b | x)$ is only one contribution to the total key probability. Equal averaging of per-token exemplar scores does not imply uniform learned key probabilities; the categorical uniformity corollary is not that claim.

Full support is an additional premise, not a consequence of replay. In Python factors, evaluation prompts admit 16–1,512 correct execution modes (mean 229.44), while the bank capacity is 16. The full-coverage corollary therefore cannot supply a domain-wide guarantee there. The retention theorem protects only admitted banked modes; it supplies no probability floor for an unbanked key.

P.8 WHEN REPLAY REVERSES CONCENTRATION, AND HOW FAST

The restoring logit field $\rho(\bar{w} - p)$ has a shared foundation with inverse probability scaling: the ideal IPS field of Sinha et al. (2026, Theorem 4.1), normalized to a fixed target w , is $\bar{w} - p$. The uniform-correct target and its forward conditional-KL formulation are also established by Lochab et al. (2026). Here the target is restricted to a discovered bank and evaluated from stored exemplars. The question below is how its restoring field competes with the fresh binary objective over a finite interval. Neither a uniform target nor its cross-entropy gradient is a new contribution of this analysis.

Fix a bank $\mathcal{B} \subseteq \mathcal{C}$ of $k \geq 2$ modes, with uniform categorical weights, and consider Equation (36). Write $c(P)$ for its fresh coefficient and define

$$M_{\mathcal{B}} = \sum_{b \in \mathcal{B}} p_b, \quad r_b = \frac{p_b}{M_{\mathcal{B}}}, \quad C_{\mathcal{B}} = \sum_{b \in \mathcal{B}} r_b^2, \quad (40)$$

$$a_\rho(t) = \rho - c(P(t))(1 - P(t)).$$

Thus $1 - C_{\mathcal{B}}$ is PCMD conditional on the bank. It equals the paper’s success-conditional PCMD when $\mathcal{B} = \mathcal{C}$; otherwise it measures only allocation inside the bank.

Theorem P.22 (Exact replay threshold for bank diversity). *Under the categorical mean-flow assumptions and a fixed uniform bank,*

$$\frac{d}{dt} \log \frac{p_i}{p_j} = -a_\rho M_{\mathcal{B}}(r_i - r_j), \quad i, j \in \mathcal{B}, \quad (41)$$

$$\dot{r}_b = -a_\rho M_{\mathcal{B}} r_b (r_b - C_{\mathcal{B}}), \quad b \in \mathcal{B}, \quad (42)$$

$$\frac{d}{dt} (1 - C_{\mathcal{B}}) = 2a_\rho M_{\mathcal{B}} \left(\sum_{b \in \mathcal{B}} r_b^3 - C_{\mathcal{B}}^2 \right). \quad (43)$$

For a nonuniform r , bank-conditional diversity strictly increases when $\rho > c(P)(1 - P)$, strictly decreases when $\rho < c(P)(1 - P)$, and has zero instantaneous derivative at equality. These conclusions require no coverage of correct modes outside the bank.

Proof. For $b \in \mathcal{B}$, the score identities give $\dot{z}_b = \rho/k - a_\rho p_b$. Subtracting two such equations gives Equation (41). The distribution r is the softmax of the logits restricted to $\mathcal{B}$, so $\dot{r}_b = r_b(\dot{z}_b - \sum_j r_j \dot{z}_j)$, proving Equation (42). Differentiating $C_{\mathcal{B}}$ gives Equation (43). Its parenthesis is $\text{Var}_{b \sim r}(r_b)$, which is strictly positive for a nonuniform distribution with positive coordinates. $\square$

A positive dose therefore need not improve diversity immediately. The thresholds for Dr.GRPO and the specified MaxRL estimator are, respectively,

$$\rho > \frac{G-1}{G}(1-P), \quad \rho > \frac{(1-P)[1-(1-P)^{G-1}]}{P}.$$

The asymptotic result for every $\rho > 0$ and this finite-time condition are compatible: the fresh concentration pressure vanishes as $P \rightarrow 1$.

Theorem P.23 (Finite recovery from accumulated replay exposure). *In the setting of Theorem P.22, suppose $a_\rho(u) \geq 0$ for $u \in [T, t]$, and define*

$$S(T, t) = \int_T^t a_\rho(u) M_{\mathcal{B}}(u) du, \quad D(t) = \log \frac{\max_{b \in \mathcal{B}} r_b(t)}{\min_{b \in \mathcal{B}} r_b(t)}.$$

Then

$$e^{D(t)} - 1 \leq (e^{D(T)} - 1)e^{-S(T, t)/k}, \quad \min_b r_b(t) \geq \frac{1}{1 + (k-1)e^{D(t)}}. \quad (44)$$

For $0 < \delta < 1/k$, put $D_\delta = \log[(1/\delta - 1)/(k-1)]$. If $D(T) > D_\delta$, the condition

$$S(T, t) \geq k \log \frac{e^{D(T)} - 1}{e^{D_\delta} - 1} \quad (45)$$

is sufficient for every $r_b(t) \geq \delta$. If $D(T) \leq D_\delta$, this conclusion already holds and is preserved on the interval. In particular, when $a_\rho \geq a_0 > 0$ and $M_{\mathcal{B}} \geq M_0 > 0$ on the interval, its required duration is at most the right-hand side of Equation (45) divided by $a_0 M_0$. At that endpoint every unconditional bank probability is at least $M_0 \delta$.

Proof. Bank-coordinate ordering is preserved by Equation (41): a difference of two bank logits satisfies a scalar equation proportional to that difference, and ties persist. Writing $r_{\max}$ and $r_{\min}$ for the extremal coordinates,

$$\dot{D} = -a_\rho M_{\mathcal{B}}(r_{\max} - r_{\min}), \quad r_{\max} - r_{\min} = r_{\max}(1 - e^{-D}) \geq \frac{1 - e^{-D}}{k}.$$

Thus $\frac{d}{dt}(e^D - 1) \leq -(a_\rho M_{\mathcal{B}}/k)(e^D - 1)$; integration proves the first bound. Since $1 \leq r_{\min} + (k-1)r_{\max}$, the second follows. Substituting D_δ and solving the first bound for the exposure proves the remaining claims. When $D(T) = 0$, uniformity is preserved directly. $\square$

The bound specifies the interval on which restoration occurs. It does not assume that bank mass is monotone or that correctness stays above its value at T . Both enter through the displayed exposure; a bound on unconditional visibility additionally needs the stated bank-mass lower bound.

Proposition P.24 (Discrete exact-gradient contraction). *Replace the continuous update by simultaneous exact mean-gradient steps, while retaining independent logits and a fixed uniform bank:*

$$z_{n+1} = z_n + \eta_n \{c(P_n) \nabla_z P_n + \rho(\bar{w} - p_n)\}, \quad \eta_n \geq 0.$$

Define D_n as above and $\gamma_n = \eta_n M_{\mathcal{B},n}[\rho - c(P_n)(1 - P_n)]$. If $0 \leq \gamma_n \leq 2$, the order of all bank probabilities is preserved at this step and

$$e^{D_{n+1}} - 1 \leq \left(1 - \frac{1 - e^{-\gamma_n}}{k}\right) (e^{D_n} - 1). \quad (46)$$

Consequently, if $0 \leq \gamma_n \leq 1$ for $n = 0, \dots, N-1$, then

$$e^{D_N} - 1 \leq (e^{D_0} - 1) \exp\left(-\frac{1 - e^{-1}}{k} \sum_{n=0}^{N-1} \gamma_n\right).$$

Proof. For two bank logits with difference $d = z_i - z_j \geq 0$,

$$d_{\text{next}} = d - \gamma_n(r_i - r_j), \quad r_i - r_j = (r_i + r_j) \tanh(d/2) \leq d/2.$$

Hence $d_{\text{next}} \geq 0$ when $\gamma_n \leq 2$. The same extremal indices therefore give $D_{n+1} = D_n - \gamma_n(r_{\max} - r_{\min})$. For $x \in [0, 1]$, convexity of $e^{-\gamma_n x}$ yields $1 - e^{-\gamma_n x} \geq (1 - e^{-\gamma_n})x$. Set $x = r_{\max} - r_{\min}$ and use $e^{D_n} x \geq (e^{D_n} - 1)/k$ from the preceding proof. This gives Equation (46). Multiplication over steps, $1 - y \leq e^{-y}$, and $1 - e^{-\gamma} \geq (1 - e^{-1})\gamma$ for $0 \leq \gamma \leq 1$ prove the last bound. $\square$

These discrete statements concern exact mean gradients, without clipping, sampling noise, optimizer state or alternating replay visits. They establish a finite-step result for the specified update, rather than a guarantee for AdamW. Uniform categorical weights are also essential to the stated target. For fixed nonuniform weights w_b , the exact conditional equation is instead

$$\dot{r}_b = r_b \left\{ \rho \left(w_b - \sum_j r_j w_j \right) - a_\rho M_{\mathcal{B}}(r_b - C_{\mathcal{B}}) \right\}. \quad (47)$$

The inverse-length weights induced by the implemented mean-token loss belong to this weighted case. The uniform contraction theorem therefore describes an equal-length categorical specialization, and does not assert uniform key probabilities for neural exemplar replay.

P.9 COMPLETE RESPONSES AND FINITE EVALUATION VISIBILITY

The learner scores a particular response string, while evaluation groups strings by execution key. Event containment provides a conditional bridge: one complete verified response is one way of producing its key. This bridge also makes the loss's length factor explicit. Here $p_\theta(b \mid x) = \Pr_{Y \sim \pi_\theta(\cdot \mid x)}[V(Y, s_x) = b]$ is the unconditional probability of producing key b , rather than the probability conditional on correctness.

Lemma P.25 (Retention from a joint exemplar potential). *Fix finitely many verified complete-response exemplars (x_j, e_j) , each realizing key b_j , with lengths $L_j > 0$ and fixed weights $\alpha_j > 0$. Let*

$$F(\theta) = \sum_j \alpha_j \frac{-\log \pi_\theta(e_j \mid x_j)}{L_j} - J(\theta), \quad J(\theta) \leq J_{\max} < \infty.$$

Suppose $F(\theta(T))$ is finite and $F(\theta(t)) \leq F(\theta(T))$ for every $t \geq T$. With $D = F(\theta(T)) + J_{\max}$, every protected exemplar satisfies

$$p_\theta(b_j \mid x_j) \geq \pi_\theta(e_j \mid x_j) \geq \exp\left(-\frac{L_j D}{\alpha_j}\right) > 0 \quad (t \geq T).$$

Proof. The weighted exemplar loss equals $F(\theta(t)) + J(\theta(t)) \leq D$. Each summand is nonnegative, so $-\log \pi_\theta(e_j \mid x_j) \leq L_j D / \alpha_j$. The complete-response event is a subset of the event producing key b_j , which proves the stated lower bound on every protected key's probability. $\square$

Exact descent of a smooth joint potential is sufficient for the energy inequality, even with shared parameters and multiple prompts. For example, the ideal categorical model permits $J(\theta) = \sum_x \nu_x \Psi_x(P_x(\theta))$ with fixed nonnegative prompt weights and $J_{\max} = \sum_x \nu_x \max_{s \in [0,1]} \Psi_x(s)$. The lemma does not establish this descent condition for stochastic, clipped, alternating AdamW updates, and it neither implies uniform neural key probabilities nor requires them. Its bound applies only to the listed exemplar–prompt pairs. Shared parameters do not extend it to held-out prompts; transfer to those prompts remains an empirical question.

The scored probability must describe the complete response, including its termination event or a fixed deterministic horizon, under the same prompt, temperature, masks, truncation rules, and verifier as the sampling law being bounded. A prefix score or a score under another decoding law does not supply this bound without a separate event or likelihood comparison. For a complete response of length L and mean-token log score s , its probability is e^{Ls} , not e^s . A change in this exemplar’s likelihood need not equal the change in its whole key’s probability.

Retention and finite evaluation visibility. At a fixed checkpoint, suppose k distinct banked keys for one prompt have probability floors $p_b \geq \delta_b > 0$. For K independent draws from that same policy, the banked distinct count $D_{\mathcal{B},K}$ satisfies

$$\mathbb{E}[D_{\mathcal{B},K}] \geq \sum_{b \in \mathcal{B}} [1 - (1 - \delta_b)^K],$$

$$\Pr(\text{some banked key is unseen}) \leq \sum_{b \in \mathcal{B}} (1 - \delta_b)^K \leq k e^{-K \delta_{\min}},$$

where $\delta_{\min} = \min_b \delta_b$. The first statement follows from indicator expectations and the second from the union bound. Thus $K \geq \log(k/\varepsilon)/\delta_{\min}$ suffices to observe all banked keys with probability at least $1 - \varepsilon$, for $0 < \varepsilon < 1$. The energy floors can make this budget impractically large; asymptotic retention does not imply visibility in an eight-draw evaluation.

Conversely, not observing a mode does not establish extinction. At a fixed checkpoint, zero sightings of a prespecified key in N independent draws give a one-sided $(1 - \alpha)$ binomial upper bound $1 - \alpha^{1/N}$ and lower bound zero (Clopper & Pearson, 1934). For $N = 32$ and $\alpha = .05$, the upper bound is approximately 8.94%. These are single-key bounds; simultaneous statements need error allocation. Counts from changing checkpoints cannot be pooled to certify a current policy, and teacher-forced scores are not independent Bernoulli observations.

P.10 WHEN A REPLAY GRADIENT IMPROVES A NEURAL EXEMPLAR

Lemma P.25 permits shared parameters by assuming descent of a joint potential. A local statement can instead expose the condition under which a realized replay update helps a particular exemplar. This condition involves interference among scores and the optimizer’s response, quantities that the categorical abstraction suppresses.

Proposition P.26 (Local replay response under shared parameters). *For a selected bank of k complete exemplars, define*

$$\ell_b(\theta) = \frac{\log \pi_\theta(e_b | x_b)}{L_b}, \quad h_b = \nabla_\theta \ell_b(\theta), \quad \bar{h} = \frac{1}{k} \sum_{j=1}^k h_j.$$

Suppose a step has the form $\Delta = \eta H(g + \lambda \bar{h}) + u$, where $\eta, \lambda \geq 0$, H is a fixed symmetric positive semidefinite matrix for this step, g is the fresh ascent direction, and u is the remaining update. Assume ℓ_b is twice differentiable and its Hessian has operator norm at most B_b along $\{\theta + s\Delta : 0 \leq s \leq 1\}$. Set

$$K_{bj} = h_b^\top H h_j, \quad \kappa_b = \frac{1}{k} \sum_j K_{bj}, \quad m_b = \eta h_b^\top H g + \eta \lambda \kappa_b + h_b^\top u - \frac{B_b}{2} \|\Delta\|_2^2.$$

Then

$$\ell_b(\theta + \Delta) - \ell_b(\theta) \geq m_b, \quad \pi_{\theta+\Delta}(e_b | x_b) \geq \pi_\theta(e_b | x_b) e^{L_b m_b}. \quad (48)$$

In particular, $m_b > 0$ guarantees that this update increases the complete exemplar’s probability, whether or not it appeared among the fresh samples.

Proof. Taylor’s theorem with the Hessian bound gives $\ell_b(\theta + \Delta) - \ell_b(\theta) \geq h_b^\top \Delta - (B_b/2)\|\Delta\|_2^2$. Substitution of the stated step and $h_b^\top H h = k^{-1} \sum_j K_{bj}$ proves the first inequality. Multiplication by L_b and exponentiation prove the second. $\square$

The required replay response is a row average κ_b of the score Gram matrix, not only its nonnegative diagonal entry K_{bb} . A sufficient interference condition is $\kappa_b > 0$, with replay strength satisfying

$$\eta \lambda \kappa_b > -\eta h_b^\top H g - h_b^\top u + \frac{B_b}{2} \|\Delta\|_2^2.$$

This inequality includes the step’s curvature cost, which itself depends on the chosen dose; arbitrarily increasing λ is not a sufficient rule.

The interference condition cannot be dropped. For example, consider three categorical logits sharing one scalar parameter, with slopes $(1, -2, 0)$. At a point with probabilities $(s, s, 1 - 2s)$, where $0 < s < 1/4$, let the first two categories be correct and banked. Their unit-length log-score gradients are $h_1 = 1 + s$ and $h_2 = -2 + s$. Uniform replay alone follows $\bar{h} = s - 1/2$, so

$$\left. \frac{d}{dt} \log p_1 \right|_{\theta=\bar{h}} = (1 + s)(s - 1/2) < 0.$$

Thus shared parameters can make replay initially reduce the probability of a rare stored mode while improving the average replay score.

For an AdamW step, replay can alter both moment estimates. The proposition therefore does not identify its H with an unchanged AdamW preconditioner. It applies to a specified decomposition of the realized step, with any remaining contribution included in u ; alternatively its Taylor bound can be evaluated directly from $h_b^\top \Delta$. Neither the interference margin nor a curvature bound is established for the experimental runs here. The statement concerns complete stored responses under the sampling law specified in App. P.9. Raising one such probability strengthens that exemplar’s contribution to its key, but need not increase total key probability or transfer to a held-out prompt.

P.11 REFERENCE KL RETAINS THE REFERENCE, NOT THE DISCOVERY

Assumption P.1(A4) sets the reference-KL coefficient to zero, which is what our runs train at (Table 6, $\beta_{\text{KL}} = 0$). The collapse theorem therefore says nothing about the regularizer most RLVR recipes carry, and a reader is entitled to ask whether that regularizer already does the work the bank is introduced for. This subsection answers in the idealized flow. The comparison separates stationary targets from recovery rates. For the Dr.GRPO and MaxRL coefficients, the full-support stationary conditional is the reference’s own. Its restoring logit force vanishes as a mode vanishes, whereas replay supplies a nonvanishing banked-coordinate force. The recovery comparison below is proved on a specified two-mode face; it is not a convergence theorem for all full-support trajectories. App. P.12 compares these mechanisms with measured runs.

Replace (A4) by

(A4’) *Reference KL.* A fixed reference μ with full support on the category set $\mathcal{A}$ enters with a positive coefficient β , and no entropy term is present alongside it.

The low-variance estimator standard in GRPO implementations is unbiased for $\text{KL}(p \parallel \mu)$ under samples drawn from p (Shao et al., 2024), so the reverse direction is the one to analyze; the forward direction is treated at the end of this subsection and behaves differently in exactly the way that matters. Write $D(z) = \text{KL}(p(z) \parallel \mu)$. The prompt-isolated categorical mean flow is

$$\dot{z} = c_G(P) \nabla_z P - \beta \nabla_z D(z), \quad \nabla_{z_b} D = p_b (\log(p_b/\mu_b) - D), \quad (49)$$

which is exact gradient flow of the potential $F_\beta(z) = -\Psi(P(z)) + \beta D(z)$, built exactly as Equation (37) was built for verified replay.

Lemma P.27 (Reverse-KL sublevels and replay barriers). *For a full-support reference, $D = \text{KL}(p \parallel \mu) \leq \log(1/\min_a \mu_a)$ on the simplex, while $R_B \rightarrow \infty$ if a banked coordinate vanishes. More precisely,*

$$\inf_{p:p_b=0} D(p) = \log \frac{1}{1 - \mu_b}.$$

Thus every finite replay-loss sublevel bounds each protected probability away from zero. A reverse-KL sublevel $\{D \leq B\}$ does so for coordinate b if $B < -\log(1 - \mu_b)$, but includes a point with $p_b = 0$ if $B \geq -\log(1 - \mu_b)$.

Proof. The global bound follows from $\sum_a p_a \log p_a \leq 0$ and $-\sum_a p_a \log \mu_a \leq \log(1/\min_a \mu_a)$. On the face $p_b = 0$, write $\tilde{\mu}_a = \mu_a/(1 - \mu_b)$ for $a \neq b$. Then $D(p) = \text{KL}(p||\tilde{\mu}) - \log(1 - \mu_b)$, whose minimum is attained at $\tilde{\mu}$. Continuity and compactness exclude that face from any strictly lower sublevel and give a positive coordinate minimum. For replay, $-\log p_b \leq kR_B$ gives the bound directly. $\square$

Descent of F_β gives $D(p(t)) \leq [F_\beta(z(T)) + \Psi_{\max}]/\beta$. This yields a floor when its right side is below the displayed face threshold, but need not do so for an arbitrary initial energy. The distinction is which sublevels exclude the boundary, not an impossibility of retention under a bounded penalty.

Proposition P.28 (The retained conditional is the reference's). *Under Assumption P.1 with (A4'), using the Dr.GRPO or centered MaxRL coefficient, the flow (49) has exactly one stationary policy of full support,*

$$p_a^* \propto \mu_a e^{\lambda R_a}, \quad \lambda = c_G(P^*)/\beta, \quad R_a = \mathbf{1}[a \in \mathcal{C}],$$

with $\lambda = (G - 1)/(G\beta)$ in closed form for the Dr.GRPO coefficient, and a unique λ for the MaxRL coefficient. Consequently

$$q^* = \mu|_{\mathcal{C}}, \quad \mathcal{H}(q^*) = \mathcal{H}(\mu|_{\mathcal{C}}), \quad \text{logit } P^* = \text{logit } \mu(\mathcal{C}) + \lambda.$$

This stationary conditional does not depend on β : the coefficient sets the correctness level there and leaves the allocation among correct modes at the reference's.

Proof. Stationarity of (49) in a coordinate with $p_b > 0$ reads $c_G(P)(R_b - P) = \beta(\log(p_b/\mu_b) - D)$, so $\log(p_b/\mu_b) = D + (c_G(P)/\beta)(R_b - P)$ and the b -independent terms are absorbed by normalization, giving the stated form with $\lambda = c_G(P)/\beta$. Conversely each $\lambda \geq 0$ determines $P(\lambda) = \mu(\mathcal{C})e^\lambda/(\mu(\mathcal{C})e^\lambda + 1 - \mu(\mathcal{C}))$, strictly increasing from $\mu(\mathcal{C})$ to 1. The consistency requirement $\lambda = c_G(P(\lambda))/\beta$ has a solution by the intermediate value theorem, since the left side increases from 0 without bound while the right side is positive and bounded; it is unique whenever c_G is non-increasing, which covers the constant Dr.GRPO coefficient and $c_G^{\text{MaxRL}}(P) = [1 - (1 - P)^{G-1}]/P$, decreasing from $G - 1$ to 1. Two correct modes give $\log(p_c/\mu_c) = \log(p_d/\mu_d)$, hence $q^* \propto \mu$ on $\mathcal{C}$, which is β -free. The logit identity is $P^*/(1 - P^*) = [\mu(\mathcal{C})/(1 - \mu(\mathcal{C}))]e^\lambda$. $\square$

The stationary target differs from full-coverage uniform replay: Corollary P.21 gives $q \rightarrow u$, whereas the KL stationary allocation is $\mu|_{\mathcal{C}}$. A broader reference changes that allocation; replay instead constructs its target from verified discoveries. The proposition does not bound transient diversity by the reference's diversity or establish convergence of every trajectory to this stationary point, and it does not apply directly to neural optimizer trajectories.

Remark P.29. The correctness prediction also depends on reference correctness $s_\mu = \mu(\mathcal{C})$: $1 - P^* = (1 - s_\mu)/(1 - s_\mu + s_\mu e^\lambda)$. For Dr.GRPO with $G = 16$ and $\beta \leq .1$, $\lambda \geq 9.375$; this amplifies the reference correctness odds by at least $e^{9.375}$ but does not give a reference-independent bound on the correctness deficit.

Lemma P.30 (Reference KL has no probability-independent boundary force). *Under (A4') the logit velocity contributed by the reference-KL term is*

$$\dot{z}_b^{\text{KL}} = \beta p_b (\log(\mu_b/p_b) + D) \xrightarrow{p_b \rightarrow 0} 0, \quad |\dot{z}_b^{\text{KL}}| \leq \beta \left(\frac{\mu_b}{e} + \log \frac{1}{\min_a \mu_a} \right),$$

whereas the verified-replay term contributes $\rho(w_b - p_b) \rightarrow \rho w_b > 0$ at the same limit. This is a property of the exactly evaluated penalty. Its score contains an unbounded log ratio, so it is not an instance of the uniformly bounded-score premise in Lemma P.33; both comparisons nevertheless have a vanishing coordinate force.

Proof. The displayed velocity is the softmax score identity used above. For the bound, $\sup_{0 < s \leq 1} s \log(\mu_b/s) = \mu_b/e$ at $s = \mu_b/e$, the same expression is bounded below by $\log \mu_b$ at $s = 1$, and $0 \leq D \leq \log(1/\min_a \mu_a)$ by Lemma P.27. The replay expression is Lemma P.18. $\square$

Table 58: **Time for a depleted correct mode to return to $p_b = 0.05$, by the depth it starts from.** Numerical integration of the categorical mean flow with $m = 4$ correct modes, $G = 16$, and a uniform reference, in the flow’s own time units. Reference KL multiplies its recovery time by 7.3–8.2 per decade of depth; verified replay adds approximately a constant per decade. Corollary P.31 proves the contrasting asymptotic orders on a separate two-mode face.

Regularizer	$s = 10^{-3}$	10^{-4}	10^{-5}	10^{-6}
Reference KL, $\beta = 0.04$	2.12e+03	1.55e+04	1.20e+05	9.83e+05
Reference KL, $\beta = 0.1$	8.49e+02	6.19e+03	4.81e+04	3.93e+05
Verified replay, $\rho = 0.01$	4.07e+02	6.37e+02	8.68e+02	1.10e+03
Verified replay, $\rho = 0.1$	4.07e+01	6.37e+01	8.68e+01	1.10e+02

Corollary P.31 (Recovery on a two-correct-mode face). *Consider the continuous extension of the categorical probability flow to the face supported on two correct modes b, c , with $p_b = s$, $p_c = 1 - s$. This boundary reduction replaces the full-support start in (A5); here $P = 1$ and the fresh correctness gradient is zero. For reference KL take fixed $\beta > 0$ and $\mu_b, \mu_c > 0$. For replay take the uniform bank $\{b, c\}$ and fixed $\rho > 0$. Fix $0 < \delta < \min\{1/2, \mu_b/(\mu_b + \mu_c)\}$. The times to reach $p_b = \delta$ from $s \downarrow 0$ satisfy*

$$T_{\text{KL}}(s) = \frac{1 + o(1)}{2\beta s \log(1/s)}, \quad T_{\text{replay}}(s) = \frac{\log(\delta/s)}{\rho} + O(1).$$

In particular their ratio diverges. The KL order is $\Theta(1/[s \log(1/s)])$, and the replay order is $\Theta(\log(1/s))$.

Proof. Write $u = p_b$ along this face. The softmax identity $\dot{u} = u(1 - u)(\dot{z}_b - \dot{z}_c)$ gives exactly

$$\dot{u}_{\text{KL}} = 2\beta u^2(1 - u)^2 \log \frac{(1 - u)\mu_b}{u\mu_c}, \quad \dot{u}_{\text{replay}} = \rho u(1 - u)(1 - 2u).$$

Both velocities are positive up to δ by its stated restriction. Separating variables expresses each hitting time as the integral from s to δ of the reciprocal velocity. For KL the integrand is $(1 + o(1))/(2\beta u^2 \log(1/u))$ as $u \downarrow 0$, and its integral is asymptotic to $1/(2\beta s \log(1/s))$ (equivalently, apply l’Hopital’s rule to the two divergent functions). For replay the integrand is $1/(\rho u) + O(1)$, giving the second expression for any fixed admissible target δ . $\square$

Table 58 is a numerical illustration in a larger categorical system, with 4 correct and three incorrect categories. Its reference is uniform, its other depleted logits start at -40 , and its target is $p_b = 0.05$. The KL times grow by a factor 7.3–8.2 per decade of initial depth; replay adds approximately $\log(10)/\rho$ per decade. These finite-depth integrations illustrate the recovery contrast but do not prove the two-mode asymptotic constants for the larger system.

The direction of the divergence decides whether it is a barrier. Forward KL to the same reference is a different object: $\text{KL}(\mu \| p) = -\sum_a \mu_a \log p_a - \mathcal{H}(\mu)$ is exactly the weighted replay loss R_w of App. P.7 with $w = \mu$, so it is unbounded at the boundary and does supply the floor $p_a \geq \exp(-C_{T,\mu}/\mu_a)$. Three differences remain, and they are the design of the bank. Its weights are the reference’s own probabilities, so a mode the base model rarely emits receives a correspondingly weak floor, where uniform replay weights every admitted key alike. Its support is all of $\mathcal{A}$, so it protects incorrect categories on the same footing as correct ones, where the bank is verifier-filtered. And it cannot be estimated from on-policy samples without importance weights or draws from μ , where the bank is a stored sample of the policy’s own verified history. Verified replay is the barrier construction with the target replaced by a policy-generated, validator-accepted, uniformly weighted one.

Correctness at the analyzed targets. For Dr.GRPO and MaxRL with a full-support reference, the interior KL stationary point has $P^* < 1$, because the reference assigns positive mass to incorrect categories. Verified-only cross-entropy instead has the proved limit $P \rightarrow 1$ when $c_G \geq 0$. These statements concern the identified stationary point and convergence theorem, respectively. They do not exclude boundary stationary points of the continuously extended reverse-KL probability flow or establish a universal correctness cost for neural KL training.

Stationary targets and finite training budgets. Full-coverage uniform replay has the same categorical limit for every $\rho > 0$, but this does not make dose selection irrelevant. The finite-time analysis above compares replay with the instantaneous fresh concentration pressure $c_G(P)(1 - P)$. Small doses can allow diversity to decline before the asymptotic target is approached.

For reference KL, increasing β changes stationary correctness through $\text{logit } P^* = \text{logit } \mu(\mathcal{C}) + c_G(P^*)/\beta$. Its boundary recovery time also depends on β ; on the two-mode face the leading scale is $1/[2\beta s \log(1/s)]$. Both coefficients therefore matter at a finite horizon. We have not swept the replay dose empirically, so the finite-time comparison is a prediction of the specified dynamics rather than a measured dose response.

Scope of the comparison. The stationary and rate statements concern independently optimized categorical logits. Schedules, changing references, momentum, and shared neural parameters require separate analysis. The implemented sampled KL term also differs from an exactly differentiated distributional KL: an absent response supplies no direct sampled score, although normalization and shared parameters can still change its probability. Reference KL has guarantees for regularized control and proximity to a reference (Geist et al., 2019; Vieillard et al., 2020). The results here distinguish its stationary target and boundary force from verified replay; they do not exclude useful finite-time diversity preservation by KL.

Corollary P.32 (Two limits and a KL stationary value in the diversity metric). *The diversity axis this paper reports is $PCMD = 1 - C(q) = 1 - \sum_{c \in \mathcal{C}} q_c^2$. In the categorical mean flow, a bounded outcome-binary objective satisfying the positivity and unique-maximum hypotheses of Corollary P.6 sends $PCMD \rightarrow 0$, and adding full-coverage uniform verified replay sends $PCMD \rightarrow 1 - 1/m$ for every dose $\rho > 0$ (Corollary P.21). With the Dr.GRPO or centered MaxRL coefficient, adding a reference KL instead gives a single interior stationary point, at which*

$$PCMD = 1 - \sum_{c \in \mathcal{C}} \left(\frac{\mu_c}{\mu(\mathcal{C})} \right)^2 \quad \text{for every } \beta > 0,$$

the frozen reference’s own success-conditional diversity (Proposition P.28).

Proof. Substitute $\mathbf{e}_{c^*}$, the uniform u on $\mathcal{C}$, and $\mu|_{\mathcal{C}}$ into $1 - \sum_c q_c^2$, using continuity of C on the simplex. $\square$

The first two statements are convergence results under their stated hypotheses; the third identifies an interior stationary point. Lemma P.27 explains when a KL sublevel excludes a face, but a general convergence proof is not supplied here. Neither the stationary value nor the two-mode recovery rate is a bound on transient neural diversity. The distinction between the reference allocation and the discovered-bank target remains useful without that stronger claim.

Two boundaries, and which instrument reaches each. The results in this appendix separate along the two boundaries of the simplex that training actually visits, and they are reached by different means.

At $P \rightarrow 0$ no verified success is available anywhere, and Lemma P.17 bounds the probability of a mixed group — the only kind that carries fresh signal — by GP . This is a *discovery* boundary, and a fresh objective is the instrument for it: Corollary P.8 gives MaxRL a factor approaching G there, and Equation (17) identifies what it is maximizing as a harmonic mixture of $\text{pass}@k$ for $k = 1, \dots, G - 1$.

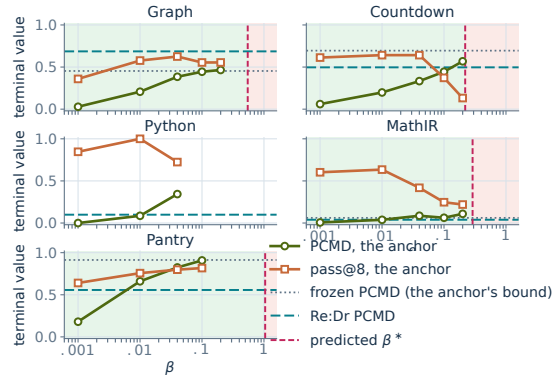
At $p_b \rightarrow 0$ for a discovered correct mode, the bounded sampled-coordinate force vanishes, while verified replay contributes $\rho(w_b - p_b)$. The reference-KL force also vanishes, at a different rate. These are coordinate-level comparisons in the stated geometry: they do not exclude retention through parameter sharing or other update rules.

The fresh and replay terms compose into $F = \rho R_B - \Psi_{\text{MaxRL}}(P)$. Its descent gives the fixed-bank retention guarantee, and the finite-time analysis quantifies when replay overcomes concentration. Their gradients can oppose one another, so composition does not imply an absence of interference. Changing fresh advantages adds arithmetic to the existing update; replay adds stored targets and gradient computation. Equation (50) and App. Q quantify that cost separately from the mathematical comparison.

Table 59: **Terminal PCMD beside categorical reference values.** Columns 11–13 give the frozen reference’s conditional diversity and the replay occupancy summaries $1 - 1/\mathbb{E}|\mathcal{B}|$. The former is the interior stationary value in Proposition P.28, not a transient upper bound. The latter is a Jensen upper bound on the mean uniform-bank limit $\mathbb{E}[1 - 1/|\mathcal{B}|]$ for nonempty fixed categorical banks; it does not certify held-out neural diversity. Re:Dr and Re:Max have separate summaries because their discovered banks differ. Every column but the β ones is the 32-disjoint-stream resample; the β columns use the same estimator on their own disjoint draws. Dashes are coefficients a domain does not yet carry, and a superscript on a β cell is a seed count below the five every other column holds, so a value resting on one run is marked where it is read rather than in a note. The last column is $\beta^* = c_G/(-\log \mu(\mathcal{C}))$, the coefficient at which Proposition P.28 puts stationary correctness at one half, computed from each domain’s own frozen correctness.

Domain	no retention		+ reference KL, by β					+ memory		reference values			
	Dr:GRPO	MaxRL	.001	.01	.04	.1	.2	Re:Dr	Re:Max	KL	Re:Dr	Re:Max	β^*
Graph	0.003	0.148	0.029	0.207	0.385	0.444	0.465	0.686	0.625	0.45	0.75	0.70	0.55
Countdown	0.004	0.022	0.060	0.198	0.333	0.444	0.568	0.497	0.531	0.69	0.56	0.52	0.22
Python	0.000	0.000	0.000	0.086	0.344	—	—	0.099	0.239	—	0.15	0.42	—
MathIR	0.003	0.008	0.007	0.038	0.086	0.062 ²	0.109 ²	0.039	0.018	0.06	0.15	0.12	0.29
Pantry	0.000	0.000	0.180	0.660	0.824	0.908 ¹	—	0.557	0.556	0.91	0.85	0.85	1.05

The results above are statements about a flow. This subsection reads them against measured cells, and the reading changes the conclusion in one place and sharpens it in two others. Table 59 reports terminal PCMD for Dr:GRPO with the same update and fresh-rollout budgets and a reference KL at 5 coefficients, 0.001, 0.01, 0.04, 0.1, 0.2, across 5 domains at Qwen2.5-0.5B and Level 1, beside the matched control and verified-replay arms. Every cell is the control configuration with β alone moved.



What the flow got right. The observed concentration is consistent with the categorical mechanism in Corollary P.6, although neural training does not satisfy that corollary’s update assumptions. The control with the same update and fresh-rollout budgets reaches PCMD at or below .004 in every domain in the table: not reduced diversity but effectively none, which is what a winner-take-all limit predicts and what no reweighting of binary outcomes was able to avoid.

Figure 40: **What a coefficient buys, and where it starts spending.** Green is where Proposition P.28 still holds stationary correctness above one half, red the range past β^* ; the damage follows that boundary in all four domains carrying one. Python has neither, solving too little when frozen to define them.

What the flow got wrong, twice, for one reason. Proposition P.28 makes the stationary conditional β -free. The measured coefficients order PCMD monotonically instead, and the ordering is stable across seeds. The same proposition puts stationary correctness at essentially one for every coefficient at or below .1, even at the cohort’s narrowest reference ($\mu(\mathcal{C}) = 0.0138$, measured on the frozen model in Countdown); the $\beta = .1$ cells instead sit near $\mu(\mathcal{C})$ itself rather than near that limit.

These discrepancies show that the stationary categorical predictions do not quantitatively describe the measured checkpoints. Finite training, shared parameters, and the sampled adaptive optimizer are all possible sources of the difference; this comparison does not identify their separate effects. The boundary-force identities and the two-mode $\Theta(1/[s \log(1/s)])$ versus $\Theta(\log(1/s))$ recovery comparison motivate mechanism checks, rather than certify the measured neural rates.

Reference diversity and bank occupancy. The table compares quantities with different meanings. For a nonempty fixed uniform categorical bank of size k , the limiting diversity is $1 - 1/k$. Across such banks, its mean is $\mathbb{E}[1 - 1/k] \leq 1 - 1/\mathbb{E}[k]$ by Jensen’s inequality. The occupancy columns report the latter summary. They are not exact mean limits when bank sizes differ, and training-bank

occupancy does not furnish a bound on held-out neural PCMD. The KL column reports the reference conditional, which is the analyzed interior stationary value rather than a bound on transient diversity.

The measurements nevertheless motivate an occupancy explanation: where replay discovers few keys, it has few distinct targets to rehearse. Where the frozen model is broader, KL can preserve useful diversity without requiring further admission. These are interpretations of the observed comparison; the categorical results do not identify which mechanism causes a particular neural outcome.

The fresh objective moves the bound, not just the rate. Re:Dr and Re:Max carry separate occupancy columns because a bank is filled by discovery, and Corollary P.8 gives MaxRL a coefficient approaching the group size exactly where correctness is scarce. If that advantage is real outside the flow it should show up as a fuller bank in the domain with the least correctness to begin with, and not elsewhere.

That is what the occupancy columns do. In Python, the domain whose control reaches the lowest correctness of the five, the Re:Max bank holds 1.73 keys per prompt against Re:Dr’s 1.18, lifting the occupancy summary from .15 to .42 and terminal PCMD from .099 to .239. In the four domains that do not start starved, the two occupancies differ by at most .6 of a key and the terminal values track them: Re:Max is ahead in Countdown, level in Pantry, and behind in Graph and MathIR, where its bank is the slightly emptier one. The prediction was specific — a discovery advantage confined to small P — and the one domain it picks out is the one where it appears.

These measurements are consistent with separating fresh learning and memory. The per-prompt mean-direction identity does not itself predict the shared neural trajectory: MaxRL alone reaches .148 PCMD in Graph and at or below .022 elsewhere, no better than Dr.GRPO alone in four domains. The larger Python bank motivates the discovery interpretation, but does not isolate it from prompt weighting, stochasticity, or optimizer effects.

Consequences for the comparison. The claim that a reference KL is the wrong instrument for diversity does not survive this table and is withdrawn. What survives is narrower and better specified. Reference KL retains what its reference had; verified replay rehearses what its bank holds; neither control retains anything. Which of the first two reaches further is a property of the domain — the frozen model’s diversity against the bank’s realized occupancy — and not a property of the method. Sorting the domains by what the best coefficient buys gives three cases rather than a ranking. In Python and Pantry an anchor carries more diversity than either memory arm at no cost in `pass@8`. In Countdown and MathIR it passes them on diversity only at a coefficient that gives correctness up, which is a different claim and is read as one below. In Graph no coefficient reaches either memory arm at all.

Three of this appendix’s results are strengthened rather than weakened by that. The bank-capacity premise of Corollary P.21 stops being a caveat and becomes the quantity that predicts where replay attains its bound. The separation between discovery and retention acquires its cases: the domains where replay does least are the ones where the bank never fills, which is a discovery failure and not a retention failure. And the coefficient window of the preceding paragraph is the one place where the two instruments are not symmetric, because the measured correctness cost at the top of the β range concerns a stationary target tradeoff, whereas the replay dose also controls finite-time restoration. A matched replay-dose sweep would be needed to compare those sensitivities empirically.

P.13 WHY A MEMORY RATHER THAN AN ANCHOR, ONCE THE REFERENCE IS COLLAPSED

App. P.12 withdrew the claim that a reference KL is the wrong instrument for diversity, because in 2 of 5 domains it reaches further than either memory arm and gives up no correctness doing it. This subsection states what remains of the case for a memory, and it is not a restatement of the withdrawn claim. The argument is about which of the two quantities a practitioner is relying on, and about what that quantity is worth in the setting this paper measures.

The case for the anchor, at its strongest. For every $\beta > 0$, the analyzed reference-KL flow has a full-support stationary point. It needs no replay bank, admission rule, or bank capacity, although it retains a reference policy, and adds a familiar loss term. Its best cell is Pantry at $\beta = 0.1$, where it reaches PCMD = 0.908 against the better memory arm’s 0.557 with `pass@8` of 0.817 against 0.797: more diversity and no less correctness. That cell rests on 1 of the five seeds its column will

carry, which is why the table marks it, but nothing in this appendix disputes the measurement, and a reader choosing a method for a domain that resembles Pantry should choose the anchor.

What the anchor is actually buying. Proposition P.28 says the quantity being bought is the reference’s own success-conditional diversity, and App. P.12 finds the measured arms at 91 to 100 percent of it. This empirical agreement is consistent with reference retention, while the stationary formula alone does not describe the finite training trajectory or establish a bound on its diversity.

This is why Pantry is the case it is. The frozen Qwen2.5-0.5B has $\text{PCMD} = 0.91$ there, and .85 is the most a capacity-16 bank assembles from 18.19 admissible keys per prompt. The anchor wins because the reference is broader than anything the bank can hold — not because copying is a better mechanism than accumulating, but because in that one cell there is more worth copying than the bank can accumulate.

How often is there that much worth copying? This paper measures the answer for deployed models. Section 2.3 reports 7 hosted deployments across the benchmark; their macro PCMD spans 0.14–0.31, with Kimi K3 highest at 0.31. Of the 101 reportable model–domain–level cells in that cohort, 7 reach $\text{PCMD} = .50$ and the highest of any of them is 0.74. Pantry’s frozen 0.91 is not a typical reference; it is higher than every cell of every frontier model we measured, including their own Pantry cells.

Read against Proposition P.28, that is the argument. An anchor pointed at a deployed model inherits a macro diversity of roughly 0.31 and typically less. Re:Dr reaches 0.686 in Graph, past what anchoring to any model in that cohort would supply on average, and it does so because the categorical replay target depends on discovered bank membership, whereas the analyzed KL stationary target depends on the reference. The two instruments are not competing for the same resource. One spends a property of the initial policy; the other spends search.

Three further asymmetries, in decreasing strength. The measured knee is the sharpest. In Countdown and MathIR every coefficient that carries the anchor past the memory arms on diversity also costs correctness, and the steepest of them is MathIR at $\beta = 0.2$, where PCMD reaches 0.109 against 0.039 while $\text{pass}@8$ falls to 0.220 from 0.816. The diversity is real and so is the price; in those domains no coefficient buys the first without the second.

The coefficient is also domain-dependent in a way the dose is not, and the table’s last column says why: β^* , the coefficient at which Proposition P.28 halves stationary correctness, is set by each domain’s own frozen correctness and runs from 0.22 to 1.05 across the domains that carry one. A practitioner must therefore locate a window whose position depends on the reference’s correctness, the group size and the estimator, per domain, before training. Replay has the same categorical limit for every positive dose, but its finite-time restoration still depends on the dose relative to fresh concentration pressure.

Last and most structural: an anchor requires a reference that is worth anchoring to, and successive rounds of RL do not supply one. The bank is assembled from the policy’s own verified successes and is available on the second round for the same reason it was on the first.

What would change this reading. The argument rests on a measured claim about references, not a theorem about mechanisms, so it is defeasible in an obvious way: a setting whose base policies carry diversity like Pantry’s, rather than like the hosted cohort’s, is a setting where anchoring is the better primitive and this appendix says so. The evidence here is also thin exactly where it is most favourable to the anchor. 3 of the 22 measured coefficient cells stand on fewer than five seeds and the thinnest on 1, the table marks each one, and the cells still filling are the high- β ones the anchor’s case rests on. MathIR’s support is keyed to derivations rather than to answers besides, so a mode certified there need not be one any policy produces. The anchor’s free wins are also the domains where the bank is furthest from holding what the prompt admits, which is the reading App. P.12 already gives them: they are discovery failures on the memory side rather than retention successes on the anchor’s. One of them does not even fit the inheritance account, since a reference that verifies nothing has no success-conditional diversity to pass on, so what the coefficient supplies there cannot be inferred from the stationary conditional formula. Zero observed successes also does not establish zero population correctness. We report them as wins regardless.

P.14 SCOPE RELATIVE TO ENTROPY, KL, ADMISSION, AND NEURAL OPTIMIZATION

Dependence on optimization geometry. For a smooth parameterization $z = z(\theta)$ with Jacobian $J_\theta = \partial z / \partial \theta$, Euclidean parameter flow instead induces $\dot{z} = c_G(P) J_\theta J_\theta^\top \nabla_z P$. The independently trained logit theorem uses $J_\theta J_\theta^\top = I$ and does not automatically extend to arbitrary shared parameters or another optimization metric. Equal binary reward alone does not force this winner selection: the categorical natural-gradient reward flow can be represented by $\dot{z}_a = c_G(P)(R_a - P)$, which gives identical velocities to correct logits and preserves their ratios. Here $R_a = \mathbf{1}[a \in \mathcal{C}]$.

The mean-gradient identity has broader scope than the collapse conclusion. For one prompt and a common autonomous preconditioner $M(\theta)$, positive coefficients give flows $c_a(P)M(\theta)\nabla_\theta P$ that follow the same orbit after a change of time. Outcome-dependent covariance, response-length weights, shared prompts, and optimizer memory can break the corresponding training-trajectory equivalence, as the preceding neural analysis shows. Parameter count alone does not identify this geometry. The empirical scale comparisons change pretrained models and schedules; they do not establish the cause of cross-scale differences or the training history of a hosted model.

A bounded advantage based on fresh samples has a different boundary behavior. The next bound includes group-dependent detached scores and distinguishes sampled updates from gradients of an exactly evaluated entropy functional.

Lemma P.33 (Bounded sampled scores have no probability-independent boundary force). *Under Assumption P.1, condition on the training history, draw $Z_1, \dots, Z_G$ independently from p , and let a_i be any detached, possibly group-dependent advantages with $|a_i| \leq B$. To interpret the bound uniformly as $p_b \rightarrow 0$, assume a finite B independent of that limit. At the on-policy point define*

$$\hat{g} = \frac{1}{G} \sum_{i=1}^G a_i (e_{Z_i} - p).$$

For every category b ,

$$|\mathbb{E}[\hat{g}_b]| \leq \mathbb{E}|\hat{g}_b| \leq 2Bp_b(1 - p_b).$$

On a group with no draw of b , exactly $\hat{g}_b = -p_b G^{-1} \sum_i a_i$; it is zero if the applied advantages sum to zero. By comparison, the verified-replay component for a fixed target w has logit velocity $\rho(w_b - p_b)$, which approaches $\rho w_b > 0$ for a banked category as $p_b \rightarrow 0$.

Proof. Use the triangle inequality and $|a_i| \leq B$. Each marginal draw satisfies $\mathbb{E}|\mathbf{1}\{Z_i = b\} - p_b| = 2p_b(1 - p_b)$, irrespective of the dependence of a_i on the group. On the absence event all indicator terms are zero. The replay formula follows by differentiating $-\sum_b w_b \log p_b$. $\square$

The bound compares the isolated logit-gradient components. It does not prove inevitable collapse, rule out retention by a bounded estimator, or guarantee that replay dominates the full update. An unsampled mode can still change through softmax normalization or shared parameters. Exact entropy is outside the uniformly bounded-score premise because its surprisal is unbounded near zero; the lemma is not an impossibility result for entropy regularization. Nor does the absence of a mode from a finite sample establish zero probability.

Exact entropy is a different comparator. For a known full correct support and uniform u , the elementary identities

$$H(q) = \log m - \text{KL}(q||u), \quad R_u(p) = \log m - \log P + \text{KL}(u||q)$$

show that exact conditional entropy and categorical replay both favor a uniform conditional target. With strictly increasing correctness potential Ψ , both $\Psi(P) + \beta H(q)$ and $\Psi(P) - \rho R_u(p)$ have the unique global optimum $P = 1, q = u$ in probability space on the simplex with $P > 0$, for positive coefficients. This is a boundary limit for finite-logit softmax policies. The optimum follows from nonnegativity of KL and increasing Ψ ; it does not assert convergence of either neural update. The KL direction distinguishes their loss level sets (Pereyra et al., 2017; Agarwal et al., 2021): replay loss diverges when a protected coordinate vanishes, whereas conditional entropy can remain $\log(m - 1)$ with one missing mode.

Exact full-output entropy is a third objective and also regularizes incorrect mass. Exact tabular entropy-regularized policy gradient has its own positive probability and convergence guarantees under specified assumptions (Mei et al., 2020); our collapse theorem contains no entropy term. The sampled semantic scores in the experiments are not exact gradients of full-output or conditional verified-key entropy. Neither their empirical comparison nor the bounded-score lemma is an impossibility result for entropy.

Discovery and optimization remain separate requirements. Retention starts after a verified key enters active memory. Actual insertion depends on generation, validation, capacity, and eligibility; a positive sampling probability alone does not establish eventual admission. Once a non-evicting capacity-16 bank fills, an unbanked key can remain ineligible. Changing membership or exemplar weights changes the loss being bounded, so fixed-bank descent cannot simply be carried across those changes. In particular, the fixed-weight argument does not cover a target whose weights continue to change with fresh observation counts.

For a fixed joint potential, a discrete bound $F(\theta_{n+1}) \leq F(\theta_n) + \epsilon_n$ with $\epsilon_n \geq 0$ and $\sum_{n \geq n_0} \epsilon_n = E < \infty$ would extend Lemma P.25, using $D = F(\theta_{n_0}) + J_{\max} + E$ for steps $n \geq n_0$. This follows by telescoping and bounds retention, not convergence. The noisy optimizer would need a separate argument establishing such control of the joint loss’s upward drift. A positive replay coefficient or small nominal learning rate does not establish it. Adam moments, clipping, momentum, prompt scheduling, and bank changes are not covered by the exact-flow proofs. The experiments have not established the required joint-energy or update-error assumptions. The mathematical contribution is the stated categorical mechanism and conditional exemplar argument; empirical support and longitudinal exemplar scores remain separate measurements.

Q WHAT VERIFIED REPLAY COSTS

Q.1 WHAT THE CONTROL MATCHES, AND WHAT IT DOES NOT

The core factorial matches initialization, data, optimizer settings, 3,072 optimizer updates, and $G = 16$ fresh rollouts per visited prompt. Both arms maintain banks and use the same configured replay traversal; the control replaces the replay score derivative with exact zeros. Replay gradients join the fresh-task gradients before the existing optimizer step. Activating replay therefore does not add an optimizer step or a fresh rollout, and the control already incurs replay computation.

This design isolates the effect of applying the replay learning signal under the common schedule. It does not match total training compute: policies can produce different rollout lengths and populate different banks, changing whether replay executes and how many tokens it processes. Equal code paths and update counts do not imply equal FLOPs or wall-clock time. More fundamentally, the control’s replay work contributes no learning signal and is not reassigned to useful baseline updates. We do not report a control that spends an equivalent budget on extra task-gradient steps, extra fresh rollouts, or a longer training horizon. The results therefore do not establish that replay is the best use of an equal total-compute budget.

The uniform-versus-frequency weighting ablation addresses a different question: how to allocate replay weight among stored modes. It retains the same bank construction, scheduling, and update rule, but its policies can also develop different workloads. It supports a contribution from balancing modes; it is not a comparison of performance at equal total compute.

Q.2 TOKEN-BASED COST ACCOUNTING AND ITS LIMITS

The retained accounting uses diagnostics from 75 replay runs: five seeds in each of five domains at Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B. Table 60 summarizes those logs without new training. It compares components of replay and a hypothetical reference-KL implementation; it is not a measured replay/control runtime comparison.

Resident state. Under full fine-tuning, a frozen reference copy requires $\Theta(|\theta|)$ parameter bytes. A replay bank stores prompt and exemplar token identifiers, whose cost depends on the number of prompts, retained keys, and token lengths rather than directly on parameter count. The esti-

Table 60: **Component cost estimates from retained replay diagnostics.** The first row per family gives mean selected-bank occupancy, reference parameter bytes, estimated bank token bytes, and their storage ratio. The second row is the likelihood-pass/reference-forward FLOP ratio of Equation (50); it excludes the detached scoring pass and is not a replay/control overhead ratio. Diagnostics are averaged within each run, then across five seeds per cell (75 runs overall).

Family	Graph	Countd.	Python	MathIR	Pantry	Ref (GB)	Bank (kB)	Ratio
Qwen2.5-0.5B	3.98	2.28	1.18	1.17	6.74	0.99	387	2,554
ratio	0.75	0.43	0.22	0.22	1.26			
Falcon3-1B	3.83	1.88	1.24	1.20	8.76	3.34	444	7,520
ratio	0.72	0.35	0.23	0.23	1.64			
Qwen2.5-3B	3.69	1.50	1.27	1.03	6.46	6.17	393	15,697
ratio	0.69	0.28	0.24	0.19	1.21			

mated mean bank footprint is at most 444 kB across these families, while the reference copy grows from 0.99 to 6.17 GB. The corresponding reference-to-bank ratios are 2,554–15,697. These are component storage estimates, not peak accelerator-memory measurements: bank bytes assume four bytes per token and exclude bookkeeping, optimizer state, and activations. A reference recovered by disabling an adapter need not incur a separate resident parameter copy.

A likelihood-pass subtotal, not total training FLOPs. Let T_{replay} include the prompt and response tokens processed by a selected bank, and T_{fresh} the corresponding fresh-group token estimate. Approximating a forward pass by $2|\theta|T$ and its backward by $4|\theta|T$, the table prices one differentiable replay forward/backward at $6|\theta|T_{\text{replay}}$ and one reference forward at $2|\theta|T_{\text{fresh}}$. Thus its reported ratio is

$$\frac{\text{replay likelihood-pass FLOPs}}{\text{reference-forward FLOPs}} \simeq \frac{3T_{\text{replay}}}{T_{\text{fresh}}} \approx \frac{3\mathbb{E}|\mathcal{B}|}{G}, \quad (50)$$

where the final approximation additionally assumes comparable fresh and stored sequence lengths. Stored exemplars and current rollouts need not have the same lengths, so occupancy alone does not determine the ratio. The token-based cell means range from 0.19 to 1.64; the subtotal is below one in 12 of 15 cells. The three exceptions are PantryPlan. This denominator is a reference forward, not the baseline’s complete training cost.

The maintained implementation also makes a detached scoring forward before the differentiable replay pass. Under the same approximation, these passes together would cost about $8|\theta|T_{\text{replay}}$, before activation recomputation and other system costs. The table’s $6|\theta|T$ subtotal omits that scoring pass and cannot certify actual historical FLOPs. Both the active and exact-zero control paths retain scoring and backward computation; this subtotal must not be interpreted as their measured cost difference.

Measurement scope. The builder averages retained rows with replay diagnostics and an available fresh-length measurement, then averages runs within each model–domain cell. It does not deduplicate repeated step records or truncate them to the 3,072-update horizon. Fresh-group token counts are reconstructed from logged response lengths and a prompt-length estimate derived from replay rows. These are descriptive workload summaries, not a cumulative compute audit. They exclude rollout generation, padding, attention-specific costs, activation recomputation, verification, bank maintenance, optimizer work, communication, and evaluation. Hardware pools also differ across cohorts. Neither these estimates nor the reference-KL comparison establishes an end-to-end speedup or resolves the missing budget control.

Q.3 THE MISSING COMPARISON AT EQUAL TOTAL COMPUTE

A separate comparison should give the baseline a fixed cumulative training budget and let it spend replay’s allocated work on useful learning. Relevant controls are additional task-gradient passes over fresh groups already collected, larger fresh-rollout groups, and additional fresh-sampling updates beyond the eight-pass horizon. These alternatives change different resources; none is represented by setting the replay derivative to zero. Their correctness and PCMD should be evaluated at the same total-compute budget, with the same initialization, task splits, seeds, evaluation requests, and hardware configuration.

Such a comparison needs either measured cumulative FLOPs or timing on controlled hardware, including rollout generation and every learner pass. It should report both the budget achieved and any

6426 overshoot from a final indivisible update, with evaluation costs treated consistently. Until that experi-
6427 ment is available, our empirical conclusion is that applying verified replay gradients improves the
6428 reported aggregate outcomes under the fixed update and fresh-sampling schedule. Whether a baseline
6429 can match those outcomes by spending the same total compute on more useful training remains open.
6430

6431
6432
6433
6434
6435
6436
6437
6438
6439
6440
6441
6442
6443
6444
6445
6446
6447
6448
6449
6450
6451
6452
6453
6454
6455
6456
6457
6458
6459
6460
6461
6462
6463
6464
6465
6466
6467
6468
6469
6470
6471
6472
6473
6474
6475
6476
6477
6478
6479

Part VII. Extended comparison

R DIVERSITY-PRESERVING RLVR: EXTENDED COMPARISON

That RLVR concentrates its outputs is not our discovery, and this appendix states what three contemporary lines establish and exactly which of our claims each one already covers. Cui et al. (2025) identify entropy collapse in reasoning RL, attribute the fall to a covariance between token probability and advantage, and fit a ceiling on downstream reward as a function of remaining entropy; Clip-Cov and KL-Cov restrict the high-covariance tokens and postpone the fall. Lochab et al. (2026) show that RLVR’s objective is indifferent to how correct probability is divided among correct answers, characterize the uniform-correct policy as the optimum of a robustness and entropy-regularized criterion, and add a conditional uniformity penalty to GRPO. Li et al. (2026b) attribute the same concentration to reverse-KL mode seeking and replace it with an approximate forward-KL objective that aligns the group to a target proportional to sampled rewards. We take the phenomenon as established by this literature and do not claim it. Our three separable claims are a success-conditional measurement, an intervention that is not a fresh-update objective, and a stated reason the two classes differ in reach.

The axis each work reads. Cui et al. (2025) track policy entropy over all sampled tokens. That quantity mixes correct and incorrect mass, so it falls both when a policy stops offering alternative correct answers and when it stops offering wrong ones, and a training run that improves only by suppressing errors registers as a loss of diversity. Lochab et al. (2026) report $\text{pass}@K$ and an equation-level count within the correct set; Li et al. (2026b) read diversity through downstream score on problems with many feasible solutions. $\text{pass}@K$ and downstream score rise with the success rate on their own: (1) makes the dependence explicit, and across the 375 untrained measurements of App. B.4 $\text{distinct}@8$ correlates .936 with $\text{pass}@8$. PCMD removes that dependence by construction — it is a function of q alone — so correctness and diversity are two axes a method can move in opposite directions, and the concentration Sec. 5.1 reports is not a restatement of a change in the success rate. MODEBENCH also fixes sameness with the executable check that assigns reward, so the equivalence used to define a mode is the one used to grant credit, with no judge or embedding threshold between them.

Where the intervention can act. All three interventions modify the fresh on-policy update. Clip-Cov and KL-Cov reweight tokens of the current rollouts; UCPO reallocates a prompt group’s positive advantage mass across that group’s verifier-positive rows; DMPO forms its target over the trajectories the group sampled. Each therefore requires a mode to be present in the current group before it can act on that mode, and that requirement is the pressure App. P formalizes. Lemma P.33 applies to this whole class whenever the per-row scores a_i are detached and uniformly bounded by some B : the expected logit velocity on category b is at most $2Bp_b(1 - p_b)$, which vanishes with p_b , and on a group containing no draw of b the component is exactly $-p_b G^{-1} \sum_i a_i$. UCPO’s estimator satisfies the premise as implemented here. Write G^+ for the number of verifier-positive rows in the group, A^+ for the per-row positive advantage a uniform split would assign, $\hat{q}_i$ for row i ’s estimated within-group mode frequency, and $\omega \in [0, 1]$ for the weight UCPO puts on inverse-frequency rather than uniform reallocation. Such a row receives $A_i = G^+ A^+ ((1 - \omega)/G^+ + \omega v_i / \sum_j v_j)$ with $v_i = 1/\hat{q}_i$ self-normalized within the group, so $|A_i| \leq G^+ |A^+|$ and the total positive mass is conserved — it is a redistribution among sampled correct rows, not an addition to an absent one. Verified replay is outside the premise because its target is not a function of the current group: for a banked key the logit velocity is $\rho(w_b - p_b)$, which approaches $\rho w_b > 0$ as $p_b \rightarrow 0$. This is a statement about which mechanism retains a signal at the boundary, and App. P.14 records what it is not: it does not prove that collapse is inevitable, does not rule out retention by a bounded estimator, and is not an impossibility result for entropy regularization, whose surprisal is unbounded near zero and therefore fails the bounded-score premise.

The target is shared; the weighting is not. UCPO’s uniform-correct target is the target our replay loss descends, and App. P.14 shows that exact conditional entropy shares that optimum too, the three differing in the direction of the KL and hence in what happens to the loss when a mode’s probability vanishes. So the disagreement with Lochab et al. (2026) is not about where to aim. DMPO is the one of the three that aims elsewhere: a target proportional to rewards over

sampled trajectories is, under a binary verifier, uniform over the correct sampled trajectories, which places mass on each mode in proportion to how many times the group produced it. That is frequency weighting, and it is the control we run directly. Holding the bank and the reuse fixed and weighting modes by rediscovery frequency instead of equally costs $+.318$ extra verified modes per prompt ($[+.272, +.358]$) with no aggregate correctness difference (App. F.6). Reuse is not the active ingredient; declining to let popularity set the rehearsal rate is.

What we measured rather than cited. Two of the three lines above are implemented directly and run as matched comparators rather than only discussed; the third is covered by the frequency-weighting control already described, and sparse RLEP-Dr is added beside them for the memory side. UCPO is implemented from its Equations 16, 47 and 51, and runs against seed-matched Dr.GRPO at Qwen2.5-0.5B and Falcon3-1B (App. F.4); its gains are local, raising Countdown correctness and PCMD there while its MathIR effects are inconclusive. An entropy-shaped objective on the success-conditional axis is implemented as Fixed Semantic MaxEnt (App. E), which supplies a bounded surprisal bonus to underrepresented verified outcomes that appear in a fresh rollout; its effects are likewise heterogeneous, and App. E states why it carries neither an exact entropy-gradient interpretation nor an individual-mode preservation guarantee. Sparse RLEP-Dr covers the memory side (Zhang et al., 2025): the same reuse of verified successes, without uniform canonical-key weighting.

Scope of the comparison. These comparators are read at the coverage they have. UCPO and sparse RLEP-Dr run at two of our three scales on MODEBENCH, whose executable domains are not the mathematical reasoning suites of Lochab et al. (2026) or the NP-Bench problems of Li et al. (2026b), and the token-level interventions of Cui et al. (2025) act at a different granularity from the outcome-level comparators we run. Nothing here ranks these methods on their own benchmarks. The claim is narrower and is the one the lemma and the weighting ablation support jointly: an objective confined to the current group and a memory over verified keys reach different modes, and uniform rehearsal over those keys is what separates our memory from replay that revisits successes as often as it found them.

Part VIII. Records

S SOURCE INTEGRITY AND REPRODUCIBILITY

S.1 ADMISSIBLE SOURCES AND MISSING CHECKPOINTS

Falcon3-1B Countdown Re:Dr seed 59 contains two complete four-draw terminal evaluation sequences with identical request identities but conflicting metrics. That source is excluded at every checkpoint and from every efficacy endpoint, paired contrast, and aggregate that reuses it; neither sequence is selected or averaged. The independently valid Dr.GRPO and GRPO seed-59 sources remain admissible. The core replay comparison therefore has 74 pairs: 14 complete five-seed blocks and Falcon Countdown at $n = 4$ (seeds 55–58). Primary partial intersections are descriptive and receive no confidence interval.

A second kind of conflict is more common and has a diagnosable cause. The job watchdog can requeue a run; the restart finds no resumable checkpoint, begins from initialization, and evaluates the initial model a second time, appending a second step-0 block to the same record. Separate attempts can land on different GPU models, and GPU model changes floating-point reduction order and therefore the sampled tokens, so the two blocks disagree and the checkpoint is invalidated under the rule below. This is why several MaxRL initial checkpoints are unavailable, leaving the longitudinal contrast with three defined seeds in Falcon3-1B Graph and one in Falcon3-1B PantryPlan. The same hardware sensitivity means that initial evaluations of the identical frozen base model agree within a GPU pool and can differ across pools, which is one reason initial evaluations are measured within each method rather than shared across arms; a before–after contrast compares a method with its own initial checkpoint and is unaffected. Of the step-0 blocks in a run’s records, exactly one is the block its admitted trajectory continues from, and App. S.2 admits that block for the 27 cells where it is unique.

Trajectory records use registered sources and approved continuations or replacements, with superseded attempts excluded. Identical duplicates are deduplicated; conflicting records invalidate the affected checkpoint rather than being selected by metric value. Missing or ambiguous checkpoints remain gaps; curves do not interpolate or carry values forward across them. An ambiguous nonterminal checkpoint does not invalidate an independently admissible terminal endpoint. Each curve declares its fixed source-admissible seed cohort; paired core curves use the admitted intersection. Accuracy and mode-count views use the same populations and gap rules. Supporting methods retain their explicit method-specific cohorts; available checkpoint counts and gaps are recorded alongside the curves. A curve point is drawn from the cohort seeds that report at that checkpoint and is ringed where that is fewer than all of them, so a checkpoint whose evidence exists for part of the cohort is read as the partial measurement it is rather than as an absence; a checkpoint no seed reports remains a gap. Machine-readable records identify the sources, exclusions, exact seed sets, checkpoint rule, and source hashes.

S.2 CONFLICTED INITIAL CHECKPOINTS, ADMITTED BY PROVENANCE

A conflicted step-0 endpoint is a requeue duplicate, not a missing evaluation, and the two attempts disagree only because they executed on different GPU models. For each affected cell the analysis admits the step-0 block that the cell’s admitted trajectory continues from — chosen by file order, step number and the presence of training rows, never by a metric value. The rule resolves 27 of the 36 conflicted cells and leaves nine, where more than one such block exists, unavailable.

Two checks bound what that admission moves. The twelve Dr.GRPO and GRPO before–after comparisons that carry Section 5.1 are unaffected and reproduce to 10^{-9} , and no terminal replay-effect block changes at all, since those compare two trained checkpoints and use no initial endpoint. What it buys is precision rather than a new direction: every affected block moves from one to four defined seeds up to the registered five and receives a nominal interval, while its sign is unchanged.

It does not license a broader collapse claim. Falcon3-1B Graph under MaxRL moves the other way, with an interval excluding zero, as does its replay variant; MaxRL concentrates on Countdown, MathIR and PantryPlan at that scale but not on Graph. We therefore state the training-induced concentration result for the RLVR-only objectives it was measured on, and report the MaxRL longitudinal cohort as the mixed result it is.

S.3 CONFLICTED REGISTERED CHECKPOINTS, ADMITTED BY PROVENANCE

The requeue that duplicates a step-0 block duplicates blocks at registered training steps in the same way, and the frozen snapshot invalidates those checkpoints under the same rule, which the training-curve figures draw as gaps. The step-0 rule above extends to any registered step: within a source file whose attempt trained past that step, a block of four draws counts as continued when the next evaluation row carries a later step, and the checkpoint is admitted only where exactly one continued block exists across the cell’s files. At step 0 this is that rule verbatim, so the 27 initial admissions are carried by the same mechanism. It reads file order, step numbers, draw indices and the presence of later training rows, never a metric value.

Of the 70 conflicted registered checkpoints away from step 0, the rule resolves 27. The remaining 43 carry more than one continued block, because the run was requeued and more than one attempt trained past that step, so nothing in the source structure separates them. They stay unavailable, and the figures draw them as gaps rather than interpolating across them. Only the terminal export of each run was retained, so the mid-training weights those checkpoints evaluated no longer exist and no re-evaluation can settle them.

S.4 DATA, MODELS, AND SCIENTIFIC RECORDS

The ModeBench dataset provides separate level/domain configurations, fixed split roles, full row identities, and construction and admission records. Its loading guide and evaluation guide identify the reported splits and preserve auxiliary data separately. The model archive links each available model–domain–method–seed export to an immutable upload revision containing original weights, configuration, tokenizer, file hashes, completion provenance, and the applicable base-model license. The experiment index maps the core scale comparisons, MaxRL factorial, Level-2 factorial, replay-weighting ablation, and supporting controls to their model families; the scientific-record index and restore guide identify preserved analysis inputs and SHA-256 verification procedures.

Each empirical figure has a machine-readable record identified in the figure manifest, containing its displayed values and frozen numerical inputs. `paper/FIGURE_MANIFEST.md` maps both paper versions to their builders, records, and reproduction commands. Detailed source and recovery inventories remain in those records and the repository’s archived audit documents. Model availability may extend beyond the frozen analysis population; it does not change a figure’s source-admission or seed-intersection rule. Terminal exports restore inference models, while intermediate trajectories require the recorded evaluation checkpoints.

S.5 EXACT CONTINUATION AND VALIDATION

Exact training continuation requires the atomic optimizer, bank, token, cursor, and request-stream state in App. D.4, in addition to model weights. State mismatch, non-finite values, missing completion evidence, changed dataset identities, and source or aggregate hash drift are hard failures. Placement-only recovery may change hardware or wall time but must preserve the scientific update. Figure reproduction uses frozen records; missing results are never replaced by shallower checkpoints or silently incorporated from a newer archive.